Mining the Social Web

Jolt生产效率大奖获奖图书

社交网站的数据挖掘与分析

REILLY®

机械工业出版社
China Machine Press

HZ BOOKS

Matthew A. Russell 著

师蓉 译

社交网站的数据挖掘与分析

Facebook、Twitter和LinkedIn产生了大量宝贵的社交数据，但是怎样才能找出谁通过社交媒介正在进行联系?他们在讨论些什么?或者他们在哪儿？本书简洁而且具有可操作性，它将揭示如何回答这些问题甚至更多的问题。你将学到如何组合社交网站数据、分析技术，如何通过可视化找到你一直在社交世界中寻找的内容，以及你闻所未闻的有用信息。

本书每章都介绍了在社交网络的不同领域挖掘数据的技术，这些领域包括博客和电子邮件。你所需要具备的就是一定的编程经验和学习基本的Python工具的意愿。

- 获得对社交网络世界的直观认识。
- 使用GitHub上灵活的脚本来获取Twitter、Facebook和LinkedIn等社交网络API中的数据。
- 学习如何应用便捷的Python工具来交叉分析你所收集的数据。
- 通过XFN探讨基于微格式的社交联系。
- 应用诸如TF-IDF、余弦相似性、搭配分析、文档摘要、派系检测之类的先进挖掘技术。
- 通过基于HTML 5和JavaScript工具包的网络技术建立交互式可视化。

马修·罗塞尔（Matthew A. Russell），Digital Reasoning Systems公司的技术副总裁和Zaffra公司的负责人，是热爱数据挖掘、开源和Web应用技术的计算机科学家。他也是《Dojo: The Definitive Guide》（O'Reilly出版社）的作者。在LinkedIn上联系他或在Twitter上关注@ptwobrussell，可随时看到他的最新动态。

"本书是《Programming Collective Intelligence》一书的深入篇，它介绍通过Python从社交网站中采集数据的一种实践方法。"

——Jeff Hammerbacher，
Cloudera首席科学家

"对于探索结构化和非结构化数据的一系列工具、技术和理论，本书给出了丰富、紧凑并实用的介绍。"

——Alex Martelli，
Google高级主管工程师，
《Python in a Nutshell》的作者

O'REILLY®
oreilly.com.cn

客服热线：(010) 88378991，88361066
购书热线：(010) 68326294，88379649，68995259
投稿热线：(010) 88379604
读者信箱：hzjsj@hzbook.com
华章网站：http://www.hzbook.com
网上购书：www.china-pub.com

O'Reilly Media, Inc.授权机械工业出版社出版

此简体中文版仅限于在中华人民共和国境内（但不允许在中国香港、澳门特别行政区和中国台湾地区）销售发行
This Authorized Edition for sale only in the territory of People's Republic of China (excluding Hong Kong, Macao and Taiwan)

ISBN 978-7-111-36960-8
9 787111 369608
定价：59.00元

社交网站的数据挖掘与分析

Matthew A. Russell 著

师蓉 译

O'REILLY®

Beijing • Cambridge • Farnham • Köln • Sebastopol • Tokyo

O'Reilly Media, Inc.授权机械工业出版社出版

机械工业出版社

欢迎加入非盈利Python编学习交流程QQ群783462347，群里免费提供500+本Python书籍！

图书在版编目（CIP）数据

社交网站的数据挖掘与分析/（美）罗塞尔（Russell, M. A.）著；师蓉译.
—北京：机械工业出版社，2012.1
(O'Reilly精品图书系列)

书名原文：Mining the Social Web
ISBN 978-7-111-36960-8

I. 社… II. ①罗… ②师… III. 数据采集 IV. TP274
中国版本图书馆CIP数据核字（2011）第280179号

北京市版权局著作权合同登记
图字：01-2011-1505号

Copyright © 2011 by O'Reilly Media, Inc.

Simplified Chinese Edition, jointly published by O'Reilly Media, Inc. and China Machine Press, 2012. Authorized translation of the English edition, 2011 O'Reilly Media, Inc., the owner of all rights to publish and sell the same.

All rights reserved including the rights of reproduction in whole or in part in any form.

英文原版由O'Reilly Media, Inc. 出版2011。

简体中文版由机械工业出版社出版 2012。英文原版的翻译得到O'Reilly Media, Inc.的授权。此简体中文版的出版和销售得到出版权和销售权的所有者——O'Reilly Media, Inc.的许可。

版权所有，未得书面许可，本书的任何部分和全部不得以任何形式重制。

封底无防伪标均为盗版
本书法律顾问
北京市展达律师事务所

书　　名/　社交网站的数据挖掘与分析
书　　号/　ISBN 978-7-111-36960-8
责任编辑/　谢晓芳
封面设计/　Karen Montgomery，张健
出版发行/　机械工业出版社
地　　址/　北京市西城区百万庄大街22号（邮政编码 100037）
印　　刷/　北京京师印务有限公司
开　　本/　178毫米×233毫米　16开本　19.75印张
版　　次/　2012年2月第1版　2012年5月第3次印刷
定　　价/　59.00元（册）

凡购本书，如有缺页、倒页、脱页，由本社发行部调换
客服热线：(010)88378991；88361066
购书热线：(010)68326294；88379649；68995259
投稿热线：(010)88379604
读者信箱：hzjsj@hzbook.com

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

O'Reilly Media, Inc.介绍

O'Reilly Media通过图书、杂志、在线服务、调查研究和会议等方式传播创新知识。自1978年开始，O'Reilly一直都是前沿发展的见证者和推动者。超级极客们正在开创着未来，而我们关注真正重要的技术趋势——通过放大那些“细微的信号”来刺激社会对新科技的应用。作为技术社区中活跃的参与者，O'Reilly的发展充满了对创新的倡导、创造和发扬光大。

O'Reilly为软件开发人员带来革命性的“动物书”；创建第一个商业网站（GNN）；组织了影响深远的开放源代码峰会，以至于开源软件运动以此命名；创立了Make杂志，从而成为DIY革命的主要先锋；公司一如既往地通过多种形式缔结信息与人的纽带。O'Reilly的会议和峰会集聚了众多超级极客和高瞻远瞩的商业领袖，共同描绘出开创新产业的革命性思想。作为技术人士获取信息的选择，O'Reilly现在还将先锋专家的知识传递给普通的计算机用户。无论是通过书籍出版，在线服务或者面授课程，每一项O'Reilly的产品都反映了公司不可动摇的理念——信息是激发创新的力量。

业界评论

“O'Reilly Radar博客有口皆碑。”

——Wired

“O'Reilly凭借一系列（真希望当初我也想到了）非凡想法建立了数百万美元的业务。”

——Business 2.0

“O'Reilly Conference是聚集关键思想领袖的绝对典范。”

——CRN

“一本O'Reilly的书就代表一个有用、有前途、需要学习的主题。”

——Irish Times

“Tim是位特立独行的商人，他不光放眼于最长远、最广阔的视野并且切实地按照Yogi Berra的建议去做了：‘如果你在路上遇到岔路口，走小路（岔路）。’回顾过去Tim似乎每一次都选择了小路，而且有几次都是一闪即逝的机会，尽管大路也不错。”

——Linux Journal

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

本书赞誉

“这是一本非读不可的书，因为现在的数据都散落在各处，令人眼花缭乱。[Matthew] Russell这位API达人、社交媒体专家（当然他更像是数据方面的科学家）在社交媒体数据挖掘方面巧妙地开创了先河。”

——Nick Ducoff，Infochimps公司CEO

“这是一本能让你了解新一代在线数据资源挖掘技术的重要指南。Russel做了件很了不起事，他为社交网站的从业人员准备了一本通俗易读的操作手册，涵盖了如何存取数据以及如何从原始数据中提取有价值的信息的简单方法。”

——Pete Warden，OpenHeatMap.com创始人

“每当我参与的项目涉及社交数据分析时，本书肯定是我的必备参考书。书中有很多有用的示例，如果你正开发与数据挖掘相关的项目，那么我郑重向你推荐本书。本书不仅对初学者有用，对数据挖掘的资深人士也极具参考价值。”

——Abe Music，Zaffra公司总裁

“作者编写本书时肯定乐在其中。他很巧妙地将传统的文本、图形挖掘库与时下流行的社交媒体应用联系到了一起。其示例既言之有物又不失简洁性，不但为读者提供了有价值的建议，也为读者进行更深入的开发和探讨提供了帮助。不论是对于刚涉足社交网络数据挖掘的新手，还是需要了解最新社交媒体API的资深研究人员来说，本书都是一本很好的指南。”

——Chris Augeri，Nebraska大学高级研究员

“对任何想涉足社交数据挖掘的人来说，这都一本非常好的书。作者进行了深入研究，从第1章开始就提供了丰富的示例。它不但易懂而且很有趣。如果你对社交网络数据的挖掘、分析和可视化感兴趣的话，那么本书是你的首选。”

——Jeffrey Humphries博士；计算机科学家

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

“在未来几年中，几乎没有任何事情可以阻碍我们通过软件自动了解人际交往方式的脚步。这个话题广博而精深。它一直是众多学术论文和学位论文的研究主题。Matthew真的是将一些容易忽略的东西联系到了一起：他对一系列庞杂而深奥的技术以及埋没在社交网站内的人际沟通方面的知识进行了很实用的讲解。本书出自一位技术高人之手——他对新工具的编程技术了如指掌。本书将为你打开软件未来十年走向的大门。”

——Tim Estes，Digital Reasoning Systems公司创始人、CEO

“本书告诉你如何最有效地利用Twitter API。”

——Raffi Krikorian，Twitter公司平台服务组

“Matthew精心挑选的这些数据源、分析技术、数据管理工具以及可视化方面的话题都非常有趣，全面体现了“如何从社交网络获取有价值的信息”领域的最新思想。他举的例子很生动，是进行深入研究很好的起点。Matthew非常在乎读者能否理解这些材料；本书随处都给出了适时、适用且真正有帮助的提示和建议。本书能激发我深入研究数据进而分析这一领域的兴趣。”

——Roger Magoulas，O’Reilly Media公司市场研究总监

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

译者序

作为译者，按图书原貌准确转述作者意图即是对图书最有力的推介。借写译者序的机会，表达一下本人对社交媒体的看法。

近年来，Twitter、Facebook、LinkedIn这些社交媒体开始蓬勃发展，且爆发出令人炫目的能量。无论是对消费者、企业或销售商，社交媒体都是一个很热门的话题，人们可以在社交媒体中看到很多最新的报道，了解世界各地的局势，推广新产品，告诉好友自己的最新状况，知道很久未曾谋面的好友的状况。人们花在社交媒体中的时间越来越多，它逐渐改变了人们的生活、交流方式，它传播的信息已经成为人们浏览互联网的重要内容，它制造了人们在社交生活中争相讨论的一个又一个热门话题，如因在微博中炫富而被广泛关注的“郭美美”事件，虽然我无意于纠缠该事件的来龙去脉，但是也要感叹一下社交媒体的无穷力量。

当我们的生活因为社交媒体变得便捷时，也遇到了很多问题。因为随之而来的是互联网中庞杂的数据和一些毫无价值的信息。在互联网中花费了大把时间仍然无法找到有用信息的情况时有发生，所以，如何有效地利用社交媒体，如何读取数据，如何从原始数据中提取出有价值的信息便成为亟待解决的问题。

这时，Matthew编写的这本书便成为你应该拥有的一本书。对于任何想要涉足社交数据挖掘分析的人来说，本书都是你的必备参考书。本书不仅介绍了挖掘数据、分析数据的技术，还提供了对数据管理工具以及可视化方面的话题的讨论。此外，为了能让读者充分了解这些技术，Matthew还提供了简单易懂的示例和非常有价值的建议。

本书前两章介绍了进行数据挖掘所需要的基本工具和背景知识，第3~9章讨论了对流行社交媒体数据的挖掘、分析和可视化，第10章对语义网进行了简单讨论。

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

很荣幸能翻译Matthew大师的作品，虽然我知道我的语言可能无法完美地展现本书的精彩之处，但我还是要庆祝经过自己的努力，不够完美但勤恳认真地完成了本书的翻译工作。

在本书的翻译过程中，特别感谢樊旺斌提供的帮助，感谢陈钢、田思源审校了部分译稿，感谢他们的鼓励和严谨。

师蓉

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

目录

前言 1

第1章 绪论：Twitter数据的处理 9

Python开发工具的安装 9

Twitter数据的收集和处理 11

小结 24

第2章 微格式：语义标记和常识碰撞 26

XFN和朋友 27

使用XFN来探讨社交关系 29

地理坐标：兴趣爱好的共同主线 37

〈以健康的名义〉对菜谱进行交叉分析 41

对餐厅评论的搜集 43

小结 45

第3章 邮箱：虽然老套却很好用 47

mbox：Unix的入门级邮箱 48

mbox+CouchDB=随意的Email分析 54

将对话线程化到一起 70

使用SIMILE Timeline将邮件“事件”可视化 79

分析你自己的邮件数据 82

小结 84

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

第4章 Twitter：朋友、关注者和Setwise操作 85

REST风格的和OAuth-Cladded API 86

干练而中肯的数据采集器 90

友谊图的构建 108

小结 116

第5章 Twitter：tweet，所有的tweet，只有tweet 118

笔PK剑：和tweet PK机枪（?!?） 118

对tweet的分析（每次一个实体） 121

并置潜在的社交网站（或#JustinBieber VS #TeaParty） 144

对大量tweet的可视化 155

小结 163

第6章 LinkedIn：为了乐趣（和利润？）将职业网络聚类 164

聚类的动机 165

按职位将联系人聚类 167

获取补充个人信息 183

从地理上聚类网络 188

小结 192

第7章 Google Buzz：TF-IDF、余弦相似性和搭配 194

Buzz=Twitter+博客（???） 195

使用NLTK处理数据 198

文本挖掘的基本原则 201

查找相似文档 208

在二元语法中发Buzz 215

利用Gmail 221

在中断之前试着创建一个搜索引擎…… 225

小结 226

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

第8章 博客及其他：自然语言处理（等）.....228

NLP：帕累托式介绍.....228

使用NLTK的典型NLP管线.....231

使用NLTK检测博客中的句子.....234

对文件的总结.....237

以实体为中心的分析：对数据的深层了解.....245

小结.....256

第9章 Facebook：一体化的奇迹.....257

利用社交网络数据.....258

对Facebook数据的可视化.....274

小结.....294

第10章 语义网：简短的讨论.....296

发展中的变革.....296

人不可能只靠事实生活.....297

期望.....301

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

前言

与其说网络是一项技术创新，不如说它是一项社会创新。我设计的是一种社会效应（帮助人们一起工作），而不是一种高科技玩具。网络的终极目标是支持并改进现实世界中存在的各种各样的“网”，我们有家庭“网”、协会“网”和公司“网”，我们会亲远疏近。

——Tim Berners-Lee，《Weaving the Web》(Harper)

是否要阅读本书

如果你有编程基础，并且对通过社交网络进行数据挖掘和分析来洞察身边的机会感兴趣的话，你就选对书了。在短短几页介绍之后，我们就会开始动手编写代码。然而，坦率地说，你对本书最主要的抱怨可能就是所有的章节都太短了。遗憾的是，当你试图抓住每天变化并且有大量机会的空间时，情况总是这样。也就是说，我非常赞同“80-20法则”（*http://en.wikipedia.org/wiki/Pareto_principle*），而且我坚信本书在践行“用80%的可用时间来探讨最有趣的20%的内容”方面，是个合理的尝试。

虽然本书篇幅有限，但是它涵盖了很多内容。总的来说，本书堪称广博而非精深，虽然当前的时机非常适合就此主题进行更深入的探讨，但是本书对这些有趣的挖掘和分析技术的深入程度比较有限。本书的编排风格，既适合你通读全书来全面了解社交网络数据方面的入门知识，也适合你根据自己的兴趣来挑选感兴趣的章节来阅读。也就是说，每一章的设计都很短小且相对独立，但是，我在每章内容材料的编排上还是精心安排了前后顺序的，这样，你在阅读全书时才会感到顺畅。

在过去的几年中，Facebook、Twitter和LinkedIn这类社交网站，已经从时尚变为主流，现在已经成为全球现象。在2010年第一季度，广受欢迎的社交网站Facebook已经超过Google，成为人们最经常访问的网站[注1]，这也证实了人们网络消费方式的明确转变。依

注1：　见第9章的第一段。

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

此事实断言“网络现在已经成为社会文化现象而不再是研究和信息的工具”可能为时尚早；然而，这一数据的确可以表明，社交网站正在以搜索引擎所不具备的方式，大规模地满足了人类的一些基本诉求。社交网络确实正在改变我们的网络生活[注2]，它们能够让技术给我们呈现出最好的（有时是最坏的）一面。社交网络的爆发只是现实世界和网络世界之间的差距不断缩小的一种方式。总的来说，本书的每一章都将社交网站与数据挖掘、分析和可视化技术的内容组织在一起，来回答以下问题：

- 谁与谁相识，他们共同的朋友是谁？
- 某人与其他人多久联系一次？
- 人与人之间的交流在多大程度上是对称的？
- 网络中最沉默/最健谈的人是谁？
- 网络中最具影响力/人气最高的人是谁？
- 人们正在聊什么（而且它有趣吗）？

要回答这些问题，通常都会涉及两人或更多的人，并且需要找出他们之间存在关系的上下文。回答这些问题所涉及的工作只是开始，更复杂的分析过程还在后面，但是你必须找个地方下手，因为社交网络API和开源工具包都很容易掌握。

笼统来说，本书把社交网站[注3]看成是由人、活动、事件、概念等组成的一幅“图”。Google和Facebook这些行业领袖已经开始逐渐推广以“图”为中心（graph-centric）的理念，而不再强调以“网络”为中心的说法了，因为它们在同时推广以“图”为基础的API。事实上，Tim Berners-Lee建议，也许他应该使用巨大全球图（Giant Global Graph，GGG）（*http://dig.csail.mit.edu/breadcrumbs/node/215*）来代替万维网（World Wide Web，WWW），因为“网”和“图”可以在定义互联网的拓扑结构的上下文中任意互换。虽然Tim Berners-Lee最初的设想能否实现仍有待观察，但是我们所熟知的网络正在因社交数据而变得越来越丰富。我们回顾过去的几年时，最明显的变化就是：由一个固有的语义网创建的第二级和第三级影响是实现真正的语义网的必要条件。两者之间的差距也变得越来越小了。

注2：Facebook的创始人马克·扎克伯格，被《时代》杂志评为2010年的年度人物（*http://www.time.com/time/specials/packages/article/0,28804,2036683_2037183_2037185,00.html*）。

注3：参阅*http://journal.planetwork.net/article.php?lab=reed0704*，换一个角度看待关注数字标识的社交网络。

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

还是不要阅读本书

从零开始构建自己的自然语言处理器、探究可视化图库的典型用法，以及任何与构建相关的最新技术这类活动，都不属于本书的范围。如果你想了解以上任何内容而购买本书的话，你一定会很失望。然而，并不能仅仅因为“在区区几百页中进行文本分析或记录匹配既不真实，也无法体现我们的最佳技术”，就认为本书无法让你找到疑难问题的合理解决方案，无法将这些方案应用于社交网站，在此过程中并无乐趣可言。当然，这也并不妨碍你在这些诱人的研究领域中培养积极的兴趣。本书的篇幅有限，只能给你开开胃，最多也就是能培养起你在数据处理方面的兴趣，并不能赋予你开天辟地的神奇力量。

也许现如今这样的提醒显得有点多此一举，但是我还是要强调一点（非常重要的一点）：本书所述内容，通常都假定你已经连接到了互联网。本书并不适合在你度假远行时随身携带，因为其中包含很多有超链接的参考内容，而且所有的代码示例都通过超链接直接与GitHub相连，GitHub（*http://github.com*）是一个非常社交化的Git（*http://git-scm.com*）存储库，它一直保持可用的最新示例代码。这种做法希望社交编码能加强志同道合的读者之间的协作，例如，有的人想一起扩展示例，也有人想一起探索有趣的问题。但愿大家能够对资源进行分叉、扩展以及改进——能结识一些志趣相投的新朋友的话更好。像API文档这种现成的资源也可以通过超链接来方便地获取，并且我们认为你更习惯使用在线内容，而不是那些注定会过期的印刷文字。

注意： 与本书配套的bug修复源代码的官方GitHub存储库是*http://github.com/ptwobrussell/Mining-the-Social-Web*。本书的官方Twitter账号是*@SocialWebMining*。

如果你需要一本能让你在sharded MySQL聚类这样的分布式计算平台或者是诸如Hadoop或Redis这样的NoSQL（*http://en.wikipedia.org/wiki/NoSQL*）技术上快速进步的参考书的话，我们并不推荐本书。我们确实使用了一些非常规的存储技术，如CouchDB（*http://couchdb.apache.org*）和Redis（*http://code.google.com/p/redis*），但它们都是在一台机器上运行的，因为这样就能很好地解决眼前的问题了。然而，如果你兴趣强烈且需要水平扩展性的话，它并不能真正连续不断地将示例移植到分布式技术中。我强烈建议你首先要掌握好基础知识，并且要保证代码首先能在一个相对简单的环境中运行，然后再将其移植到一个更复杂的分布式系统中，这样，一旦数据访问不再是在本地时，你就可以依此对算法进行较大的调整以保持其高性能。如果你想要这么做的话，Dumbo（*http://github.com/klbostee/dumbo/wiki*）是用于研究的很好选择。请继续关注本书的Twitter账号（*@SocialWebMining*）来获取有关Dumbo的扩展示例。

虽然我们会尽量遵守运营管理相关网站的条款并领会其精神，但是你可能会对从社交网

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

站所获得的数据进行加工，本书对于这些做法的法律后果并没有提供任何意见。有些无奈的是，许多最流行的社交网站的授权许可条款禁止在它们提供的平台之外使用这些数据，但目前来说，这种做法是意料之中的事。大部分社交网站就像是带围墙的花园，但是从他们的立场（以及他们的投资者的立场）来说，这些公司提供的很多价值目前依赖于控制平台和保护他们用户的隐私；这种平衡很难维持，而且短期内不可能有较大的改观。

最后的小提示是：本书略微倾向于*nix环境[注4]，因为有些可视化的内容可能会给Windows用户带来些小麻烦。然而，一旦出现这类问题，建议采用合理的替代方案或临时性方案，例如启动VirtualBox（*http://www.virtualbox.org*）在Linux环境中运行示例。幸运的是，并不经常发生这种情况，当出现这些问题时，你可以忽略相应的章节继续阅读，这并不会影响你阅读的乐趣。

工具和先决条件

本书唯一的先决条件就是你需要主动地学习一些Python知识，并且做好了亲自动手处理社交数据的准备。本书的任何技术或者示例都不需要太多数据分析、高性能运算、分布式系统、机器学习或者任何其他特别的背景知识。一些示例可能会涉及你以前没有使用过的结构，如线程池（*http://en.wikipedia.org/wiki/Thread_pool_pattern*），但不必担心——我们使用Python编程。Python直观的语法、优秀的数据处理生态系统软件包和核心数据结构（实际上是JSON）（*http://www.json.org*），使它成为一种优秀的教学工具，虽然功能强大却很容易使用。在其他情况下，如处理自然语言时，我们会使用一些处理高级事务的包，但是我们将会从应用程序设计者的角度来使用这些技术。由于在其他编程语言中也很可能存在非常相似的绑定，因此如果你愿意的话，这应该是移植代码示例的必备练习套路。（但愿这可以在GitHub中用得上！）除了上面的介绍之外，本书没有过多纠结于使用Python的利弊，因为它是非常适合该项工作的工具。如果你是编程新手或者从来没有见过Python语法，那么你只要保证没有跳过前几页内容即可。如果你正在寻找可靠的介绍资料，网上有很多不错的文档，Python的官方教程（*http://docs.python.org/tutorial*）就是很好的起点。

本书试图从各种可视化工具和工具包中，有选择性地介绍一系列有用的可视化工具，既有电子表格类的消费类产品，也有Graphviz（*http://www.graphviz.org*）类的工业产品，还有Protovis（*http://vis.stanford.edu/protovis*）这种尖端的HTML5（*http://en.wikipedia.org/wiki/HTML5*）技术。每一章都会适当地介绍一些新的可视化技术，但我们会尽量顺其自然，让它讲得通。你会对从这些工具中构建轻量级原型的想法感到满意的。也就是

注4：*nix是Linux/Unix环境的术语，在此处等同于非Windows。

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

说，本书的大部分可视化内容只是现成的示例或者只是对API项目做了很小的改动，所以只要肯学，你就能做到。

本书约定

下面是本书关于印刷字体方面的一些约定：

斜体（Italic）
: 用于电子信箱、URL、文件名、路径名以及用于强调第一次介绍的新的术语。

`等宽字体（Constant width）`
: 用于文件内容以及命令输出，来表示模块、方法、语句以及命令。

`等宽粗体（Constant width bold）`
: 用于程序代码段，来显示应该由用户输入的命令或文字，有时则用于强调程序代码的一部分。

`等宽斜体（Constant width italic）`
: 用于程序代码段中可替换的部分以及一些注释。

`<等宽字体>（<Constant width>）`
: 表示应该以真实程序代码取代的语法单元。

注意： 表示和附近文字相关的技巧、建议或一般性注释。

警告： 表示和附近文字相关的警告和注意事项。

代码示例的使用

各章中大多数编号的示例都可以从GitHub（*https://github.com/ptwobrussell/Mining-the-social-web*）上下载——它是本书的官方代码库。我们鼓励你持续关注这个库，以便获得最新的修正了bug的代码，以及由作者和其他社交编码社区编写的更多示例。

本书旨在帮助你完成你的工作。一般来说，可以在程序和文档中使用本书的代码。如果你复制了代码的关键部分，那么你就需要联系我们获得许可。例如，利用本书的几段代码编写程序是不需要许可的。售卖或出版O'Reilly书中示例的CD-ROM确实需要我们的许可。引用本书回答问题以及引用示例代码不需要我们的许可。将本书的大量示例代码用于你的产品文档中需要许可。

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

如果你在参考文献中提到我们，我们会非常感激，但我们也并不强求。参考文献通常包括标题、作者、出版社和ISBN。例如：“《Mining the Social Web》by Matthew A Russell. © 2011 Matthew Russell，978-1-449-38834-8。”

如果你认为你对代码示例的使用已经超出以上的许可范围，我们很欢迎你通过*permissions@oreilly.com*联系我们。

联系我们

有关本书的任何建议和疑问，可以与下面的出版社联系：

美国：

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472

中国：

北京市西城区西直门南大街2号成铭大厦C座807室（100035）
奥莱利技术咨询（北京）有限公司

我们会在本书的网页中列出勘误表、示例和其他信息。可以通过*http://oreilly.com/catalog/9781449388438/*访问该页面。

读者可以通过Get Satisfaction（*http://getsatisfaction.com/oreilly*）向作者和出版社申请常见的帮助。

读者也可能通过GitHub上的问题追踪系统（*http://github.com/ptwobrussell/Mining-the-Social-Web/issues*），对示例代码以及本书的任何其他内容归档分类。

要评论或询问本书的技术问题，请发送邮件到：

bookquestions@oreilly.com

有关我们的书籍、会议、资源中心以及O'Reilly网络，可以访问我们的网站：

http://www.oreilly.com
http://www.oreilly.com.cn

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

致谢

至少可以说，编写一本技术书需要作出很多牺牲。首先，在家里，我放弃了与妻子Baseeret和女儿Lindsay Belle相处的很多时间，这比我乐于承认的时间还要多。虽然我的抱负是有朝一日能在一定程度上征服世界（这只是暂时的，坦白说，我正在尽力摆脱这种状态），但是我还是要对你们的爱表示感激。

我深信你所做的一切决定最终都会影响到你的一生（尤其是你的职业生涯），但是谁也不可能孤独前行，我们要懂得感恩。撰写本书时，我真的很庆幸能与世界上最聪明的一帮人合作，其中包括像Mike Loukides这样聪明的技术编辑，以及O'Reilly这样极富天赋的制作团队，还有帮助我完成本书的很多热心的审阅者。我要特别感谢Abe Music、Pete Warden、Tantek Celik、J.Chris Anderson、Salvatore Sanfilippo、Robert Newson、DJ Patil、Chimezie Ogbuji、Tim Golden、Brian Curtin、Raffi Krikorian、Jeff Hammerbacher、Nick Ducoff和Cameron Marlowe对本书所用材料的审查或者对本书提出的有见地的建议，所有这些都最终体现在了本书的品质中。在此我也要感谢Tim O'Reilly的热心，他允许我将他的一些Twitter和Google Buzz数据在第4、5、7章中进行了仔细的研究；这些内容肯定为这些章节增趣不少。我不可能一一介绍曾经直接或间接地帮助过我或者帮助过本书出版的人，在此一并表示感谢。

最后，要感谢你阅读这本书，你也许会买一本留作纪念。虽然我已尽了我最大的努力，但你还是会发现本书存在的一些疏漏之处；然而，我坚信，虽然疏漏在所难免，但本书定会让你觉得值得你花上几个晚上或几周的时间来细细研读它，而且最终你也的确会有所收获。

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

第1章

绪论：Twitter数据的处理

我们本应从具体的社交网站API（Application Programming Interface，应用编程接口）、无模式设计（schemaless design）或者其他话题展开讨论，但还是让我们先通过示例来看看对社交网站的数据进行收集和分析是多么简单。本章相当于一个“驾车指南”，旨在激发你开始对后继章节所谈论的话题展开深入思考，随后的各章将会逐一对这些话题进行详细的讨论。我们首先会准备好开发环境，然后马上进入对Twitter数据的收集和分析。

Python开发工具的安装

本书的示例代码是用Python语言编写的，因此，如果你已经在系统中安装了最新版本的Python和easy_install，显然你已经对Python有所了解了，那你就可以跳过这一部分内容。如果你还没有安装Python，说明目前你还是个Python菜鸟。但不必担心，很快你就会成为一名Python高手了，因为Python很容易自学。不论是什么操作系统平台，用户都可以从*http://www.python.org/download/*中找到相应的下载和安装说明，但我强烈推荐Windows用户安装ActivePython（*http://www.activestate.com/activepython*），它会在Windows命令提示符（后文称为“终端”）中自动将Python添加到你的路径中，而且它还自带了easy_install工具包，我们一会儿会讨论它。书中的示例都是用最新的Python 2.7编写、测试的，但在更新的版本上应该也能正确地运行。编写本书时，Python 2仍然是主流版本（*http://wiki.python.org/moin/Python2orPython3*），建议你也使用该版本，如果你认为Python 3已经完全移植了你需要的全部依赖项，而且你也乐意调试版本升级引起的任何问题，那就另当别论了。

一旦安装了Python，就可以在终端中输入python来启动一个解释器了。请试试示例1-1。

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

示例1-1：第一个真正的Python解释器会话

```
>>> print "Hello World"
Hello World
>>> #this is a comment
...
>>> for i in range(0,10): # a loop
...     print i, # the comma suppresses line breaks
...
0 1 2 3 4 5 6 7 8 9
>>> numbers = [ i for i in range(0,10) ] # a list comprehension
>>> print numbers
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> if 10 in numbers: # conditional logic
...     print True
... else:
...     print False
...
False
```

另一个你会用到的工具是easy_install[注1]，它类似于Linux系统中的包管理器，有了它，就可以轻松地安装Python包，而不必自己下载源代码，然后再构建、安装了。可以从*http://pypi.python.org/pypi/setuptools*中下载最新版的easy_install，对不同的操作系统平台，网站上都有详细的安装说明。一般来说，*nix系统的用户会用sudo easy_install命令将对应模块写入Python的完整安装目录中。假设Windows用户已经按照我的建议使用了ActivePython，它的安装程序已经自动包含了easy_install。

注意： Windows用户也可以从一篇名为“Installing easy_install…could be easier”（参见*http://blog.sadphaeton.com/2009/01/20/python-development-windows-patr-2-installing-easyinstallcould-be-easier.html*）的博文中获得帮助。文中讨论了当你运行easy_install时，可能会常遇到的一些有关C代码编译的常见问题。

一旦正确配置了easy_install，就可以运行下列命令来安装NetworkX（本书通篇都会用它来构建和分析图）了，并且会得到类似的输出结果：

```
$ easy_install networkx
Searching for networkx

...truncated output...

Finished processing dependencies for networkx
```

注1：虽然本书的示例使用了大家熟知的easy_install，但在Python社区内，pip（*http://pip.openplans.org/*）工具也开始渐渐流行起来，对此我们应该多加关注。能用easy_install处理的软件包，也都可以用pip处理。

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

NetworkX安装好后，你可能认为只要从解释器中导入它即可正确运行，但有时候有些包会出点意外。例如，可能会发生以下情况：

```
>>> import networkx
Traceback (most recent call last):

... truncated output ...

ImportError: No module named numpy
```

只要出现ImportError错误就说明有一个包缺失。在本例中，我们安装的networkx模块有一个未作处理的依赖项——numpy，它是一个高度优化的科学计算工具集。通常，再次调用easy_install就可以解决这个问题，处理numpy缺失的方法与安装NetworkX的方法一样。只要关闭解释器，并且在终端输入easy_install numpy即可安装该依赖项：

```
$ easy_install numpy
Searching for numpy

...truncated output...

Finished processing dependencies for numpy
```

既然numpy已经安装好了，就应该可以打开一个新的解释器，运行import networkx命令，并用它来构建图表，如示例1-2所示。

示例1-2：使用NetworkX创建一个由节点和边构成的图

```
>>> import networkx
>>> g=networkx.Graph()
>>> g.add_edge(1,2)
>>> g.add_node("spam")
>>> print g.nodes()
[1, 2, 'spam']
>>> print g.edges()
[(1, 2)]
```

现在，你已经安装好了Python核心开发工具，并且已经准备好做一些更有趣的任务。如果你已经掌握了这一节的主要内容，那么在深入学习之前最好查阅一下官方Python教程（*http://docs.python.org/tutorial*）。

Twitter数据的收集和处理

你不可能是连Twitter都不知道的火星人吧！Twitter是一个高度社交化的实时微博客服务，它允许发布不超过140个字符的短消息，这些消息称为tweet。与Facebook、LinkedIn这些关系对等的社交网络不同，Twitter中的非对等关系有“好友”和“关注者”之分。假设你有一个Twitter账号，那么你的好友就是你关注的用户，而你的关注者

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

则是关注你的那些用户。虽然你可以选择关注所有关注你的用户，但通常你不会这样做，因为你希望在自己的首页Timeline[注2]更新中显示那些自己觉得有趣的tweet。无论是从其庞大的用户数量，还是其作为一种营销手段,以及近期作为第三方消息服务的传输层来说，Twitter都是一种重要的现象。它提供了大量的API，虽然不注册你也可以使用其中的大多数API，但如果你建立自己的网络并对其进行挖掘的话会更有趣。在你做重型开发之前，先花点时间看一下Twitter自由的服务条款（*http://twitter.com/tos*）、API文档（*http://apiwiki.twitter.com*）和API法则（*http://twitter.com/apirules*），这样可以确保你有效地利用Twitter数据来做你想做的任何事情。本书后续的章节假设你已经有一个可以用于挖掘数据的Twitter账号和足够多的好友/关注者。

注意： 本书的官方Twitter账号是*@SocialWebMining*。

摆弄Twitter的API

和Twitter网络API有关的最小封装可以通过一个叫做twitter的软件包（*http://github.com/sixohsix/twitter*）按照easy_install的如下规范安装：

```
$ easy_install twitter
Searching for twitter

...truncated output...

Finished processing dependencies for twitter
```

这个包还包含一个容易使用的命令行实用工具和IRC机器人程序，因此在安装完这个模块之后，就可以通过在shell中简单地输入twitter来了解如何使用这个命令行实用工具。然而，我们关注的是交互式Python解释器的使用。我们将通过一些示例来了解它，但要注意，随时可以在终端运行pydoc来浏览这些文档。*nix用户可以简单地输入pydoc twitter.Twitter来查阅Twitter类的相关文档，而Windows用户需要输入python -mpydoc twitter.Twitter才能实现此功能。如果你要经常查看某些模块的文件，可以选择pydoc中的-w选项来生成一个HTML页面，并把它添加到你的浏览器的收藏夹中。同样值得了解的是，在一个模块或类中运行pydoc，与在解释器中运行help()命令得到的内联文档相同。你可以试着在解释器中输入help(twitter.Twitter)，看看结果是否相同。

现在，让我们通过Twitter的搜索API（*http://dev.twitter.com/doc/get/search*）提供给我们的动态功能来找出人们正在讨论的话题。打开解释器开始搜索，你可以依照示例1-3，根据需要使用help()函数来尽可能解答你自己的问题。

注2： *http://support.twitter.com/entries/164083-what-is-a-timeline*。

欢迎加入非盈利Python编程学习交流程QQ群783462347，群里免费提供500+本Python书籍！

示例1-3：检索Twitter搜索功能

```
>>> import twitter
>>> twitter_search = twitter.Twitter(domain="search.twitter.com")
>>> trends = twitter_search.trends()
>>> [ trend['name'] for trend in trends['trends'] ]
[u'#ZodiacFacts', u'#nowplaying', u'#ItsOverWhen', u'#Christoferdrew',
u'Justin Bieber', u'#WhatwouldItBeLike', u'#Sagittarius', u'SNL', u'#SurveySays',
u'#iDoit2']
```

你可能会惊讶Twitter模块的使用模式居然如此简单明了：用一个基本的URL将Twitter类实例化，然后调用Twitter对象中与URL上下文相对应的方法。例如，twitter_search.trends()函数可以启动一个HTTP（Hyper Text Transport Protocol，超文本传输协议）调用来获取*http://search.twitter.com/trends.json*，也可以在Web浏览器中输入该网址得到相同的结果。作为示例1-1的延伸，需要交代一下背景信息，本章的第一稿是在周六晚上起草的，所以《SNL》（周六夜现场，美国王牌喜剧秀）在列表中出现并非偶然。现在最好把这份Twitter API官方文档（*http://dev.twitter.com.doc*）添加到你的收藏夹中，因为你经常会用到它。

鉴于《SNL》是当红节目，接下来应该做的就是通过搜索API搜索tweet来抓取包含相关文本的搜索结果，然后再将它们以可读的JSON（JavaScript Object Notation，JavaScript对象标记）（*http://json.org*）结构打印出来，如示例1-4所示。

示例1-4：通过Twitter搜索结果分页

```
>>> search_results = []
>>> for page in range(1,6):
...     search_results.append(twitter_search.search(q="SNL", rpp=100, page=page))
```

这段代码抓取并存储了查询（q）SNL获得的5组（页）连续结果，每页包含100条结果（rpp: results per page）。我们在循环中以*http://search.twitter.com/search.json?&q=SNL&rpp=100&page=1*的形式所执行的等价REST查询[注3]很值得再次观摩。REST API和twitter模块之间的弱映射关系使得与Twitter服务交互的Python代码的编写变得很容易。执行搜索后，search_results列表中会包含5个对象，每个对象都由100组结果组成。可以用Python 2.6中内置的json包将结果以一种可读的方式输出，这样便于查看，如示例1-5所示。

示例1-5：以JSON结构输出的漂亮的Twitter数据效果

```
>>> import json
>>> print json.dumps(search_results, sort_keys=True, indent=1)
[
 {
  "completed_in": 0.088122000000000006,
```

注3：　如果你不熟悉REST，可以查看3.2节的补充信息“REST风格的网络服务”的简短介绍。

```
  "max_id": 11966285265,
  "next_page": "?page=2&max_id=11966285265&rpp=100&q=SNL",
  "page": 1,
  "query": "SNL",
  "refresh_url": "?since_id=11966285265&q=SNL",
  "results": [
   {
    "created_at": "Sun, 11 Apr 2010 01:34:52 +0000",
    "from_user": "bieber_luv2",
    "from_user_id": 106998169,
    "geo": null,
    "id": 11966285265,
    "iso_language_code": "en",
    "metadata": {
     "result_type": "recent"
    },
    "profile_image_url": "http://a1.twimg.com/profile_images/809471978/DSC00522...",
    "source": "&lt;a href=&quot;http://twitter.com/&quot;&gt;web&lt;/a&gt;",
    "text": " ...truncated... im nt gonna go to sleep happy unless i see @justin...",
    "to_user_id": null
   }
            ... output truncated - 99 more tweets ...

  ],
  "results_per_page": 100,
  "since_id": 0
 },

   ... output truncated - 4 more pages ...
]
```

警告： 请注意，从2010年底开始，每个搜索结果中的`from_user_id`字段都不再对应tweet作者真实的Twitter id了。详情请参阅Twitter API Issue #214（*http://code.google.com/p/twitter-api/issues/detail?id=214*）。这一改变对本章的示例代码并没有影响，但是当你自己开始创作时要特别注意这一点。

以后的章节会详细地讨论这一查询（见第5章）；此时需要重点关注的是：tweet是由响应结果输入的。我们将用下面的方法将这500个tweet文本提取到列表中。示例1-6展示了一个双重列表推导式，它是用来表示嵌套循环的。

示例1-6：一个简单的Python列表推导式

```
>>> tweets = [ r['text'] \
...     for result in search_results \
...         for r in result['results'] ]
```

本书经常会用到列表推导式。如果是单行编写的话看起来可能会很混乱，但是以嵌套循环打印出来的话，你很容易就能理解它的含义了。本示例中`tweets`的结果相当于定义一个叫做`tweets`的空表，并用它调用同类型嵌套循环中的`tweets.append(r['text'])`，如

示例1-6所示。详情请参阅Python官方教程中“数据结构”一节（*http://docs.python.org/tutorial/datastructures.html*）。列表推导式非常强大，因为它们通常可以显著地提高嵌套列表的性能，并以直观（一旦你熟悉了它们后就会觉得直观）简单的语法显示出来。

频率分析和词汇多样性

可用于非结构化文本的一种最直观的度量，是一个叫做词汇多样性的度量。简单来说，它用来表示文本中唯一标记的数量除以文本中标记总数量得到的结果。它是其自身很基本、很重要的度量。它可以按照示例1-7那样计算。

示例1-7：tweet词汇多样性的计算

```
>>> words = []
>>> for t in tweets:
...     words += [ w for w in t.split() ]
...
>>> len(words) # total words
7238
>>> len(set(words)) # unique words
1636
>>> 1.0*len(set(words))/len(words) # lexical diversity
0.22602928985907708
>>> 1.0*sum([ len(t.split()) for t in tweets ])/len(tweets) # avg words per tweet
14.476000000000001
```

警告： 在Python 3.0之前，除法运算符会提供floor函数并且返回一个整数值（除非其中的一个操作数是浮点数）。使用1.0乘以分子或分母来避免截尾误差。

解释“词汇多样性的值大约是0.23”的一种方法是：聚集到一起的tweet中，每4个单词中就有1个是唯一的。已知每条tweet平均大约有14个单词，也就是说每条tweet中只有3个单词是唯一的。如果不引入任何其他信息，我们就可以认为这意味着“每条tweet携带着约20%的唯一信息”。此刻需要了解的是，有多少“杂质”是由于用户为了保持每条tweet少于140个字符而使用的罕见缩写，以及tweet用语中哪些是最常用的、哪些是最不常用的。单词的分布和它们的使用频率都是很有用的。虽然这些很容易计算，但我们最好安装一个工具，它可以提供内置的频率分布和许多其他文本分析工具。

自然语言工具包（Natural Language ToolKit，NLTK）是在本书中使用的一个流行模块：它提供了大量的用于各种文本分析的工具，包括常见度量的计算、信息提取和自然语言处理（Natural Language Processing，NLP）。虽然与商界和学术界的不断发展相比，NLTK并不一定是最先进的，但它仍然可以为你提供一个坚实而广阔的基础——如果这是你第一次尝试处理自然语言的话，那就更绰绰有余了。如果你的项目已经足够成熟，而NLTK提供的质量或效率不能满足你的要求，那么根据你愿意投入的时间和金钱，你

大约会有3个选择：通过进行对比实验和评测来寻找开源空间作为合适的替代手段，通过白皮书和原型来“攒”你自己的工具包，或者获取某个商业产品的许可证。这些选择既不便宜（假设你相信时间就是金钱）也不简单。

可以通过easy_install规范安装NLTK，但你需要重启解释器来使用它。在退出当前工作会话之前，可以通过cPickle模块来保存（“pickle”）数据，如示例1-8所示。

示例1-8：保存数据

```
>>> f = open("myData.pickle", "wb")
>>> import cPickle
>>> cPickle.dump(words, f)
>>> f.close()
>>>
$ easy_install nltk
Searching for nltk

...truncated output...

Finished processing dependencies for nltk
```

警告： 当你试图运行import nltk命令时，如果遇到“ImportError: No module named yaml”问题，只要执行easy_install pyYaml指令，应该就可以解决了。

安装完NLTK后，最好花点时间在它的官方网站（*http://www.nltk.org*）中查看它的文档。其中包括由Steven Bird、Ewan Klein和Edward Loper编写的《Natural Language Processing with Python》（O'Reilly出版社）的全文，它是NLTK的权威参考书。

人们正在讨论什么

挖掘Twitter数据最主要的原因是试图回答“人们正在讨论什么”的问题。回答这个问题最简单的方法是进行基本频率分析。NLTK提供一个API来简化频率分析，所以我们可以省点事，让NLTK处理这些细节吧！示例1-9展示了通过创建频率分布得出的结果，然后看看50个最常见和50个最不常见的单词是什么。

示例1-9：使用NLTK执行基本频率分析

```
>>> import nltk
>>> import cPickle
>>> words = cPickle.load(open("myData.pickle"))
>>> freq_dist = nltk.FreqDist(words)
>>> freq_dist.keys()[:50] # 50 most frequent tokens
[u'snl', u'on', u'rt', u'is', u'to', u'i', u'watch', u'justin', u'@justinbieber',
u'be', u'the', u'tonight', u'gonna', u'at', u'in', u'bieber', u'and', u'you',
u'watching', u'tina', u'for', u'a', u'wait', u'fey', u'of', u'@justinbieber:',
u'if', u'with', u'so', u"can't", u'who', u'great', u'it', u'going',
u'im', u':)', u'snl...', u'2nite...', u'are', u'cant', u'dress', u'rehearsal',
u'see', u'that', u'what', u'but', u'tonight!', u':d', u'2', u'will']
```

```
>>> freq_dist.keys()[-50:] # 50 least frequent tokens
[u'what?!', u'whens', u'where', u'while', u'white', u'whoever', u'whoooo!!!!',
u'whose', u'wiating', u'wii', u'wiig', u'win...', u'wink.', u'wknd.', u'wohh', u'won',
u'wonder', u'wondering', u'wootwoot!', u'worked', u'worth', u'xo.', u'xx', u'ya',
u'ya&lt;3miranda', u'yay', u'yay!', u'ya\u2665', u'yea', u'yea.', u'yeaa', u'yeah!',
u'yeah.', u'yeahhh.', u'yes,', u'yes;)', u'yess', u'yess,', u'you!!!!!',
u"you'll", u'you+snl=', u'you,', u'youll', u'youtube??', u'youu&lt;3',
u'youuuuu', u'yum', u'yumyum', u'~', u'\xac\xac']
```

注意： Python 2.7添加了便于计数操作的collections.Counter类（*http://docs.python.org/library/collections.html#collections.Counter*）。在你不熟悉NLTK安装，或者你只想试用一下Python标准库中最新和最强大的类时，你会发现它大有用处。

从示例1-9的结果可以看出，常见标记比罕见标记携带更多有用的信息。虽然有些工作需要使用机器才能得到更详细的确认，但常见标记指的是人、时间和活动等实体，而罕见标记大多是噪声，不能得出任何有意义的结论。

关于最常见标记，你最先注意到的可能是列表顶部的“snl”。它是原始搜索查询的基础，所以这不足为奇。更有趣的是，当你浏览其余的标记时：显然有很多对Justin Bieber的讨论，你可以从@justinbieber、justin和bieber这些标记中得到验证。熟悉《SNL》的人都知道“tina”和“fey”的出现也并非巧合，因为Tina Fey长期参与这个节目。但愿浏览这些标记并形成“Justin Bieber是个受欢迎的人”的推测不是太困难（作为人类来说），而且很多人因为他在星期六晚上的出场而感到非常兴奋，而我们恰好在那时执行了这个搜索查询。

此刻你可能在想：“那又怎么样？我也可以浏览一些tweet并且推断出同样多的结果。”虽然这可能可行，但你愿意自己或者花钱雇人来7×24小时做这件事吗？而且如果你是在一个完全陌生的领域工作，并且不得不跳过一些短信息广告的随机抽样的话，结果又会如何？问题的关键是，频率分析是一个虽然非常简单但十分强大的工具，我们不应该仅仅因为它如此浅显就轻视它。相反，正因为它如此浅显、简单，我们应该最先试用它。因此，这里给你最基本的一点忠告就是：一个非常简单的技术应用会在回答“人们现在正在讨论什么？”这个问题上帮你很大的忙。

作为最终的观察结果，关于正在进行的对话的性质，“rt”的存在也是一个非常重要的线索。RT标记是一个很特殊的符号，通常被添加到一条消息的开头，表明你正在以另一个人的身份转发Twitter消息。从这个标记的高频率，可以合理地推断出很多类似的tweet中包含眼前的主题。事实上，这个观察结果是我们进行下一步分析的基础。

注意： RT标记可以被添加到一条信息的开头，表明它正在被转播，或者以Twitter的说法来说是正在被“转发”。例如，“RT @SocialWebMining Justin Bieber is on SNL 2nite. w00t?!?” 这条tweet表明发送者正在转发通过*@SocialWebMining*得到的消息。转发的这条tweet的等价形式是“Justin Bieber is on SNL 2nite. w00t?!? Ummm...(via *@SocialWebMining*)”。

从tweet中提取关系

因为社交网络首先是现实世界中人与人之间的关系，所以用于存储社交网络数据的一种非常方便的格式就是“图”。让我们使用NetworkX来构建一幅“图”，用它来连接转发了消息的Twitter用户。我们所用的图中包含有方向性信息，并用其指示消息流动的方向，所以应该更精确地称它为有向图（digraph）（参见*http://en.wikipedia.org/wiki/Directed_graph*）。虽然Twitter的API确实提供了一些决定和分析“被相互转发的状态”的功能，但对我们目前的使用情况来说，这些API并不是特别适合，因为我们不得不使用很多API调用来反复访问服务器，这是对有限的API调用限额的浪费。

注意： 编写本书时，对于身份验证的请求，Twitter强加了“每小时350个API调用”的速率限制；匿名请求被限定在每小时150个。更多细节请参阅*http://dev.twitter.com/pages/rate-limiting*。第4章和第5章将会讨论最大限度地使用限额的技术，以及用于收集数据的一些其他有创意的做法。

此外，我们可以使用tweet自身的线索，使用一个简单的正则表达式来正确地提取转发的消息。按照约定，Twitter用户名要以@符号开头，而且只能包含字母、数字和下划线。因此，按照转发规则来说，我们只需搜索如下模式：

- 由用户名跟随的RT
- 由用户名跟随的via

虽然第5章介绍了专门用于从tweet中解析实体的模块，但示例1-10说明你不需要任何特殊的库，就可以使用re模块来编译[注4]模式，并且以轻量级的方式提取出tweet的始发者。

示例1-10：使用正则表达式来查找转发消息

```
>>> import re
>>> rt_patterns = re.compile(r"(RT|via)((?:\b\W*@\w+)+)", re.IGNORECASE)
>>> example_tweets = ["RT @SocialWebMining Justin Bieber is on SNL 2nite. wOOt?!?",
...     "Justin Bieber is on SNL 2nite. wOOt?!? (via @SocialWebMining)"]
>>> for t in example_tweets:
...     rt_patterns.findall(t)
...
```

注4：目前的情况下，编译正则表达式意味着将它转换为字节码，这样就可以使用C语言编写的匹配引擎来执行它。

```
[('RT', ' @SocialWebMining')]
[('via', ' @SocialWebMining')]
```

有时候怕不够明显，调用findall()会返回一个元组列表，对于该模式中的每个组来说，这个列表中的每个元组都包含匹配的文本或一个空字符串；但要注意：regex确实在提取的实体上留下了一个前导空格，但调用strip()就可以轻松搞定，如示例1-11所示。因为两条示例tweet都不会同时包含在圆括号表达式中包含的两个组中，所以每个元组中都有一个字符串是空的。

注意： 正则表达式是个很基础的编程概念，本书不会对其进行解释。re模块文档（*http://docs.python.org/library/re.html*）可以加快你的理解，如果你想了解更多相关知识，可以随时查阅Friedl的杰作：《Mastering Regular Expressions》（*http://oreilly.com/catalog/9780596528126*）（O'Reilly）。

由于API返回的tweet数据结构提供了现在发送消息的用户名和最新发现的可以提取转发消息始发者的能力，因此很容易将消息加载到NetworkX图中。让我们创建一幅图，用节点表示用户名，用两个节点之间的一条有向边表示节点之间存在转发关系。该边上标有tweet ID的有效负载和tweet文本内容。

示例1-11表明了生成这类图的过程。最基本的步骤就是要总结出套路：提取包含在转发消息中的用户名，将tweet页面展开成更容易在循环中处理的统一列表，最后，遍历这些tweet并且在图中加入边。虽然稍后我们会生成这幅图的图像，但有必要说明的是可以通过分析图的特性来了解它们，而不需要将它们可视化。

示例1-11：创建并分析一幅用来描述谁转发了谁的图

```
>>> import networkx as nx
>>> import re
>>> g = nx.DiGraph()
>>>
>>> all_tweets = [ tweet
...                 for page in search_results
...                     for tweet in page["results"] ]
>>>
>>> def get_rt_sources(tweet):
...     rt_patterns = re.compile(r"(RT|via)((?:\b\W*@\w+)+)", re.IGNORECASE)
...     return [ source.strip()
...              for tuple in rt_patterns.findall(tweet)
...                  for source in tuple
...                      if source not in ("RT", "via") ]
...
>>> for tweet in all_tweets:
...     rt_sources = get_rt_sources(tweet["text"])
...     if not rt_sources: continue
...     for rt_source in rt_sources:
...         g.add_edge(rt_source, tweet["from_user"], {"tweet_id" : tweet["id"]})
```

```
...
>>> g.number_of_nodes()
160
>>> g.number_of_edges()
125
>>> g.edges(data=True)[0]
(u'@ericastolte', u'bonitasworld', {'tweet_id': 11965974697L})
>>> len(nx.connected_components(g.to_undirected()))
37
>>> sorted(nx.degree(g))
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5, 6, 6, 9, 37]
```

NetworkX提供的内置操作是理解这些数据的一个有用的起点，但重要的是记住：我们只考虑了发生在Twitter中关于《SNL》整个谈话的很小的一部分——成千上万（或者更多）条tweet中的500条。例如，图中的节点数告诉我们在500条tweet中，彼此之间的转发关系涉及160个用户，有125条边连接着这些节点。160/125的比（约为1.28）是一条重要的线索，它告诉我们节点的平均度（*http://en.wikipedia.org/wiki/Degree_(graph_theoty)*）近似于1——意味着虽然一些节点与多个节点相连接，但平均起来大约是一个节点一个连接。

通过调用connected_components，我们可以看到该图由37幅子图组成，且不是完全连通的。degree的输出乍看好像有点神秘，但事实上它证实了我们已搜集到的发现：只要将其作为理解“节点在图连接的有多么好”的一种方式即可，而不必提供实际的图。在这种情况下，大多数值为1，意味着所有这些节点的度都是1，都只与图中的一个节点连接。一些值介于2～9之间，表明这些节点连接着2～9个其他节点。极端的异常值是度为37的节点。之所有出现这样的图，主要是因为它是由不相交的节点组成的，但还是有一个高度连接的节点。图1-1用柱形图说明度的分布。趋势线表明该分布严格遵循幂法则（*http://en.wikipedia.org/wiki/Power_law*），并有一条很“重”或很“长”的尾巴。虽然本书并没无意强调长尾分布的特点，但你会发现我们遇到的很多分布都会呈现出这种特性，而且如果你迫切想要主动深入了解的话，那么我鼎力支持！齐普夫定律（*http://en.wikipedia.org/wiki/Zipf's_law*）是个不错的入手点。

在本书中，我们会花更多时间，使用自动化的试探法来理解数据；本章作为绪论，旨在让你了解一下基本知识，并且让你思考可以轻松利用数据的方式。然而，在结束本章之前，我们对图进行一下可视化，并以此来确认我们的直觉并未误导我们。

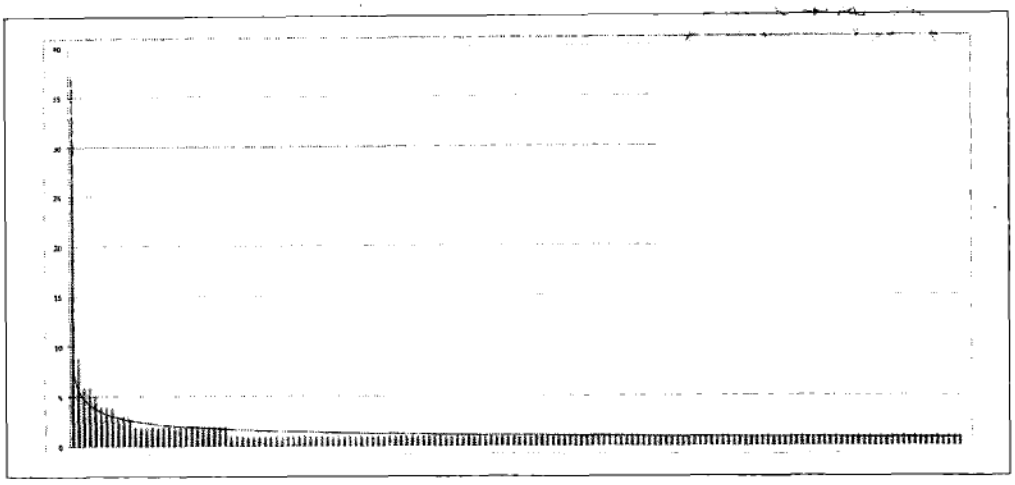

图1-1：一幅可以说明图中每个节点的度的分布图，它反映了图的连接关系

tweet图的可视化

Graphviz是可视化社区的主要工具。本节介绍一种可以可视化tweet数据图的方法：将它们导出到DOT语言（*http://www.graphviz.org/doc/info/lang.html*）中，DOT语言是Graphviz使用的一种基于文本的简单格式。所有平台的Graphviz二进制文件都可以从它的官方网站（*http://www.graphviz.org*）下载，而且不论是什么平台，它们都很容易安装。一旦Graphviz安装成功，*nix用户就可以使用`easy_install pygraphviz`，满足NetworkX需要的依赖项PyGraphviz（*http://networkx.lanl.gov/pygrahiviz/*）来生成DOT。Windows用户最有可能在安装PyGraphviz时遇到困难[注5]，但这并无大碍，因为相对于我们要生成的这部分DOT语言输出，修改几行代码就是小意思了。

示例1-12展示了适用于这两种平台的方法。

示例1-12：不论什么平台，都很容易生成DOT语言的输出

```
OUT = "snl_search_results.dot"

try:
    nx.drawing.write_dot(g, OUT)
except ImportError, e:
```

注5：从NetworkX Ticket #117（*http://network.lanl.gov/trac/ticket/117*）可以看出，这是一个长期存在的问题，但不知道为什么，即使经过这么多年挫折，仍然没有得到解决。基本的问题与在`easy_install`过程中需要编译C代码有关。这个问题这么长时间没得到解决的部分原因可能是：通过生成DOT语言输出很容易就可以解决这个问题。

```
# Help for Windows users:
# Not a general-purpose method, but representative of
# the same output write_dot would provide for this graph
# if installed and easy to implement

dot = ['"%s" -> "%s" [tweet_id=%s]' % (n1, n2, g[n1][n2]['tweet_id']) \
    for n1, n2 in g.edges()]
f = open(OUT, 'w')
f.write('strict digraph {\n%s\n}' % (';\n'.join(dot),))
f.close()
```

生成的DOT输出样式如示例1-13所示。

示例1-13：DOT语言输出的示例

```
strict digraph {
"@ericastolte" -> "bonitasworld" [tweet_id=11965974697];
"@mpcoelho" -> "Lil_Amaral" [tweet_id=11965954427];
"@BieberBelle123" -> "BELIEBE4EVER" [tweet_id=11966261062];
"@BieberBelle123" -> "sabrina9451" [tweet_id=11966197327];
}
```

已经得到了DOT语言输出，下一步便是将其转换成图像。Graphviz提供了各种布局算法来可视化导出的图；过去常用来以循环风格的布局提供图形的工具`circo`，用在这里应该不错，因为有数据表明，图将会以“中心辐射”式拓扑结构显示自我图（ego graph）（*http://networkx.lanl.gov/examples/drawing/ego_graph.html*）的形状，它由一个高度连接的中心节点与其他很多度为1的节点连在一起。在*nix平台上，下面的命令将会把从NetworkX导出的*snl_search_results.dot*文件转换为*snl_search_results.dot.png*文件，可以在图像查看器中打开后一个文件（操作的结果如图1-2所示）：

```
$ circo -Tpng -Osnl_search_results snl_search_results.dot
```

Windows用户可以使用GVedit应用程序呈现文件，如图1-3所示。要获取更多的Graphviz选择，请参阅联机文档（*http://www.graphviz.org/Documentation.php*）。对整个图形文件的目测证实该图的特点符合我们之前的分析，而且我们可以形象地证实度数最大的节点是@Justinbieber，他是这么多讨论的主题（而且，如果万一你错过了那期《SNL》节目，且不了解当晚的嘉宾主持是谁的话，你就能知道谁是主题）。请记住，如果我们得到了更多tweet，我们很可能会看到比我们一直在分析的、已经被证明的500条tweet样本更互连的子图。对该图的进一步分析作为自选习题留给读者，因为本章旨在让你准备好开发环境，并且以有趣的话题来吸引你的注意。

Graphviz还会在本书中出现，如果你认为自己是（或者想成为）一名数据科学家，它应该是你必须掌握的工具。也就是说，我们也会研究其他有用的方法将图可视化。后面的章节将介绍其他社交网络数据和分析方法。

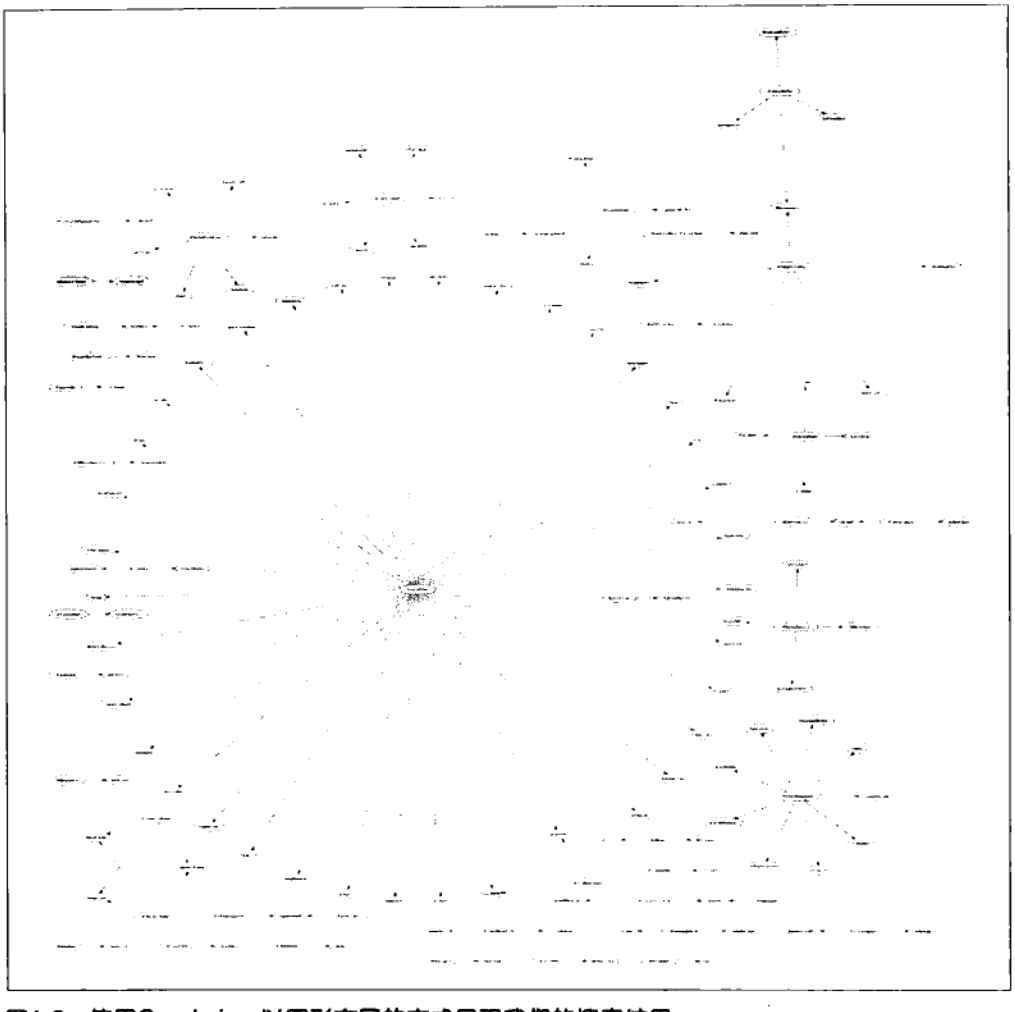

图1-2：使用Graphviz，以圆形布局的方式呈现我们的搜索结果

综合应用：用Protovis可视化转发的tweet

本节介绍一个综合示例，这个示例综合了本章大部分内容并添加了可视化内容。除了在控制台输出一些有用的信息外，它还接受搜索词作为命令行参数，抓取、分析并且弹出Web浏览器来将数据可视化为基于HTML5的交互图。可以从本书的官方代码库*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/introduction__retweet_visualization.py*中获取这个示例。很希望你能去下载。在本书后面的几章中，我们会再次使用Protovis（*http://vis.stanford.edu/protovis/*），它是这个示例的基础可视化工具包，图1-4说明了该脚本的Protovis输出。示例脚本的样本只是开始——我们还可以有其他更多的脚本。

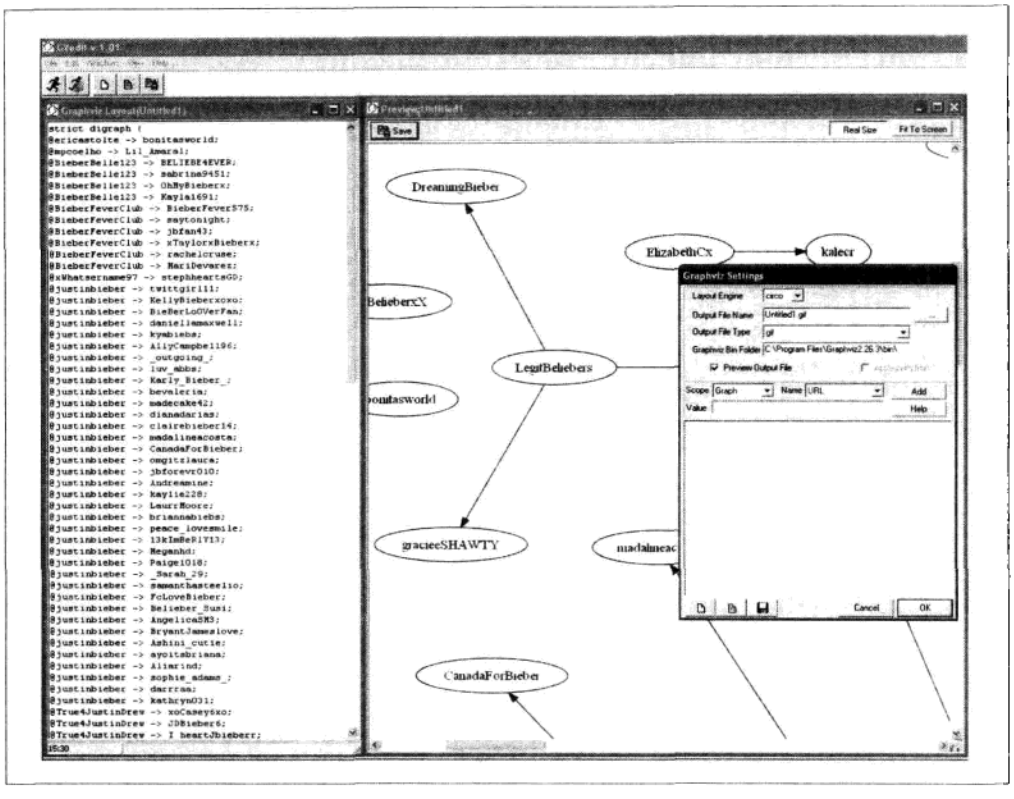

图1-3：Windows用户可以使用GVedit，而不用与Graphviz在命令提示符中交互

小结

本章不仅是个铺垫，而且说明了使用Python的交互式解释器来探索和可视化Twitter数据是多么简单。在阅读后继章节之前，最重要的是调试好Python开发环境，在此我强烈建议你花点时间了解Twitter API和Graphviz。如果你想要画一条切线，你可能应该试试canviz（*http://code.google.com/p/canviz/*），它可在Web浏览器<canvas>元素上绘制Graphviz图。你可能还需要研究IPython（*http://ipython.scipy.org/moin/*），它是一个“更好”的Python解释器，提供了自动补全、历史记录跟踪等功能。从现在起，在本书中我们所做的大部分工作将涉及可运行的脚本，但重要的是在尝试新想法、调试时，你要尽可能高效。

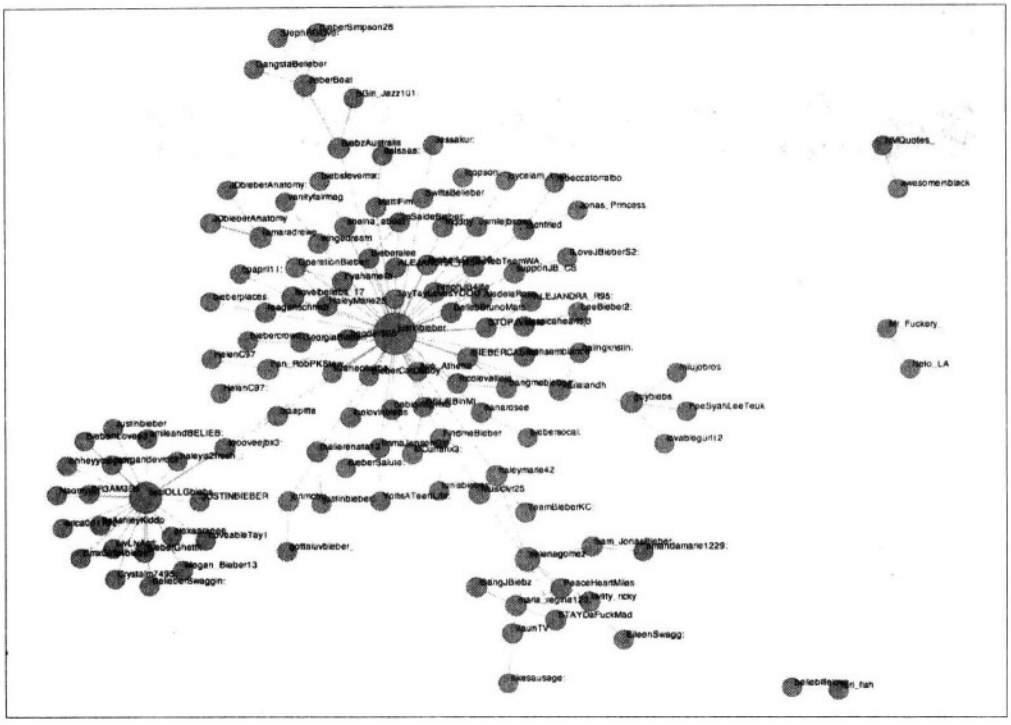

图1-4：以力导向布局呈现的交互式Protovis图，能可视化“JustinBieber”查询的转发关系

第2章

微格式：语义标记和常识碰撞

在网络的不断发展中，微格式是向前迈出的重要一步，因为它提供了一种有效的机制，可以将“更智能的数据”嵌入到网页中，并且易于内容创造者实现。简单来说，微格式（*http://www.microformats.org*）只是简单的约定，它以一种完全增值的方式明确地将结构化数据包含在网页中。本章首先简要地介绍微格式的全貌，然后深入研究一些示例，其中包含XFN（XHTML Friends Network）（*http://gmpg.org/xfn/*）、geo（*http://microformats.org/wiki/geo*）、hRecipe（*http://microforamts.org/wiki/hrecipe*）和hReview（*http://microformats.org/wiki/hReview*）的具体用途。我们将详细地从博客友情链接中挖掘人际关系，从网页中提取坐标，从foodnetwork.com（*http://foodnetwork.com*）中解析菜谱，并且分析其中一些菜谱的评论。虽然本章的示例代码清单并不打算作为“全面规范”，但也足以让你了解这些内容。

虽然将“被geo或hRecipe这类微格式修饰的数据”称为“社交数据”有点夸张，但它仍然十分有趣，而且必然会在社交数据混合应用中发挥更大的作用。编写本书时，几乎有一半的Web开发人员称他们使用了微格式（*http://www.webdirections.org/sotw10/markup/*），microformats.org社区刚刚庆祝了它的5岁生日（*http://microformats.org/2010/07/08/microformats-org-at-5-hcards-rich-snippets*），Google称富文本摘要中包含94%的时间、微格式（*http://microformats.org/2010/07/08/microformats-org-at-5-hcards-rich-snippets*）。如果说Google的这一结论能说明点什么的话，那么我们可以说，我们看到了微格式的显著增长。事实上，根据ReadWriteWeb，Google希望看到至少50%的网页包含某种形式的语义标记，并且非常鼓励公司进行“同行之间的良性竞争”来支持这些行为（*http://www.readwriteweb.com/archives/google_semantic_web_push_rich_snippets_usage_grow.php*）。无论从哪方面考虑，如果你重视Web空间，将来你都会看到更多的微格式，所以让我们开始学习它吧！

XFN和朋友

语义网爱好者宣称，FOAF（*http://en.wikipedia.org/wiki/FOAF_(software)*）（朋友的朋友——描述人与人之间的关系、他们的活动等的实体论）之类的技术有一天也许会成为驱动健壮而又分散的社交网络的催化剂，这可以理解为Facebook这样的严格控制的平台的对立。虽然所谓的语义网技术（如FOAF）似乎还没有成为使他们无处不在的引爆点，但这并不太令人惊讶。如果你十分了解Web短暂的历史，那么你会认识到创新频现，而且网站运营的高度分散性并不十分有利于应对突如其来的变革（参阅第10章）。然而，变化似乎是以一种渐进的方式频繁、流畅地发生。微格式所经历的用来填补Web中无效的“智能数据”的方式，就是“桥接现有技术和即将出现的新兴标准”的一个非常好的示例。在这种特殊情况下，要想让相当模糊的网络之间的差距缩小并不可能实现，它主要是基于人类可读的HTML 4.01标准（*http://www.w3.org/TR/REC-html40/*），使用信息更明确、对机器解读更友好的语义网。

微格式的魅力在于它们提供了一种嵌入数据的方式，这些数据与社交网络、日历、简历和共享书签相关，而且它们更多地是以完全向后兼容的方式立即嵌入到已经存在的HTML标记中的。因为*geo*这类微格式而变得多样化的整个生态系统正在悄然建立，而其余的技术正在缓慢发展，而且正在实现新兴的搜索引擎、社交媒体网站和博客平台的普及。编写本书时，微格式社区正在进行突飞猛进的发展，包括Google宣告他们已经开始支持hRecipe作为其富文本摘要倡议的一部分（*http://googlewebmastercentral.blogspot.com/2010/04/better-recipes-on-web-introducing.html*）。表2-1提供了一些流行的微格式和浏览网站时可能会遇到的一些相关举措。更多示例请参阅*http://microformats.org/wiki/examples-in-the-wild*。

表2-1：在网页中嵌入结构化数据的一些流行技术

技术	用途	人气	标记规范	类型
XFN	表示超链接中人类可读的关系	广泛使用，尤其被广泛应用在博客平台中	语义HTML、XHTML	微格式
geo	嵌入人与物体的地理坐标	广泛使用，尤其是被MapQuest和Wikipedia这样的网站广泛使用	语义HTML、XHTML	微格式
hCard	标识个人、公司和其他联系信息	广泛使用	语义HTML、XHTML	微格式

表2-1：在网页中嵌入结构化数据的一些流行技术（续）

技术	用途	人气	标记规范	类型
hCalendar	嵌入iCalendar数据	逐渐发挥作用（*http://microformats.org/2010/04/28/google-adds-support-for-hcalendar-and-hrecipe-rich-snippets*）	语义HTML、XHTML	微格式
hResume	嵌入简历和个人简历信息	被LinkedIn[a]这样的网站广泛使用	语义HTML、XHTML	微格式
hRecipe	标识食谱	被foodnet.com这样的利基网站广泛使用	语义HTML、XHTML	微格式
微数据	将名称/值对嵌入到HTML5授权的网页中	一项新技术，但已经开始发挥作用（*http://googlewebmastercentral.blogspot.com/2010/03/microdata-support-for-rich-snippets.html*）	HTML5	W3C倡议
RDFa	按照主题专家创造的专业词汇，将明确的事实嵌入到XHTML网页中	随意地依靠专业词汇；FOAF之类的词汇正在稳步发展而其他的仍然默默无闻	XHTML[b]	W3C倡议
开放图谱协议	将现实世界事物的介绍嵌入到XHTML网页中	正在稳步发展，因为Facebook平台的使用而潜力无限（*http://www.facebook.com/press/info.php?statistics*）	XHTML（基于RDFa）	Facebook平台倡议

a. LinkedIn以hResume（http://*steve.ganz.name/blog/2007/01/linkedin-launches-hresume.html*）格式呈现超过75百万全球用户的公开简历（*http://press.linkedin.com*）。

b. 在编写这篇文章时，正在积极尝试将RDFa嵌入到语义标记和HTML5中。参阅W3C HTML+RDFa 1.1工作草案（*http://www.w3.org/TR/rdfa-in-html/*）。

你可能还会遇到很多微格式，但是一个好的经验是看那些著名的网站（如Google、Yahoo!和Facebook）在做什么。微格式从它们那里获得的支持越多，就越可能成功，并对数据挖掘越有用。

语义标记

因为微格式用于将语义知识嵌入到网页中，所以要求XHTML或语义HTML是有意义的。但语义HTML或者语义标记和XHTML的区别究竟是什么呢？一种方法是从内容和表现相分离的角度来考虑。语义HTML标记是通过“以内容为主而不是以表现为主”的标签来强调网页中信息的含义。遗憾的是，HTML作为最初的定义和最早的发展，几乎没有促进语义标记的发展。另一方面，XHTML是格式良好的XML，是在20世纪90年代后期制定的，用于解决出现在HTML中的、由于内容和表现相分离而产生的重要问题。当时的想法是原文可以由完全不以表现为主的XML编写，但是工具可以将内容转换为容易被浏览器利用和呈现的内容。被转换后的内容被称为XHTML。XHTML与HTML几乎相同，只是它仍然是有效的XML，并且要求所有的元素都是封闭的或者自我封闭的、正确嵌套的，并且是用小写字母定义的。

从设计方面来说，XHTML似乎正是Web需要的。从这个建议来看，可以得到很多好处，而且几乎没有任何损失：结构良好的XHTML内容可以证明对XML模式有效，而且能享受XML的所有其他优点，如使用命名空间（RDFa这类语义网技术依赖的设备）的自定义属性。仅存的问题是：它没有流行起来。无论是IE浏览器的问题，Web开发者之间关于提供给浏览器正确的MIME类型的混乱、可用的XML开发工具的质量，还是期望整个网络采取一个简短的超时来执行转换的事实，也许都是我们应该避免的有争议的内容。但现实是我们之前预计的都没有发生。因此，我们仍然生活在一个基于HTML 4.01标准的语义标记的世界里（虽然经过了十年的时间，但HTML 4.01标准仍然在蓬勃发展），而基于XHTML的技术（如RDFa）仍然不是主流。大多数网络开发者都屏住了呼吸，希望HTML5能够创造一个期待已久的融合。

使用XFN来探讨社交关系

我们已经初步了解了微格式是如何融入整个网络空间的，现在让我们学习XFN的一些实际应用，它可能是目前为止你遇到的最流行的微格式了。正如你所知的，XFN通过在锚标签的`rel`属性中添加一些关键字来标识你和其他人之间的关系。经常在博客中，尤其是在“博客友情链接”插件中，就像WordPress提供的那些插件中使用XFN[注1]。考虑示例2-1所示的HTML内容，它可能会出现在博客友情链接中。

注1：请参阅*http://codex.wordpress.org/Links_Add_New_SubPanel*来获取流行插件的示例。

示例2-1：XFN标记示例

```
<div>
    <a href="http://example.org/matthew" rel="me">Matthew</a>
    <a href="http://example.com/users/jc" rel="friend met">J.C.</a>
    <a href="http://example.com/users/abe" rel="friend met co-worker">Abe</a>
    <a href="http://example.net/~baseeret" rel="spouse met">Baseeret</a>
    <a href="http://example.net/~lindsaybelle" rel="child met">Lindsay Belle</a>
    </div>
```

阅读这些rel标签中的内容，可以很明显地看出不同人之间的关系。即，Matthew有两个朋友、一个配偶和一个孩子。他与其中的一个朋友一起工作，而且在“现实生活”中与每个人都会会面（这并不是严格意义上的在线关系）。XFN只是使用一些定义明确的词汇（*http://gmpg.org/xfn/1*）。好消息是当在足够大的规模中使用XFN时，它虽然看似简单，却非常强大，因为它是特意编写的结构化信息。除非你遇到一些完全过时的内容（如两个最好的朋友变成了敌人，却忘了更新他们的博客友情链接），XFN都能完全正确地指出两个人是什么关系。由于大多数博客平台都支持XFN，因此可以发现很多信息。坏消息是XFN并没有真正告诉你这些基本资料以外的任何事，所以你需要使用其他信息源和技术来发现基于这些事实的推论，如：“Matthew和Baseeret结婚了，而且有一个叫Lindsay Belle的孩子。”但是你总是要从某一点开始，不是吗？

让我们快速编写一个简单的脚本来收集XFN数据，这类似于rubhub（*http://rubhub.com*）提供的服务，它是一个使用XFN来抓取并检索大量网站的社交搜索引擎。如果你想在进入下一节之前研究一下全面规范的话，可能还需要使用其中一种在线XFN工具（*http://gmpg.org/xfn/creator*）。

对XFN数据的广度优先抓取

让我们通过挖掘一些XFN数据，并基于它创建社交图谱来进入社交领域。由于XFN可以嵌入到任何网页中，因此我们不得不做一些Web抓取。然而，它并没有多难，它几乎是你见过的最简单的Web抓取了，而且BeautifulSoup包可以大大减轻我们的负担。示例2-2的代码使用Ajaxian（*http://ajaxian.com*）作为图的基础，它是关于现代网络发展的大众博客。在运行easy_install BeautifulSoup命令之后再尝试运行它。

示例2-2：抓取网页中的XFN内容（microformats_xfn_scrape.py）

```
# -*- coding: utf-8 -*-

import sys
import urllib2
import HTMLParser
from BeautifulSoup import BeautifulSoup

# Try http://ajaxian.com/
URL = sys.argv[1]
```

```
XFN_TAGS = set([
    'colleague',
    'sweetheart',
    'parent',
    'co-resident',
    'co-worker',
    'muse',
    'neighbor',
    'sibling',
    'kin',
    'child',
    'date',
    'spouse',
    'me',
    'acquaintance',
    'met',
    'crush',
    'contact',
    'friend',
    ])

try:
    page = urllib2.urlopen(URL)
except urllib2.URLError:
    print 'Failed to fetch ' + item

try:
    soup = BeautifulSoup(page)
except HTMLParser.HTMLParseError:
    print 'Failed to parse ' + item

anchorTags = soup.findAll('a')
for a in anchorTags:
    if a.has_key('rel'):
        if len(set(a['rel'].split()) & XFN_TAGS) > 0:
            tags = a['rel'].split()
            print a.contents[0], a['href'], tags
```

警告： 从3.1.X版本开始，BeautifulSoup开始使用HTMLParser（*http://docs.python.org/library/htmlparser.html*）来代替SGMLParser，因而，常见的抱怨是：比起3.0.x版本中的SGMLParser来，BeautifulSoup似乎不太健壮。BeautifulSoup的创造者提出了很多建议（*http://www.crummy.com/software/beautiful soup/3.1-problems.html*），如果3.1.x不够健壮，满足不了你的需求，那么你最好选择使用3.0.x版本。另外，你可能遇到示例2-2所示的HTMLParseError，并丢弃这些内容。

运行包含XFN信息的URL代码会返回姓名、关系类型，且一个人的每个朋友都会返回一个URL。示例输出如下：

```
Dion Almaer http://www.almaer.com/blog/ [u'me']
Ben Galbraith http://weblogs.java.net/blog/javaben/ [u'co-worker']
Rey Bango http://reybango.com/ [u'friend']
```

```
Michael Mahemoff http://softwareas.com/ [u'friend']
Chris Cornutt http://blog.phpdeveloper.org/ [u'friend']
Rob Sanheim http://www.robsanheim.com/ [u'friend']
Dietrich Kappe http://blogs.pathf.com/agileajax/ [u'friend']
Chris Heilmann http://wait-till-i.com/ [u'friend']
Brad Neuberg http://codinginparadise.org/about/ [u'friend']
```

假设每个朋友的URL都包含XFN或者其他有用的信息，那么追踪这些链接，以系统的方式生成更多的社交图谱信息就十分简单了。这正是下一个代码示例实现的方法：它以广度优先的方式创建一幅图，这就是说，它所做的工作类似于示例2-3中伪代码所描述的内容。

示例2-3：广度优先搜索的伪代码

```
Create an empty graph
Create an empty queue to keep track of nodes that need to be processed

Add the starting point to the graph as the root node
Add the root node to a queue for processing

Repeat until some maximum depth is reached or the queue is empty:
  Remove a node from the queue
  For each of the node's neighbors:
    If the neighbor hasn't already been processed:
      Add it to the queue
      Add it to the graph
      Create an edge in the graph that connects the node and its neighbor
```

既然这种建立图谱的方法有利于在双向节点之间创建边，所以如果存在这样的边，就不需要任何额外的操作了。它可以用于社交场合，因为它本来就适用于标识共同的朋友，而不需要提供任何附加逻辑。示例2-4是对示例2-2中代码的改进，通过追踪嵌入了XFN数据的超链接，采用广度优先的方法来构建NetworkX图。即使运行两段示例代码中深度较浅的代码，这取决于你身边朋友的邻近社区中XFN的流行程度，可以通过生成该图的图像文件，并检查它或者在它上面运行图的各种度量看到它（见图2-1）。

注意： Windows用户可能想要复习一下1.2.3节，来解释为什么nx.drawing.write_dot的解决方案是示例2-4必需的。

示例2-4：使用深度优先搜索抓取XFN连接（microformats_xfn_crawl.py）

```
# -*- coding: utf-8 -*-

import sys
import os
import urllib2
from BeautifulSoup import BeautifulSoup
```

```
import HTMLParser
import networkx as nx

ROOT_URL = sys.argv[1]

if len(sys.argv) > 2:
    MAX_DEPTH = int(sys.argv[2])
else:
    MAX_DEPTH = 1

XFN_TAGS = set([
    'colleague',
    'sweetheart',
    'parent',
    'co-resident',
    'co-worker',
    'muse',
    'neighbor',
    'sibling',
    'kin',
    'child',
    'date',
    'spouse',
    'me',
    'acquaintance',
    'met',
    'crush',
    'contact',
    'friend',
    ])

OUT = "graph.dot"

depth = 0

g = nx.DiGraph()

next_queue = [ROOT_URL]

while depth < MAX_DEPTH:

    depth += 1
    (queue, next_queue) = (next_queue, [])

    for item in queue:
        try:
            page = urllib2.urlopen(item)
        except urllib2.URLError:
            print 'Failed to fetch ' + item
            continue

        try:
            soup = BeautifulSoup(page)
        except HTMLParser.HTMLParseError:
            print 'Failed to parse ' + item
            continue
```

```
        anchorTags = soup.findAll('a')

        if not g.has_node(item):
            g.add_node(item)

        for a in anchorTags:
            if a.has_key('rel'):
                if len(set(a['rel'].split()) & XFN_TAGS) > 0:
                    friend_url = a['href']
                    g.add_edge(item, friend_url)
                    g[item][friend_url]['label'] = a['rel'].encode('utf-8')
                    g.node[friend_url]['label'] = a.contents[0].encode('utf-8')

                    next_queue.append(friend_url)

        # Further analysis of the graph could be accomplished here

if not os.path.isdir('out'):
    os.mkdir('out')

try:
    nx.drawing.write_dot(g, os.path.join('out', OUT))
except ImportError, e:

    # Help for Windows users:
    Exploring
    # Not a general purpose method, but representative of
    # the same output write_dot would provide for this graph
    # if installed and easy to implement

    dot = []
    for (n1, n2) in g.edges():
        dot.append('"%s" [label="%s"]' % (n2, g.node[n2]['label']))
        dot.append('"%s" -> "%s" [label="%s"]' % (n1, n2, g[n1][n2]['label']))

    f = open(os.path.join('out', OUT), 'w')
    f.write('''strict digraph {
    %s
    }''' % (';\n'.join(dot), ))
    f.close()

# *nix users could produce an image file with a good layout
# as follows from a terminal:
# $ circo -Tpng -Ograph graph.dot
# Windows users could use the same options with circo.exe
# or use the GVedit desktop application
```

注意： 有向图中节点和边的属性label的使用并不是随意的。节点和边中包含的属性被序列化为DOT语言，Graphviz工具将标签识别为特殊的属性（*http:www.graphviz.org/doc/info/attrs.html#d:label*）。

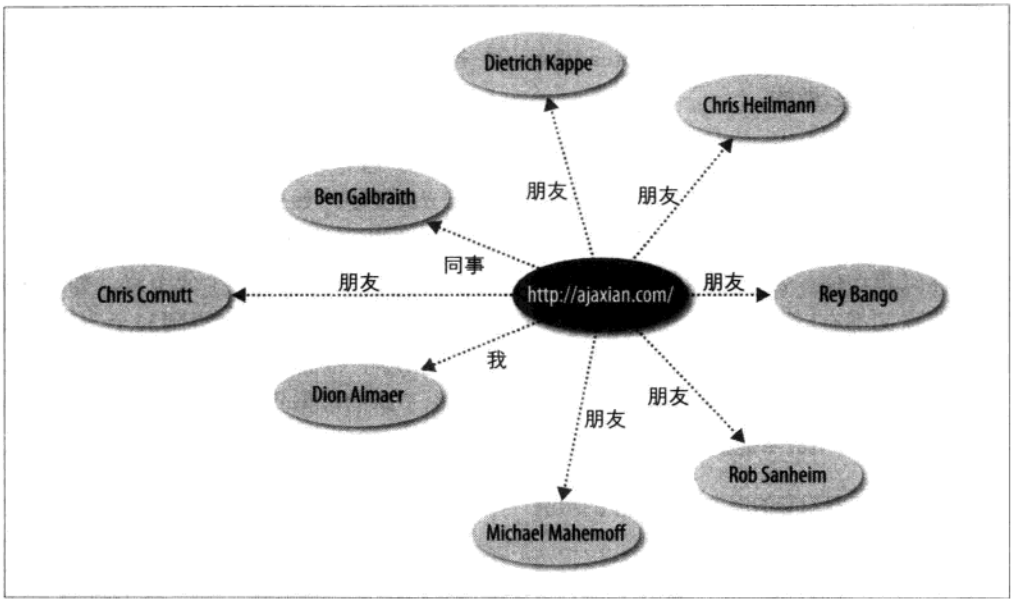

图2-1：Ajaxian XFN数据的有向图——请注意，http://ajaxian.com和Dion Almaer之间的边标签“我”表明博客是Dion的。

这幅图虽然很简单，却非常有趣。它连接了8个人，其中，Dion Almaer是共同的主线。然而，请注意，抓取一层或多层可能会引入图中“与其他所有人都连接”的节点。单看图的话，我们无法根据“同事”和“朋友”之间的关系，判别Dion与Ben Galbraith的关系是否更为密切，但是如果他在“被他的超链接标识的目标”中提供了任何信息的话，我们就可以抓取Ben的XFN信息，搜索其他同事标签来构建“谁与谁共事”的社交网络。更多挖掘数据的知识请查看第6章，因为它与同事和工作搭档相关。

对广度优先技术的简单分析

一般我们不会停顿这么长时间来分析该方法，但是由于这个示例是我们编写的第一个真正的算法，而且我们会在本书中多次见到它，因此值得更仔细地对它进行分析。一般来说，当你检查算法时，必须考虑两个标准：效率和有效性。换一种说法就是：性能和质量。

任何算法的标准性能分析通常都包括分析它在最坏情况下的时间复杂度和空间复杂度，即对于一个大型数据集，执行程序所花的时间和需要的内存。我们采用的广度优先方法实质上是广度优先搜索，只是我们并没有真正执行搜索，因为结束条件并没有把图扩展到最大深度或直到我们遍历完所有节点。如果搜索了一些具体的东西，而不只是无限地抓取链接，它就可以被视为真正的广度优先搜索了。然而，在实践中，由于计算资源的

有限性，几乎所有的搜索都会强加一些结束条件。因此，广度优先搜索更常见的变通做法称为有界广度优先（bounded breadth-first）搜索，它强制了搜索的最大深度，本示例中我们也是这么做的。

对于一个广度优先搜索（或广度优先抓取）来说，最坏情况下的时间复杂度和空间复杂度都可以通过b^d限定，其中，b是图的分支因子，d是深度。如果你在纸上勾画出一个示例，并且思考它，这个分析将是非常有意义的。如果图中的每个节点都有5个邻节点，而且你只想访问一层，那么最终你会得到6个节点：根节点和它的5个邻节点。如果这5个邻节点又都有5个邻节点，而且你想要再扩展一层，那么最终你会得到31个节点：根节点，根节点的5个邻节点，每个根节点的邻节点的5个邻节点。这种分析看似迂腐，但处理大数据时，能够完成粗略的分析，是以“你了解的数据集比以往的数据集重要得多”为基础的。表2-2提供了一个概述，用于说明小规模的b和d的b^d是如何增长的。

表2-2：计算不同深度的图的分支因子的示例

分支因子	深度=1的节点	深度=2的节点	深度=3的节点	深度=4的节点	深度=5的节点
2	3	7	15	31	63
3	4	13	40	121	364
4	5	21	85	341	1365
5	6	31	156	781	3906
6	7	43	259	1555	9331

虽然之前的讨论主要属于该算法的理论，但值得注意的最终考虑因素是：对于一个固定大小的数据集，该算法的实际性能到底如何。对代码的简单分析表明，从“绝大多数时间都用在等待`urlopen`方法返回处理的内容”的观点来看，它主要是I/O约束。3.3节的示例介绍并展示了使用线程池来提高性能，并以此作为复杂度的交换。

在分析方面，最后需要考虑的是结果的综合质量。从质量分析的角度来看，对图输出的基本直观检查表明连接人的图是真实存在的。任务完成了吗？是的，但仍然有改进的空间。需要考虑的是URL的微小变化可能会导致同一个人的多个节点出现在图中。例如，如果Matthew在一个超链接中引用了URL *http://example.com/~matthew*，在另一个URL中引用了*http://www.example.com/~matthew*，尽管它们指的很可能是Web上相同的资源，但是图中的这两个节点也会不同。幸运的是，XFN定义了`rel="me"`这个特殊值，可以用于统一身份（*http://gmpg.org/xfn/and/#idconsolidation*）。Google的社交图谱API（*http://code.google.com/apis/socialgraph/*）采用了这种方法来连接用户不同的个人资料，而且存在许多使用`rel="me"`来允许用户连接多个外部网站中个人资料的服务示例。解决URL

的另一个（小得多的）问题是结束位置上末尾斜线的使用或省略。大多数精心设计的网站都能自动将一个URL重定向到另一个URL，所以这个细节通常不是问题。

幸运的是，其他人也认识到了这类问题，而且决定做些什么。SocialGraph Node Mapper（*http://code.google.com/p/google-sgnodemapper/*）是一个有趣的开源项目，它标准化了一些涉及末尾斜线的URL、“www”的存在等，但是它也认识到不同的社交网站可能显示不同的URL，它们都可能链接到同一个人。例如，对于某个给定的社交网站来说，*http://blog.example.com/matthew*和*http://status.example.com?user=matthew*可能被解析为同一个人。

地理坐标：兴趣爱好的共同主线

geo和hRecipe这类微格式在挖掘社交网络方面不是特别有用，但如果我们因此就不讨论它们的话，那就大错特错了。虽然非特定环境下的独立geo数据未必有社交属性，但是重要而并非显而易见的关系通常都来自不同的数据集，它们通过相同的地理环境连接在一起。geo数据是无处不在的，它在社交混搭应用甚至是这些应用的命名中都起着非常重要的作用，因为我们可以根据空间中的某个特定的点将人们聚集在一起。“现实生活”和Web生活之间的界限正在日益缩小，与现实世界中的个人连接的任何数据都会变成社交数据。例如，基于一个人的确切地址、她在家喜欢做的食物，甚至是那些菜谱的配方，你可以了解她的很多情况。这一节通过查找、分析、可视化一些geo和hRecipe数据的示例来学习，因为你可能会想尽早找到它的用武之地。

维基百科文章+Google地图=自驾游是否成立

把“将地理位置信息嵌入到网页中一种最简单并且使用最广泛的微格式”称为geo是非常恰当的。该规范的灵感来自于vCard（*http://www.ietf.org/rfc/rfc2426.txt*）中具有相同名称的一个属性，它提供了指定位置的方法。geo中有两种嵌入微格式的方法。示例2-5中的HTML片段说明了两种描述田纳西州中最好的小镇富兰克林的技术。

示例2-5：geo标记示例

```
<!-- The multiple class approach -->
<span style="display: none" class="geo">
    <span class="latitude">36.166</span>
<span class="longitude">-86.784</span>
    </span>

<!-- When used as one class, the separator must be a semicolon -->
<span style="display: none" class="geo">36.166; -86.784</span>
```

正如你所看到的，该微格式简单地将纬度值和经度值封装在具有相应类名的标记中，并且将它们都打包在具有geo类的一个标签中。很多流行的网站（包括维基百科、雅虎本

地、MapQuest本地和其他网站）都使用geo和其他微格式，在他们的网页中显示结构化的数据。

注意： 使用geo的习惯做法是要隐藏用户所携带的信息。可以使用两种传统的CSS完成该工作：style="display:none"和style="visibility:hidden"。前者完全删除了网页中该元素的位置，以便布局表现得好像它不存在一样。后者隐藏了内容，但是保留了它在页面中占用的空间。

示例2-6是一个简单的程序，它从MapQuest本地页面解析出geo微格式数据，用于说明如何从实现了geo微格式的内容中提取坐标。

示例2-6：从MapQuest本地提取geo数据（microformats_mapquest_geo.py）

```
# -*- coding: utf-8 -*-

import sys
import urllib2
from BeautifulSoup import BeautifulSoup
import HTMLParser

# Pass in a URL such as http://local.mapquest.com/franklin-tn

url = sys.argv[1]
try:
    page = urllib2.urlopen(url)
except urllib2.URLError, e:
    print 'Failed to fetch ' + url
    raise e
    exit()

try:
    soup = BeautifulSoup(page)
except HTMLParser.HTMLParseError:
    print 'Failed to parse ' + url
    exit()

geoTag = soup.find(True, 'geo')

if geoTag and len(geoTag) > 1:
    lat = geoTag.find(True, 'latitude').string
    lon = geoTag.find(True, 'longitude').string
    print 'Location is at', lat, lon
elif geoTag and len(geoTag) == 1:
    (lat, lon) = geoTag.string.split(';')
    (lat, lon) = (lat.strip(), lon.strip())
    print 'Location is at', lat, lon
else:
    print 'No location found'
```

使用微格式的含义微妙而有些深刻：虽然人们可能阅读关于某地的文章，如富兰克林镇、田纳西州，而且只凭直觉就可以知道这页地图上的一点指示该镇的位置，但是如果

没有给机器人配置不同模式的专门逻辑，机器人是无法轻易地得到这个结论的。这种页面分析是个棘手的主题，通常情况下你可能认为你已经想到了所有的可能性，但你会发现还是漏了一种。将正确的语义嵌入页面中消除了二义性，可降低爬虫和开发者（如，你自己）的门槛，该页面有效地标记了非结构化数据，在某种程度上，即使我们的伙伴Robby机器人（*http://en.wikipedia.org/wiki/Robby_the_Robot*）也可以理解。这是生产者和消费者双赢的局面，但愿最终的效果是所有人都能有所创新。

通过microform.at和Google地图绘制geo数据图

一旦你发现嵌入了有趣的geo数据的网页，你想做的第一件事就是将它可视化。例如，让我们来看看维基上的“List of National Parks of the United States”（*http://en.wikipedia.org/wiki/List_of_U.S._national_parks*）这篇文章。它显示了国家公园漂亮的表格视图，而且使用geo格式标记了它们，但是快速将数据加载到可以目测的交互式工具中不是更好吗？microform.at（*http://microform.at*）是一个了不起的小型网络服务，它可以从给定的URL中提取几种微格式，并且以各种有用的格式将它们传送回来。它显示了各种检测和与网页中的微格式数据交互的选项，如图2-2所示。

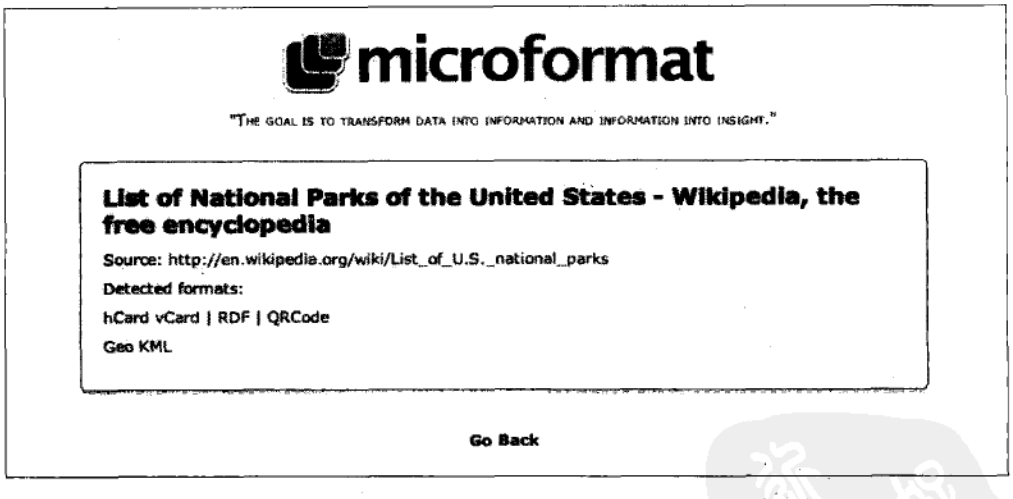

图2-2：维基上名为“List of National Parks of the United States”的文章的http://microform.at结果

如果可以选择，KML（*http://code.google.com/apis/kml/documentation/*）（Keyhole Markup Language，Keyhole标记语言）输出可能是最简单的可视化geo数据的方法。可以下载Google Earth并且在本地加载KML文件，或者直接在Google Maps搜索栏中输入包含KML数据的URL来提交它，而不需要付出任何其他努力。对于显示在microform.

at上的结果，点击“KML”链接会触发文件下载，可以在Google Earth中使用该文件，但是可以通过右击将它复制到剪贴板上，并且将它传递给Google Earth。图2-3显示了Google Earth对*http://microform.at/?type=geo&url=http%3A%2F%2Fen.wikipedia.org%2Fwiki%2FList_of_U.S._national_parks*的可视化——前面提到的维基百科文章的KML结果，这只是使用type和url查询字符串参数的基础URL *http://microform.at*。

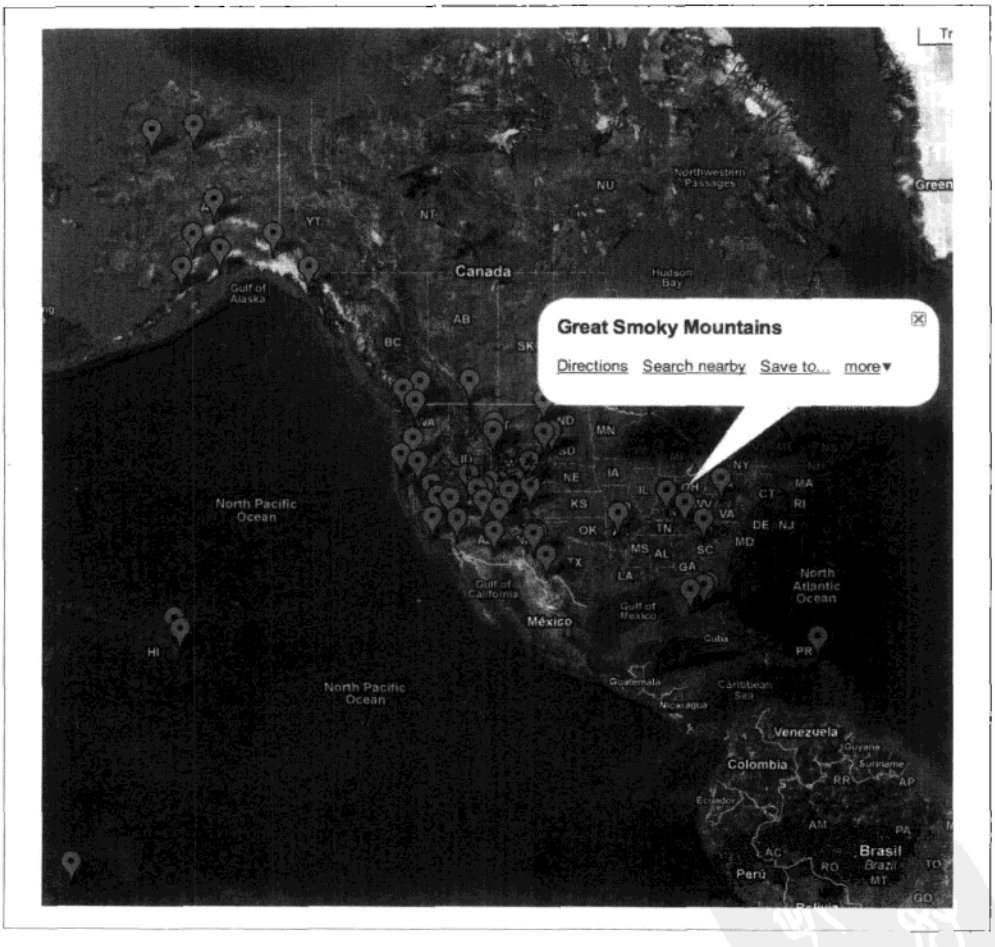

图2-3：当传递了microform.at中的KML结果后，显示了美国所有国家公园的Google Maps结果

以维基百科中“包含了geo数据这类语义标记”的文章开始，能够详细地将其可视化出来，这是一种强大的分析能力，因为它以如此小的代价就迅速地传递出了其内涵。像Firefox Operator插件（*https://addons.mozilla.org/en-US/firefox/addon/4106*）这样的浏览器扩展就是为了进一步减小所需付出代价。一章里只可以说这么多，花一小时左右的时间来将本节的国家公园数据与LinkedIn职业网络中的联系方式结合起来，来看看如何才

能让你的下一次（可能是人为的）出差更有乐趣呢？（查看6.2节的示例，了解如何通过使用k-均值技术寻找聚类，并计算这些聚类的中心，来获取并分析geo数据。）

（以健康的名义）对菜谱进行交叉分析

因为Google的富文本摘要主动使用了微格式，所以对微格式的认识在不断增长，很多最流行的美食网站已经使用hRecipe和hReview来展示菜谱和评论了。考虑虚拟的网上约会服务的潜力，它抓取博客和其他社交中心，试着将人们配对起来进行晚餐约会。人们可以合理地预期，使用足够多的链接到具体人的geo和hRecipe信息，将会对其初次约会“成功率”产生巨大的影响。可以依据两个标准来配对：他们住的地方有多远和他们喜欢吃什么。例如，两个喜欢用有机原料煮素食的人之间的晚餐约会比烧烤爱好者和素食爱好者之间的约会好得多。饮食喜好和是否使用某种过敏原或有机原料可以成为有用的线索，可以强化正确的经营理念。虽然我们并不打算推出一个新的在线数据服务，但是万一你决定采纳这个想法并支持它，我们就会开始这么做。

Food Network（*http://www.foodnetwork.com*）是真正使用了微格式改进整个网络倡议的在线网站之一，它将菜谱信息显示在hRecipe微格式中，并且在菜谱的评论中使用了hReview微格式[注2]。本节说明了搜索引擎（或是你）如何从Food Network网页的菜谱和评论中解析出结构化数据，并对其进行索引或分析。虽然我们并没有分析菜谱或评论中的自由文本，或者永久地存储提取的信息，但是如果你感兴趣的话，后面的几章会告诉你怎么做。特别是，第3章将介绍CouchDB，它可以存储、共享（并分析）从支持微格式的Web内容中提取出来的数据，第7章将介绍自然语言处理（Natural Language Processing，NLP）的一些基础知识，可以用它来深入了解评论中的内容（很凑巧的是，在本章中已出现了一道菜谱。）

示例2-7是对“用于解析hRecipe格式数据”的示例2-6的改编。

示例2-7：解析泰式菜谱的hRecipe数据（microformats_foodnetwork_hrecipe.py）

```
# -*- coding: utf-8 -*-

import sys
import urllib2
import json
import HTMLParser
import BeautifulSoup
```

注2：2010年中期，Food Network以和Yelp相同的方式使用了hReview，下一节将介绍这些内容；然而，2011年1月上旬，Food Network提供的工具更改为只包含hreview-aggregate（*http://microformats.org/wiki/hreview-aggregate*）。

```
# Pass in a URL such as
# http://www.foodnetwork.com/recipes/alton-brown/pad-thai-recipe/index.html

url = sys.argv[1]

# Parse out some of the pertinent information for a recipe
# See http://microformats.org/wiki/hrecipe


def parse_hrecipe(url):
    try:
        page = urllib2.urlopen(url)
    except urllib2.URLError, e:
        print 'Failed to fetch ' + url
        raise e

    try:
        soup = BeautifulSoup.BeautifulSoup(page)
    except HTMLParser.HTMLParseError, e:
        print 'Failed to parse ' + url
        raise e

    hrecipe = soup.find(True, 'hrecipe')

    if hrecipe and len(hrecipe) > 1:
        fn = hrecipe.find(True, 'fn').string
        author = hrecipe.find(True, 'author').find(text=True)
        ingredients = [i.string
                       for i in hrecipe.findAll(True, 'ingredient')
                           if i.string is not None]

        instructions = []
        for i in hrecipe.find(True, 'instructions'):
            if type(i) == BeautifulSoup.Tag:
                s = ''.join(i.findAll(text=True)).strip()
            elif type(i) == BeautifulSoup.NavigableString:
                s = i.string.strip()
            else:
                continue
            if s != '':
                instructions += [s]

        return {
            'name': fn,
            'author': author,
            'ingredients': ingredients,
            'instructions': instructions,
            }
    else:
        return {}

recipe = parse_hrecipe(url)
print json.dumps(recipe, indent=4)
```

对于一个示例URL（如Alton Brown著名的泰式菜谱），可以得到示例2-8所示的结果。

示例2-8：从示例2-7中得到的泰式菜谱解析结果

```
{
    "instructions": [
        "Place the tamarind paste in the boiling water and set aside ...",
        "Combine the fish sauce, palm sugar, and rice wine vinegar in ...",
        "Place the rice stick noodles in a mixing bowl and cover with ...",
        "Press the tamarind paste through a fine mesh strainer and add ...",
        "Place a wok over high heat. Once hot, add 1 tablespoon of the ...",
        "If necessary, add some more peanut oil to the pan and heat until ..."
    ],
    "ingredients": [
        "1-ounce tamarind paste",
        "3/4 cup boiling water",
        "2 tablespoons fish sauce",
        "2 tablespoons palm sugar",
        "1 tablespoon rice wine vinegar",
        "4 ounces rice stick noodles",
        "6 ounces Marinated Tofu, recipe follows",
        "1 to 2 tablespoons peanut oil",
        "1 cup chopped scallions, divided",
        "2 teaspoons minced garlic",
        "2 whole eggs, beaten",
        "2 teaspoons salted cabbage",
        "1 tablespoon dried shrimp",
        "3 ounces bean sprouts, divided",
        "1/2 cup roasted salted peanuts, chopped, divided",
        "Freshly ground dried red chile peppers, to taste",
        "1 lime, cut into wedges"
    ],
    "name": "Pad Thai",
    "author": "Recipe courtesy Alton Brown, 2005"
    }
```

虽然它并不是真正的社交分析形势，但分析同一菜谱的变化，观察外观之间是否有任何关联或者是否缺少某种原料、菜谱的评分/评论是十分有趣的。例如，你可以试着搜罗一些不同的泰式菜谱，确定菜谱中最常见的原料有哪些，最不常见的有哪些。

对餐厅评论的搜集

这一节通过简要地介绍hReview，总结了我们对微格式（和泰国菜）的研究。Yelp是实现hReview的主流服务，这样就能显示顾客对餐厅的评分了。示例2-9说明了如何获取Yelp实现的hReview信息。可以尝试的示例URL包含在示例代码中，它代表了一家泰国餐厅，如果你有机会去参观的话，那么你绝对不想错过它。

警告： 虽然规范很稳定，但hReview的实现似乎不尽相同，而且所包含的偏差也千奇百怪。特别是，示例2-9没有将reviewer解析为hCard，因为Yelp实现本身并未包含该信息。

示例2-9：对泰式菜谱的hReview数据解析（microformats_yelp_hreview.py）

```
# -*- coding: utf-8 -*-

import sys
import re
import urllib2
import json
import HTMLParser
from BeautifulSoup import BeautifulSoup

# Pass in a URL that contains hReview info such as
# http://www.yelp.com/biz/bangkok-golden-fort-washington-2

url = sys.argv[1]

# Parse out some of the pertinent information for a Yelp review
# Unfortunately, the quality of hReview implementations varies
# widely so your mileage may vary. This code is *not* a spec
# parser by any stretch. See http://microformats.org/wiki/hreview

def parse_hreviews(url):
    try:
        page = urllib2.urlopen(url)
    except urllib2.URLError, e:
        print 'Failed to fetch ' + url
        raise e

    try:
        soup = BeautifulSoup(page)
    except HTMLParser.HTMLParseError, e:
        print 'Failed to parse ' + url
        raise e

    hreviews = soup.findAll(True, 'hreview')

    all_hreviews = []
    for hreview in hreviews:
        if hreview and len(hreview) > 1:

            # As of 1 Jan 2010, Yelp does not implement reviewer as an hCard,
            # per the spec

            reviewer = hreview.find(True, 'reviewer').text

            dtreviewed = hreview.find(True, 'dtreviewed').text
            rating = hreview.find(True, 'rating').find(True, 'value-title')['title']
            description = hreview.find(True, 'description').text
            item = hreview.find(True, 'item').text

            all_hreviews.append({
                 'reviewer': reviewer,
                 'dtreviewed': dtreviewed,
                 'rating': rating,
                 'description': description,
                 })
    return all_hreviews

reviews = parse_hreviews(url)
```

```
# Do something interesting like plot out reviews over time
# or mine the text in the descriptions...

print json.dumps(reviews, indent=4)
```

示例2-9中的截取样例结果如示例2-10所示。它们包括评论者，将后者从hCard微格式节点中解析出来，并作为它自己的对象。

示例2-10：对应于示例2-9的示例hReview样例结果

```
[
    {
        "reviewer": "Nick L.",
        "description": "Probably the best Thai food in the metro area...",
        "dtreviewed": "4/27/2009",
        "rating": "5"
    },

    ...truncated...

]
```

遗憾的是，不论是Yelp、Food Network，还是其他网站，都没有提供足够具体的信息来将评论者以一种有意义的方式集合起来，但愿这种情况很快就能得到改观，给社交网络开发出更多的可能性。同时，你可以充分利用得到的数据，划分出一家餐厅的平均评分，来看看这家餐厅的水平是改善了还是下降了。另一种想法可能是挖掘文本中的description字段。查看第7章和第8章来了解它的原理。

注意： 为了简洁，此处省略了将JSON数据传输到单一的输出列表并将它加载到电子表格中的繁重工作。但是如果你感觉特别懒的话，可以去下载这些数据的电子表格。

当你将奇客（geek）和食物数据结合起来时，你会发现可能出现无穷无尽的创新，最近出版的畅销书《Cooking for Geeks》（O'Reilly）（*http://www.cookingforgeeks.com*）就是很好的证明。因为食品网站发展到提供附加的API，所以我们可以在这个领域中看到更多的创新。

小结

如果你没深入了解本章，但有一点一定要记住：微格式是一种装饰标记的方法，是用来显示菜谱、联系方式和人际关系这些特定类型的结构化信息。微格式的潜力无限，因为它们允许我们采用现有的内容，并从中生成显式的、与预期标准一致的数据。希望在未来的几个月里能看到微格式的显著增长和对微格式的兴趣。你还可以看到基于HTML5的微数据（*http://www.w3c.org/TR/html5/microdata.html*）惊人的创新，因为HTML5获得了市场占有率。如果你发现你自己有大量的空闲时间，一定要看看Google的社交图谱API

（*http://code.google.com/apis/socialgraph/*）。不要与第9章讨论的Facebook的基于RDFa的（*http://en.wikipedia.org/wiki/RDFa*）开放图谱协议（*http://developers.facebook.com/docs/api*）或相关的图形API（*http://developers.facebook.com/docs/api*）混淆，这些API包括XFN（*http://gmpg.org/xfn/*）、FOAF（*http://www.foaf-project.org*）和其他公开声明的连接的索引。

第3章

邮箱：虽然老套却很好用

本章介绍一些基本的邮件（虽然社交网络有很多优点，但是在未来的几年中，邮件仍将是互联网的主打产品）分析工具和技术，以回答以下问题：

- 谁发送的邮件最多？
- 一天中是否存在某个特定的时间（或是一周中的某天），这时候发件人最可能收到对一个问题的回复？
- 几个人之间，谁发送的邮件最多？
- 讨论最热烈的话题是什么？

虽然社交媒体网站积累了越来越多的准实时社交数据，但它还是存在重大的缺点。和Email不同，社交网络数据是由服务提供者集中管理的，服务提供者可以创建规则，规定访问它的方式，以及你能做什么、不能做什么[注1]。另一方面，邮件数据主要是分散管理的，它以富邮件列表的形式分散在Web上，是对一连串有趣话题的讨论。如果你使用Google和Yahoo!这类服务提供商提供的服务来检索邮件列表，它们就会限制邮件列表数据的使用，但可以用稍微简单的方法来挖掘内容，这会提高成功率：可以通过订阅列表并等待数据的积累，这样一来就能轻易地采集数据；也可以让列表所有者提供给你一套存档内容，这样也可以收集到数据。另一个有趣的因素就是，不像社交媒体网站，企业对它们自己的邮箱都有完全的控制权，这样就可以进行聚集分析，并以此发现某种趋势。

注1：完全可以理解的是，任何人都（还）没有足够好的理由第一个出来为百万美元的诉讼案件进行辩护，并查明一些社交网站是否有一些必须遵守的使用条款。因此，（迄今为止的）现状就是遵守游戏规则，并在可能侵权时放弃。

你可能已经猜到了，要想找到作为例证的真实的社交数据集并不容易（如果还能找到的话），但幸运的是，本章有最为真实的数据集：公开的Enron邮件数据集（*http://www.cs.cmu.edu/~enron/*）。

mbox：Unix的入门级邮箱

假如你之前还没有见过mbox文件，那么我可以告诉你，从根本上说它是大型的串联邮件消息文本，很容易由基于文本的工具获取。每封邮件的开头都是由特殊的*From_*line标记的，并将它格式化为"From user@example.com Fri Dec 25 00:06:42 2009"模式，其中日期/时间戳是asctime（*http://opengroup.org/onlinepubs/007908775/xsh/asctime.html*）格式，它是一种标准的固定长度的时间戳表示。

注意： mbox格式是很常见的，大多数邮件客户端都提供了“导出”或者“另存为”选项，可以将数据导出为此格式，而不关心底层实现。

在mobox文件中，两封邮件之间的边界是由两个新行前面的（除了第一次出现）*From_*line决定的。示例3-1是一个虚构的mbox的一小部分，它包含两封邮件。

示例3-1：示例mbox文件的一小部分

```
From santa@northpole.example.org Fri Dec 25 00:06:42 2009
Message-ID: <16159836.1075855377439@mail.northpole.example.org>
References: <88364590.8837464573838@mail.northpole.example.org>
In-Reply-To: <194756537.0293874783209@mail.northpole.example.org>
Date: Fri, 25 Dec 2001 00:06:42 -0000 (GMT)
From: St. Nick <santa@northpole.example.org>
To: rudolph@northpole.example.org
Subject: RE: FWD: Tonight
Mime-Version: 1.0
Content-Type: text/plain; charset=us-ascii
Content-Transfer-Encoding: 7bit

Sounds good. See you at the usual location.

Thanks,
-S

-----Original Message-----
From: Rudolph
Sent: Friday, December 25, 2009 12:04 AM
To: Claus, Santa
Subject: FWD: Tonight

Santa -

Running a bit late. Will come grab you shortly. Standby.

Rudy
```

```
Begin forwarded message:

> Last batch of toys was just loaded onto sleigh.
>
> Please proceed per the norm.
>
> Regards,
> Buddy
>
> --
> Buddy the Elf
> Chief Elf
> Workshop Operations
> North Pole
> buddy.the.elf@northpole.example.org

From buddy.the.elf@northpole.example.org Fri Dec 25 00:03:34 2009
Message-ID: <88364590.8837464573838@mail.northpole.example.org>
Date: Fri, 25 Dec 2001 00:03:34 -0000 (GMT)
From: Buddy <buddy.the.elf@northpole.example.org>
To: workshop@northpole.example.org
Subject: Tonight
Mime-Version: 1.0
Content-Type: text/plain; charset=us-ascii
Content-Transfer-Encoding: 7bit

Last batch of toys was just loaded onto sleigh.

Please proceed per the norm.

Regards,
Buddy

--
Buddy the Elf
Chief Elf
Workshop Operations
North Pole
buddy.the.elf@northpole.example.org
```

但是可以证明mbox中至少还存在另一封邮件，在示例3-1中，我们可以看到两封邮件。按时间先后来说，第一封邮件是由Buddy撰写的，发送给*workshop@northpole.example.org*，告诉他玩具已经装好了。mbox中的另一封邮件是Santa对Rudolph的回复。示例mbox中没有显示的是一封中间邮件，Rudolph给Santa转发了Buddy的邮件，解释了他迟到的原因。虽然我们可以通过阅读这些邮件文本推断出发生的事情，但是我们也可以从Message-ID、References和In-Reply-To标头中得到重要线索。这些标头非常直观，为显示线程化讨论和那类事情的算法提供了基础。稍后我们会研究在“电子邮件线程化”中使用了这些字段的一个著名算法，但是重点是每封邮件都有唯一的邮件ID，包含了对这封正在被回复的确切邮件是一个回复的情况的引用，而且可以在答复链中引用多封其他邮件，它们是正在进行的讨论进程的一部分。

注意： 除了使用一些Python模块来干这些没有技术含量的活之外，我们不会讨论电子邮件的相关细节，如多部分邮件、MIME（*http://en.wikipedia.org/wiki/MIME*）、7位内容传输编码等。如果你想仔细研究一下的话，毫不费力就能找到全面涵盖这些主题的极好的参考资料。

这些标头极为重要。即使是在这个简单的示例中，你也可以看到当你在解析一封邮件的正文时，事情如何变得混乱：Rudolph的客户端使用“>”符号引用转发的内容，而Santa的邮件客户端过去常常直接回复，没有引用任何内容，但是使用了一个人类可读的邮件标头。即使不是完全没有可能，但因为其包含的二义性，尝试从mailbox数据中解析出确切的对话流仍很困难。当你想查看这些邮件时，如果你感兴趣的是比研究原始数据存储更方便的技术的话，大多数邮件客户端都能够显示比你平常看到的更多的扩展邮件标头。示例3-2说明了示例3-1的邮件流，图3-1显示了Apple Mail的示例标头。

图3-1：大多数邮件客户端都允许通过一个选项菜单来查看扩展标头

示例3-2：示例3-1的邮件流

```
Fri, 25 Dec 2001 00:03:34 -0000 (GMT) - Buddy sends a message to the workshop
Friday, December 25, 2009 12:04 AM - Rudolph forwards Buddy's message to Santa
                                     with an additional note
Fri, 25 Dec 2001 00:06:42 -0000 (GMT) - Santa replies to Rudolph
```

幸运的是，有很多可以做的，而你不必从根本上重新实现一个邮件客户端。而且，如果

你只是想要浏览邮箱，可以简单地将它导入到邮件客户端并浏览它，对吧！虽然只是固定的套路，但也值得花一点时间来研究你的邮件客户端是否有一个“以mbox格式导入/导出数据”的选项，这样你才可以使用本章的工具来对它进行交叉分析。

Python提供了一些解析mbox数据的基本工具，示例3-3的脚本介绍了一种将mbox数据转换为一系列JSON对象的基本方法。

警告： 请注意，示例3-3的好几个地方包含了`decode('utf-8', 'ignore')`函数。当使用email或网页这些基于文本的数据时，由于特别的字符编码，很容易会遇到臭名昭著的`UnicodeDecodeError`，发生了什么或者如何解决这个问题通常并不明显。简略的回答是可以在任何字符串值中运行`decode()`函数，并且把它传递给第二个参数，该参数指定了在碰到`UnicodeDecodeError`时应该怎么办。默认值是`'strict'`，它会引发异常，但是也可以根据需要使用`'ignore'`或`'replace'`来代替它。

示例3-3：将mbox转换为更为方便的JSON结构（mailboxes_jsonify_mbox.py）

```
# -*- coding: utf-8 -*-

import sys
import mailbox
import email
import quopri
from BeautifulSoup import BeautifulSoup

try:
    import jsonlib2 as json # much faster then Python 2.6.x's stdlib
except ImportError:
    import json

MBOX = sys.argv[1]

def cleanContent(msg):

    # Decode message from "quoted printable" format

    msg = quopri.decodestring(msg)

    # Strip out HTML tags, if any are present

    soup = BeautifulSoup(msg)
    return ''.join(soup.findAll(text=True))

def jsonifyMessage(msg):
    json_msg = {'parts': []}
    for (k, v) in msg.items():
        json_msg[k] = v.decode('utf-8', 'ignore')

    # The To, CC, and Bcc fields, if present, could have multiple items
    # Note that not all of these fields are necessarily defined

    for k in ['To', 'Cc', 'Bcc']:
        if not json_msg.get(k):
```

```
            continue
        json_msg[k] = json_msg[k].replace('\n', '').replace('\t', '').replace('\r'
                , '').replace(' ', '').decode('utf-8', 'ignore').split(',')

    try:
        for part in msg.walk():
            json_part = {}
            if part.get_content_maintype() == 'multipart':
                continue
            json_part['contentType'] = part.get_content_type()
            content = part.get_payload(decode=False).decode('utf-8', 'ignore')
            json_part['content'] = cleanContent(content)

            json_msg['parts'].append(json_part)
    except Exception, e:
        sys.stderr.write('Skipping message - error encountered (%s)' % (str(e), ))
    finally:
        return json_msg

# Note: opening in binary mode is recommended
mbox = mailbox.UnixMailbox(open(MBOX, 'rb'), email.message_from_file)
json_msgs = []
while 1:
    msg = mbox.next()
    if msg is None:
        break
    json_msgs.append(jsonifyMessage(msg))

print json.dumps(json_msgs, indent=4)
```

警告： 从Python 2.6.x开始，jsonlib2（可以通过easy_install获得）这类基于C语言的第三方模块要比标准库模块快得多，但是Python 2.7附带了与jsonlib2这类模块等同的更新（*http://bugs.python.org/issue4136*）。为了便于说明，就不多介绍在相当大的JSON结构（大约为100MB）使用标准库和jsonlib2之间的区别了。

这段脚本解析出了Email中最相关的信息，并构建出了可移植的JSON对象。我们能做的还有很多，但是它解决了最常见的问题，包括解码quoted-printable（QP）文本（*http://en.wikipedia.org/wiki/Quoted-printable*）和去除HTML标签的简单机制。quopri模块用于解决QP格式，它是一种用于将8位内容转换为7位通道的编码[注2]。示例3-3的简略的示例输出如示例3-4所示。

示例3-4：来自示例3-1的示例mbox、由示例3-3产生的示例JSON输出

```
[
    {
        "From": "St. Nick <santa@northpole.example.org>",
        "Content-Transfer-Encoding": "7bit",
        "To": [
```

注2：如果你对具体细节感兴趣的话，请查看*http://en.wikipedia.org/wiki/Quoted-printable*的有关概述或RFC 2045（*http://tools.ietf.org/html/rfc2045*）。

```
            "rudolph@northpole.example.org"
        ],
        "parts": [
            {
                "content": "Sounds good. See you at the usual location.\n\nThanks,...",
                "contentType": "text/plain"
            }
        ],
        "References": "<88364590.8837464573838@mail.northpole.example.org>",
        "Mime-Version": "1.0",
        "In-Reply-To": "<194756537.0293874783209@mail.northpole.example.org>",
        "Date": "Fri, 25 Dec 2001 00:06:42 -0000 (GMT)",
        "Message-ID": "<16159836.1075855377439@mail.northpole.example.org>",
        "Content-Type": "text/plain; charset=us-ascii",
        "Subject": "RE: FWD: Tonight"
    },
    {
        "From": "Buddy <buddy.the.elf@northpole.example.org>",
        "Content-Transfer-Encoding": "7bit",
        "To": [
            "workshop@northpole.example.org"
        ],
        "parts": [
            {
                "content": "Last batch of toys was just loaded onto sleigh. \n\n...",
                "contentType": "text/plain"
            }
        ],
        "Mime-Version": "1.0",
        "Date": "Fri, 25 Dec 2001 00:03:34 -0000 (GMT)",
        "Message-ID": "<88364590.8837464573838@mail.northpole.example.org>",
        "Content-Type": "text/plain; charset=us-ascii",
        "Subject": "Tonight"
    }
]
```

既然使用了新发现的技术将邮件数据解析为可访问的格式，迫切想要开始分析它就是很自然的事情了。本章剩余的部分将使用公开的Enron邮件数据作为mbox下载。*enron.mbox.gz*文件是一个mbox，它是由来自原始Enron语料库中的、出现在“inbox”文件夹中的消息构造的，而*enron.mbox.json.gz*文件使用示例3-3的脚本将相同的数据转成JSON。虽然没有作为一个示例在这里显示，但是可以下载这段脚本，它使用*mailboxes_convert_enron_inbox_to_mbox.py*脚本（*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/mailboxes__convert_enron_inbox_to_mbox.py*）将原始Enron数据转换为mbox格式。请注意，如果你在线搜索，你可能会在*http://www.cs.cmu.edu/~enron/*上得到最接近Enron数据的数据，它是一种非标准的格式，更适合研究，包括日历信息、笔记等。 用于将数据集的一部分转换为显式标记为mbox格式的“Inbox数据”的脚本，可以从*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/mailboxes__convert_enron_inbox_to_mbox.py*上获得。本章剩余的部分假设你使用的是可以下载的mbox数据。

mbox+CouchDB=随意的Email分析

使用正确的工具可以大大简化数据分析中涉及的工作。虽然分析结构化数据最明显的方法是：多花点时间创建一个先验模式，将数据导入该模式，因为我们总是会忘记一些事，所以要修改该模式，然后多次重复这一过程，这肯定不是一件非常轻松的事[注3]。由于电子邮件消息以文档为中心的性质和JSON数据结构之间的自然映射，我们将会把我们的mbox数据导入CouchDB（*http://couchdb.apache.org/*）中，它是一个面向文档的数据库，提供了map/reduce功能，该功能可以用于在数据上创建索引，执行回答“某某人发送了几封邮件”或“某某天发送了几封邮件”这类问题的聚合频率分析。

注意： 关于CouchDB，它适合哪种整体数据存储场景，与基本分析最无关的很多其他独特功能的全面讨论都不在本书讨论的范围内。然而，即使你从来没有听说过CouchDB，你也完全可以理解本节内容。

使用CouchDB分析文档的另一个好处是它提供了一个完全基于REST的接口（*http://en.wikipedia.org/wiki/Representational_State_Transfer*），如果你允许其他人复制（并分析）你的数据库，它可以集成到任何基于Web的架构中，并提供方便使用的复制功能。

REST风格的Web服务

REST（*http://en.wikipedia.org/wiki/Representational_State_Transfer*）（Representational State Transfer，表象化状态转换）是一种架构风格，是Roy Fielding在他的博士论文中定义的（*http://www.ics.uci.edu/~taylor/documents/2002-REST-TOIT.pdf*），但是你不需要阅读这篇论文来了解它。虽然整本书都是关于REST的，但是了解几条关键原则就足够了：客户端和服务器应该是无状态通信的，URI应该可以通过标准HTTP动词（如GET、PUT、POST、HEAD和DELETE）描述承受动作的资源。例如，分层URL *http://example/com/blog*可以描述“博客”资源。该资源的GET会抓取它，而上下文/blog/foo中的PUT会为用户“foo”创建一个新博客。上下文/blog/foo中的DELETE会删除该博客，POST /blog/foo会在foo的博客中添加一篇博文。如果你想深入了解REST，并且想要看一些处理不那么明显的情况的实际范例，那么应该读一下《RESTful Web Services》（*http://oreilly.com/catalog/9780596529260*）（O'Reilly）。

本节剩余的部分假设你能够在你的系统中设置并安装CouchDB的二进制版本，或者从源代码编译并安装它。你可能想了解一下CouchOne（*http://www.couchone.com*），它是由CouchDB的创造者和主要提交者组成的公司，提供了大多数平台的二进制下载，还提供

注3：　CouchDB用户往往会滥用“轻松”这个词，而且以下的讨论会尽可能真实。

了免费的CouchDB托管选项，你会发现它很有用。Cloudant（*http://coudant.com*）可能是你会考虑的另一种在线托管选项。

刚开始时，你可能会认为CouchDB只是一个键－值存储，其中，键是任意标识符，值是你自己设计的基于JSON的文档。图3-2通过Futon说明了一个文档集合，以及集合中的单个文档，它是CouchDB的基于Web的管理界面。可以通过*http://localhost:5984/_utils/*在本地安装中来访问Futon。

图3-2：Enron数据库的聚合视图和其中的一个文档

安装好CouchDB之后，需要执行的另一个管理任务就是：通过常用的easy_install couchdb方法安装Python客户端模块couchdb。可以通过pydoc或*http://packages.python.org/CouchDB/*在线读取couchdb的更多内容。现在已经安装好了CouchDB和Python客户端，是时候放松一下，编写一些代码来提取我们的JSONified mbox数据——但是如果这是你第一次接触CouchDB，那么花一点时间来熟悉一下Futon吧！如果随性的东西并不能让你放松，你应该暂停一下，浏览一下《CouchDB: The Definitive Guide》（O'Reilly）前面几章的内容，你可以在*http://books.couchdb.org/relax*上阅读它的电子版本。

将文档批量加载到CouchDB中

在未经压缩的*enron.mbox*文件上运行示例3-3的脚本并将输出重定向到一个文件中，会产生一个相当大的JSON结构（大约为200MB）。示例3-5提供的脚本说明了如何使用couchdb-python将数据按原样加载到CouchDB中，你应该已经通过easy_install couchdb安装了该模块。（你必须承认，与要求加载关系模式的可选择方案，这段代码看起来很惬意。）请注意，如果你正在使用笔记本电脑这类典型的消费类硬件产品，那么完成该段脚本可能需要几分钟[注4]。起初看来，几分钟似乎有些慢，但是当你考虑到JSON结构包含被输出为CouchDB文档的4万多个对象，也就是每秒300次文档事务处理时，这实际上是一台普通的笔记本电脑的合理性能。性能监控工具，如*nix系统中的top或Windows系统中的任务管理器，应该会显示CPU的一个内核（*http://en.wikipedia.org/wiki/Multi-core_processor*）被挂起来了，这表示最大的吞吐量。在开发过程中，最好使用一个较小的文档集，而不是使用完整的数据集。你可以轻松地取出十几条JSONified Enron消息，应用在本章的示例中。

注意： 虽然CouchDB的底层编码语言Erlang以支持高并发性出名，但这并不意味着CouchDB或者其他使用Erlang建立的应用平台一定可以保持CPU中所有内核都能满负荷运转。例如，在CouchDB中，对数据库的写操作是指“仅追加”操作，它被序列化到磁盘上的B树中，甚至很多批量写入都不能轻易地分散到多个内核中。在这一方面，数据库和内核的写操作存在着一一映射关系。然而，可以同时在多个数据库中执行批量写操作，可以使多个内核的吞吐量达到最大值。

示例3-5：用于说明将JSON数据加载到CouchDB中的简单脚本（mailboxes_load_json_mbox.py）

```
# -*- coding: utf-8 -*-

import sys
import os
import couchdb
```

注4：为了便于比较，在MacBook Pro 2.8GHz酷睿2双核处理器中，使用采用存储设置安装的CouchDB 0.11.0执行示例3-5大约需要两分钟。

```
try:
    import jsonlib2 as json
except ImportError:
    import json

JSON_MBOX = sys.argv[1] # i.e. enron.mbox.json
DB = os.path.basename(JSON_MBOX).split('.')[0]

server = couchdb.Server('http://localhost:5984')
db = server.create(DB)
docs = json.loads(open(JSON_MBOX).read())
db.update(docs, all_or_nothing=True)
```

随着数据的加载，现在我们很想坐下来放松一下，但是使用Futon浏览数据会引起一些关于“谁、多长时间、什么时候通信”的问题。让我们开始使用CouchDB的map/reduce功能，这样我们就可以回答其中一些问题了。

注意： 如果你完全不了解map/reduce功能，不必苦恼，了解它的更多内容是本节的一部分内容。

简单来说，映射函数采用一批文档，为每个文档映射出一个新的键/键值对，而化简函数采用一个文档集合，并以某种方式化简它们。例如，计算平方的算术和：$f(x)=x_1^2+x_2^2+\cdots+x_n^2$，可以被表示为一个取所有值平方的映射函数，对每个输入值产生了一对一的映射，而化简函数只是简单地将映射函数的输出加起来，并将它归并为一个值。虽然这种编程模式非常适合于简单的并行化问题，但它并不适合于所有问题。使用CouchDB解决某个问题必须要求做出该计算模式适合于该任务的假设。

合理的排序

如果你仔细观察图3-2，你就会发现你刚刚装入CouchDB中的文档是按键排序的。默认情况下，键是特殊的_id值，它是由CouchDB自动分配的，所以现在的排列次序几乎毫无意义。为了以更合理的方式浏览Futon中的数据，并执行有效的范围查询，我们需要执行映射操作来创建按照其他值排列的视图。按日期排序似乎是一个好主意，它为某类时序分析提供了方便，所以让我们从这里开始，看看会发生什么。但是首先，需要改变一个小配置，这样就可以在Python中编写map/reduce函数来执行该任务了。

CouchDB尤其吸引人，因为它是由支持高并发性[注5]和高容错率的Erlang语言编写的。通过map/reduce函数查询并转换数据的即装即用语言实际上就是JavaScript。请注意，我们确实可以选择编写JavaScript的map/reduce函数，并从CouchDB提供的内置JavaScript

注5：一般来说，“高并发性”的特点意味着多个操作可以同一时间（并行）在一个微处理器的多个内核中运行，产生最大的CPU吞吐量。

函数中得到一些益处——如_sum、_count和_stats。但是从开发环境的语法检查/高亮中得到的好处可能更有用，而且比起盯着被包装成Python代码中的三重引号字符串值的JavaScript函数来说，这样更好看。而且，假设你已经安装了couchdb，指定Python视图服务器只需要调整一个小配置，这样你就可以在Python中编写你的CouchDB代码了。更精确地说，在CouchDB语法中，map/reduce函数称为视图函数，它们驻留在一个叫做设计文档（design document）（*http://guide.couchdb.org/draft/design.html*）的特殊文档中。

将下面的一行（或者Widnows可兼容的等价式）插入到CouchDB的*local.ini*配置文件的相应部分，其中，couchpy可执行文件是视图服务器，它会在安装couchdb模块时自动安装。在完成更改后，不要忘记重新启动CouchDB：

```
[query_servers] python=/path/to/couchpy
```

注意： 如果在确定*nix环境中的couchpy可执行文件位置时遇到问题，试着使用终端会话中的which couchpy来找到它的绝对路径。对于Windows用户，使用ActivePython提供的easy_install应该可以将*couchpy.exe*文件放置在C:\PythonXY\Scripts中。当执行easy_install couchdb命令显示该信息时，观察一下输出。

如果你不想编写大量代码来将可变的日期字符串解析成标准格式，那么你还得执行easy_install dateutil命令。它是一个非常实用的包，可以标准化大多数日期格式，因为当你遇到凌乱的Email语料库中的日期戳时，你不必知道你正在面临着什么。示例3-6是按文档的日期/时间戳映射它们的脚本，在Python中配置好CouchDB之后，应该就可以在导入的示例数据中运行它。示例输出是与该查询匹配的文件列表，为了简洁一些，这里把它省略了。

示例3-6：使用Python，按日期/时间戳映射文档的简单映射程序（mailboxes_map_json_mbox_by_date_time.py）

```
# -*- coding: utf-8 -*-

import sys
import couchdb
from couchdb.design import ViewDefinition
try:
    import jsonlib2 as json
except ImportError:
    import json

DB = sys.argv[1]
START_DATE = sys.argv[2] #YYYY-MM-DD
END_DATE = sys.argv[3] #YYYY-MM-DD

server = couchdb.Server('http://localhost:5984')
db = server[DB]
```

```
def dateTimeToDocMapper(doc):

    # Note that you need to include imports used by your mapper
    # inside the function definition

    from dateutil.parser import parse
    from datetime import datetime as dt
    if doc.get('Date'):
        # [year, month, day, hour, min, sec]
        _date = list(dt.timetuple(parse(doc['Date']))[:-3])
        yield (_date, doc)

# Specify an index to back the query. Note that the index won't be
# created until the first time the query is run

view = ViewDefinition('index', 'by_date_time', dateTimeToDocMapper,
                      language='python')
view.sync(db)

# Now query, by slicing over items sorted by date

start = [int(i) for i in START_DATE.split("-")]
end = [int(i) for i in END_DATE.split("-")]
print 'Finding docs dated from %s-%s-%s to %s-%s-%s' % tuple(start + end)

docs = []
for row in db.view('index/by_date_time', startkey=start, endkey=end):
    docs.append(db.get(row.id))
print json.dumps(docs, indent=4)
```

要了解代码，最重要的是了解dateTimeToDocMapper()函数的作用，它是一个自定义生成器[注6]。它接受文档作为参数，并生成相同的文档；然而，该文档是由方便的日期值键入的，适合于轻易地操作和排序。请注意，CouchDB中的映射函数没有副作用；不论映射函数中发生了什么或者映射函数产生了什么，作为参数传递的文档都保持不变。在CouchDB说法中，dateTimeToDocMapper()函数是名叫“by_date_time”的视图，它是“index”设计文档的一部分。在Futon的右上角，可以把组合框的值从“All documents”改为“Design documents”，自己验证一下，如图3-3所示。也可以以相同的方式查看ViewDefinition中的pydoc。

当第一次运行代码时，应该要花费大约5分钟的时间来执行map函数，这期间会挂起一个内核。几乎80%的时间都用于创建索引，而剩余的时间则用于执行查询。然而，请注意，该索引只需要建立一次，所以后续查询是相对高效的，对于指定的时间范围，大约会花20秒取出并返回约2200个文档，也就是大约每秒钟110个文档。

注6：笼统来说，生成器是返回迭代器的函数。参阅*http://docs.python.org/tutorial/classes.html*。

图3-3：查看Futon中的设计文档

从map函数得到的最重要的信息是：所有文档都是按日期索引的，在Futon中，这可以通过选择“View”组合框中的“by_date_time”索引值来自己验证一下，如图3-4所示。虽然按某些标准将文档排序并不是世界上最有趣的活动，但是它是很多聚合分析函数的必

要步骤，如计算整个语料库中满足某个标准的文档总数。下一节将说明这种分析的一些基本构建块。

图3-4：按日期/时间值排序的文档

映射/化简启发的频率分析

当遇到一个新的数据集时，制表频率（Tabulating frequency）通常是你面临的第一个颇具探索性的任务，因为它可以以很小的代价告诉你很多知识。本节研究几种可以使用CouchDB来创建基于频率的索引的方法。

按日期/时间范围变动的频率

虽然by_date_time索引可以很好地按日期/时间创建对文档排序的索引，但是对于统计由共同边界（如年、月、周等）传输的文档频率，它并不是有用的索引。虽然我们的确可以编写计算db.view('in dex/by_date_time', startkey=start, endkey=end)调用频率的客户端代码，但是让CouchDB做这个工作更有效（也更轻松）。所需要的只是对映射函数的简单修改，以及对化简函数的简单介绍。map函数会对每个日期/时间戳产生等于1的值，而reducer会统计相等键的总数。示例3-7说明了该方法。请注意，在运行示例之前，需要运行easy_install prettytable命令，prettyable包能产生漂亮的表格输出。

注意： 了解化简函数只能作用于映射到相同键上的值是非常重要的。对映射/化简的全面介绍超出本书的范围，但是网上有很多有用的资源，包括维基上的CouchDB介绍（*http://wiki.apache.org/couchdb/Introduction_to_CouchDB_views*）和一个基于Java-Script的交互工具（*http://blog.mudynamics.com/wp-content/uploads/2009/04/icouch.html*）。reducer的详细内容会在5.1节介绍。本章的所有示例都不必使用rereduce参数，但是，要提醒你的是，它是存在的，而且在一些情况下是非常重要的，会在所有的示例中将它明确地列出来。

示例3-7：使用mapper和reducer来统计按日期编写的邮件数（mailboxes_count_json_mbox_by_date_time.py）

```
# -*- coding: utf-8 -*-

import sys
import couchdb
from couchdb.design import ViewDefinition
from prettytable import PrettyTable

DB = sys.argv[1]

server = couchdb.Server('http://localhost:5984')
db = server[DB]

def dateTimeCountMapper(doc):
    from dateutil.parser import parse
    from datetime import datetime as dt
    if doc.get('Date'):
        _date = list(dt.timetuple(parse(doc['Date']))[:-3])
        yield (_date, 1)

def summingReducer(keys, values, rereduce):
    return sum(values)

view = ViewDefinition('index', 'doc_count_by_date_time', dateTimeCountMapper,
                      reduce_fun=summingReducer, language='python')
view.sync(db)

# Print out message counts by time slice such that they're
# grouped by year, month, day

fields = ['Date', 'Count']
pt = PrettyTable(fields=fields)
[pt.set_field_align(f, 'l') for f in fields]

for row in db.view('index/doc_count_by_date_time', group_level=3):
    pt.add_row(['-'.join([str(i) for i in row.key]), row.value])

pt.printt()
```

注意： 如果你需要调试Python中的mappers或reducer，你会发现在控制台中输出结果并不起作用。你可以使用的一种有效的方法是将消息追加到一个文件中。但要记住，在附加模式中以'a'选项打开文件：例如，open('debug.org', 'a')。

总结一下，我们创建了doc_count_by_date_time视图，并把它保存在“index”设计文档中；它由为每个文档产生日期键的mapper和接收任何输入并返回它们的总和的reducer组成。你可能正在寻找的缺失链接是summingReducer函数如何工作。在这个示例中，reducer简单计算了拥有匹配键的文档的所有值的累加和。一般来说，CouchDB reducer的工作方式是：传送给它们键和键值的对应列表，并且通常用一个自定义函数执行一些聚合操作（如在这个例子中，计算它们的总和）。rerecude参数是一种工具，它通过处理reducer本身化简结果，使增量式映射/化简成为可能。本章的示例需要包含化简的无特殊处理的情况，但是如果你需要了解关于化简的更多内容，可以阅读5.1节的相关内容或*http://wiki.apache.org/couchdb/Introduction_to_CouchDB_views*。

group_level参数允许我们通过与键相关的任何日期/时间粒度，执行频率分析。从概念上来说，CouchDB使用该参数分出该键的前*N*个部分，在匹配的键/键值对上自动执行化简操作。目前，这意味着计算前*N*个匹配部分的键的总和。需要注意的是，如果传递给reducer完整的日期/时间键，它被具体到第二部分，我们几乎同时查询完全在同一时刻发送的Email，这可能没用。只传递日期/时间键的前两部分将查询按年发送的Email总数，传递前两部分将查询按月发送的Email总数等。传送给db.view函数的group_level关键字参数是如何控制CouchDB把键的哪部分传递给化简函数。运行完计算基本索引的代码后，就可以在Futon中研究该功能，如图3-5所示。

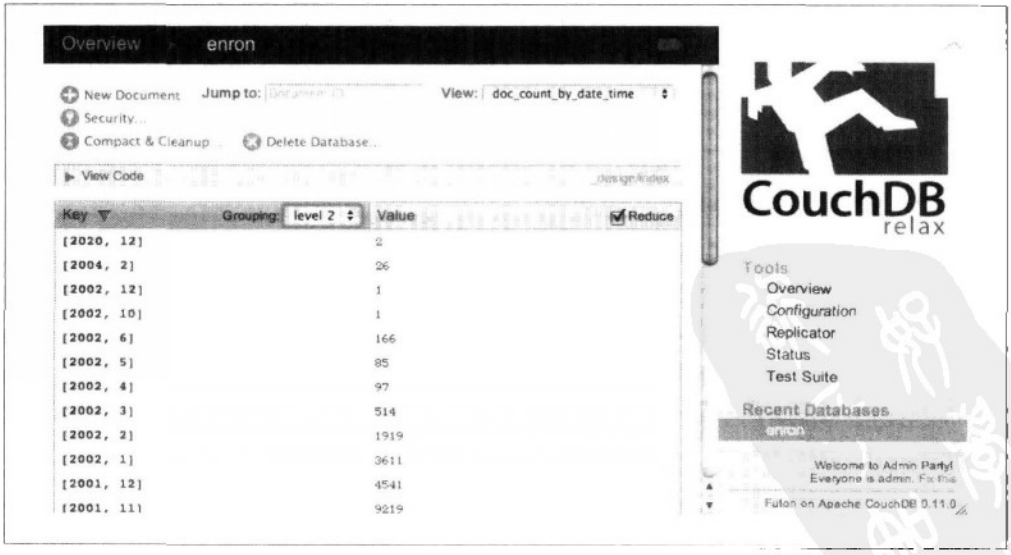

图3-5：可以使用Futon研究group_by选项

在创建返回doc_count_by_date_time视图的索引期间，执行dateTimeCountMapper()函数所花费的时间与你之前看到的映射函数一样，它通过改变group_level参数，产生运行

很多有用的查询的能力。请记住，很多备选策略（如切断从纪元开始的毫秒数）也是可用的，而且它们可能有助于执行不是以年、月、周等为边界的查询。必须生成从纪元开始的毫秒作为mapper中的键，并把这些关键字参数传送到db.view的startkey和endkey中。Reducer并不是必需的。

如果你想做与日期无关的基于时间的分析（如在一天中任意给定时间内发送的文件数）又会怎样呢？视图查询的分组行为利用了备份数据库的B树底层结构，所以不能直接通过键中的任意项，以一种类似于Python列表片段的方式来分组，如k[4:-2]或k[4:5]。对于这类查询，可能想使用一个合适的前缀来构建新键，如[hour,minute,second]。过滤客户端需要遍历整个文档集合，但无须提供为后续查询构建索引的额外收益。

B-树是最好的吗

B-树是CouchDB和多数其他数据库系统的基本数据结构，因为从长远来看，即使不断面临最坏的状况，它们也显示出对核心操作（插入、更新、删除）的对数性能。B-树必须保持平衡，并保持数据有序排列。这些特点会产生高效的查找，因为底层实现只需要最低限度的磁盘读取。由于仍然可以快速地寻道传统的基于磁盘的硬盘驱动器（数毫秒左右），这就意味着同样可以快速地访问B-树存储的大量数据。

人们对于“B-树”这个词的来源并没有统一的认识，但是普遍认同“B”代表B-树的发明者Bayer。网上有更多B-树和它们常见变形的信息。如果你决定深入研究CouchDB，那么你应该尽可能多了解它们的理论和实际实现，因为在CouchDB的设计中，它们是不可或缺的。

按发件人/收件人字段统计的频率

其他指标，如某人最初编写了多少封邮件、两个特定的组之间发生了多少次直接通信等，对于作为电子邮件分析的一部分来说，都是高度相关的统计信息。示例3-8说明了如何通过一封邮件中的To和From字段来计算两个人通信的次数。显然，包含Cc和Bcc字段也可以产生特殊的值，这取决于你提出的问题。该清单和示例3-6之间唯一的实质性区别是：映射函数可能会产生多个键/键值对。你可以最低限度地修改这个模式来计算多种聚合操作。

示例3-8：按发件人和收件人来映射并化简（mailboxes_count_json_mbox_by_sender_recipient.py）

```
# -*- coding: utf-8 -*-

import sys
import couchdb
```

```
from couchdb.design import ViewDefinition
from prettytable import PrettyTable

DB = sys.argv[1]

server = couchdb.Server('http://localhost:5984')
db = server[DB]

def senderRecipientCountMapper(doc):
    if doc.get('From') and doc.get('To'):
        for recipient in doc['To']:
            yield ([doc['From'], recipient], 1)

def summingReducer(keys, values, rereduce):
    return sum(values)

view = ViewDefinition('index', 'doc_count_by_sender_recipient',
                      senderRecipientCountMapper, reduce_fun=summingReducer,
                      language='python')
view.sync(db)

# print out a nicely formatted table
fields = ['Sender', 'Recipient', 'Count']
pt = PrettyTable(fields=fields)
[pt.set_field_align(f, 'l') for f in fields]

for row in db.view('index/doc_count_by_sender_recipient', group=True):
    pt.add_row([row.key[0], row.key[1], row.value])

pt.printt()
```

按值对文档排序

在一个数据库中CouchDB按键对文档持久排序在很多情况下是很有用的，但是也有例外的情况，比如，需要按值排序时。按频率对示例3-7的结果排序很有用，这样我们就可以看到“前*N*个”结果了。对于小型数据集来说，一个好的选择是只执行客户端排序。几乎每一种编程语言都可以可靠地执行快速排序（*http://en.wikipedia.org/wiki/Quick_sort*）算法，它的平均复杂度显示了合理的组合[注7]。

然而，对于较大的数据集或更多利基情况来说，你可能要考虑一些备选方案。一种方法是转置化简的文档（例如，示例3-5中所示的文档），将它们加载到另一个数据库中，这样就可以自动按键排序了。虽然这种方法有点绕弯子，但是它不需要付出很多精力来实现，而且，如果要导出的文档数很大的话，在把不同的数据库映射到不同的内核之间的操作中，它的确具有性能优势。示例3-9说明了这种方法。

注7：通常，最坏的情况下，要对一个集合快速排序需要进行$n*\lg(n)$次比较，在事先不知道数据的任何信息而编写算法时，你所能做的只有这些。

示例3-9：通过使用转置mapper来按键对文档排序，并把它导入到另一个数据库中（mailboxes_sort_by_value_in_another_db.py）

```
# -*- coding: utf-8 -*-

import sys
import couchdb
from couchdb.design import ViewDefinition
from prettytable import PrettyTable

DB = sys.argv[1]

server = couchdb.Server('http://localhost:5984')
db = server[DB]

# Query out the documents at a given group level of interest
# Group by year, month, day

docs = db.view('index/doc_count_by_date_time', group_level=3)

# Now, load the documents keyed by [year, month, day] into a new database

db_scratch = server.create(DB + '-num-per-day')
db_scratch.update(docs)

def transposeMapper(doc):
    yield (doc['value'], doc['key'])

view = ViewDefinition('index', 'num_per_day', transposeMapper, language='python')
view.sync(db_scratch)

fields = ['Date', 'Count']
pt = PrettyTable(fields=fields)
[pt.set_field_align(f, 'l') for f in fields]

for row in db_scratch.view('index/num_per_day'):
    if row.key > 10: # display stats where more than 10 messages were sent
        pt.add_row(['-'.join([str(i) for i in row.value]), row.key])
```

虽然“在客户端将它们排序并将它们导入到数据库”是相对简单的方法，但这些方法在相对简单的情况下可以得到良好的扩展；有一个真正具有行业优势的基于Lucene的解决方案，它可以用于各种各样的全面索引中。下一节将研究如何使用couchdb-lucene来执行基于文本的索引（Lucene的主要内容），但是按值对文档排序、索引地理位置等其他可能性也是力所能及的。

couchdb-lucene：全文索引以及更多

Lucene（*http://lucene.apache.org/java/docs/index.html*）是一个基于Java的高性能全文索引搜索引擎库；它最常见的用途是将关键字搜索功能合并到应用程序中。couchdb-lucene项目本质上是围绕Lucene最核心功能的Web服务封装，这使它能够索引CouchDB文档。couchdb-lucene的运行方式与独立的Java虚拟机（Java Virtual Machine，JVM）

进程相似，它通过HTTP与CouchDB通信，所以，如果你有可用的硬件，那么你就能在一个完全独立的机器上运行它了。

即使是未对Lucene和couchdb-lucene进行深入讲解的初级教程，多少也会让你有点摸不着头脑。但是，我们将通过一个简单的示例说明如何运行这些工具，你会发现对现在的任务来说，全文索引是必不可少的。如果你现在不需要全文索引，那么先跳过本节内容，以后再看。为了便于比较，请注意，通过编写与关键字和文档相关的映射函数，执行基于文本的索引当然是可行的，如示例3-10所示。

示例3-10：切分文档的mapper

```
def tokenizingMapper(doc):
    tokens = doc.split()
    for token in tokens:
        if isInteresting(token): # Filter out stop words, etc.
            yield token, doc
```

然而，很快你就会发现：如果你想要建立一个良好的计分函数来按相关性或其他超出基本频率分析的方法来排列文档的话，你需要多了解一些基本的信息检索（Information Retrieval，IR）概念。好在Lucene有很多优点，你可能不需要编写你自己的映射函数，使用couchdb-lucene就可以完成全文索引了。

注意： 和使用couchdb模块的前几节内容不同，这一节使用httplib来直接运行CouchDB的REST API，而且包含用JavaScript编写的视图函数。尝试用其他方法似乎会让人有点费解。

安装couchdb-lucene的二进制快照和说明都可以从*http://github.com/rnewson/couchdb-lucene*中得到。必要的详细配置细节可以从*README*文件中找到，安装完Java后，这个文件就会存在了，需要执行couchdb-lucene的运行脚本，该操作会启动一个Web服务器，并对CouchDB的*local.ini*配置文件做一些小改动，这样couchdb-lucene就可以通过HTTP与CouchDB通信了。需要的关键配置变化已经记录在couchdb-lucene的*README*文件中了。示例3-11说明了这些内容。

注意： 对获取couchdb-lucene服务封装感兴趣的Windows用户可能会被告知：截至2010年10月，这个讨论主线已经过期了：*http://www.apacheserver.net/CouchDB-lucene-windows-service-wrapper-at1031291.htm*。

示例3-11：将CouchDB配置为couchdb-lucene

```
[couchdb]
os_process_timeout=300000 ; increase the timeout from 5 seconds.

[external]
fti=/path/to/python /path/to/couchdb-lucene/tools/couchdb-external-hook.py
```

```
[httpd_db_handlers]
_fti = {couch_httpd_external, handle_external_req, <<"fti">>}
```

简而言之，我们将超时增加到了5分钟，并定义了一个称为fti（full text index，全文索引）自定义行为，每当在数据库中调用_fti内容时，都会调用couchdb-external-hook.py脚本（由couchdb-lucene函数提供）中的功能。该脚本转而与运行Lucene的JVM（Java进程）交互来提供全文索引。这些细节很有趣，但是你不必受制于这些细节，除非你坚持要弄清楚它们。

couchdb-lucene配置完成之后，试着执行示例3-12的脚本，它在Subject字段中执行默认的索引操作和数据库中每个文档的内容。请记住，关于每封邮件的主题和内容中的基于文本的索引，你完全可以使用正常的Lucene查询语法来获取你的搜索的粒度控制。然而，默认的关键字功能通常就足够了。

示例3-12：使用couchdb-lucene来获取全文索引

```
# -*- coding: utf-8 -*-

import sys
import httplib
from urllib import quote
import json

DB = sys.argv[1]
QUERY = sys.argv[2]
# The body of a JavaScript-based design document we'll create

dd = \
    {'fulltext': {'by_subject': {'index': '''function(doc) {
                                var ret=new Document();
                                ret.add(doc.Subject);
                                return ret
                            }'''},

    'by_content': {'index': '''function(doc) {
                               var ret=new Document();
                               for (var i=0; i < doc.parts.length; i++) {
                                   ret.add(doc.parts[i].content);
                               }
                               return ret
                           }'''}}}

# Create a design document that'll be identified as "_design/lucene"
# The equivalent of the following in a terminal:
# $ curl -X PUT http://localhost:5984/DB/_design/lucene -d @dd.json

try:
    conn = httplib.HTTPConnection('localhost', 5984)
    conn.request('PUT', '/%s/_design/lucene' % (DB, ), json.dumps(dd))
    response = conn.getresponse()
finally:
    conn.close()
```

```
if response.status != 201: # Created
    print 'Unable to create design document: %s %s' % (response.status,
            response.reason)
sys.exit()

# Querying the design document is nearly the same as usual except that you reference
# couchdb-lucene's _fti HTTP handler
# $ curl http://localhost:5984/DB/_fti/_design/lucene/by_subject?q=QUERY

try:
    conn.request('GET', '/%s/_fti/_design/lucene/by_subject?q=%s' % (DB,
                quote(QUERY)))
    response = conn.getresponse()
    if response.status == 200:
        response_body = json.loads(response.read())
        print json.dumps(response_body, indent=4)
    else:
        print 'An error occurred fetching the response: %s %s' \
            % (response.status, response.reason)
finally:
    conn.close()
```

你一定想查看全部的在线couchdb-lucene文档（*http://github.com/rnewson/couchdb-lucene*），但重点是你创建了一个特殊格式的设计文档，它包含了一个保存索引名称和基于JavaScript的索引函数的fulltext字段。索引函数返回一个Document对象，后者包含Lucene应该索引的字段。请注意，当建立索引时，couchdb-lucene（不是CouchDB）会定义该Document对象。对于臭名昭著的“rapor”[注8]，示例查询结果会返回示例3-13所示的响应。请注意，可以使用id值，直接从CouchDB中查找进一步分析的文档。

示例3-13：对Enron数据集中“raptor”的示例查询结果

```
/* Sample query results for
http://localhost:5984/enron/_fti/_design/lucene/by_content?q=raptor */

{ "etag" : "11b7c665b2d78be0",
  "fetch_duration" : 2,
  "limit" : 25,
  "q" : "default:raptor",
  "rows" : [ { "id" : "3b2c340c28782c8986737c35a355d0eb",
       "score" : 1.4469228982925415
     },
     { "id" : "3b2c340c28782c8986737c35a3542677",
       "score" : 1.3901585340499878
     },
     { "id" : "3b2c340c28782c8986737c35a357c6ae",

       "score" : 1.375900149345398

     },
```

注8：在Enron的上下文中，Raptors是用来隐瞒亏损的金融设备。Enron使用它们，在账本中隐瞒了数百亿负债。

```
    /* ... output truncated ... */

    { "id" : "2f84530cb39668ab3cdab83302e56d65",
      "score" : 0.8107569217681885

    }
  ],
"search_duration" : 0,
"skip" : 0,
"total_rows" : 72
}
```

响应的全部细节都可以从文档中得到，但是最有趣的部分是包含文档ID值的行集，可以使用它们访问CouchDB中的文档。从数据流方面来说，应该注意的是：本质上并不是直接与CouchDB交互；它只是定义了一个特制的设计文件，并给CouchDB发出了一个查询，它了解设计文档和查询中的基于`fulltext`和`_fti`的存在分别应该怎么做。对于Lucene中计算文档得分的具体方法的讨论超出了本书的范围，但是可以在它的评分文档（scoring documentation）（*http://lucene.apache.org/java/3_0_0/scoring.html*），更具体地说，是在Similarity（*http://lucene.apache.org/java/3_0_1/scoring.html*）类中阅读它的更多细节。不修改Lucene的源代码，而自定义Lucene的得分属性（如果你需要这么做的话）是不太可能的，但是你应该可以通过`couchdb-lucene`中的各种API参数，通过“boost”（*http://lucene.apache.org/java/2_4_0/queryparsersyntax.html#Boosting%20a%20Term*）这类参数影响得分，而不是处理源代码。

如果你非常熟悉Enron事件中的术语“raptor”，那么你可能在搜索结果中找到排名最靠前的邮件中有用的前几行：

> The quarterly valuations for the assets hedged in the Raptor structure were valued through the normal quarterly revaluation process. The business units, RAC and Arthur Andersen all signed off on the initial valuations for the assets hedged in Raptor. All the investments in Raptor were on the MPR and were monitored by the business units, and we prepared the Raptor position report based upon this information... .

如果你不知道从哪里开始查找，可以使用简单的关键字。刚才引用的邮件的主题是“RE: Raptor Debris”。知道还有谁在讨论Raptor的主题不是很有趣吗？巧的是，这正是下一节的主题。

将对话线程化到一起

作为对线程化对话的第一次尝试，你可能会以试探邮件的主题标头（Subject Header）中的一些基本字符串开始，并且最终从检查收件人、发件人和试图拼接起来的日期戳切入正题。幸运的是，邮件服务器可能比你想象的更先进，从3.1节中，你知道可以用

Message-ID、In-Reply-To和References标头从一个mailbox的邮件中提取对话。邮件线程算法通常被称为“jwz线程”[注9]，它把所有的这些都考虑进来了，并提供了一种合理的方法来解析邮件线程。该算法的所有细节都可以从*http://www.jwz.org/doc/threading.html*中在线得到。我们将使用的实现是Mail Trends（*http://code.google.com/p/mail-trends/*）项目中一个算法的非常简单的更改[注10]，它提供了一些其他有用的即装即用工具。由于从2010年开始，Google代码中已经没有任何与该项目有关的内容了，因此不能确定Mail Trends是否仍在积极维护，但是该项目为邮件分析提供了一个有用的起点，对jwz线程的回收可以证明这一点。

让我们来看一看示例3-14中的完整流程，然后我们会深入研究更多的细节。

示例3-14：通过“jwz线程”，从mbox数据中创建会话线程（mailboxes_threading.py）

```
# -*- coding: utf-8 -*-

import sys
import couchdb
from mailboxes__jwzthreading import thread, Message
from mailboxes__CouchDBBulkReader import CouchDBBulkReader
from datetime import datetime as dt
from prettytable import PrettyTable

try:
    import jsonlib2 as json
except:
    import json

DB = sys.argv[1]
NUM_PROC_THREADS = 3 # Recommendation: ~1 thread/core

# Pull the data as efficient as possible from CouchDB by using a thread
# pool to get as close as possible to being I/O bound.
# A single request to _all_docs works except that it CPU bound to a single core

now = dt.now()
print >> sys.stderr, 'Bulk reading from CouchDB...'
br = CouchDBBulkReader(DB, NUM_PROC_THREADS)
docs = br.read()
print >> sys.stderr, '\t%s' % (dt.now() - now, )

now = dt.now()
print >> sys.stderr, 'Threading in Memory...'
threads = thread([Message(doc) for doc in docs])
print >> sys.stderr, '\t%s' % (dt.now() - now, )

# Write out threading info into another database.
# Note that writes to CouchDB are serialized to append-only
```

注9：“jwz”是对其作者Jamie Zawinski的引用。

注10：换句话说，可以修改该代码，使它更加面向对象，改变输入/输出格式来使用我们的JSONified消息对象，通过只考虑线程算法需要的几个字段，会显著地降低内存配置。

```
# databases, so threading is unlikely to help here, and besides,
# the average document size is very small, making this a quick operation

now = dt.now()
print >> sys.stderr, 'Bulk writing to CouchDB...'
server = couchdb.Server('http://localhost:5984')
db = server.create(DB + '-threads')
results = db.update([{'thread': thread} for thread in threads],
                    all_or_nothing=True)
print >> sys.stderr, '\t%s' % (dt.now() - now, )

# Some basic stats

print >> sys.stderr, 'Total number of threads: %s' % len(threads)
print >> sys.stderr

# Compute (_id, len(thread)) tuples
# You could also compute thread length directly in CouchDB using a simple reducer
# function

stats = sorted(zip([result[1] for result in results], [len(t) for t in threads]),
               key=lambda x: x[1])

fields = ['Thread Id', 'Thread Length']
pt = PrettyTable(fields=fields)
[pt.set_field_align(f, 'l') for f in fields]

for stat in stats:
    pt.add_row(stat)

pt.printt()
```

整体流程是：从CouchDB中批量读取邮件，在内存中执行线程化，然后将线程信息写入一个独立的数据库中，其中每个线程都是一个文档，它包含返回到原始邮件内容的引用。最后，输出一些关于讨论线程长度的基本统计信息。你亦可使用映射/化简方法来计算线程长度的统计值（查看3.2.3节）。对话线程是一个像示例3-15那样的文档。

示例3-15：示例线程化结果

```
{
    "_id": "b6d4f96224bc546acd34c405e6fff62f",
    "_rev": "1-1bf63dcdd94067ad647afe2ea3ade63c",
    "thread": [
        {
            "external_id": "24a30d62545728e26eb3311d63ae6e02",
            "subject": "FW: Sitara EOL Bridge Problem Today"
        },
        {
            "external_id": "bb808c9081912f5861295bf1d105dd02",
            "subject": "FW: Sitara EOL Bridge Problem Today"
        },
        {
            "external_id": "3b2c340c28782c8986737c35a332cd88",
            "subject": "FW: Sitara EOL Bridge Problem Today"
        }
```

```
]
}
```

该清单最有趣的部分是对mailboxes_jwzthreading和CouchDBBulkReader的引用。mailboxes_jwzthreading中的API将从CouchDB中抓取的每封邮件文档转化为相对不同的形式，然后将它们传送给线程算法，该算法会处理核心工作，并且返回方便的JSON格式，可以将该JSON格式提取回CouchDB中。可以从*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/mailboxes__jwzthreading.py*中获取mailboxes_jwzthreading的细节。

由于人们更赞成从couchdb模块中批量读取内容，因此经常会省略CouchDBBulkReader的使用，但是CouchDBBulkReader通过使用内部线程池同时对CouchDB调度多个请求，显著地提升了性能。根本的问题是：对于服务器的单个读或单个写请求来说，CouchDB只使用单内核。最初这可能会让你吃惊，因为 Erlang（底层实现语言）是支持高并发的，但是项目的维护人员[注11]有充分的理由做出这个设计决策。好在可以同时对服务器发送多个请求，并轻松地启动多个内核。这是CouchDBBulkReader采用的方法，示例3-16是对该方法的介绍。简单来说，先作出只抓取数据库中每个文档ID值的初始请求（比较快的操作，因为最终整理的数据量很小），然后把这些值排序并分成相等的块。每一块都被分配给一个抓取特定范围内的整个文档的线程。

按照经验，不要对线程池的每个内核都使用多个处理线程，在*nix系统中使用top这类的性能监控工具，或者在Windows中使用任务管理器来跟踪到底使用了系统中的多少资源。理想情况下，如果你使用双核机器，你会发现与CouchDB相关的守护进程beam.smp 200%被挂起，让你有效地进行“CPU限制”，但是高于150%的比例仍然是一个显著的进步。你应该可以观察到，批量读取几乎消耗了检索数据需要的全部时间，以至于几乎没有时间执行实际的线程，并将线程文档写回到CouchDB中。一个有趣的练习是考虑将线程算法导入couchdb模块使用的映射/化简示例中，或者甚至用JavaScript重写它。

注意： 示例3-16使用的threadpool包可以通过常用的方法得到：easy_install threadpool。

示例3-16：使用线程池来最大化CouchDB的读取吞吐量（mailboxes_CouchDB-BulkReader.py）

```
# -*- coding: utf-8 -*-

from datetime import datetime as dt
from math import ceil
```

注11： CouchDB基本上不可能一直支持利用多内核来执行批量写操作，因为写入方式被序列化到磁盘中了。然而，不难想象对某种读操作使用多核的补丁，由于底层数据结构是树，而它本质上就适合遍历多个节点。

```
import httplib
import urllib
import time
import threadpool
try:
    import jsonlib2 as json
except:
    import json

class CouchDBBulkReader:

    def __init__(
        self,
        db,
        num_threads,
        host='localhost',
        port=5984,
        ):

        self.db = db
        self.num_threads = num_threads
        self.host = host
        self.port = port
        self.results = []

        id_buckets = self._getDocIds()
        self.pool = threadpool.ThreadPool(self.num_threads)
        requests = threadpool.makeRequests(self._getDocs, id_buckets,
                self._callback, self._errCallback)
        [self.pool.putRequest(req) for req in requests]
        self.pool.wait()

    def read(self):
        while True:
            try:
                time.sleep(0.5)
                self.pool.poll()
            except threadpool.NoResultsPending:
                return self.results
            except KeyboardInterrupt:
                print 'Keyboard interrupt. Exiting'
                sys.exit()
            finally:
                self.pool.joinAllDismissedWorkers()

    # Called to quickly get all of the document ids which can be sorted and dibbied out
    def _getDocIds(self):

        # Helper function to tersely compute a list of indices that evenly distribute
        # the items in it

        def partition(alist, indices):
            return [alist[i:j] for (i, j) in zip([0] + indices, indices
                    + [None])][:-1]


        try:
            conn = httplib.HTTPConnection(self.host, self.port)
```

```
            conn.request('GET', '/%s/_all_docs' % (self.db, ))
            response = conn.getresponse()
            if response.status != 200: # OK
                print 'Unable to get docs: %s %s' % (response.status,
                        response.reason)
                sys.exit()

        ids = [i['id'] for i in json.loads(response.read())['rows']
            if not i['id'].startswith('_')]
        ids.sort()
    finally:
        conn.close()

    partition_size = int(ceil(1.0 * len(ids) / self.num_threads))
    indices = []

    _len = len(ids)
    idx = 0
    while idx < _len:
        idx += partition_size
        indices.append(idx)

    return partition(ids, indices)

def _getDocs(self, ids):
    try:
        (startkey, endkey) = (ids[0], ids[-1])
        conn = httplib.HTTPConnection(self.host, self.port)
        conn.request('GET',
                     '/%s/_all_docs?startkey="%s"&endkey="%s"&include_docs=true'
                     % (self.db, startkey, endkey))
        response = conn.getresponse()
        if response.status != 200: # OK
            print 'Unable to get docs: %s %s' % (response.status,
                        response.reason)
                sys.exit()
            return response.read()
        finally:
            conn.close()

    def _errCallback(self, request, result):
        print 'An Error occurred:', request, result
        sys.exit()

    def _callback(self, request, result):
        rows = json.loads(result)['rows']
        self.results.extend([row['doc'] for row in rows])
```

已经有计算会话线程的工具了，现在让我们回过头来看看Enron数据。

看谁在说话

运行示例3-14的代码会创建一个数据库，它提供了把邮件ID值分组为会话线程的快速访问。因为数据库已经包含了原始的邮件，Lucene支持的另一个数据库包含了关键字搜

索功能，现在你可以真正开始做一些有趣的事情了。现在让我们通过考虑计算与提到Raptor的对话线程相关的邮件地址集合的任务，重新访问上一节的Raptor，或换一种说法，对于参与讨论的不同集合的人使用这个术语。生成的数据结构与示例3-17相似。

示例3-17：将对话线程化到一起产生的理想结果

```
{
    "participants : [ person-1@example.com,
                      person-2@example.com,
                      ...
                    ],
    "message_ids" : [ "id1",
                      "id2",
                      ...
                    ],
    "subject"     : "subject"
    }
```

我们将采取的方法包括以下三个步骤：

- 查询couchdb-lucene，获取与Raptor这类感兴趣的术语相关的邮件ID。
- 查看与任何邮件ID相关的对话线程。
- 计算邮件地址的唯一集合，它们出现在与任何线程中的邮件相关的标头字段中。

示例3-18重复使用了一些以前的代码，并说明了一种可能将它们联系在一起的实现。

示例3-18：一种将mbox数据中的会话线程化到一起的健壮方法（mailboxes_participants_in_conversations.py）

```
# -*- coding: utf-8 -*-

import sys
import httplib
from urllib import quote
from urllib import urlencode
import json

DB = sys.argv[1] # enron
QUERY = sys.argv[2]

# Query couchdb-lucene by_subject and by_content

message_ids_of_interest = []
for idx in ['by_subject', 'by_content']:

    try:
        conn = httplib.HTTPConnection('localhost', 5984)
        conn.request('GET', '/%s/_fti/_design/lucene/%s?q=%s&limit=50000' % (DB,
                     idx, quote(QUERY)))
        response = conn.getresponse()
        if response.status == 200:
            response_body = json.loads(response.read())
```

```
            message_ids_of_interest.extend([row['id'] for row in
                    response_body['rows']])
        else:
            print 'An error occurred fetching the response: %s %s' \
                % (response.status, response.reason)
            sys.exit()
    finally:
        conn.close()

# Remove any duplicates

message_ids_of_interest = list(set(message_ids_of_interest))

# Perform discussion thread filtering in memory. It's a relatively
# small amount of data

try:
    conn = httplib.HTTPConnection('localhost', 5984)
    conn.request('GET', '/%s/_all_docs?include_docs=true' % (DB + '-threads', ))
    response = conn.getresponse()
    if response.status != 200: # OK
        print 'Unable to get docs: %s %s' % (response.status, response.reason)
        sys.exit()

    threads = [dict([('thread_id', row['doc']['_id']), ('message_ids',
               [t['external_id'] for t in row['doc']['thread']])]) for row in
               json.loads(response.read())['rows']]
finally:
    conn.close()

# Find only the threads that have a message_id appearing in the list of message ids
# fetched from the Lucene index

threads_of_interest = [t for t in threads for message_id in t['message_ids']
                       if message_id in message_ids_of_interest]

# Remove duplicates

seen = []
idx = 0
while idx < len(threads_of_interest):
    if threads_of_interest[idx]['thread_id'] in seen:
        threads_of_interest.pop(idx)
    else:
        seen.append(threads_of_interest[idx]['thread_id'])
        idx += 1

# Cull out message ids for threads of interest

message_ids_for_threads_of_interest = [t['message_ids'] for t in
                                       threads_of_interest]

# Flatten out the list of lists into just a list and remove duplicates

message_ids_for_threads_of_interest = list(set([message_id for message_ids in
    message_ids_for_threads_of_interest for message_id in message_ids]))
```

```
# Query CouchDB for the email addresses in various headers of interest using a bulk
# request

try:
    conn = httplib.HTTPConnection('localhost', 5984)
    post_params = json.dumps({'keys': message_ids_for_threads_of_interest})
    conn.request('POST', '/%s/_all_docs?include_docs=true' % (DB, ), post_params)
    response = conn.getresponse()
    if response.status != 200: # OK
        print 'Unable to get docs: %s %s' % (response.status, response.reason)
        sys.exit()

    full_docs = [row['doc'] for row in json.loads(response.read())['rows']]
finally:
    conn.close()

# Finally, with full messages of interest on hand, parse out headers of interest and
# and compute unique sets of email addresses for each thread by decorating
# threads_of_interest

for thread in threads_of_interest:
    participants = []
    for message_id in thread['message_ids']:
        doc = [d for d in full_docs if d['_id'] == message_id][0]

        try:
            participants.append(doc.get('From'))
            participants.extend(doc.get('To'))
        if doc.get('Cc'):
            participants.extend(doc.get('Cc'))
        if doc.get('Bcc'):
            participants.extend(doc.get('Bcc'))
    except:
        pass # Maybe a X-To header, etc. as opposed to To?

    thread['participants'] = list(set(participants))
    thread['subject'] = doc['Subject']
print json.dumps(threads_of_interest, indent=4)
```

该脚本的示例输出如示例3-19所示。

示例3-19：会话线程中“Raptor”搜索查询的示例结果

```
[
    {
        "thread_id": "b6d4f96224bc546acd34c405e6c471c5",
        "participants": [
            "j.kaminski@enron.com",
            "rakesh.bharati@enron.com"
        ],
        "message_ids": [
            "24a30d62545728e26eb3311d63effb47"
        ],
        "subject": "FW: Note on Valuation"
    },
    {
```

```
        "thread_id": "b6d4f96224bc546acd34c405e6dbc0d4",
        "participants": [
            "mary.fischer@enron.com",
            "danny.wilson@enron.com",
            "a..lee@enron.com",
            "john.swafford@enron.com",
            "facundo.caminos@enron.com"
        ],
        "message_ids": [
            "24a30d62545728e26eb3311d633cf6b3"
        ],
        "subject": "Tax Accruals on the Raptor Companies"
    },
    {
        "thread_id": "b6d4f96224bc546acd34c405e6eb7adf",
        "participants": [
            "mark.ruane@enron.com",
            "rick.buy@enron.com"
        ],
        "message_ids": [
            "3b2c340c28782c8986737c35a357c6ae"
        ],
        "subject": "FW: E-11 Raptor"
    },

    ... output truncated ...

    ]
```

简而言之，该脚本提供了一些工具，可以基于关键字试探法来确定谁参与了哪些对话。可以将mbox文件加载到一个合适的邮件客户端，并手动搜索该信息，但是示例中所演示的方法一般情况下都是有用的，适合很多自动或半自动化的分析。

使用SIMILE Timeline将邮件“事件”可视化

可视化邮件数据的方法有很多。可以按时间操作邮件，将数据显示为直方图来检查一天中大多数邮件往来发生的时间，在发件人和收件人之间创建关系图，按会话线程过滤，将查询结果加载到时间轴中，或者使用其他技术。这一节说明了SIMILE Timeline（*http://simile-widgets.org/wiki/Timeline*）的即装即用用法，它是一种易于使用（但是非常强大）的工具，可以可视化以事件为中心的数据。SIMILE Timeline尤其适用于研究邮件数据，因为它允许我们将每封邮件的传输作为单独的事件，同时将较大的会话线程可视化为扩展事件，它会在相当长的一段时间内发生。我们也可以轻易地为每封邮件指定链接，这样当我们点击Timeline中的邮件时，它就会提供Futon中的邮件全文。

我们会坚持实用的分析方法，避免建立成熟的Web应用程序来可视化邮件数据，但是只需要付出很少的努力，就可以构建出更健壮的东西。我们会选择简单地修改示例3-18的输出格式，这样它就能生成与Timeline兼容的JSON了。从现在开始，所需要的只是将简

单的网页指向本地文件系统中的SIMILE事件源JSON输出（*http://simile-widgets.org/wiki/Timeline_EventSources*）来加载Timeline。示例目标输出如示例3-20所示。

示例3-20：SIMILE Timeline要求的数据格式（mailboxes_participants_in_conversations_adapted_for_simile.py）

```
{
    "dateTimeFormat": "iso8601",
    "events": [
        {
            "start": "2002-02-06T08:20:49-08:00",
            "description": "Message involving sarah.palmer@enron.com",
            "link": "http://localhost:5984/_utils/document.html?enron/bb...",
            "durationEvent": false,
            "title": "Enron Mentions -- 02/06/02"
        },
        {
            "start": "2001-05-22T16:20:25-07:00",
            "description": "Message involving j.kaminski@enron.com, ...",
            "link": "http://localhost:5984/_utils/document.html?enron/24a...",
            "durationEvent": false,
            "title": "RE: Pricing of restriction on Enron stock"
        },
        ...
    ]
}
```

示例3-21说明了示例3-18的基本增强，对于产生可以被SIMILE Timeline使用的输出，它是必要的（如图3-6所示）。除了每个会话线程的事件之外，它对每封邮件都创建了一个事件。

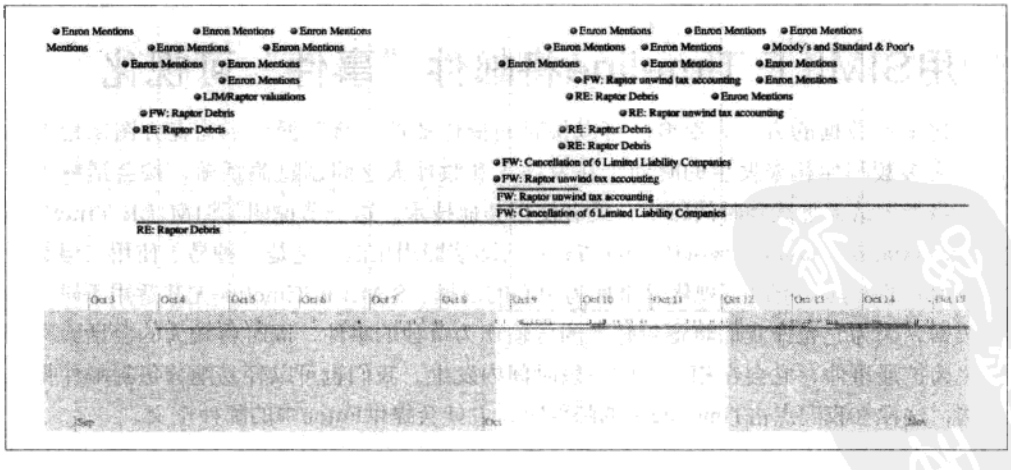

图3-6：由SIMILE Timeline可视化的“Raptor”查询的示例结果：可以在两个方向“无限”滚动

示例3-21：示例3-18的增强输出，生成可以被SIMILE Timeline使用的输出

```
# Finally, with full messages of interest on hand, parse out headers of interest
# and compute output for SIMILE Timeline

events = []
for thread in threads_of_interest:

    # Process each thread: create an event object for the thread as well as
    # for individual messages involved in the thread

    participants = []
    message_dates = []
    for message_id in thread['message_ids']:
        doc = [d for d in full_docs if d['_id'] == message_id][0]
        message_dates.append(parse(doc['Date']).isoformat())
        try:
            participants.append(doc.get('From'))
            participants.extend(doc.get('To'))
            if doc.get('Cc'):
                participants.extend(doc.get('Cc'))
            if doc.get('Bcc'):
                participants.extend(doc.get('Bcc'))
        except:
            pass # Maybe a X-To or X-Origin header, etc. as opposed to To?

        # Append each individual message in the thread

        event = {}
        event['title'] = doc['Subject']
        event['start'] = parse(doc['Date']).isoformat()
        event['durationEvent'] = False
        event['description'] = 'Message involving ' \
            + ', '.join(list(set(participants)))
        event['link'] = 'http://localhost:5984/_utils/document.html?%s/%s' % (DB,
                doc['_id'])
        events.append(event)

    # Find start and end dates for the messages involved in the thread

    if len(thread['message_ids']) > 1:
        event = {}
        event['title'] = doc['Subject']
        message_dates.sort()
        event['start'] = parse(message_dates[0]).isoformat()
        event['end'] = parse(message_dates[-1]).isoformat()
        event['durationEvent'] = True
        event['description'] = str(len(thread['message_ids'])) \
            + ' messages in thread'
        events.append(event) # append the thread event

if not os.path.isdir('out'):
    os.mkdir('out')

f = open(os.path.join('out', 'simile_data.json'), 'w')
f.write(json.dumps({'dateTimeFormat': 'iso8601', 'events': events}, indent=4))
f.close()
```

```
print >> sys.stderr, 'Data file written to: %s' % f.name

# Point SIMILE to the data file
```

关于Timeline有很多在线说明（*http://simile-widgets.org/timeline/*）和大量文档。在Timeline上绘制邮件的示例仅仅是入门的最低限度；这只是可能的开始。《Getting Started with Timeline》教程（*http://www.simile-widgets.org/wiki/Getting_Started_with_Timeline*）是好的开始方法。假设你的数据支持它要求的查询，`mailboxes_participants_in_conversations_adapted_for_simile.py`脚本就是总控键（turn-key），因为它解析数据、将它转储到HTML模板中，自动在Web浏览器中打开它。

分析你自己的邮件数据

Enron邮件数据在邮件分析上是很好的例证，但是你也一定想仔细研究一下你自己的邮件数据。幸运的是，很多流行的邮件客户端都提供了“导出到mbox”选项，这使你非常容易将你的邮件数据转换为一种适合于通过本章描述的工具来分析的格式。例如，可以选择Apple Mail中的一些邮件，从File（文件）菜单中选择Save AS（另存为），然后选择Raw Message Source（原始信息源）作为格式选择，来将邮件导出为mbox文件（参见图3-7）。在网上搜索一下，应该就能找到大多数主流客户端“导出为mbox”的操作方式了。

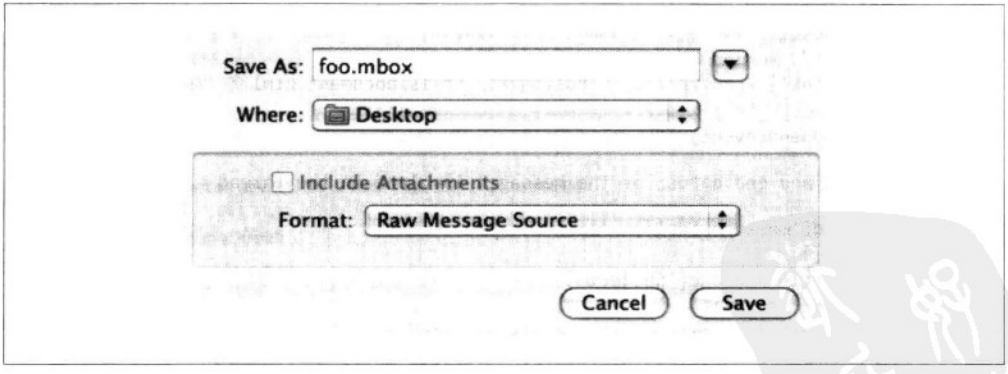

图3-7：大多数邮件客户端都提供了将你的邮件数据导出为mbox存档文件的选项

如果你只使用在线邮件客户端，你可以选择将你的数据传输到一个邮件客户端，并导出它，但是你可能更愿意直接从服务器中导出数据，完全自动化mbox文件的创建过程。几乎所有的在线邮件服务都支持POP3协议（Post Office Protocol version 3，邮局协议版本3），大多数服务也支持IMAP协议（Internet Message Access Protocol，Internet邮件访问协议），导出邮件的Python脚本编写起来并不困难。可以用于从任何地方导出邮件

数据的一个非常强大的命令行工具是用Python编写的getmail（*http://pyropus.ca/software/getmail/*）。Python标准库中包含两个模块：poplib（*http://docs.python.org/library/poplib.html*）和imaplib（*http://docs.python.org/library/imaplib.html*），它们提供了很好的基础，所以如果你在线搜索，也可能会发现很多有用的脚本。很容易启动和运行getmail。例如，为了导出你的Gmail收件箱数据，只需要下载并安装它，然后使用一些基本选项来建立*getmailrc*文件。示例3-22展示了对*nix环境的一些设置。Windows用户需要改变[destination]路径和有效路径的[options]message_log值。

示例3-22：对*nix环境的示例getmail设置

```
[retriever]
type = SimpleIMAPSSLRetriever
server = imap.gmail.com
username = ptwobrussell
password = blarty-blar-blar

[destination]
type = Mboxrd
path = /tmp/gmail.mbox

[options]
verbose = 2
message_log = ~/.getmail/gmail.log
```

修改好配置后，接下来只需要从终端调用getmail就可以了。一旦有了本地mbox，就可以使用本章学过的技术来分析它。

```
$ getmail
getmail version 4.20.0
Copyright (C) 1998-2009 Charles Cazabon. Licensed under the GNU GPL version 2.
SimpleIMAPSSLRetriever:ptwobrussell@imap.gmail.com:993:
  msg     1/10972 (4227 bytes) from ... delivered to Mboxrd /tmp/gmail.mbox
  msg     2/10972 (3219 bytes) from ... delivered to Mboxrd /tmp/gmail.mbox
  ...
```

7.6节将研究使用imaplib来抓取你的Gmail数据并分析它，作为第7章示例的一部分，它主要关注Google技术。

Graph Your（Gmail） Inbox Chrome扩展

分析webmail有几个有用的工具包，最近出现的一个最有前途的工具包是Graph Your Inbox Chrome扩展（*http://graphyourinbox.com*）。为了使用这个扩展，只需要安装来授权它访问你的邮件数据，运行一些Gmail查询，其余部分就可以自动处理了。可以搜索“pizza”这样的关键字和“2010”这样的时间值，或者运行更高级的查询，如“from:matthew@example.org”和“label:Strata”。这个扩展只可能越来越好，因为它是新出现的并且一直都很受欢迎。图3-8显示了一个示例截图。

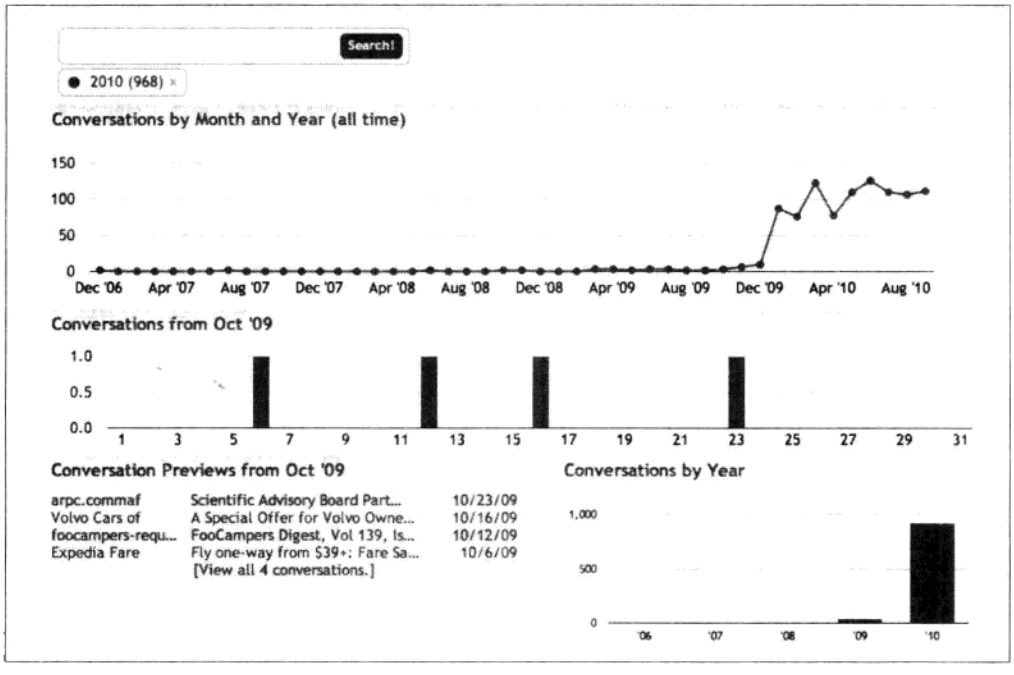

图3-8：Graph Your Inbox Chrome扩展提供了Gmail活动的简明摘要

7.6节提供了一个如何使用Python的smtplib模块来访问你的账号（或者其他使用SMTP的邮件账号）并挖掘邮件中文本信息的概述。若你对邮件标头信息感兴趣并且准备深入挖掘文本，一定要阅读一下这一节的内容。

小结

本章讨论了很多基础知识，但是我们只接触了邮件处理的皮毛。我们的焦点一直在mbox上，它是一种简单、方便的文件格式，有高度的可移植性，可以使用很多Python工具、包进行简单分析。有很多挖掘mbox的开源技术，Python是交叉分析它们的重要语言。在这些工具和CouchDB这样的存储和工具上投入一点点，你就可以轻松地共享你的数据，并得到长足的发展。

第4章

Twitter：朋友、关注者和Setwise操作

除非过去的几年里你与世隔绝，否则你一定听说过Twitter——一种微博服务，可以用来发布简短的（不超过140个字符）的状态更新。不论你是喜欢它、讨厌它，或者对它没有感觉，都不能否认Twitter已经改变了人们在Web上的沟通方式。本章试图介绍一些基本的分析函数，可以通过Twitter API实现这些函数来回答一些有趣的问题，例如：

- 我有多少好友/关注者？
- 谁是我关注了，但没有关注我的人？
- 谁关注了我，我却没有关注他？
- 我的网络中最友好和最不友好的人是谁？
- 谁是我的“共同好友”（我关注了他，他也关注了我）？
- 如果在我所有的关注者和他们所有的关注者中，有人转发了我的tweet，那么我的影响力是什么？

警告： Twitter的API在不断发展。强烈建议你关注Twitter API账号：*@TwitterAPI*，并检查本书和你看到的不符合官方文档（*http://dev.twitter.com/doc*）的实际行为之间的差别。

本章分析了Twitter用户之间的关系，而下一章将进一步研究tweet的实际内容。为本章开发的代码相对健壮，因为它考虑了常见的问题，如臭名昭著的Twitter速率限制（*http://dev.twitter.com/pages/rate-limiting*）[注1]、网络I/O错误、可能管理大量数据等。

注1：编写这部分内容时，Twitter将OAuth请求限制在350个/小时，将匿名请求限制在150个/小时。Twitter相信这些限制应该可以满足所有客户端应用，如果它们不能满足，那么很可能是你开发的应用程序有误。

最终结果是一个相当强大的命令行工具，我们可以轻易地改变它，以便让它满足自定义使用（*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/TwitterSocialGraphUtility.py*）。

注意： 拥有获取并挖掘你自己的Twitter数据的工具是十分必要的。然而，请注意，将Twitter历史数据存档到美国国会图书馆的提议（*http://www.loc.gov/tweet/how-tweet-it-is/html*）可能很快就会引起与获取相关的不便、烦恼，以及各种分析形式的API速率限制问题。Infochimps（*http://infochimps.org*）这类公司也正在兴起，它们提供了一种（从其他内容中）获取各种Twitter数据的媒介。在Infochimps（*http://infochimps.org/search?query=twitter*）上查询Twitter数据会出现各种结果，从有关世界杯的Twitter消息的归档到对smileys使用方式的分析等都有涉猎。

REST风格的和OAuth-Cladded API

2010年将作为Twitter的过渡期而被人记住， Twitter在这一年已经完全成熟了。基本HTTP认证代替了OAuth[注2]，文档改善了，API状态变得透明，不一而足。随着收购Summize（*http://blog.twitter.com/2008/07/finding-perfect-match.html*）而引入的Twitter搜索API逐渐取代了“传统的”REST（*http://en.wikipedia.org/wiki/Representational_State_Transfer*）API，由于生产现状的原因，人们开始越来越多地使用信息流API。如果Twitter在酒单上，你可能会拿起2010年份的瓶子说：“不错的年份——这个年份的酒相当好。”网络中隐藏着很多有用的信息，本章瞄准的是重现那些绝对必要的信息，其他信息则不会涉及。

本章的大部分开发都围绕着获取用户的好友和关注者的社交图谱API、获取用户列表中更多用户信息（姓名、地理位置、最后一条tweet等）的API和获取Twitter消息数据的API。整本书都是（一字一句地）关于探讨更多的可能性的，但是一旦你摸清了门道，你的想象力将会自动开始探索之旅。另外，总要给“有兴趣的读者”留一些练习，对不对？

用于Twitter的Python客户端直接命名为`twitter`（我们已经在第1章用过它了）。它提供了Twitter的REST风格的Web服务的最小封装。几乎没有介绍该模块的文档，因为你只是以相同的方式构建了请求，将URL与Twitter的在线文档拼凑起来。例如，来自终端的检索Tim O'Reilly用户信息的请求只包含对`/users/show`资源的调度请求作为`curl`命令，如下：

```
$ curl 'http://api.twitter.com/1/users/show.json?screen_name=timoreilly'
```

注2： 截止2010年10月，Twitter实现了OAuth 1.0a，但是你应该可以在2011年看到它对OAuth 2.0的支持。

注意： curl是一种便利的工具，它使用各种协议来将数据传输到服务器中（或者从服务器中传输出来），而且它特别适用于从终端发出HTTP请求。作为大多数*nix系统的标准配置，它通常存在于PATH中，但是Windows用户可能需要下载并配置它（*http://curl.haxx.se/latest.cgi?curl=win32-ssl-devel-msvc*）。

这个请求有两个微妙之处。首先，Twitter有API版，所以出现在URL内容中的/1表示正在使用API 1。其次，user_id可能已经取代了screen_name。显然，这条curl命令的映射相当于示例4-1中的Python脚本。

示例4-1：提取一个Twitter用户的扩展信息

```
# -*- coding: utf-8 -*-

import twitter
import json

screen_name = 'timoreilly'
t = twitter.Twitter(domain='api.twitter.com', api_version='1')
response = t.users.show(screen_name=screen_name)
print json.dumps(response, sort_keys=True, indent=4)
```

如果你还没有查看Twitter的在线文档（*http://dev.twitter.com/doc*），那么值得明确指出的是：/users/show API调用不需要认证，并且它有一些具体特点，这取决于用户是否在隐私设置中“保护了”他的Twitter消息。/users/loopup API调用和/users/show类似，只是它需要认证，而且允许传入一个以逗号分割的screen_name或user_id值列表，这样你就可以执行批量查找了。为了获得使用Twitter API的更多授权，你需要了解OAuth，它是下一节内容的主题。

不，你不能知道我的密码

OAuth代表“开放授权”。为了尽可能地高瞻远瞩，这一节内容提供了非常粗略的OAuth 2.0（*http://tools.ietf.org/html/draft-ietf-oauth-v2-10*）概述，它是一种新兴的授权方案，Facebook已经开始实现它了（*http://developers.facebook.com/docs/authentication/*）（参阅第9章），Twitter打算“很快”支持它（*http://www.slideshare.net/episod/chirp-2010-too-many-secrets-but-never-enough-oauth-at-twitter*）。它最终会成为新的行业标准。编写本书时，Twitter和很多其他Web服务都支持OAuth 1.0a，它由RFC 5849（*http://tools.ietf.org/html/rfc5849*）定义。然而，这种现象将会被改变，因为OAuth 2.0可以简化开发者的工作，并且能促进更好的用户体验（*http://hueniverse.com/2010/05/introducing-oauth-2-0/*）。虽然这一节的术语和细节都是针对OAuth 2.0的，但是OAuth 1.0a中的基本工作流程与此十分相似。这两种方案都是“三条腿的”，因为它们涉及客户端应用程序中信息的交换（通常称为“dance”），它需要访问受保护

的资源、资源所有者（如社交网络）和最终用户，它需要对客户端应用程序授权来访问受保护的资源（不需要提供给它用户名/密码组合）。

注意： Twitter当前的API版本是1.0（且仅支持1.0）和目前Twitter 支持OAuth 1.0a完全是巧合。当Twitter准备好后，它的API 1也可能支持OAuth 2.0。

所以，简单地说，OAuth提供给你一种方法，来授权一个应用程序访问存储在另一个应用程序中的数据，而无需共享你的用户名和密码。IETF（Internet Engineering Task Force，Internet工程任务组）OAuth 2.0协议（*http://tools.ietf.org/html/draft-ietf-oauth-v2-10*）规范并没有那么可怕，你应该花点时间仔细阅读它，因为OAuth无处不在——尤其是在社交网络中。下面是涉及的主要步骤：

- 你（最终用户）想授权某种应用程序（客户端）来访问由Web服务器（资源拥有者）托管的一些数据（范围）
  - — 你很明智，知道不应该直接把你的凭证交给应用程序。
- 客户端将你重定向到资源拥有者，而不是询问你的密码，你让资源拥有者直接给客户端授权范围。
  - — 一旦最终用户提供了授权，客户端就能使用唯一的客户端标识符和联系它的方式来标识自身。
- 假设最终用户授权客户端，就会通知客户端，并且给它一个授权码来确定最终用户已经授权它访问某个范围。
  - — 但是如果我们在这里停止的话，会存在一个小问题：由于客户端已经使用了不必是密钥的标识符来标识自身，恶意客户端也可以用欺骗手段来标识自身，并且伪装成它正在由一个信任的发布者创建，成功地欺骗最终用户对它授权。
- 客户端给资源拥有者显示它刚才收到的授权码和它的客户端标识符以及相应的客户端密钥，取回一个访问令牌（access token）。客户端标识符、客户端密钥和授权码的组合确保资源拥有者可以正确地标识客户端和它的授权。
  - — 访问令牌是有选择地“短期有效的”，需要由客户端刷新。
- 客户端使用这个访问令牌代表最终用户提出请求，直到该访问令牌被撤销或过期。

规范的1.4.1节（*http://tools.ietf.org/html/draft-ietf-oauth-v2-10#section-1.4.1*）提供了更多细节和图4-1的基础。

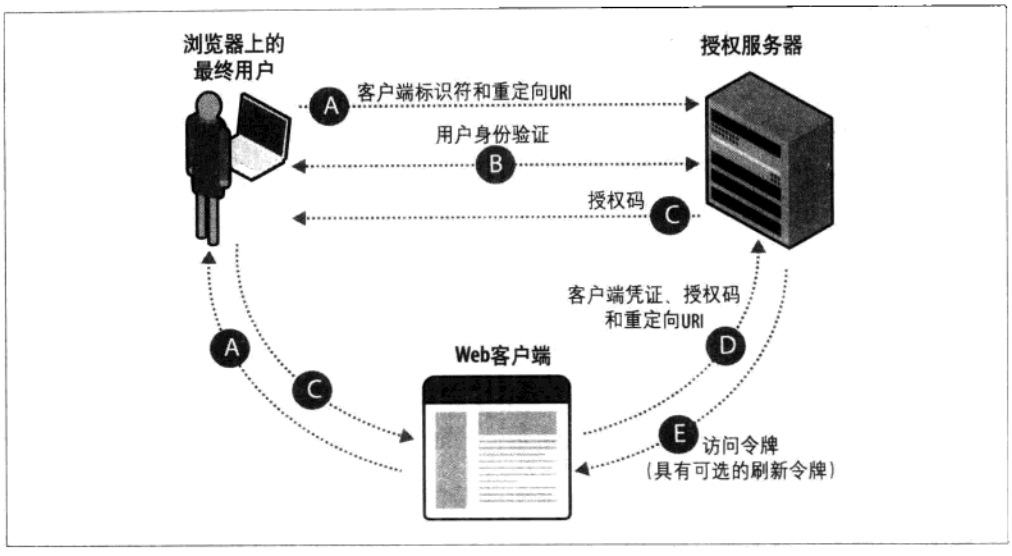

图4-1：OAuth 2.0的Web服务器流程图。从Web客户端开始，按照箭头的字母顺序，在完成A～E步骤后可以返回到Web客户端。

此外，OAuth 2.0是比较新的技术，截止2010年底，在作为RFC规范正式发布之前，仍然在仔细推敲它的各种细节。想要了解它，最好的办法是下工夫阅读规范，勾画一些流程图，向自己提问，扮演成恶意用户等。要了解与OAuth相关的见解，可以阅读一下Ryan Paul的Ars Technica文章："Twitter: A Case Study on How to Do OAuth Wrong"（*http://arstechnica.com/security/guides/2010/09/twitter-a-case-study-on-how-to-do-oauth-wrong.ars*）。也可以看一下Eran Hammer-Lahav（OAuth 2.0协议工作草案的第一作者）的回复（*http://hueniverse.com/2010/09/all-this-twitter-oauth-security-nonsense/*）和密码专家Ben Adida相当详细的反驳（*http://benlog.com/articles/2010/09/02/an-unwarranted-bashing-of-twitters-oauth/*）。

注意： 作为包含Twitter中称为"单用户用例"的最新工具，Twitter也提供了最新的快速通道获取需要的凭证来发出请求，而不需要实现整个OAuth流程。可以从*http://dev.twitter.com/pages/oauth_single_token*中阅读更多内容。然而，本章和下一章都假设你想要实现标准OAuth流程。

干练而中肯的数据采集器

原则上，抓取Twitter数据非常简单：提出请求，存储响应，按需重复。但是，现实世界中的各种东西都会妨碍我们，如网络I/O、臭名昭著的失败鲸[注3]，还有那些讨厌的API速率限制。幸运的是，解决这些问题并不难，只要你多做一些规划，并预先考虑到可能（并且一定会）出错的事情即可。

当执行长期运行的、能够逐渐摆脱速率限制的程序时，编写健壮的代码显得尤其重要；你要处理可能发生的各种异常情况，并尽最大的努力来纠正错误，在尽了最大努力，情况仍然不容乐观的情况下，保存当前状态，并在它们停止的地方指示出如何回到原来的状态。也就是说，当为Twitter这类平台编写数据采集代码时，必须假设它会给你出难题。必须处理非典型情况，而且它们通常是正常情况，而不是例外。

我们开发的代码是“加固的”，因为它能处理最常见的错误，并将它作为模式，这样，你就可以轻易地扩展它来处理可能出现的新状况。这就是说，即使采集很少的Twitter数据时，也可能会遇到两个特定的HTTP错误：401错误（未授权）和503错误（过载）。前者在你试图访问用户保护的数据时发生，而后者基本上是不可预测的。

当Twitter返回一个HTTP错误时，`twitter`模块就会抛出一个`TwitterHTTPError`异常，和其他Python异常的处理方式相同，可以用`try/except`块来处理它。示例4-2说明了一段精简的代码块，用于采集一些朋友的ID，并处理较常见的异常情况。

注意： 需要创建一个Twitter app（*http://twitter.com/apps/new*），以便获取在本书的Twitter示例中使用的一个用户密钥。这很容易，只需花一点儿时间就可以创建了。

示例4-2：使用OAuth来认证并抓取一些好友数据（friends_followers_get_friends.py）

```
# -*- coding: utf-8 -*-

import sys
import time
import cPickle
import twitter
from twitter.oauth_dance import oauth_dance

# Go to http://twitter.com/apps/new to create an app and get these items

consumer_key = ''
consumer_secret = ''

SCREEN_NAME = sys.argv[1]
friends_limit = 10000
```

注3：当Twitter过载时，就会发生HTTP 503错误。在浏览器中，错误页面显示的是臭名昭著的“失败鲸”图像。参阅*http://twitter.com/503*。

```
(oauth_token, oauth_token_secret) = oauth_dance('MiningTheSocialWeb',
        consumer_key, consumer_secret)
t = twitter.Twitter(domain='api.twitter.com', api_version='1',
                    auth=twitter.oauth.OAuth(oauth_token, oauth_token_secret,
                    consumer_key, consumer_secret))

ids = []
wait_period = 2 # secs
cursor = -1

while cursor != 0:
    if wait_period > 3600: # 1 hour
        print 'Too many retries. Saving partial data to disk and exiting'
        f = file('%s.friend_ids' % str(cursor), 'wb')
        cPickle.dump(ids, f)
        f.close()
        exit()

    try:
        response = t.friends.ids(screen_name=SCREEN_NAME, cursor=cursor)
        ids.extend(response['ids'])
        wait_period = 2
    except twitter.api.TwitterHTTPError, e:
        if e.e.code == 401:
            print 'Encountered 401 Error (Not Authorized)'
            print 'User %s is protecting their tweets' % (SCREEN_NAME, )
            elif e.e.code in (502, 503):
            print 'Encountered %i Error. Trying again in %i seconds' % (e.e.code,
                wait_period)
            time.sleep(wait_period)
            wait_period *= 1.5
            continue
        elif t.account.rate_limit_status()['remaining_hits'] == 0:
            status = t.account.rate_limit_status()
            now = time.time() # UTC
            when_rate_limit_resets = status['reset_time_in_seconds'] # UTC
            sleep_time = when_rate_limit_resets - now
            print 'Rate limit reached. Trying again in %i seconds' % (sleep_time,
                    )
            time.sleep(sleep_time)
            continue

    cursor = response['next_cursor']
    print 'Fetched %i ids for %s' % (len(ids), SCREEN_NAME)
    if len(ids) >= friends_limit:
        break

# do something interesting with the IDs

print ids
```

twitter.oauth模块提供了read_token_file和write_token_file这两个方便的函数，二者可用于存储、检索OAuth令牌和OAuth令牌密钥，所以不必每次都手动输入PIN来验证身份。

注意： 在OAuth 2.0用法中，“client”和OAuth 1.0中的“consumer”作用一样，因此，在上面的清单中使用了consumer_key和consumer_secret变量名。

清单中有一些值得注意的事项：

- 可以通过在*http://dev.twitter.com/apps/new*上注册一个Twitter应用程序来获取你自己的consumer_key和consumer_secret。这两项和“OAuth dance”中返回的凭证可以提供给应用程序访问你的账户数据（在本例中是你的朋友列表）的权限。
- Twitter的社交图谱API的在线文档（*http://dev.twitter.com/doc/get/friends/ids*）指出：对好友/关注者数据请求的每次调用都会返回多达5000个ID。在返回多于5000个ID的情况下，会返回一个不等于0的光标值，可以用它导航到下一批ID。在本例中，“突然中断”最多有1万个ID值，但是friends_limit可以是一个任意大的数值。
- 由于/friends/ids资源每次都返回多达5000个ID，因此只要速率限制没有达到350个请求/小时，普通用户账户就可以检索多达175万个ID。虽然对于一个Twitter用户来说，有这么多朋友很反常，但是对于一个高人气用户来说，有很多关注者是很正常的。
- 虽然不能确定来自哪份官方文档或示例代码，但结果中的ID看起来是按时间倒序排列的，所以第一个值是你最近关注的用户，而最后一个值是你最先关注的用户。通过t.followers.ids请求关注者可能会以相同的顺序返回结果。

到目前为止，只介绍了一部分Twitter API。虽然它们足以回答关于你的账户或者其他一些未受保护的账户的一些有趣的问题，但是还有很多其他API。不久我们会查看更多API，但是首先，让我们继续介绍一个简单的子程序来重构示例4-2。

一个非常简短的重构子程序

由于几乎所有包含Twitter数据的代码清单都会重复执行OAuth dance，并提出健壮的请求，这些请求可以接受假设可能会出错的一连串事情，因此非常值得建立执行这些任务的模式。我们采取的方法是：对login()和makeTwitterRequest()函数隔离OAuth逻辑，这样示例4-2看起来就像下面的重构版本示例4-3：

示例4-3：重构示例4-2来对OAuth使用两个常见的实用工具，并发出API请求（friends_followers_get_friends_refactored.py）

```
# -*- coding: utf-8 -*-

import sys
import time
import cPickle
```

```
import twitter
from twitter__login import login
from twitter__util import makeTwitterRequest

friends_limit = 10000

# You may need to setup your OAuth settings in twitter__login.py
t = login()

def getFriendIds(screen_name=None, user_id=None, friends_limit=10000):
    assert screen_name is not None or user_id is not None

    ids = []
    cursor = -1
    while cursor != 0:
        params = dict(cursor=cursor)
        if screen_name is not None:
            params['screen_name'] = screen_name
        else:
            params['user_id'] = user_id

        response = makeTwitterRequest(t, t.friends.ids, **params)

        ids.extend(response['ids'])
        cursor = response['next_cursor']
        print >> sys.stderr, \
            'Fetched %i ids for %s' % (len(ids), screen_name or user_id)
        if len(ids) >= friends_limit:
            break

    return ids

if __name__ == '__main__':
    ids = getFriendIds(sys.argv[1], friends_limit=10000)

    # do something interesting with the ids

    print ids
```

从现在起，我们将继续使用twitter__login（*https://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/twitter__login.py*）和twitter__util（*https://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/twitter__util.py*）来尽可能保持示例的清晰简洁。在继续阅读之前，值得花些时间在线仔细阅读这些模块的源。它们将会反复出现，twitter__util中很快就会有一些常用的便利函数。

下一节将介绍Redis，它是一个强大的数据结构服务器，由于其性能和简单性，很快就受到了大家的热捧。

Redis：数据结构服务器

我们已经注意到，当你要执行可能长时间运行的程序来从Web上获取数据时，提前规划是很重要的，因为可能会出现很多问题。但是一旦你得到了所有的数据，你要如何处理

它们呢？起初，你可能只是将它存储到磁盘中。但是在我们刚才考虑的情况中，可能会产生与下面的结构类似的目录结构：

```
./
screen_name1/
    friend_ids.json
    follower_ids.json
    user_info.json
screen_name2/
    ...
...
```

除非你采集了一个非常受欢迎的用户的所有好友/关注者，否则这些看起来都很合理——然后，根据你的平台，你可能要面对包含数百万个子目录的目录，相对来说，它是不可用的，因为你不能轻易地（尽可能）在一个终端中浏览它。将所有信息保存到磁盘中也需要你维护某种注册表，它记录了所有的昵称，因为对数百万文件产生目录列表所需要的时间可能不会生成一个理想的性能配置文件。如果使用这些数据的应用程序变得线程化，你可能会以多个写者需要在同一时间访问同一文件的方式结束，所以你必须开始处理文件锁定这类事情。这可能不是你的本意。现在我们真正需要的是一个系统，它可以非常轻松地存储基本的键/键值对以及一个简单的键编码方案——磁盘备份目录将是好的开始。下一段代码通过连接用户ID、分割符和数据结构名来说明键的结构：

```
s={}
s["screen_name1$friend_ids"]=[1,2,3,…]
s["screen_name1$friend_ids"] # returns [1,2,3,…]
```

但是如果映射函数可以自动计算集合操作不是很酷吗？这样我们只需要告诉它做这些事情：

```
s.intersection("screen_name1$friend_ids","screen_name1$follower_ids")
```

来自动计算一个Twitter用户的“互粉”（即，计算出他们的哪些好友关注了他们）。一个称为Redis（*http://code.google.com/p/redis/*）的开源项目正好提供了这种功能。Redis很容易安装、极其快速（用C语言编写）、规模适中，一直在积极维护，而且有附带可用文档的强大的Python客户端。体验Redis就像安装它（*http://code.google.com/p/redis/*）并启动它的服务器一样简单。（Windows用户可以通过抓取由servicestack.net维护的二进制文件（*http://code.google.com/p/servicestack/wiki/RedisWindowsDownload*）来安装它。）然后，只要运行easy_install redis来获取漂亮的Python客户端即可，它提供了它必须提供的所有功能的简单访问。例如，将上一段代码转化为下面的Redis代码：

```
import redis

r=redis.Redis(host='localhost',port=6379,db=0) # Default params
[r.sadd("screen_name1$friend_ids",i) for i in [1,2,3,…]]
```

```
r.smembers("screen_name1$friend_ids") # Returns [1,2,3,…]]
```

请注意，虽然sadd和smembers是基于集合的操作，但Redis包含了针对各种其他数据结构类型的操作，如集合、列表和哈希表。集合操作被证明是特别有趣的，因为它们可以回答本章开头提出的很多问题。值得花点时间来阅读Redis Python客户端文档，这样可以更深入了解它的性能。回想一下，可以仅执行pydoc redis.Redis这样的命令来从终端快速浏览文档。

注意： 查阅“Redis: under the hood”（*http://www.pauladamsmith.com/articles/redis_under_the_hood.html*）来深入了解Redis在内部是如何工作的。

基本的集合操作

你可能会遇到的最常见的集合操作是并、交和差运算。回想一下，集合和列表的区别是：集合是无序的且只包含唯一的元素，而列表是有序的，可以包含重复元素。从Python 2.6起，Python为集合提供了内置支持，通过set数据结构来实现。表4-1是一些常见的集合操作示例，它包含了很少的好友和关注者：

```
好友={Abe, Bob}, 关注者={Bob, Carol}
```

表4-1：好友和关注者的示例集合操作

操作	结果	注释
好友 ∪ 关注者	Abe、Bob、Carol	某人的全部社交网络
好友 ∩ 关注者	Bob	某人共同的好友
好友－关注者	Abe	某人关注了别人，但是他却没有被对方关注
关注者－好友	Carol	关注了某人，却没有被他关注

如上所述，Redis提供了处理常见集合操作的本地操作。接下来会用到的一些最相关的操作包括：

`smembers`
: 返回集合中所有的成员

`scard`
: 返回集合的基数（集合中元素的个数）

`sinter`
: 计算集合的交集

`sdiff`
: 计算集合的差集

`mget`
: 返回多个键的字符串值

`mset`
: 存储多个键的字符串值

`sadd`
: 在一个集合中添加成员（如果集合不存在，就创建一个集合）

`keys`
: 返回与类正则表达式模式匹配的所有键

浏览Python中内置集合数据类型中的Pydoc应该可以让你确信它与Redis API之间密切的映射。

我们将像1874年那样分析

虽然集合论中涉及的概念很古老，但大家普遍认为Georg Cantor是集合论的创始人。1874年，他在论文“On a Characteristic Property of All Real Algebraic Numbers”中正式将集合论作为研究工作的一部分，他回答了与无穷概念有关的问题。例如：

- 自然数（零和正整数）是否多于整数（正整数和负整数）？
- 有理数（可以被表示为小数的数）的数量是否多于整数？
- 无理数（不能被表示为小数的数，如pi，$\sqrt{2}$等）是否多于有理数？

Cantor提出的与前两个问题相关的无穷的重点是：自然数、整数和有理数的基数是相等的，因为可以映射这些数字，使它们用一个明确的起点形成一个可以在一个方向上无限延伸的序列。即使没有终点，也可以说这些集合的基数是可数无穷的，因为如果你有足够的时间来统计它们的话，就能得到一个可以明确遵循的确切序列。数学家把一个可数无穷集合的基数称为$\aleph_0$，它是无穷大的正式定义。考虑可数无穷集合背后传递基本概念的数字模型。每个模型都表明了一个可以无限延伸的起点。

- 自然数：0，1，2，3，4，…
- 正整数：1，2，3，4，5，…
- 负整数：−1，−2，−3，−4，−5，…

- 整数：0，1，−1，2，−2，3，−3，4，−4，…
- 有理数：0/0，0/1，−1/1，−1/0，−1/−1，0/−1，1/−1，1/0，1/1，…

请注意，有理数模式是：你可以从直角平面的原点开始，创建一条螺旋线，其中每个*x*/*y*坐标对都被表示为是有理数的小数。（两种情况下它都是没有定义的，因为对于作为整体的集合基数来说，除以零是没有意义的。）

然而，最终证明：无理数集合的基数并不等于$\aleph_0$，因为不可能以可数的方式排列它们，并形成返回到包含基数$\aleph_0$的一个一一对应关系。Cantor使用了著名的对角论证法（*http://en.wikipedia.org/wiki/Cantor's_diagonal_argument*）作为证明。对角论证法的要点是：只有当你认为你已经映射出一个可以使无理数可数的序列，才可以证明整组数丢失——并且当你将这些丢失的数放入序列中时，仍然会有一组数丢失。最终证明你永远不可能形成必要的一一对应关系。因此，无理数集合的基数与自然数集合、正整数集合和无理数集合的基数是不同的，因为从这些集合中不能获得一一对应关系。

那么无理数集合的基数是什么呢？基数为$\aleph_0$的集合的幂级数是所有无理数集合的基数。这个值称为$\aleph_1$。此外，基数$\aleph_1$的集合的幂级数称为$\aleph_2$，依次类推。计算一组无穷数的幂级数确实是一个难以理解的概念，但是你晚上睡不着觉时，可以好好琢磨一下。

使用基本的朋友/关注者度量来增强性能

Redis应该可以对查询进行有效的处理，并分析某类查询中大量的Twitter数据。只需要使用一些附加逻辑对示例4-2做简单的修改，就可以在Redis中存储数据了。示例4-4是计算一些基本的朋友/关注者统计信息的更新。使用Redis自带的函数来计算集合操作。

示例4-4：采集、存储并计算朋友和关注者的统计信息（friends_followers__friend_follower_symmetry.py）

```
# -*- coding: utf-8 -*-

import sys
import locale
import time
import functools
import twitter
import redis
from twitter__login import login

# A template-like function for maximizing code reuse,
# which is essentially a wrapper around makeTwitterRequest
```

```
# with some additional logic in place for interfacing with
# Redis
from twitter_util import _getFriendsOrFollowersUsingFunc

# Creates a consistent key value for a user given a screen name
from twitter_util import getRedisIdByScreenName

SCREEN_NAME = sys.argv[1]

MAXINT = sys.maxint

# For nice number formatting
locale.setlocale(locale.LC_ALL, '')

# You may need to setup your OAuth settings in twitter_login.py

t = login()

# Connect using default settings for localhost
r = redis.Redis()

# Some wrappers around _getFriendsOrFollowersUsingFunc
# that bind the first two arguments

getFriends = functools.partial(_getFriendsOrFollowersUsingFunc,
                               t.friends.ids, 'friend_ids', t, r)

getFollowers = functools.partial(_getFriendsOrFollowersUsingFunc,
                                 t.followers.ids, 'follower_ids', t, r)

screen_name = SCREEN_NAME

# get the data

print >> sys.stderr, 'Getting friends for %s...' % (screen_name, )
getFriends(screen_name, limit=MAXINT)

print >> sys.stderr, 'Getting followers for %s...' % (screen_name, )
getFollowers(screen_name, limit=MAXINT)

# use redis to compute the numbers

n_friends = r.scard(getRedisIdByScreenName(screen_name, 'friend_ids'))

n_followers = r.scard(getRedisIdByScreenName(screen_name, 'follower_ids'))

n_friends_diff_followers = r.sdiffstore('temp',
                                        [getRedisIdByScreenName(screen_name,
                                        'friend_ids'),
                                        getRedisIdByScreenName(screen_name,
                                        'follower_ids')])
r.delete('temp')

n_followers_diff_friends = r.sdiffstore('temp',
                                        [getRedisIdByScreenName(screen_name,
                                        'follower_ids'),
                                        getRedisIdByScreenName(screen_name,
                                        'friend_ids')])
r.delete('temp')
```

```
n_friends_inter_followers = r.sinterstore('temp',
        [getRedisIdByScreenName(screen_name, 'follower_ids'),
        getRedisIdByScreenName(screen_name, 'friend_ids')])
r.delete('temp')

print '%s is following %s' % (screen_name, locale.format('%d', n_friends, True))
print '%s is being followed by %s' % (screen_name, locale.format('%d',
                                      n_followers, True))
print '%s of %s are not following %s back' % (locale.format('%d',
        n_friends_diff_followers, True), locale.format('%d', n_friends, True),
        screen_name)
print '%s of %s are not being followed back by %s' % (locale.format('%d',
        n_followers_diff_friends, True), locale.format('%d', n_followers, True),
        screen_name)
print '%s has %s mutual friends' \
    % (screen_name, locale.format('%d', n_friends_inter_followers, True))
```

除了使用functools.partial（*http://docs.python.org/library/functools.html*）来从一段常见的参数绑定代码中创建getFriends和getFollowers之外，示例4-4应该非常简单。还有一件非常微妙的事值得注意：示例4-4中没有对r.save的调用，这意味着当数据持久化到磁盘时，*redis.conf*命令中的设置。默认情况下，Redis根据在指定的时间间隔中是否发生了一些变化所确定的清单，将数据存储在内存中，且将数据异步快照到磁盘中。异步写入的风险是：如果发生某些意外情况（如系统崩溃或断电），可能会丢失数据。Redis提供了“只能添加”（append only）选项，允许你在*redis.conf*中防止这种可能性的发生。

注意：我们强烈建议你使用*redis.conf*中的appendonly选项来防止数据丢失。要获得有用的细节，请参阅“Append Only File HOWTO”（*http://code.google.com/p/redis/wiki/AppendOnlyFileHowto*）。

考虑以下与Tim O'Reilly的关注者的网络有关的输出。请记住，速率限制是350个OAuth请求/小时，你可以期待该代码的运行时间少于一小时，因为要收集所有关注者的ID值，只需要大约300个API调用：

```
timoreilly is following 663
timoreilly is being followed by 1,423,704
131 of 633 are not following timoreilly back
1,423,172 of 1,423,704 are not being followed back by timoreilly
timoreilly has 532 mutual friends
```

请注意，虽然你可以选择接受收集少数关注者来避免速率限制强加的等待，但API文档没有指出使用前*N*页数据的值将产生一个真正随机的抽样，并且数据好像是按时间倒序返回的——所以，你可能无法以一种可预测的方法来推断你的逻辑是否取决于它。例如，如果返回的前1万个关注者正好包含532个共同的朋友，则从那些点外推将会导致歪曲的分析，因为这些结果并不代表较多的人数。对于相当受欢迎的Twitter用户（如Britney Spears），拥有远超过500万的关注者，大约1000个API调用将用来在4小时之内

抓取所有的关注者。等待这类数据通常是值得的，并且你可以使用Twitter的信息流API来保持数据的与时俱进，这样你永远都不必重新体验这个痛苦的过程了。

注意： 某类分析的常见错误来源是：忘记了和你的样本相关的总体人数。例如，随机抽取Tim O'Reilly的1万个好友和关注者，实际上包含了他所有的好友，而只包含他的一小部分关注者。根据分析的复杂度，与总人数相关的样本量（*http://en.wikipedia.org/wiki/Sampling_(statistics)*）可以确定一个实验的结果是否有统计意义，以及你对它的置信水平。

给定这些基本的好友/关注者状态，引导我们走向其他有趣的分析的一些问题也会随之而来。例如，没有关注Tim O'Reilly的那131个人是谁？考虑到好友和关注者的各种可能性，“谁没有关注我？”这个问题比较有趣，而且可以认为它提供了某人兴趣的很多见解。所以，我们该如何回答这个问题呢？

对用户ID列表的凝视并不能吸引人，所以将这些用户ID解析成实际的用户对象才是明显的第一步。示例4-5通过将常见的错误处理代码封装为可重用的形式来扩展示例4-4。它也提供了一个函数，该函数演示了如何使用/users/loopup API将这些ID值解析为昵称，它接受多达100个用户ID或昵称，并且返回与之前通过/users/show看到的一样的基本用户信息。

示例4-5：从ID中解析基本用户信息，如昵称（friends_followers_get_user_info.py）

```
# -*- coding: utf-8 -*-

import sys
import json
import redis
from twitter__login import login

# A makeTwitterRequest call through to the /users/lookup
# resource, which accepts a comma separated list of up
# to 100 screen names. Details are fairly uninteresting.
# See also http://dev.twitter.com/doc/get/users/lookup
from twitter__util import getUserInfo

if __name__ == "__main__":
    screen_names = sys.argv[1:]

    t = login()
    r = redis.Redis()

    print json.dumps(

      getUserInfo(t, r, screen_names=screen_names),
      indent=4
    )
```

虽然并没有全部复制，但从twitter_util导入的getUserInfo()函数本质上只是一个使用

昵称列表向/users/loopup资源发出的makeTwitterRequest请求。下面的代码段将会说明这一点：

```
def getUserInfo(t, r, screen_names):
    info = []
    response = makeTwitterRequest(t,
                                  t.users.lookup,
                                  screen_name=','.join(screen_names)
                                 )

    for user_info in response:
        r.set(getRedisIdByScreenName(user_info['screen_name'], 'info.json'),
              json.dumps(user_info))
        r.set(getRedisIdByUserId(user_info['id'], 'info.json'),
              json.dumps(user_info))

        info.extend(response)

    return info
```

值得注意的是getUserInfo在两个不同的键存储了相同的用户信息：用户ID和昵称。存储这两个键使我们轻易地找出给定用户ID值的昵称，并找出给定昵称的ID值。把用户ID值转换为昵称是一个非常有用的操作，因为用于获取好友和关注者的社交图谱API只返回ID值，在它们被解析为昵称和基本的用户信息之前，它们的值并不直观。虽然和其他方法相比，该方法涉及冗余存储，但是它的方便性毋庸置疑。如果存储是个问题的话，随意采取一个精简的方法即可。

Tim O'Reilly关注的用户信息的示例如示例4-6所示，它说明了可以得到的Twitter用户的信息类型。可以处理的数据没有限制。我们既不会挖掘用户描述和没有关注Tim的用户的Twitter消息，也不会将它们打印出来，但是如果你希望进行这类分析，那么应该会有足够的数据。

示例4-6：Tim O'Reilly以JSON数据表示的用户对象示例

```
{
    "id": 2384071,
    "verified": true,
    "profile_sidebar_fill_color": "e0ff92",
    "profile_text_color": "000000",
    "followers_count": 1423326,
    "protected": false,
    "location": "Sebastopol, CA",
    "profile_background_color": "9ae4e8",
    "status": {
        "favorited": false,
        "contributors": null,
        "truncated": false,
        "text": "AWESOME!! RT @adafruit: a little girl asks after seeing adafruit ...",
        "created_at": "Sun May 30 00:56:33 +0000 2010",
        "coordinates": null,
```

```
        "source": "<a href=\"http://www.seesmic.com/\" rel=\"nofollow\">Seesmic</a>",
        "in_reply_to_status_id": null,
        "in_reply_to_screen_name": null,
        "in_reply_to_user_id": null,
        "place": null,
        "geo": null,
        "id": 15008936780
    },
    "utc_offset": -28800,
    "statuses_count": 11220,
    "description": "Founder and CEO, O'Reilly Media. Watching the alpha geeks...",
    "friends_count": 662,
    "profile_link_color": "0000ff",
    "profile_image_url": "http://a1.twimg.com/profile_images/941827802/IMG_...jpg",
    "notifications": false,
    "geo_enabled": true,
    "profile_background_image_url": "http://a1.twimg.com/profile_background_...gif",
    "name": "Tim O'Reilly",
    "lang": "en",
    "profile_background_tile": false,
    "favourites_count": 10,
    "screen_name": "timoreilly",
    "url": "http://radar.oreilly.com",
    "created_at": "Tue Mar 27 01:14:05 +0000 2007",
    "contributors_enabled": false,
    "time_zone": "Pacific Time (US & Canada)",
    "profile_sidebar_border_color": "87bc44",
    "following": false
}
```

用于处理HTTP错误和批量获取用户信息的重构逻辑会在下面的几节中提供。请注意，`handleTwitterHTTPError()`函数故意不包含每一种可能的错误情况的处理，因为可能要采取的行动会随着情况的变化而不同。例如，如果因为有人拔掉网线而触发了urllib.URL（超时操作），就要给用户提示一个具体的行动方针。

示例4-5揭示了一些好消息和一些不太好的消息。好消息是将用户ID解析到包含署名、位置信息、最新tweet等的用户对象是信息的宝库。不太好的消息是从速率限制方面来看，这是相当“昂贵”的，因为只能批量备份100个数据。对于Tim O'Reilly的好友来说，只有7个API调用。但是对于他的关注者来说，有超过1.4万个调用，如果速率限制限定在350个调用/小时的话（并且收集时没有故障），这需要花费差不多两天时间来收集。

然而，如果有某人的好友和关注者ID值的一个完整集合，你就可以随机进行抽样，并且可以尽情地计算统计意义的度量。Redis提供了完全符合要求的`srandmember()`函数。传送给它一个集合名（如`timoreilly$follow_ids`），它会返回这个集合的一个随机成员。

通过计算共同好友和关注者来计算相似性

我们可以追求的另一种唾手可得的成果是：计算两个或者更多Twitter用户的共同好友和关注者。在给定的空间内，对于某些原因，这些用户可能不很有趣。一个原因是：他们是连接不同网络的“共同主线”；你可能将它解释成一种相似性度量。例如，如果两个用户都关注了很多相同的人，你就可以推断这两个用户有相似的兴趣。从这里开始，你可能会开始分析嵌入在共同朋友的tweet中的信息，来深入了解这些人的共同特点，如果有所收获的话，你就能得出一些结论了。结果证明计算共同的好友和关注者只是一个集合操作。示例4-7说明了Redis中对存储了集合交集结果的sinterscore()函数的使用，并且引入了locale.format来实现漂亮的打印，这样可以很容易地阅读输出的内容。

示例4-7：查找多个Twitter用户的共同好友/关注者，它的输出更容易阅读（friends_followers_friends_followers_in_common.py）

```
# -*- coding: utf-8 -*-

import sys
import redis

from twitter__util import getRedisIdByScreenName

# A pretty-print function for numbers
from twitter__util import pp

r = redis.Redis()

def friendsFollowersInCommon(screen_names):
    r.sinterstore('temp$friends_in_common',
                  [getRedisIdByScreenName(screen_name, 'friend_ids')
                      for screen_name in screen_names]
                  )

    r.sinterstore('temp$followers_in_common',
                  [getRedisIdByScreenName(screen_name, 'follower_ids')
                      for screen_name in screen_names]
                  )

    print 'Friends in common for %s: %s' % (', '.join(screen_names),
            pp(r.scard('temp$friends_in_common')))
    print 'Followers in common for %s: %s' % (', '.join(screen_names),
        pp(r.scard('temp$followers_in_common')))

    # Clean up scratch workspace

    r.delete('temp$friends_in_common')
    r.delete('temp$followers_in_common')

if __name__ == "__main__":
    if len(sys.argv) < 3:
        print >> sys.stderr, "Please supply at least two screen names."
        sys.exit(1)

    # Note:
    # The assumption is that the screen names you are
```

```
    # supplying have already been added to Redis.
    # See friends_followers__get_friends__refactored.py

    friendsFollowersInCommon(sys.argv[1:])
```

请注意，虽然工作集合中的值是ID值，但是也可以使用Redis中的randomkey()函数抽取好友和关注者，并使用示例4-5中的getUserInfo()函数来解析有用的信息，如昵称、最新的tweet、地理位置等。

影响的度量

当有人通过类似Twitter的服务分享消息时，自然会想知道通过被转发，信息会渗透到整个网络的多大范围。假设一个人的关注者越多，他的tweet被转发的可能性就越大是合理的。可以说最初编写的tweet被转发的百分比更高的用户比不常被转发的用户具有更大的影响力。tweet被转发的比例更高的用户，即使这些tweet不是他们原创的，也可以说他们是专家——人际关系特别好且乐于分享信息的人[注4]。度量两个或更多用户相对影响力的一个简单方法是：简单地比较其关注者的数量，因为每个关注者都会直接查看他们的tweet。从示例4-6中我们已经知道，我们可以通过/users/lookup和/users/show API来获取一个用户的关注者（和朋友）的数量。从这两个API中提取这些信息非常简单：

```
    for screen_name in screen_names:
        _json = json.loads(r.get(getRedisIdByScreenName(screen_name, "info.json")))
        n_friends, n_followers = _json['friends_count'], _json['followers_count']
```

统计关注者的数量很有趣，但是我们能做的还有更多。例如，某个给定的用户可能不像信息专家（如Tim O'Reilly）那样受欢迎，但是如果他是你的关注者，而且他转发了你的tweet，那么你就突然接近了全网络的人，而且一旦他们确定你很有趣，他们也会开始关注你。因此，计算用户的潜在影响力更好的方法是：不仅比较他们关注者的数量，而且深入挖掘网络的好几层。事实上，我们可以使用示例2-4中介绍的广度优先方法。

示例4-8说明了一个广义的crawl()函数，它接受昵称清单、抓取深度和控制检索多少好友和关注者的参数。friends_limit和followers_limit参数控制从社交图谱API中获取多少项（5000批），而friends_sample和followers_sample控制检索多少个用户对象（100批）。getUserInfo()的更新函数也包含在反映抽样参数的通道中。

示例4-8：爬取好友/关注者关系（friend_followers_crawl.py）

```
# -*- coding: utf-8 -*-

import sys
import redis
import functools
```

注4：参阅Malcolm Gladwell的《The Tipping Point》（Back Bay Books）来深入了解专家。

```
from twitter__login import login
from twitter__util import getUserInfo
from twitter__util import _getFriendsOrFollowersUsingFunc

SCREEN_NAME = sys.argv[1]

t = login()
r = redis.Redis()

# Some wrappers around _getFriendsOrFollowersUsingFunc that
# create convenience functions

getFriends = functools.partial(_getFriendsOrFollowersUsingFunc,
                               t.friends.ids, 'friend_ids', t, r)
getFollowers = functools.partial(_getFriendsOrFollowersUsingFunc,
                                 t.followers.ids, 'follower_ids', t, r)

def crawl(
    screen_names,
    friends_limit=10000,
    followers_limit=10000,
    depth=1,
    friends_sample=0.2, #XXX
    followers_sample=0.0,
    ):

    getUserInfo(t, r, screen_names=screen_names)
    for screen_name in screen_names:
        friend_ids = getFriends(screen_name, limit=friends_limit)
        follower_ids = getFollowers(screen_name, limit=followers_limit)

        friends_info = getUserInfo(t, r, user_ids=friend_ids,
                                   sample=friends_sample)

        followers_info = getUserInfo(t, r, user_ids=follower_ids,
                                     sample=followers_sample)

        next_queue = [u['screen_name'] for u in friends_info + followers_info]

        d = 1
        while d < depth:
            d += 1
            (queue, next_queue) = (next_queue, [])
            for _screen_name in queue:
                friend_ids = getFriends(_screen_name, limit=friends_limit)
                follower_ids = getFollowers(_screen_name, limit=followers_limit)

                next_queue.extend(friend_ids + follower_ids)

                # Note that this function takes a kw between 0.0 and 1.0 called
                # sample that allows you to crawl only a random sample of nodes
                # at any given level of the graph

                getUserInfo(user_ids=next_queue)

if __name__ == '__main__':
    if len(sys.argv) < 2:
        print "Please supply at least one screen name."
```

```
    else:
        crawl([SCREEN_NAME])

        # The data is now in the system. Do something interesting. For example,
        # find someone's most popular followers as an indiactor of potential influence.
        # See friends_followers__calculate_avg_influence_of_followers.py
```

假设为获取某个用户的好友ID和关注者ID，你已经对`friends_limit`和`followers_limit`运行了足够多的`crawl()`函数，剩下的就是采用足够大的随机抽样，并计算有趣的度量，如一层关注者的平均数。查看他的前*N*个关注者来了解他可能会影响谁也是很有趣的。示例4-9说明了一种从Redis中取出数据并计算出Tim O'Reilly最受欢迎的关注者的可能的方法。

示例4-9：计算一个Twitter用户最受欢迎的关注者（friends_followers__calculate_avg_influence_of_followers.py）

```
# -*- coding: utf-8 -*-

import sys
import json
import locale
import redis

from prettytable import PrettyTable

# Pretty printing numbers
from twitter__util import pp

# These functions create consistent keys from
# screen names and user id values
from twitter__util import getRedisIdByScreenName
from twitter__util import getRedisIdByUserId

SCREEN_NAME = sys.argv[1]

locale.setlocale(locale.LC_ALL, '')

def calculate():
    r = redis.Redis() # Default connection settings on localhost

    follower_ids = list(r.smembers(getRedisIdByScreenName(SCREEN_NAME,
                        'follower_ids')))

    followers = r.mget([getRedisIdByUserId(follower_id, 'info.json')
                        for follower_id in follower_ids])
    followers = [json.loads(f) for f in followers if f is not None]

    freqs = {}
    for f in followers:
        cnt = f['followers_count']
        if not freqs.has_key(cnt):
            freqs[cnt] = []

        freqs[cnt].append({'screen_name': f['screen_name'], 'user_id': f['id']})

    # It could take a few minutes to calculate freqs, so store a snapshot for later use
```

```
    r.set(getRedisIdByScreenName(SCREEN_NAME, 'follower_freqs'),
        json.dumps(freqs))

    keys = freqs.keys()
    keys.sort()

    print 'The top 10 followers from the sample:'

    fields = ['Date', 'Count']
    pt = PrettyTable(fields=fields)
    [pt.set_field_align(f, 'l') for f in fields]

    for (user, freq) in reversed([(user['screen_name'], k) for k in keys[-10:]
                                  for user in freqs[k]]):
        pt.add_row([user, pp(freq)])

    pt.printt()

    all_freqs = [k for k in keys for user in freqs[k]]
    avg = reduce(lambda x, y: x + y, all_freqs) / len(all_freqs)

    print "\nThe average number of followers for %s's followers: %s" \
        % (SCREEN_NAME, pp(avg))

# psyco can only compile functions, so wrap code in a function

try:
    import psyco
    psyco.bind(calculate)
except ImportError, e:
    pass # psyco not installed

calculate()
```

注意： 在很多常见的复杂运算情况下，psyco（*http://psyco.sourceforge.net*）模块可以动态编译代码并产生显著的速度增长。如果要执行超过1秒的计算，它完全是可选的但绝对值得认真研究。

输出覆盖Tim O'Reilly的关注者中大约15万（大约10%）的样本大小。对于统计分析来说，与人数相关的高样本大小确保了一个小的误差界限和一个很高的置信水平[注5]。也就是说，可以认为结果很有代表性，虽然它与关于人数的绝对真理有很大的不同：

```
The Top 10 followers from the sample:
aplusk 4,993,072
BarackObama 4,114,901
mashable 2,014,615
MarthaStewart 1,932,321
Schwarzenegger 1,705,177
zappos 1,689,289
```

注5：　在99%的置信水平它约是误差界限的0.14倍。

```
Veronica 1,612,827
jack 1,592,004
stephenfry 1,531,813
davos 1,522,621
The average number of followers for timoreilly's followers: 445
```

有趣的是，列表中出现了一些熟悉的名字，包括一些历来最受欢迎的Twitter用户：Ashton Kutcher（*@aplusk*）、Barack Obama、Martha Stewart和Arnold Schwarzenegger。删除前10个关注者，并重新计算会把Tim关注者的平均关注者数降低到大约284个。然而，删除少于10个关注者的关注者会把该数字显著提高到1000多个！请注意，这个范围内的关注者有好几万个，而且简单地阅读他们的个人资料确实会得到一些现实情况：这些用户很多都是垃圾账户、保护他们tweet的用户等。删除前10名关注者以及自身的关注者少于10个的关注者可能会是一个合理的度量；做这两件事得到的结果约是800，这个结果仍然很高。据说被某个受欢迎的Twitter用户转发Twitter消息的理念有一些优点，该Twitter用户与其他受欢迎的Twitter用户有很多联系。

友谊图的构建

本章使用集合作为主要的数据结构来存储和处理数据，因为我们主要操作ID值这样的项集，集合操作提供了一些非常强大的操作。虽然总有一些适合大多数工作的通用工具，但没有适合所有工作的工具。你可以选择付出代价方式；一般来说，当把数据存储到专用索引（正如在上一章学习的CouchDB的情况）时，代价是预先支付的，或者是在查询时，当你查找它而且无法享受这些索引所带来的好处时付出代价。如果必须兼顾两者，就会以冗余存储和与数据的非规范化相关的复杂性的方式来支付代价——更不必说源代码本身的维护。

当你开始提适合于网络拓扑的问题时，你可能想把你的Redis数据导出到NetWorkX这类图形数据库中（第1章介绍过）。可以构建很多可能的图，但是让我们假设你对分析存在于某人社交网络中的友谊感兴趣，所以你想要一幅图来指出谁是谁的朋友。由于社交图谱提供了深入的见解，示例4-8提供了爬取Twitter关系的方法，花点时间来获取一个或多个有趣的Twitter用户的友谊数据。本节剩余的部分将介绍一些分析它的示例。

假设你已经得到了友谊数据并且将它存储到了Redis中，示例4-10说明了如何构建针对某个用户拥有的所有的“共同友谊”图。从根本上说，只要系统地在嵌套循环中遍历全部用户，并且在正在被考虑的圈子中存在朋友关系的任何两个人之间创建边即可。一旦数据可以被NetworkX使用，就突然有了一个完整的图解算法和实用工具宝库。下一节会探讨可以从友谊图中挖掘出来的一些有价值的东西。

示例4-10：将Redis中的好友/关注者数据导出到NetworkX中进行简单的图分析（friends_followers_redis_to_networkx.py）

```
# -*- coding: utf-8 -*-

# Summary: Build up a digraph where an edge exists between two users
# if the source node is following the destination node
import os
import sys
import json
import networkx as nx
import redis

from twitter__util import getRedisIdByScreenName
from twitter__util import getRedisIdByUserId

SCREEN_NAME = sys.argv[1]

g = nx.Graph()
r = redis.Redis()

# Compute all ids for nodes appearing in the graph

friend_ids = list(r.smembers(getRedisIdByScreenName(SCREEN_NAME, 'friend_ids')))
id_for_screen_name = json.loads(r.get(getRedisIdByScreenName(SCREEN_NAME,
                                'info.json')))['id']
ids = [id_for_screen_name] + friend_ids

for current_id in ids:
    print >> sys.stderr, 'Processing user with id', current_id

    try:
        current_info = json.loads(r.get(getRedisIdByUserId(current_id, 'info.json'
                                  )))
        current_screen_name = current_info['screen_name']
        friend_ids = list(r.smembers(getRedisIdByScreenName(current_screen_name,
                          'friend_ids')))

        # filter out ids for this person if they aren't also SCREEN_NAME's friends too,
        # which is the basis of the query

        friend_ids = [fid for fid in friend_ids if fid in ids]
    except Exception, e:
        print >> sys.stderr, 'Skipping', current_id

    for friend_id in friend_ids:
        try:
            friend_info = json.loads(r.get(getRedisIdByUserId(friend_id,
                                           'info.json')))
        except TypeError, e:
            print >> sys.stderr, '\tSkipping', friend_id, 'for', current_screen_name
            continue

        g.add_edge(current_screen_name, friend_info['screen_name'])

# Pickle the graph to disk...
```

```
if not os.path.isdir('out'):
    os.mkdir('out')

filename = os.path.join('out', SCREEN_NAME + '.gpickle')
nx.write_gpickle(g, filename)
print 'Pickle file stored in: %s' % filename

# You can un-pickle like so...

# g = nx.read_gpickle(os.path.join('out', SCREEN_NAME + '.gpickle'))
```

带着为爬取并存储在Redis中的友谊数据构建一种便捷的图表示的任务，现在开始进入有趣的部分：分析它。

派系检测与分析

示例4-10提供了一个简短的程序，它通过从Redis中导入数据并通过等幂的add_edge操作重复在图中添加边，来说明如何开始查找存在于指定用户中的友谊党派（*http://en.wikipedia.org/wiki/Clique*）。例如，如果Abe是Bob、Carol和Dale的朋友，Bob和Carol也是朋友，图中最大（“最高的”）的派系就存在于Abe、Bob和Carol之间。然而，如果Abe、Bob、Carol和Dale都是共同的朋友，那么图将会是完全连通的，而且最大的派系大小为4。在图中添加节点会创建其他派系，但是它不一定会影响图中最大的派系。在社交网的上下文中，派系很吸引人，因为它指出了图中共同友谊的最大集合。对于两个社交网络，比较它们的最大友谊党派会提供团体动力等的深入见解。

图4-2说明了一幅示例图，已经突出显示了其中最大的派系。可以说本图有4个派系。

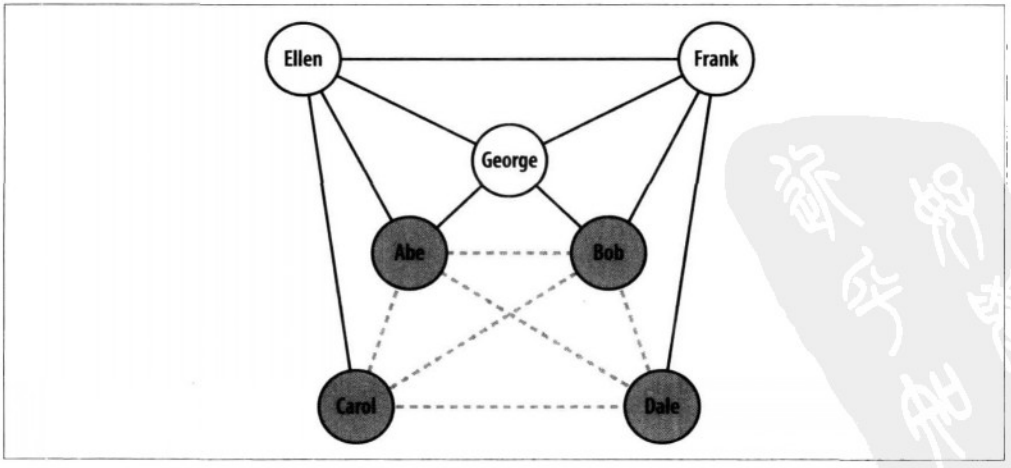

图4-2：包含大小为4的最大完备子图的示例图

注意： 从技术上来说，最大派系和最大完备子图之间是有区别的。最大完备子图是图中最大的派系（或如果它们大小相同，则是图中的派系）。另一方面，最大派系不是任何派系的子图。最大完备子图也是最大派系，因为它不是其他派系的子图。然而，图中通常存在各种其他最大派系，而且不一定和最大完备子图共享节点。例如，图4-2说明了大小为4的一幅最大完备子图，但是图中也有几个大小为3的最大派系。

查找派系是一个NP完全问题（意味着指数的运行时），但是NetworkX确实提供了find_cliques方法（*http://networkx.lanl.gov/reference/generated/networkx.find_cliques.html#networkx.find_cliques*），该方法提供了一个处理主要任务的可靠实现方式。需要注意的只是运行可能需要很长时间，因为图超过了相当小的规模。

可以以许多方法修改示例4-11来计算有趣的内容，只要事先收集到必要的友谊信息即可。假设事先构建了上节中所示的图，它会告诉你如何使用NetworkX的API查找并分析派系。

示例4-11：使用NetworkX来查找图中的派系(friends_followers_clique_analysis.py)

```
# -*- coding: utf-8 -*-

import sys
import json
import networkx as nx

G = sys.argv[1]

g = nx.read_gpickle(G)

# Finding cliques is a hard problem, so this could
# take awhile for large graphs.
# See http://en.wikipedia.org/wiki/NP-complete and
# http://en.wikipedia.org/wiki/Clique_problem

cliques = [c for c in nx.find_cliques(g)]

num_cliques = len(cliques)

clique_sizes = [len(c) for c in cliques]
max_clique_size = max(clique_sizes)
avg_clique_size = sum(clique_sizes) / num_cliques

max_cliques = [c for c in cliques if len(c) == max_clique_size]

num_max_cliques = len(max_cliques)
max_clique_sets = [set(c) for c in max_cliques]
people_in_every_max_clique = list(reduce(lambda x, y: x.intersection(y),
                                         max_clique_sets))

print 'Num cliques:', num_cliques
print 'Avg clique size:', avg_clique_size
print 'Max clique size:', max_clique_size
print 'Num max cliques:', num_max_cliques
print
```

```
print 'People in all max cliques:'
print json.dumps(people_in_every_max_clique, indent=4)
print
print 'Max cliques:'
print json.dumps(max_cliques, indent=4)
```

脚本中的示例输出遵循Tim O'Reilly的600+友谊，并且显示出一些有趣的见解。网络中共有75万多个派系，平均派系的大小为14，最大派系的大小为26。也就是说，与Tim的好友完全连接的最大人数是26，其中有6个不同的派系，如示例4-12的示例输出所示。同一个人占这6个派系总人数的20/26。你可能会认为这些成员是Tim好友网络的基础。一般来说，能够在图中发现派系（尤其是最大派系和最大完备子图）可以让你深入了解复杂数据。对一个大派系成员之间tweet内容的相似性/多样性分析将会是tweet分析中非常宝贵的经验。（更多的tweet分析，请见下一章。）

示例4-12：示例4-11的示例输出，说明了最大完备子图的常见成员

```
Num cliques: 762573
Avg clique size: 14
Max clique size: 26
Num max cliques: 6

People in all max cliques:
[
    "kwerb",
    "johnbattelle",
    "dsearls",
    "JPBarlow",
    "cshirky",
    "leolaporte",
    "MParekh",
    "mkapor",
    "steverubel",
    "stevenbjohnson",
    "LindaStone",
    "godsdog",
    "Joi",
    "jayrosen_nyu",
    "dweinberger",
    "timoreilly",
    "ev",
    "jason_pontin",
    "kevinmarks",
    "Mlsif"
]

Max cliques:
[
    [
        "timoreilly",
        "anildash",
        "cshirky",
        "ev",
```

```
        "pierre",
        "mkapor",
        "johnbattelle",
        "kevinmarks",
        "MParekh",
        "dsearls",
        "kwerb",
        "Joi",
        "LindaStone",
        "dweinberger",
        "SteveCase",
        "leolaporte",
        "steverubel",
        "Borthwick",
        "godsdog",
        "edyson",
        "dangillmor",
        "Mlsif",
        "JPBarlow",
        "stevenbjohnson",
        "jayrosen_nyu",
        "jason_pontin"
    ],

    ...

    ]
```

显然，除了检测派系之外，我们能做的还有更多。在地图上绘制包含在派系中的用户的地理位置，来查看紧密相连的人际网络和当地地理位置之间是否存在任何联系，分析他们的个人资料数据和/或tweet内容中的信息也有可能。下一章都是关于tweet的分析，所以当你读到这部分内容时，你可能会再次冒出这一想法。但是首先，让我们简要探讨一下Infochimps提供的有趣的Web API，我们可以把它作为我们已经做过的分析的完整性检查。

Infochimp“强链接”API

Infochimps（*http://infochimps.org*）是一个提供大型数据目录的组织。在它的很多产品之间是一个巨大的Twitter历史数据存档和用于操作数据的各种增值API。其中更有趣的Twitter度量和分析API（*http://api.infochimps.com/describe/soc/net/tw*）是强链接API，它返回一个与查询的用户交流最频繁的用户列表。要访问这个API，只需要注册一个免费账户来获取一个API密钥，然后在URL上执行简单的GET请求即可。唯一的缺点是结果提供的是用户ID值，需要把它解析成昵称才有用[注6]。幸运的是，使用本章前面的现有代码很容易做到这一点。示例4-13的脚本说明了如何使用Infochimps“强链接”API，并使用Redis来从用户ID值中解析出昵称。

注6：一个Infochimps“Who Is” API（*http://api.infochimps.com/describe/soc/net/tw/whois*）即将出现，将会提供将用户ID解析为昵称的方法。

示例4-13：从Infochimps“强链接”API中抓取数据（friends_followers_infochimps_strong_links.py）

```
# -*- coding: utf-8 -*-

import sys
import urllib2
import json
import redis

from twitter__util import getRedisIdByUserId

SCREEN_NAME = sys.argv[1]
API_KEY = sys.argv[2]
API_ENDPOINT = \
    'http://api.infochimps.com/soc/net/tw/strong_links.json?screen_name=%s&apikey=%s'

r = redis.Redis() # default connection settings on localhost

try:
    url = API_ENDPOINT % (SCREEN_NAME, API_KEY)
    response = urllib2.urlopen(url)
except urllib2.URLError, e:
    print 'Failed to fetch ' + url
    raise e

strong_links = json.loads(response.read())

# resolve screen names and print to screen:

print "%s's Strong Links" % (SCREEN_NAME, )
print '-' * 30
for sl in strong_links['strong_links']:
    if sl is None:
        continue
    try:
        user_info = json.loads(r.get(getRedisIdByUserId(sl[0], 'info.json')))
        print user_info['screen_name'], sl[1]
    except Exception, e:
        print >> sys.stderr, "ERROR: couldn't resolve screen_name for", sl
        print >> sys.stderr, "Maybe you haven't harvested data for this person yet?"
```

有趣的是，包含在Tim的6个最大完备子图中的20个人，只有*@kevinmarks*出现在结果集中。这似乎意味着你在Twitter中和好友联系紧密并不意味着你经常直接与他们交流。然而，快速浏览一下下一章，你会发现在表5-2中，Tim经常转发的一些Twitter用户确实出现在了强链接列表中，即*@ahier*、*@gnat*、*@jamesoreilly*、*@pahlkadot*、*@OReillyMedia*和*@monkchips*。（请注意，*@monkchips*是强链接列表中的第二名，如示例4-14所示。）Infochimps确实没有明确指出什么是它的强链接API计算，只说明它是建立在实际的tweet内容基础上的，对这些内容的分析是下一章的主题。现在学到的经验是Twitter中紧密连接的友谊并不一定暗示频繁的交流。（而且，Infochimps提供了绝对值得一看的有趣的API。）

示例4-14：示例4-13中来自于Infochimps强链接API的示例结果

```
timoreilly's Strong Links
------------------------------
jstan 20.115004
monkchips 11.317813
govwiki 11.199023
ahier 10.485066
ValdisKrebs 9.384349
SexySEO 9.224745
tdgobux 8.937282
Scobleizer 8.406802
cheeky_geeky 8.339885
seanjoreilly 8.182084
pkedrosky 8.154991
OReillyMedia 8.086607
FOSSwiki 8.055233
n2vip 8.052422
jamesoreilly 8.015188
pahlkadot 7.811676
ginablaber 7.763785
kevinmarks 7.7423387
jcantero 7.64023
gnat 7.6349654
KentBottles 7.40848
Bill_Romanos 7.3629074
make 7.326427
carlmalamud 7.3147154
rivenhomewood 7.276802
webtechman 7.1044493
```

交互式3D图的可视化

虽然深入研究图布局算法有点超出了本书的范围，但是很难放弃介绍Ubigraph（*http://ubietylab.net/ubigraph/*）的机会，它是3D交互式图的一个可视化工具。虽然可视化派系没有任何的分析价值，但是它是了解这部分内容的好方法。Ubigraph易于安装，并且和许多流行的编程语言绑定在一起，如Python。示例4-15说明了使用Ubigraph绘制一个网络中最大完备子图之间常见成员的图。坏消息是Windows用户可能需要考虑运行基于Linux的虚拟机，因为不可能任何时候都能构建Ubigraph服务器。好消息是即使你没有高深的Linux知识，将它们组合起来也并不困难。

示例4-15：使用Ubigraph可视化图数据（friends_followers_ubigraph.py）

```
# -*- coding: utf-8 -*-

import sys
import json
import networkx as nx

# Packaged with Ubigraph in the examples/Python directory
import ubigraph
```

```
SCREEN_NAME = sys.argv[1]
FRIEND = sys.argv[2]

g = nx.read_gpickle(SCREEN_NAME + '.gpickle')

cliques = [c for c in nx.find_cliques(g) if FRIEND in c]
max_clique_size = max([len(c) for c in cliques])
max_cliques = [c for c in cliques if len(c) == max_clique_size]

print 'Found %s max cliques' % len(max_cliques)
print json.dumps(max_cliques, indent=4)

U = ubigraph.Ubigraph()
U.clear()
small = U.newVertexStyle(shape='sphere', color='#ffff00', size='0.2')
largeRed = U.newVertexStyle(shape='sphere', color='#ff0000', size='1.0')

# find the people who are common to all cliques for visualization

vertices = list(set([v for c in max_cliques for v in c]))
vertices = dict([(v, U.newVertex(style=small, label=v)) for v in vertices if v
                not in (SCREEN_NAME, FRIEND)])

vertices[SCREEN_NAME] = U.newVertex(style=largeRed, label=SCREEN_NAME)
vertices[FRIEND] = U.newVertex(style=largeRed, label=FRIEND)

for v1 in vertices:
    for v2 in vertices:
        if v1 == v2:
            continue
        U.newEdge(vertices[v1], vertices[v2])
```

总而言之，只要创建一幅图并加入顶点和边即可。然而，请注意，和NetworkX不同的是，节点并不是由它们的标签来定义的，所以必须注意不要在同一幅图中添加重复的节点。前面的例子说明了通过遍历一个经过略微定制的字典中的顶点来创建图，这样可以更容易看到某些节点。按照前面示例的模式，这个特殊的列表将会可视化由一个特定用户（该例中为Tim O'Reilly）和一个好友（由FRIEND定义）共享的最大完备子图中的常见成员，也就是说，最大的派系包含两个约束节点。图4-3显示了运行中的Ubigraph屏幕截图。和本书中其他所有内容一样，这仅是最低限度的介绍，旨在激励你走出去后再大显身手。

小结

本章才开始接触到Twitter数据处理的皮毛，其关注的重点是可以被发现的好友/关注者的关系。将本章中所有功能联系在一起的一个命令行工具的示例实现可以在*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/TwitterSocialGraphUtility.py*中得到，并且应该可以根据你自己的需要对其进行简单的改写。（如果你这样做，请考虑在GitHub上晒一晒，并将你所做的更改秀给大家。）一些有趣的想法请参见如下的练习：

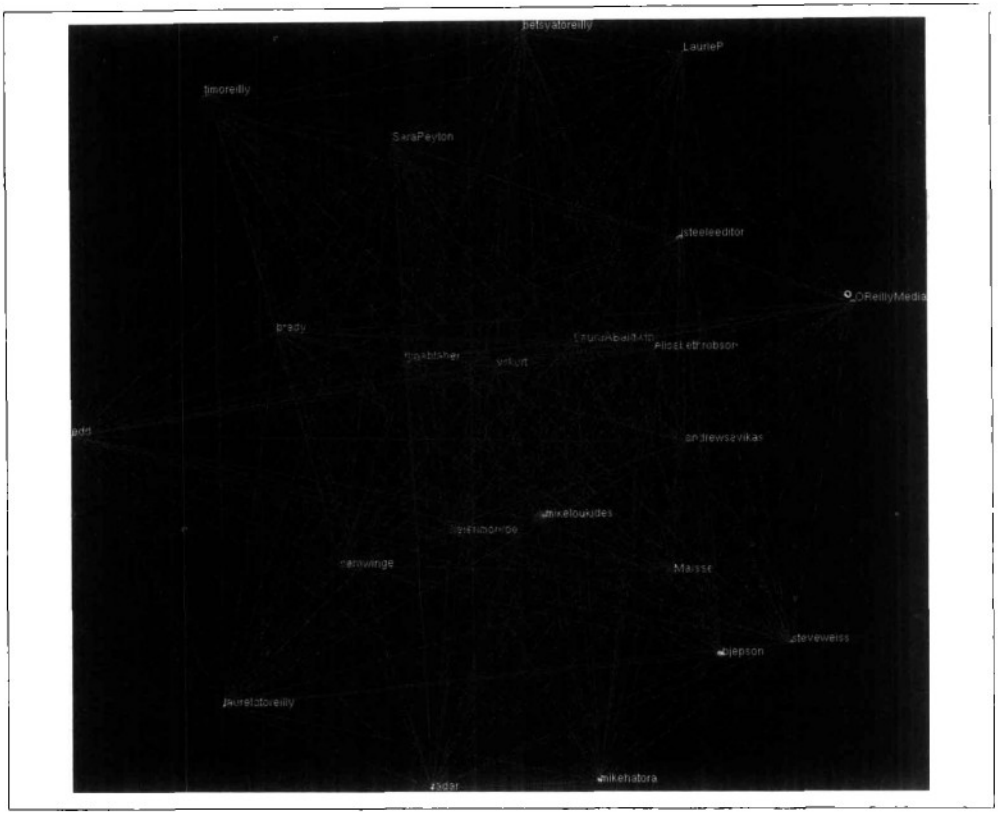

图4-3：对出现在最大完备子图中共同成员的一个交互式3D可视化截屏，同时包含@timoreilly和@mikeloukides（这里没有正确截图；你真正需要交互地尝试这种截图！）

- 本章没有利用geo数据。试着绘制出你的关注者所处地理位置的直方图，然后在在线地图上将它可视化。第6章主要使用geo数据，并提供可能会使用的一些映射技术的适当介绍。
- 试着对你的好友/关注者用户对象中的描述字段进行一些基本词频分析。第7章和第8章提供了你认为可能很有用的工具和技术。
- 试着创建你的每个关注者tweet数量的直方图，来看看谁是最爱闲聊的。该数据可以从用户对象的`statues_count`中得到。
- 本章没有讨论列表（一种集合用户的方法，通常是因为他们共享的一些共同主线）。查看列表中的API文档（*http://dev.twitter.com/doc*），编写一些代码来试着确定谁已经把你（或你欣赏的其他Twitter用户）加入列表，以及列表中用户之间的相似性是什么。一个好的起点是：通过Twitter的公共Web接口，直接查看出现在列表中的用户。

第5章

Twitter：tweet，所有的tweet，只有tweet

tweet和RT坐在篱笆上。tweet离开了。谁留下来了？

本章会大量使用CouchDB的map/reduce功能，使用tweet中的实体（*@mentions*，#hashtags等）来试着回答“大家在讨论什么？”这个问题。由于每天远远超过5000万条tweet的总吞吐量（*http://mashable.com/2010/02/22/twitter-50-million-tweets/*）和偶尔超过3000条/秒的tweet峰值速度（*http://blog.twitter.com/2010/06/another-big-record-part-deux.html*），挖掘tweet内容的潜力巨大，从本章开始，我们将开始挖掘tweet。第4章主要关注存在于朋友和关注者之间的社交图谱联系，而本章通过检查Twitter用户的tweet中的实体（*http://dev.twitter.com/pages/tweet_entities*），来尽可能了解他们。你也可以回到Redis中查看它们之间的关系，来访问从第4章获取的数据和用于图谱分析的NetworkX。tweet这么多，我们的时间却这么少——让我们开始吧！

注意： 阅读本章之前，强烈建议你先阅读第3章和第4章。本章讨论的大部分内容都建立在这两章的基础上，包括本章中再次使用的Redis和CouchDB。

笔PK剑：和tweet PK机枪（?!?）

如果文胜于武，那么应该如何评论tweet呢？有很多Twitter挽救了生命的有趣事件，最臭名昭著的就是James Karl Bulk著名的“被捕了”tweet（*http://techcrunch.com/2008/04/16/twitter-saves-man-from-egyptian-justice/*），这使他在被埃及政府拘留后，迅速被释放了出来。找到类似的事件并不难，而且Twitter已经被无数次用于高尚的募捐等慈善活动了。有一个信息表达渠道有时真的会产生巨大的差异。然而，你的主时间轴（tweet流）和公共时间轴常常会充满不太生动或不太耐人寻味的信息。这时，从搜索API中删掉这

些“垃圾”可以帮助你看到你想看的内容。由于多达50%的tweet中都至少包含一个tweet作者特意构思的实体（*http://dev.twitter.com/pages/tweet_entities*），因此它们为tweet分析提供了非常合理的起点。事实上，Twitter已经认识到它们的价值，已经开始直接在时间轴API调用中显示它们了，从2010年初开始，它们正在逐渐成为整个Twitter中最标准的API。考虑示例5-1中的tweet，它是通过选择在查询中指定的`include_entities=true`参数，从时间轴API调用中取得的。

示例5-1：从说明了tweet实体的搜索API中获得的样例tweet

```
{
    "created_at" : "Thu Jun 24 14:21:11 +0000 2010",
    "entities" : {
                    "hashtags" : [
                        {   "indices" : [ 97, 103 ],
                            "text" : "gov20"
                        },
                        {   "indices" : [ 104, 112 ],
                            "text" : "opengov"
                        }
                    ],
                    "urls" : [
                        {   "expanded_url" : null,
                            "indices" : [ 76, 96 ],
                            "url" : "http://bit.ly/9o4uoG"
                        }
                    ],
                    "user_mentions" : [
                        {   "id" : 28165790,
                            "indices" : [ 16, 28 ],
                            "name" : "crowdFlower",
                            "screen_name" : "crowdFlower"
                        }
                    ]
     },
    "id" : 16932571217,
    "text" : "Great idea from @crowdflower: Crowdsourcing the Goldman ... #opengov",
    "user" : {
        "description" : "Founder and CEO, O'Reilly Media. Watching the alpha ...",
        "id" : 2384071,
        "location" : "Sebastopol, CA",
        "name" : "Tim O'Reilly",
        "screen_name" : "timoreilly",
        "url" : "http://radar.oreilly.com",
       }
}
```

默认情况下，一条tweet可以通过`status`对象中的`user`字段，指出其作者的很多有用的信息，但是tweet实体使我们进一步了解了tweet内容。通过简单地查看这条样例tweet，我们可以肯定地推断：*@timoreilly*可能对开放政府和Government 2.0的转变话题感兴趣，它由tweet中的hashtag表示。也可以肯定地推断：*@crowdflower*可能与Government 2.0有

关，此URL可能指向与此相关的内容。因此，如果要以自动的方式发现tweet作者的一些其他信息，可以考虑从*@timoreilly*转到*@crowdflower*，研究该用户的tweet或个人信息，搜索包含在tweet中的hashtag来看看会弹出什么其他信息，或者点击链接并分析页面，了解这条tweet的更多背景。

由于分析tweet实体可以获取这么多价值，你会非常想在API或者从越来越容易挖掘的Twitter数据历史存档中看到它们。然而，不必手动将它们从文本中解析出来（当tweet包含任意Unicode字符时，手动解析并不容易），只要运行`easy_install twitter-text-py`命令[注1]即可，这样你就可以关注更有趣的问题了。示例5-2的脚本说明了Extractor类的基本用法，它产生的结构与时间轴API显示的结构类似。在tweet实体成为默认值之前，用这种方式自动嵌入实体有百利而无一害。

注意： 截止2010年10月，API中的tweet实体正在变得越来越常见，但并没有被官方“授权”，也不是规范。本章假设你想知道自己如何将它们解析出来，但是你应该认识到即时了解Twitter最新的API可能会让你省点事。在你从Infochimps或GNIP这样的机构中挖掘历史档案时，人工提取tweet实体可能有很大的帮助。

示例5-2：使用tweet_text包帮助提取tweet实体（the_tweet_extract_tweet_entities.py）

```
# -*- coding: utf-8 -*-

import sys
import json
import twitter_text
import twitter
from twitter__login import login

# Get a tweet id clicking on a status right off of twitter.com.
# For example, http://twitter.com/#!/timoreilly/status/17386521699024896

TWEET_ID = sys.argv[1]

# You may need to setup your OAuth settings in twitter__login.py
t = login()

def getEntities(tweet):

    # Now extract various entities from it and build up a familiar structure

    extractor = twitter_text.Extractor(tweet['text'])

    # Note that the production Twitter API contains a few additional fields in
    # the entities hash that would require additional API calls to resolve
```

注1：twitter-text-py模块是twitter-text-rb模块的一个端口（都可以通过GitHub得到），Twitter会将它用于生产中（*http://dev.twitter.com/pages/tweet_entities*）。

```
    entities = {}
    entities['user_mentions'] = []
    for um in extractor.extract_mentioned_screen_names_with_indices():
        entities['user_mentions'].append(um)

    entities['hashtags'] = []
    for ht in extractor.extract_hashtags_with_indices():

        # massage field name to match production twitter api

        ht['text'] = ht['hashtag']
        del ht['hashtag']
        entities['hashtags'].append(ht)

    entities['urls'] = []
    for url in extractor.extract_urls_with_indices():
        entities['urls'].append(url)

    return entities

# Fetch a tweet using an API method of your choice and mixin the entities

tweet = t.statuses.show(id=TWEET_ID)

tweet['entities'] = getEntities(tweet)

print json.dumps(tweet, indent=4)
```

现在，有了tweet实体概述和一些有趣的可能性，让我们开始采集并分析一些tweet吧！

对tweet的分析（每次一个实体）

CouchDB是很好的采集tweet的存储介质，因为，和第3章的电子邮件一样，它们被方便地表示为基于JSON的文档，适合于映射/化简分析。我们的下一个示例脚本从时间轴中采集tweet，该脚本相对强壮，而且很容易被理解，因为所有的模块和大部分代码已经在前面几章介绍过了。确认它的一个巧妙的考虑是它使用了简单的映射/化简作业来计算tweet的最大ID值，将它作为查询限制传递，这样就可以避免从Twitter API中提取重复数据了。更多详情，请查看时间轴API中与since_id参数相关的信息。

注意到从用户时间轴中得到的最新tweet的最大数量约是3200条，而主时间轴[注2]返回约800个状态也是有益的；因此，提取所有可用的数据并不是很昂贵（从统计速率限制数量方面来说）。当第一次与时间轴API交互时，不太直观的是请求公众时间轴数据只返回20条tweet的事实，这些tweet每60秒更新一次。为了收集更多数据，需要使用信息流

注2：Twitter API文档指出，好友时间轴与主时间轴类似，只是它不包含向后兼容的转发tweet。

API。

例如，如果你想多了解一下“硅谷最受欢迎的聪明人”[注3]Tim O'Reilly，你需要确保CouchDB正在运行，然后按下面的方法调用示例5-3的脚本：

```
$ python the_tweet_varvest_timeline.py uses 16 timoreilly
```

虽然为了你的分析需要，大约得收集3200条tweet，但这只需要很短的时间就能完成。

示例5-3：从用户或公众时间轴中采集tweet（the_tweet_harvest_timeline.py）

```
# -*- coding: utf-8 -*-

import sys
import time
import twitter
import couchdb
from couchdb.design import ViewDefinition
from twitter__login import login
from twitter__util import makeTwitterRequest

def usage():
    print 'Usage: $ %s timeline_name [max_pages] [user]' % (sys.argv[0], )
    print
    print '\ttimeline_name in [public, home, user]'
    print '\t0 < max_pages <= 16 for timeline_name in [home, user]'
    print '\tmax_pages == 1 for timeline_name == public'
    print 'Notes:'
    print '\t* ~800 statuses are available from the home timeline.'
    print '\t* ~3200 statuses are available from the user timeline.'
    print '\t* The public timeline updates once every 60 secs and returns 20 statuses.'
    print '\t* See the streaming/search API for additional options to harvest tweets.'

    exit()

if len(sys.argv) < 2 or sys.argv[1] not in ('public', 'home', 'user'):
    usage()
if len(sys.argv) > 2 and not sys.argv[2].isdigit():
    usage()
if len(sys.argv) > 3 and sys.argv[1] != 'user':
    usage()

TIMELINE_NAME = sys.argv[1]
MAX_PAGES = int(sys.argv[2])

USER = None

KW = { # For the Twitter API call
    'count': 200,
    'skip_users': 'true',
    'include_entities': 'true',
```

注3：查看2010年5月《Inc.》杂志的封面（*http://www.inc.com/magazine/20100501/the-oracle-of-silicon-valley.html*）。

```
    'since_id': 1,
    }
if TIMELINE_NAME == 'user':
    USER = sys.argv[3]
    KW['id'] = USER # id or screen name
if TIMELINE_NAME == 'home' and MAX_PAGES > 4:
    MAX_PAGES = 4
if TIMELINE_NAME == 'user' and MAX_PAGES > 16:
    MAX_PAGES = 16
if TIMELINE_NAME == 'public':
    MAX_PAGES = 1

t = login()

# Establish a connection to a CouchDB database

server = couchdb.Server('http://localhost:5984')
DB = 'tweets-%s-timeline' % (TIMELINE_NAME, )

if USER:
    DB = '%s-%s' % (DB, USER)

try:
    db = server.create(DB)
except couchdb.http.PreconditionFailed, e:

    # Already exists, so append to it, keeping in mind that duplicates could occur

    db = server[DB]

    # Try to avoid appending duplicate data into the system by only retrieving tweets
    # newer than the ones already in the system. A trivial mapper/reducer combination
    # allows us to pull out the max tweet id which guards against duplicates for the
    # home and user timelines. It has no effect for the public timeline

    def idMapper(doc):
        yield (None, doc['id'])

    def maxFindingReducer(keys, values, rereduce):
        return max(values)

    view = ViewDefinition('index', 'max_tweet_id', idMapper, maxFindingReducer,
                          language='python')
    view.sync(db)
    KW['since_id'] = int([_id for _id in db.view('index/max_tweet_id')][0].value)

# Harvest tweets for the given timeline.
# For friend and home timelines, the unofficial limitation is about 800 statuses although
# other documentation may state otherwise. The public timeline only returns 20 statuses
# and gets updated every 60 seconds.
# See http://groups.google.com/group/twitter-development-talk/browse_thread/
# thread/4678df70c301be43
# Note that the count and since_id params have no effect for the public timeline

page_num = 1
while page_num <= MAX_PAGES:
    KW['page'] = page_num
```

```
    api_call = getattr(t.statuses, TIMELINE_NAME + '_timeline')
    tweets = makeTwitterRequest(t, api_call, **KW)
    db.update(tweets, all_or_nothing=True)
    print 'Fetched %i tweets' % len(tweets)
    wait_period = 2
    page_num += 1
```

已经有了收集tweet的基本工具，现在让我们开始看看我们能发现哪些有用的信息吧。

对（Tim的）Tweet的利用

这一节探讨了一些最常见的问题，它们是从Tim O'Reilly的用户时间轴中挖取出的简单堆架式实体。虽然Tim是为了教育的目的才同意将自己置于“显微镜”之下供众位读者观摩分析，但是你可以放心地在你自己的tweet中重用这些脚本，或者供其他对此感兴趣Twitter用户使用。或者，可以从收集来作为研究基础的数量相当大的公众时间轴数据开始。一些我们将会讨论的有趣的问题包括：

- 最常出现在Tim的tweet中的用户实体，有多少也是他的好友？
- Tim的tweet中最常出现的实体是什么？
- Tim最常转发谁的tweet？
- Tim的tweet中有多少条被转发了？
- Tim的tweet中有多少至少包含一个实体？

和很多包含相对未知数据源的其他情况一样，要了解更多信息，首先需要统计它包含的内容的总数。就tweet数据而言，统计用户mention、hashtag和URL的总数是好的开始。下一节通过利用一些基本的映射/化简功能计算tweet实体的总数。虽然tweet实体已经存在于时间轴数据中了，但是本章剩余部分的一些代码示例假设你已经从其他地方（搜索API、信息流API等）获取了数据。因此，解析出tweet实体，使它们保持良好一致的形式。依靠Twitter提取tweet实体只需要调整几行代码。

Tim的tweet中包含哪些实体

可以通过访问*http://localhost:5984/_utils*，使用CouchDB的Futon浏览Tim的用户时间轴数据，但是你很快会发现自己对这些结果并不满意，你想要获取的是Tim发了什么信息的总述，而不是某些事的mention列表。初步的答案是映射/化简作业值得一试，使用示例5-4所示的脚本很容易就可以计算出所需的结果。

示例5-4：从tweet中提取实体，执行简单的频率分析（the_tweet_count_entities_in_tweets.py）

```
# -*- coding: utf-8 -*-
```

```
import sys
import couchdb
from couchdb.design import ViewDefinition
from prettytable import PrettyTable

DB = sys.argv[1]

server = couchdb.Server('http://localhost:5984')
db = server[DB]

if len(sys.argv) > 2 and sys.argv[2].isdigit():
    FREQ_THRESHOLD = int(sys.argv[2])
else:
    FREQ_THRESHOLD = 3

# Map entities in tweets to the docs that they appear in

def entityCountMapper(doc):
    if not doc.get('entities'):
        import twitter_text
        def getEntities(tweet):

            Now extract various entities from it and build up a familiar structure

            extractor = twitter_text.Extractor(tweet['text'])

            # Note that the production Twitter API contains a few additional fields in
            # the entities hash that would require additional API calls to resolve

            entities = {}
            entities['user_mentions'] = []
            for um in extractor.extract_mentioned_screen_names_with_indices():
                entities['user_mentions'].append(um)

            entities['hashtags'] = []
            for ht in extractor.extract_hashtags_with_indices():

                # Massage field name to match production twitter api

                ht['text'] = ht['hashtag']
                del ht['hashtag']
                entities['hashtags'].append(ht)

            entities['urls'] = []
            for url in extractor.extract_urls_with_indices():
                entities['urls'].append(url)

            return entities

        doc['entities'] = getEntities(doc)

    if doc['entities'].get('user_mentions'):
        for user_mention in doc['entities']['user_mentions']:
            yield ('@' + user_mention['screen_name'].lower(), [doc['_id'], doc['id']])
    if doc['entities'].get('hashtags'):
        for hashtag in doc['entities']['hashtags']:
            yield ('#' + hashtag['text'], [doc['_id'], doc['id']])
    if doc['entities'].get('urls'):
```

```
        for url in doc['entities']['urls']:
            yield (url['url'], [doc['_id'], doc['id']])

def summingReducer(keys, values, rereduce):
    if rereduce:
        return sum(values)
    else:
        return len(values)

view = ViewDefinition('index', 'entity_count_by_doc', entityCountMapper,
                      reduce_fun=summingReducer, language='python')
view.sync(db)

# Print out a nicely formatted table. Sorting by value in the client is cheap and easy
# if you're dealing with hundreds or low thousands of tweets
entities_freqs = sorted([(row.key, row.value) for row in
                         db.view('index/entity_count_by_doc', group=True)],
                         key=lambda x: x[1], reverse=True)

fields = ['Entity', 'Count']
pt = PrettyTable(fields=fields)
[pt.set_field_align(f, 'l') for f in fields]

for (entity, freq) in entities_freqs:
    if freq > FREQ_THRESHOLD:
        pt.add_row([entity, freq])

pt.printt()
```

请注意，虽然不使用rereduce参数，也可以创建一个不怎么有用的索引来计算频率计数，但是构建更有用的索引提供了使用rereduce的机会，对于任何重要的映射/化简作业来说，它都是重要的考虑因素。

对rereduce的说明

示例5-4是我们看到的第一个显式使用了rereduce参数的清单，所以它可能有利于解释究竟发生了什么。请记住，除了减少一些mapper的输出之外（必须按键分组），一些reducer也可能将几个reducer的输出传递给rereduce。在统计和计算总和这类常用功能中，输入来自哪里也许并没有什么差别，只要能反复使用加法这类交换运算即可。然而，有时候，初化简和再化简之间的差别很重要。这时，rereduce参数就很有用。

对于示例5-4中所示的summingReducer函数，考虑mapper中的以下样例输出（每个元组的值部分的实际内容是什么并不重要）：

```
[ ["@foo", [x1, x2]], ["@foo", [x3, x4]], ["@bar", [x5, x6]],
```

```
["@foo", [x7, x8]], ("@bar", [x9, x10]] ]
```

为了简单起见，让我们假设存储这些元组的基础B树中的每个节点一次只能存储两项（CouchDB实现的实际值已经过千）。从概念上来说，当rereduce是False时，reducer在第一次传递过程中作用于如下输入。再提醒一遍，值是按键分组的：

```
# len(  [  [x1, x2],  [x3, x4]  ]  ) == 2
summingReducer(["@foo", "@foo"],  [  [x1, x2], [x3, x4]  ], False)

# len( [  [x7, x8] ]   ) => 1
summingReducer(["@foo"], [  [x7, x8]  ], False)

# len(  [  [x5, x6], [x9, x10]  ]  ) == 2
summingReducer(["@bar", "@bar"], [  [x5, x6],  [x9, x10]  ], False)
```

当rereduce为True时，在reducer中第二次传递期间，因为化简函数操作的是已经按键分组的、已化简的输出，没有相关的键值比较是必要的。由于前面的len操作只是统计了每个键（一个tweet实体）出现的次数，通过reducer的其他传递需要计算这些值的总数来计算最终结果，如下面的rereduce阶段所示：

```
# values from previously reducing @foo keys: sum([2, 1]) == 3
summingReducer(None, [ 2, 1 ], True)

# values from previously reducing @bar keys: sum([2]) == 2
summingReducer(None, [ 2 ], True)
```

总的来说：作为一个中间步骤，len第一次用于统计实体出现的次数；然后，在最后一步中，Sum函数计算这些中间结果。

简单来说，这段脚本使用mapper来为每个文档生成(entity，[couchdb_id,tweet_id])形式的元组，然后使用reducer统计不同的实体出现的次数。由于可能使用相对较小的项集合，还有第3章介绍的一些其他的排序方法，然后，简单地对客户端的数据排序并使用某个频率阈值。阈值为15的示例输出如表5-1所示，它同时表示在图5-1中，这样你就可以了解基本的分布了。

表5-1：从@timoreilly的tweet中获取的、按频率排序的实体

实体	频率
#gov20	140
@OReillyMedia	124
#Ebook	89
@timoreilly	77
#ebooks	55

表5-1：从@timoreilly的tweet中获取的、按频率排序的实体（续）

实体	频率
@slashdot	45
@jamesoreilly	41
#w2e	40
@gnat	38
@n2vip	37
@monkchips	33
#w2s	31
@pahlkadot	30
@dalepd	28
#g2e	27
#ebook	25
@ahier	24
#where20	22
@digiphile	21
@fredwilson	20
@brady	19
@mikeloukides	19
#pdf10	19
@nytimes	18
#fooeast	18
@andrewsavikas	17
@CodeforAmerica	16
@make	16
#pkedrosky	16
@carlmalamud	15
#make	15
#opengov	15

那么，Tim这些天在关注什么呢？他最喜欢的一些主题出现在列表中并不奇怪（例如，#gov20的出现次数高达140次，远远高于其他实体），但更耐人寻味的是考虑一些不太明显的用户提及，这可能表明密切的关系。实际上，可以假设Tim发现他提到的用户很

有趣。你甚至可以推断他被这些用户影响或者甚至相信这些用户。（可以设计很多有趣的算法试着确定这些类型的关系，它们有不同程度的确定性。）虽然在这一节计算的

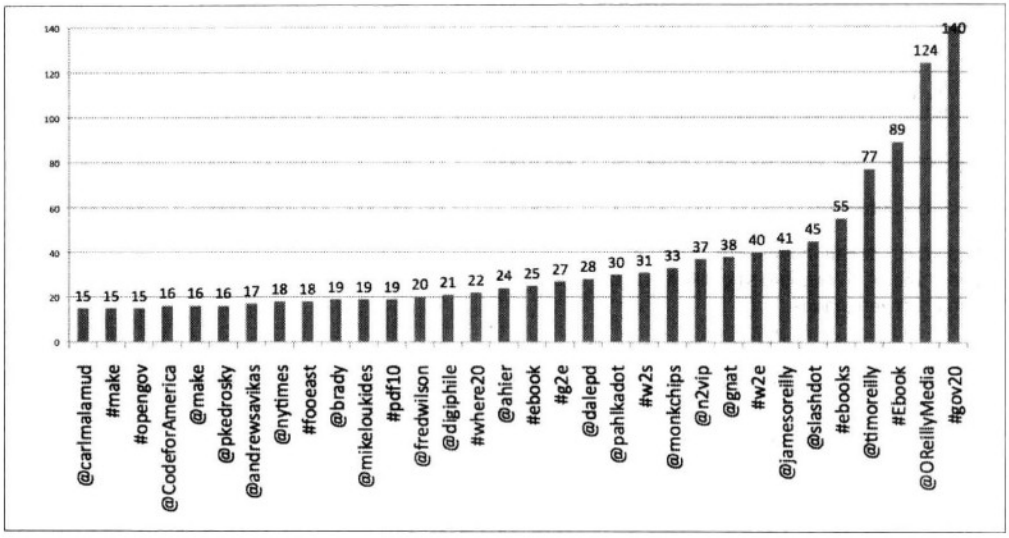

图5-1：@timoreilly转发的最新tweet示例实体的频率

原始记数很有趣，但是使用基于时间的过滤方法是另一种吸引人的可能性。例如，可以通过在示例5-4的映射/化简代码中使用基于时间的过滤器来获得有用的启示，这样我们就可以计算过去的*N*天中Tim在谈论什么话题；而不必分析他所有的约3200条tweet（发这么多tweet需要的时间远大于*N*天）。（一些Twitter用户几年都没有发3200条tweet。）在3.4节介绍的SIMILE Timeline上划分tweet、浏览它们来了解Tim最近最常发什么tweet并不困难。

频繁出现的用户实体意味着友谊吗

示例4-4提供了说明如何获取Twitter用户的好友和关注者清单，并且使用Redis来存储结果。假设你已经使用示例4-4的清单抓取了Tim的好友和关注者，那么你就可以从Redis中得到结果了，而且很容易就可以计算出最频繁发tweet消息的*N*个用户实体中到底有多少也是Tim的好友。示例5-5说明了使用第4章中存储在Redis中的信息来解析用户ID昵称的过程；Redis内存中的集合运算用来计算最常出现在Tim的用户时间轴中的哪些用户实体也是他的好友。

示例5-5：查找也是好友的@mention tweet实体（the_tweet_how_many_user_entities_are_friends.py）

```
# -*- coding: utf-8 -*-
```

```
import json
import redis
import couchdb
import sys
from twitter__util import getRedisIdByScreenName
from twitter__util import getRedisIdByUserId

SCREEN_NAME = sys.argv[1]
THRESHOLD = int(sys.argv[2])

# Connect using default settings for localhost

r = redis.Redis()

# Compute screen_names for friends
friend_ids = r.smembers(getRedisIdByScreenName(SCREEN_NAME, 'friend_ids'))
friend_screen_names = []
for friend_id in friend_ids:
    try:
        friend_screen_names.append(json.loads(r.get(getRedisIdByUserId(friend_id,
                                   'info.json')))['screen_name'])
    except TypeError, e:
        continue # not locally available in Redis - look it up or skip it

# Pull the list of (entity, frequency) tuples from CouchDB

server = couchdb.Server('http://localhost:5984')
db = server['tweets-user-timeline-' + SCREEN_NAME]

entities_freqs = sorted([(row.key, row.value) for row in
                         db.view('index/entity_count_by_doc', group=True)],
                         key=lambda x: x[1])

# Keep only user entities with insufficient frequencies

user_entities = [(ef[0])[1:] for ef in entities_freqs if ef[0][0] == '@'
                 and ef[1] >= THRESHOLD]

# Do a set comparison

entities_who_are_friends = \
    set(user_entities).intersection(set(friend_screen_names))

entities_who_are_not_friends = \
    set(user_entities).difference(entities_who_are_friends)

print 'Number of user entities in tweets: %s' % (len(user_entities), )
print 'Number of user entities in tweets who are friends: %s' \
    % (len(entities_who_are_friends), )
for e in entities_who_are_friends:
    print '\t' + e
print 'Number of user entities in tweets who are not friends: %s' \
    % (len(entities_who_are_not_friends), )
for e in entities_who_are_not_friends:
    print '\t' + e
```

频率阈值为15（如示例5-6所示）的输出是可以预见的，然而还是会有一些有价值的新发现。

示例5-6：示例5-5的示例输出，显示了也是@timoreilly好友的@mention tweet实体

```
Number of user entities in tweets: 20
Number of user entities in tweets who are friends: 18
    ahier
    pkedrosky
    CodeforAmerica
    nytimes
    brady
    carlmalamud
    pahlkadot
    make
    jamesoreilly
    andrewsavikas
    gnat
    slashdot
    OReillyMedia
    dalepd
    mikeloukides
    monkchips
    fredwilson
    digiphile
Number of user entities in tweets who are not friends: 2
    n2vip
    timoreilly
```

总的来说，有20个用户实体超过了值为15的频率阈值，其中的18人被证明是好友。由于大多数出现在Tim的tweet中的人也是他的好友，因此可以放心地说：Tim和这些人之间存在着某种高度的信任关系。花点时间比较一下这个列表和4.3节中的练习结果。然而，同样有趣的可能是注意到和*@n2vip*一样，Tim自己也出现在了最常在tweet中谈论的实体中。更仔细地研究包含*@n2vip*的tweet上下文可能会很有用。是否有一些理论可以支持“在某人的tweet流中多次提到的用户，但是他们并非好友”？让我们找找看。

从第3章的内容，你已经知道了在CouchDB数据库中使用Lucene的全文索引功能是多么快速简单，对示例3-12的最小改编和扩展是为了进一步快速了解提到*@n2vip*的tweet。示例5-7说明了它是如何完成的。

示例5-7：使用couchdb-lucence查询tweet数据（the_tweet_couchdb_lucene.py）

```
# -*- coding: utf-8 -*-

import sys
import httplib
from urllib import quote
import json
import couchdb

DB = sys.argv[1]
```

```
QUERY = sys.argv[2]

# The body of a JavaScript-based design document we'll create

dd = \
    {'fulltext': {'by_text': {'index': '''function(doc) {
                            var ret=new Document();
                            ret.add(doc.text);
                            return ret
                        }'''}}}

try:
server = couchdb.Server('http://localhost:5984')
db = server[DB]
except couchdb.http.ResourceNotFound, e:
print """CouchDB database '%s' not found.
Please check that the database exists and try again.""" % DB
    sys.exit(1)

try:
conn = httplib.HTTPConnection('localhost', 5984)
conn.request('GET', '/%s/_design/lucene' % (DB, ))
response = conn.getresponse()
finally:
conn.close()

# If the design document did not exist create one that'll be
# identified as "_design/lucene". The equivalent of the following
# in a terminal:
# $ curl -X PUT http://localhost:5984/DB/_design/lucene -d @dd.json
if response.status == 404:
try:
conn = httplib.HTTPConnection('localhost', 5984)
conn.request('PUT', '/%s/_design/lucene' % (DB, ), json.dumps(dd))
response = conn.getresponse()

if response.status != 201:
    print 'Unable to create design document: %s %s' % (response.status,
    response.reason)
    sys.exit(1)
finally:
    conn.close()

# Querying the design document is nearly the same as usual except that you reference
# couchdb-lucene's _fti HTTP handler
# $ curl http://localhost:5984/DB/_fti/_design/lucene/by_subject?q=QUERY

try:
conn.request('GET', '/%s/_fti/_design/lucene/by_text?q=%s' % (DB,
    quote(QUERY)))
response = conn.getresponse()
if response.status == 200:
response_body = json.loads(response.read())
else:
print 'An error occurred fetching the response: %s %s' \
% (response.status, response.reason)
    print 'Make sure your couchdb-lucene server is running.'
```

```
    sys.exit(1)
finally:
    conn.close()

doc_ids = [row['id'] for row in response_body['rows']]
# pull the tweets from CouchDB and extract the text for display
tweets = [db.get(doc_id)['text'] for doc_id in doc_ids]
for tweet in tweets:
    print tweet
    print
```

该段脚本的缩略输出如示例5-8所示，表示@*n2vip*出现在@*timoreilly*的这么多tweet中，是因为他们两个正在进行Twitter对话。

示例5-8：示例5-7的样例输出

```
@n2vip Thanks. Great stuff. Passing on to the ebook team.
@n2vip I suggested it myself the other day before reading this note.
RT @n2vip Check this out if you really want to get your '#Churchill on', a ...
@n2vip Remember a revolution that began with a handful of farmers and tradesmen ...
@n2vip Good suggestion re having free sample chapters as ebooks, not just pdfs...
@n2vip I got those statistics by picking my name off the influencer list in ...
RT @n2vip An informative, non-partisan FAQ regarding Health Care Reform at ...
@n2vip Don't know anyone who is advocating that. FWIW, it was Rs who turned ...
@n2vip No, I don't. But a lot of the people arguing against renewables seem ...
@n2vip You've obviously never read an ebook on the iPhone. It's a great reading ...
@n2vip I wasn't suggesting that insurance was the strange world, just that you ...
@n2vip In high tech, there is competition from immigrant workers. Yet these two ...
@n2vip How right you are. We really don't do a good job teaching people ...
@n2vip The climategate stuff is indeed disturbing. But I still hold by what ...
@n2vip FWIW, I usually do follow links, so do include them if appropriate. Thanks.
@n2vip I don't mind substantive disagreement - e.g. with pointers to real info ...
@n2vip Totally agree that ownership can help. But you need to understand why ...
@n2vip Maybe not completely extinct, but certainly economically extinct. E.g. ...
@n2vip I wasn't aware that it was part of a partisan agenda. Too bad, because ...
RT @n2vip if only interesed in his 'Finest Hour' speech, try this - a ...
@n2vip They matter a lot. I was also struck by that story this morning. Oil ...
@n2vip I understand that. I guess "don't rob MYsocialized medicine to fund ...
RT @n2vip Electronic medical record efforts pushing private practice docs to ...
@n2vip I think cubesail can be deployed in 2 ways: to force quicker re-entry ...
RT @ggreenwald Wolf Blitzer has major epiphany on public opinion and HCR that ...
```

拼接另一半对话

似乎@*n2vip*的很多次提及都与他至少参与了Tim的两次对话这个事实有关，快速浏览这些评论可以看出，这些对话似乎有点政治性质（这并不奇怪，因为在本章前面的练习中已经注意到，#gov20 hashtag出现了多次。）有趣的后继练习是扩展示例5-7来提取tweet中的`in_reply_to_status_id`字段，重组出更完整和可读的对话。

重构“最小化API调用次数的线程”的一个技巧就是简单地抓取@*n2vip*的所有tweet，这些tweet ID比感兴趣的最小tweet ID值大，并且是一次性抓取全部ID而不是一个个地收集状态ID。示例5-9说的就是这一做法。

示例5-9：重构tweet会话线程（the_tweet_reassemble_discussion_thread.py）

```
# -*- coding: utf-8 -*-

import sys
import httplib
from urllib import quote
import json
import couchdb
from twitter__login import login
from twitter__util import makeTwitterRequest

DB = sys.argv[1]
USER = sys.argv[2]

try:
    server = couchdb.Server('http://localhost:5984')
    db = server[DB]
except couchdb.http.ResourceNotFound, e:
    print >> sys.stderr, """CouchDB database '%s' not found.
Please check that the database exists and try again.""" % DB
    sys.exit(1)

# query by term

try:
    conn = httplib.HTTPConnection('localhost', 5984)
    conn.request('GET', '/%s/_fti/_design/lucene/by_text?q=%s' % (DB,
                 quote(USER)))
    response = conn.getresponse()
    if response.status == 200:
        response_body = json.loads(response.read())
    else:
        print >> sys.stderr, 'An error occurred fetching the response: %s %s' \
            % (response.status, response.reason)
        sys.exit(1)
finally:
    conn.close()

doc_ids = [row['id'] for row in response_body['rows']]

# pull the tweets from CouchDB

tweets = [db.get(doc_id) for doc_id in doc_ids]

# mine out the in_reply_to_status_id fields and fetch those tweets as a batch request

conversation = sorted([(tweet['_id'], int(tweet['in_reply_to_status_id']))
                       for tweet in tweets if tweet['in_reply_to_status_id']
                       is not None], key=lambda x: x[1])
min_conversation_id = min([int(i[1]) for i in conversation if i[1] is not None])
```

```
max_conversation_id = max([int(i[1]) for i in conversation if i[1] is not None])

# Pull tweets from other user using user timeline API to minimize API expenses...

t = login()

reply_tweets = []
results = []
page = 1
while True:
    results = makeTwitterRequest(t,
        t.statuses.user_timeline,
        count=200,
        # Per <http://dev.twitter.com/doc/get/statuses/user_timeline>, some
        # caveats apply with the oldest id you can fetch using "since_id"
        since_id=min_conversation_id,
        max_id=max_conversation_id,
        skip_users='true',
        screen_name=USER,
        page=page)
    reply_tweets += results
    page += 1
    if len(results) == 0:
        break

# During testing, it was observed that some tweets may not resolve or possibly
# even come back with null id values -- possibly a temporary fluke. Workaround.
missing_tweets = []
for (doc_id, in_reply_to_id) in conversation:
    try:
        print [rt for rt in reply_tweets if rt['id'] == in_reply_to_id][0]['text']
    except Exception, e:
        print >> sys.stderr, 'Refetching <<tweet %s>>' % (in_reply_to_id, )
        results = makeTwitterRequest(t, t.statuses.show, id=in_reply_to_id)
        print results['text']

    # These tweets are already on hand
    print db.get(doc_id)['text']
    print
```

该段代码的很多部分应该很眼熟。有了验证Twitter API、在本地存储和检索数据、从Twitter中获取远程数据的能力，可以用最少的“业务逻辑”做很多事情。此段脚本的缩略示例输出如示例5-10所示，显示了*@timoreilly*和*@n2vip*之间的讨论流。

示例5-10：示例5-9的样例输出

```
Question: If all Ins. Co. suddenly became non-profit and approved ALL Dr. ...
@n2vip Don't know anyone who is advocating that. FWIW, it was Rs who turned ...

@timoreilly RT @ggreenwald Wolf Blitzer has major epiphany on public opinion ...
RT @ggreenwald Wolf Blitzer has major epiphany on public opinion and HCR that ...

@timoreilly RE: Cubesail - I don't get it, does the sail collect loose trash ...
@n2vip I think cubesail can be deployed in 2 ways: to force quicker re-entry ...

@timoreilly How are you finding % of your RT have links? What service did you ...
```

```
@n2vip I got those statistics by picking my name off the influencer list in ...

@timoreilly a more fleshed-out e-book 'teaser' chapter idea here: http://bit.ly/aML6eH
@n2vip Thanks. Great stuff. Passing on to the ebook team.

@timoreilly Tim, #HCR law cuts Medicare payments to fund, in part, broader ...
@n2vip I understand that. I guess "don't rob MYsocialized medicine to fund ...

@timoreilly RE: Auto Immune - a "revolution" that is measured by a hand-full ...
@n2vip Remember a revolution that began with a handful of farmers and tradesmen ...

Do oil spills in Africa not matter? http://bit.ly/bKqvO1 @jaketapper @yunjid ...
@n2vip They matter a lot. I was also struck by that story this morning. Oil ...
```

Tim最常转发谁的tweet

古语有云（并非如此）“转发tweet是最高形式的恭维”，“Tim最常转发谁的tweet”这个问题的另一种说法是：“Tim最常恭维谁？”或者，因为他不可能转发他不感兴趣的内容，所以我们可以这样问：“Tim认为谁在谈论重要的事情？”合理的假设是：出现在Tim的公共时间轴中的很多用户实体可能是Tim转发的tweet的作者。让我们进一步探讨这个想法，计算Tim的tweet中有多少条是转发的消息，在这些转发的tweet中，哪个tweet作者最常被转发。有很多收集转发的tweet消息的API方法，但是由于我们已经从之前的练习中获得了数千条tweet，因此让我们分析这些tweet，作为从我们的API调用中取得最大值的方法。

转发的tweet有两种基本模式：

- RT *@user* Mary有一只小羊
- Mary有一只小羊（通过*@user*）

两种情况下的含义都很简单：有人给*@user*提供了特殊的信息。示例5-11通过作为简单的映射/化简例程一部分的正则表达式匹配器，提取出Tim转发了其他Twitter用户tweet的次数。

警告： 请注意，在本书的编写过程中，Twitter的API一直在快速发展。示例5-11通过从tweet文本中提取“RT”这类线索，来统计转发的tweet条数。当你阅读这部分内容时，可能已经有更有效的方法来计算一段时间内其他用户的tweet被转发的次数。

示例5-11：计算Twitter用户被某人转发的次数（the_tweet_count_retweets_of_other_user.py）

```
# -*- coding: utf-8 -*-

import sys
import couchdb
```

```
from couchdb.design import ViewDefinition
from prettytable import PrettyTable

DB = sys.argv[1]

try:
    server = couchdb.Server('http://localhost:5984')
    db = server[DB]
except couchdb.http.ResourceNotFound, e:
    print """CouchDB database '%s' not found.
Please check that the database exists and try again.""" % DB
    sys.exit(1)

if len(sys.argv) > 2 and sys.argv[2].isdigit():
    FREQ_THRESHOLD = int(sys.argv[2])
else:
    FREQ_THRESHOLD = 3

# Map entities in tweets to the docs that they appear in

def entityCountMapper(doc):
    if doc.get('text'):
        import re
        m = re.search(r"(RT|via)((?:\b\W*@\w+)+)", doc['text'])
        if m:
            entities = m.groups()[1].split()
            for entity in entities:
                yield (entity.lower(), [doc['_id'], doc['id']])
        else:
            yield ('@', [doc['_id'], doc['id']])

def summingReducer(keys, values, rereduce):
    if rereduce:
        return sum(values)
    else:
        return len(values)

view = ViewDefinition('index', 'retweet_entity_count_by_doc', entityCountMapper,
                      reduce_fun=summingReducer, language='python')
view.sync(db)

# Sorting by value in the client is cheap and easy
# if you're dealing with hundreds or low thousands of tweets

entities_freqs = sorted([(row.key, row.value) for row in
                         db.view('index/retweet_entity_count_by_doc',
                         group=True)], key=lambda x: x[1], reverse=True)

fields = ['Entity', 'Count']
pt = PrettyTable(fields=fields)
[pt.set_field_align(f, 'l') for f in fields]

for (entity, freq) in entities_freqs:
    if freq > FREQ_THRESHOLD and entity != '@':
        pt.add_row([entity, freq])

pt.printt()
```

如果你认为结果与表5-1中的原始实体计数值基本一致的话，你可能会感到惊讶。表5-2中的列表也使用了阈值15来对比它们之间的差异。

表5-2：出现在@timoreilly转发的tweet中的最常见实体，其他列用来说明@timoreilly转发的tweet总数的标准化

实体	@timoreilly 转发@user tweet的次数	@user发布的tweet总条数	标准的转发评分
@monkchips	30	33215	0.000903206
@ahier	18	14849	0.001212203
@slashdot	41	22081	0.0018568
@gnat	28	11322	0.002473061
@mikeloukides	15	2926	0.005126452
@pahlkadot	16	3109	0.005146349
@oreillymedia	97	6623	0.014645931
@jamesoreilly	34	4439	0.007659383
@dalepd	16	1589	0.010069226

所以，Tim最常转发谁的tweet/恭维谁？不要惊讶，大部分的列表项是与他的公司密切相关的同事和他的公司，其中，次数最多的是*@oreillymedia*。目测一下tweet转发次数和原始实体数，就能发现相互之间存在很明显的相关性。然而，请记住，表5-2中“被*@timoreilly* 转发的*@user tweet*次数”这一列显示的结果并没有标准化。例如，*@dalepd*被转发的次数远远少于*@oreillymedia*，但是对tweet中内嵌的用户对象中的`statuses_count`字段（在API结果的其他地方）的简单检查表明*@dalepd*发tweet的频率远远小于*@oreillymedia*。实际上，如果你用每个用户转发的tweets总数除以tweets总数来标准化数据的话，那么*@dalepd*的排名仅次于*@oreillymedia*——这意味着除了*@oreillymedia*之外，Tim是表5-2中最常转发tweet的人，但是如果你按原始转发tweet频率排序的话，并不一定会出现这样的结果。图5-2用气泡图说明了这个分析。确定为什么有的人转发tweet没有像他们自己所说的那样频繁，这也是个很有趣的话题。

因为并不难确定Tim最常转发谁的tweet，所以用另一种方式问同样的问题怎么样：谁最常转发Tim的tweet？如果没有明确的目标用户群，由于Twitter API的速率限制，要想从总体上回答这个问题就比较困难，因为Tim约有150万个关注者。然而，如果你将它缩小到合理的小目标人群，就会更容易处理。下一节将会研究其中一些选项。

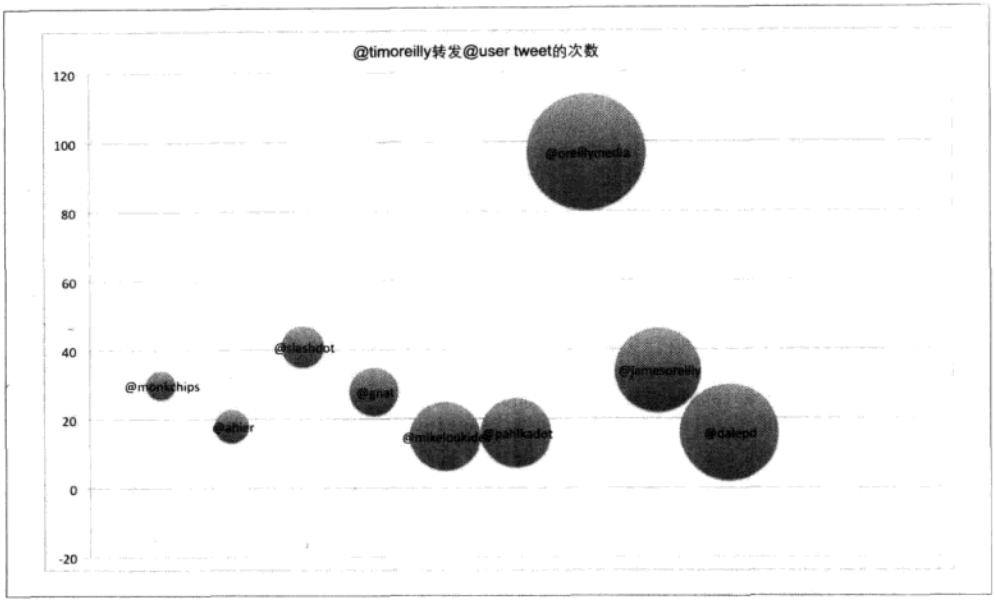

图5-2：该气泡图的y轴表示用户被转发的原始次数；气泡的面积表示与用户发的tweet总条数相比，他被转发次数的标准分数

Tim的影响力

询问你的多少条tweet被转发了实际上是度量你自己影响力的另一种方法。如果你发了很多条tweet，但是没人转发你的tweet，完全可以说你几乎没有任何影响力——至少作为Twitter用户来说是这样。实际上，发现自己有很多关注者而被转发的次数却很少是自相矛盾的，因为你获得关注和你的tweet被别人转发原因是一样的：你不仅很有趣而且很有影响力。

一个非常简单、便宜的基本指标是计算被转发的tweet比率。比率为1表示你编写的每条tweet都被转发了，表明你有很强的影响力（很可能会有二次、三次转发，这样很容易就能达到数百万用户），而比率接近于0表示较弱的影响力。当然，由于Twitter的性质，任何用户被转发的tweet比率都不可能为1，若只是由于对话的机制（*@replies*），则比率将会下降。Twitter提供了`statuses/retweets_of_me`资源，认证用户可以使用它来了解自己的哪些tweet被转发了。然而，我们并没有访问这个API来分析Tim被转发的tweet，所以我们需要寻找其他出路。

示例5-12使用tweet中的`retweet_count`字段来计算被转发了给定次数的tweet总数，作为现在看起来应该像一个简单的映射/化简组合的一部分。样例输出被格式化为示例5-12后面的图。

示例5-12：查找最常被转发的tweet（the_tweet_count_retweets_by_others.py）

```
# -*- coding: utf-8 -*-

import sys
import couchdb
from couchdb.design import ViewDefinition
from prettytable import PrettyTable
from twitter__util import pp

DB = sys.argv[1]

try:
    server = couchdb.Server('http://localhost:5984')
    db = server[DB]
except couchdb.http.ResourceNotFound, e:
    print """CouchDB database '%s' not found.
Please check that the database exists and try again.""" % DB
    sys.exit(1)

# Map entities in tweets to the docs that they appear in

def retweetCountMapper(doc):
    if doc.get('id') and doc.get('text'):
        yield (doc['retweet_count'], 1)

def summingReducer(keys, values, rereduce):
    return sum(values)

view = ViewDefinition('index', 'retweets_by_id', retweetCountMapper,
                      reduce_fun=summingReducer, language='python')
view.sync(db)

fields = ['Num Tweets', 'Retweet Count']
pt = PrettyTable(fields=fields)
[pt.set_field_align(f, 'l') for f in fields]

retweet_total, num_tweets, num_zero_retweets = 0, 0, 0
for (k,v) in sorted([(row.key, row.value) for row in
                     db.view('index/retweets_by_id', group=True)
                     if row.key is not None],
                    key=lambda x: x[0], reverse=True):
    pt.add_row([k, v])

    if k == "100+":
        retweet_total += 100*v
    elif k == 0:
        num_zero_retweets += v
    else:
        retweet_total += k*v

    num_tweets += v

pt.printt()

print '\n%s of %s authored tweets were retweeted at least once' % \
    (pp(num_tweets - num_zero_retweets), pp(num_tweets),)
```

```
print '\t(%s tweet/retweet ratio)\n' % \
      (1.0*(num_tweets - num_zero_retweets)/num_tweets,)

print 'Those %s authored tweets generated %s retweets' % (pp(num_tweets), pp(retweet_total),)
```

图5-3显示了被格式化为更直观的图的示例5-12的样例结果，图5-3使用对数刻度来压缩*y*轴。*x*轴的值与被*y*轴表示的确定的转发次数相对应。总的“曲线面积”超过了3000——被分析的tweet总数。例如，最左边的柱形表示Tim的533条tweet完全没有被转发，50条tweet被转发了50次，最右边的柱形表示超过60多条tweet被转发了100多次[注4]。

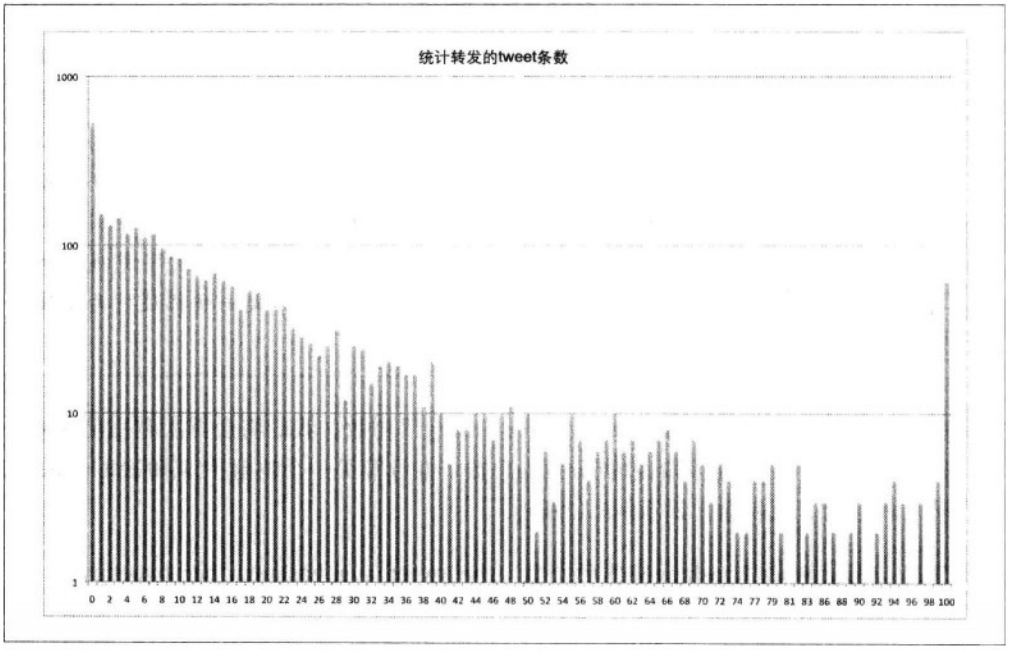

图5-3：示例5-12的示例结果

这个分布并不是太让人惊讶，因为它一般按照幂法则发展，有很多tweet被快速传播、被转发了几百次。粗略的估算是这3000多条tweet中，有2536条至少被转发了一次（比率约为0.80），总共生成了5万多条转发的tweet（比例因子约为16）。

至少可以说，这些数字证实了Tim作为有影响力的信息专家的地位。

注4：请注意，截止2010年10月底，retweet_count字段的最大值都是100。对于这一批数据来说，有两条tweet被转发了100次，59条tweet被转发了“100+”次。

Tim的tweet中有多少包含hashtag

“包含hashtag实体的tweets本质上比不包含hashtag实体的tweet具有更高的价值”这个理论似乎很合理，因为某人特意将聚合信息嵌入到了这些tweet中，而实质上只是将它们转换为半结构化信息。例如，这样的假设看上去似乎很有道理：在每条tweet中平均有两个以上hashtag的用户对于“传播知识”是很感兴趣的，同时他们也很清楚信息的力量；然而每条tweet中hashtag的平均数在0.1以下的用户对此不太感兴趣。

公众分类法

人类智力的一个基本方面是将事情分类并推导出分层结构，其中，每个元素都“属于”分层结构中较高层次的父节点或者是它的“子元素”。先不考虑分类法和本体论之间区别的哲学辩论，分类法本质上是将元素分类为父/子关系的分层结构。公众分类法（*http://en.wikipedia.org/wiki/Folksonomy*）这个术语大约是在2004年杜撰出来的，用于作为描述协作标签领域和出现在各种Web生态系统中的社交索引成就的手段，从它融合“公众”和“分类法”的意义上来说，它是一种文字游戏。所以，当你允许人们用标签将内容分类时，公众分类法本质上只是一种描述作为集体智慧机制出现的标签分散领域的新奇方法。

对你来说，现在很容易计算每条tweet中的hashtag平均数。我们将会再次利用前面的一些代码，计算某个映射/化简阶段的hashtag总数，并计算另一个映射/化简阶段的tweet总数，然后将这两个数相除，如示例5-13所示。

示例5-13：统计tweet中hashtag实体的个数（the_tweet_avg_hashtags_per_tweet.py）

```
# -*- coding: utf-8 -*-

import sys
import couchdb
from couchdb.design import ViewDefinition

DB = sys.argv[1]

try:
    server = couchdb.Server('http://localhost:5984')
    db = server[DB]
except couchdb.http.ResourceNotFound, e:
    print """CouchDB database '%s' not found.
Please check that the database exists and try again.""" % DB
    sys.exit(1)

# Emit the number of hashtags in a document

def entityCountMapper(doc):
    if not doc.get('entities'):
        import twitter_text
```

```
        def getEntities(tweet):

            # Now extract various entities from it and build up a familiar structure

            extractor = twitter_text.Extractor(tweet['text'])

            # Note that the production Twitter API contains a few additional fields in
            # the entities hash that would require additional API calls to resolve

            entities = {}
            entities['user_mentions'] = []
            for um in extractor.extract_mentioned_screen_names_with_indices():
                entities['user_mentions'].append(um)

            entities['hashtags'] = []
            for ht in extractor.extract_hashtags_with_indices():

                # Massage field name to match production twitter api

                ht['text'] = ht['hashtag']
                del ht['hashtag']
                entities['hashtags'].append(ht)

            entities['urls'] = []
            for url in extractor.extract_urls_with_indices():
                entities['urls'].append(url)
            return entities

        doc['entities'] = getEntities(doc)

    if doc['entities'].get('hashtags'):
        yield (None, len(doc['entities']['hashtags']))

def summingReducer(keys, values, rereduce):
    return sum(values)

view = ViewDefinition('index', 'count_hashtags', entityCountMapper,
                      reduce_fun=summingReducer, language='python')
view.sync(db)

num_hashtags = [row for row in db.view('index/count_hashtags')][0].value

# Now, count the total number of tweets that aren't direct replies

def entityCountMapper(doc):
    if doc.get('text')[0] == '@':
        yield (None, 0)
    else:
        yield (None, 1)

view = ViewDefinition('index', 'num_docs', entityCountMapper,
                      reduce_fun=summingReducer, language='python')
view.sync(db)

num_docs = [row for row in db.view('index/num_docs')][0].value

# Finally, compute the average
```

```
print 'Avg number of hashtags per tweet for %s: %s' % \
       (DB.split('-')[-1], 1.0 * num_hashtags / num_docs,)
```

对于我们刚才抓取的一批tweet，该脚本的运行表明Tim的每条tweet中平均包含0.5个hashtag，它不是对某人的直接答复。也就是说，他的一半tweet中包括hashtag。对于经常发tweet的用户来说，大多数时候，包含hashtag都给整体Twitter搜索索引和不断变化的公众分类法作出了重大贡献。作为后续练习，计算每条tweet中链接实体的平均数可能会非常有趣，或许甚至是按照链接，通过检查链接的网页中的标题或内容，试着找出Tim兴趣的新信息。（在后面的几章中，特别是第7章和第8章，我们将介绍更多作为分析网页基本技能的文本挖掘。）

并置潜在的社交网站（或#JustinBieber VS #TeaParty）

数据挖掘最吸引人的方面是：它可以让你从现有的信息中发现新知识。的确如古语所说“知识就是力量”，在这个信息量稳步增长而没有下降趋势的时代尤其如此。作为有趣的练习，让我们看看从Twitter数据海洋中，我们可以发现哪些潜在的社交网络。在此，我们采用的基本方法是：通过搜索特定的hashtag，以特定的方式收集不少于两种主题的关键数据，然后使用上一节（5.2.4节）编写的指标来了解网络之间的相似性。

因为没有“愚蠢的问题”，所以让我们领会著名的经济学家Steven D.Levitt[注5]的精神，提出这样的问题：“#TeaParty和#JustinBieber的共同点是什么？”[注6]

示例5-14提供了收集关于特定主题的大约1.5万条最新tweet，并将它们存储到CouchDB中的简单机制。和本章中的其他清单一样，它包括简单的映射/化简逻辑来逐渐更新事件中的tweet，你可能想长时间运行它来收集更多数据，这会多于搜索API短期内可以提供的数据。你可能想要研究这类任务的信息流API（*http://dev.twitter.com/pages/streaming_api*）。

注5：Steven D. Levitt是《Freakonomics:A Rogue Economist Explores the Hidden Side of Everything》（Harper）的作者之一，该书系统地使用数据来回答“学校老师和相扑选手的共同点是什么？”这类看似激进的问题。

注6：这个问题的部分灵感来源于Radar的文章：“Data Science democratized”（*http://radar.oreilly.com/2010/07/data-science-democratized.html*），其中提到了研究这个问题的陈述报告（*http://www.slideshare.net/bigdatacamp/stefan-groschupf-of-datameer-gives-lightning-talk-at-big-datacamp-4667387*）。

示例5-14：获取某个查询的tweet（the_tweet_search.py）

```
# -*- coding: utf-8 -*-

import sys
import twitter
import couchdb
from couchdb.design import ViewDefinition
from twitter__util import makeTwitterRequest

SEARCH_TERM = sys.argv[1]
MAX_PAGES = 15

KW = {
    'domain': 'search.twitter.com',
    'count': 200,
    'rpp': 100,
    'q': SEARCH_TERM,
    }

server = couchdb.Server('http://localhost:5984')
    DB = 'search-%s' % (SEARCH_TERM.lower().replace('#', '').replace('@', ''), )

try:
    db = server.create(DB)
except couchdb.http.PreconditionFailed, e:

    # already exists, so append to it, and be mindful of duplicates

    db = server[DB]

t = twitter.Twitter(domain='search.twitter.com')

for page in range(1, 16):
    KW['page'] = page
    tweets = makeTwitterRequest(t, t.search, **KW)
    db.update(tweets['results'], all_or_nothing=True)
    if len(tweets['results']) == 0:
        break
    print 'Fetched %i tweets' % len(tweets['results'])
```

下面几节都基于3000条左右tweet中的每个主题，假设你已经运行了这段脚本来收集#TeaParty和#JustinBieber的相关数据（或者其他让你感兴趣的主题）。

警告： 根据你的终端偏好，你可能需要转义某些字符（如hash符号），因为如果不转义它们的话，它们可能会被shell解释。例如，在Bash中，需要将#TeaParty的hashtag查询转义为\#TeaParty，确保shell把这个hash符号解释为查询术语的一部分，而不是把它作为一条注释的开始。

#JustinBieber和#TeaParty的tweet中最常共同出现的实体

描述两个不同群体最简单、最有效的方法是分析出现在tweet聚合池中的实体。除了可以告诉你每个群体讨论的其他主题之外，你还可以比较共同出现的实体来作出非常基

本的相似性度量。示例5-4已经提供了在实体分析中执行第一次传递所需的逻辑。假设你已经为#JustinBieber和#TeaParty运行了搜索查询，那么你应该会有两个名为“search-justinbieber”和“search-teaparty”的CouchDB数据库，你可以传递参数来产生你自己的结果。每个实体频率超过20的hashtag的样例结果如表5-3和表5-4所示；图5-4显示了这些表的基本频率分布图。因为y轴包含一些极值，为了使其更易阅读，把它调整为对数刻度。

表5-3：出现在包含#TeaParty的tweet中最常见的实体

实体	频率
#teaparty	2834
#tcot	2426
#p2	911
#tlot	781
#gop	739
#ocra	649
#sgp	567
#twisters	269
#dnc	175
#tpp	170
#GOP	150
#iamthemob	123
#ucot	120
#libertarian	112
#obama	112
#vote2010	109
#TeaParty	106
#hhrs	104
#politics	100
#immigration	97
#cspj	96
#acon	91
#dems	82
#palin	79

表5-3：出现在包含#TeaParty的tweet中最常见的实体（续）

实体	频率
#topprog	78
#Obama	74
#tweetcongress	72
#jcot	71
#Teaparty	62
#rs	60
#oilspill	59
#news	58
#glennbeck	54
#FF	47
#liberty	47
@welshman007	45
#spwbt	44
#TCOT	43
http://tinyurl.com/24h36zq	43
#rnc	42
#military	40
#palin12	40
@Drudge_Report	39
@ALIPAC	35
#majority	35
#NoAmnesty	35
#partriottweets	35
@ResistTyranny	34
#tsot	34
http://tinyurl.com/386k5hh	31
#conservative	30
#AZ	29
#TopProg	29
@JIDF	28

表5-3：出现在包含#TeaParty的tweet中最常见的实体（续）

实体	频率
@STOPOBAMA2012	28
@TheFlaCracker	28
#palin2012	28
@thenewdeal	27
#AFIRE	27
#Dems	27
#asamom	26
#GOPDeficit	25
#wethepeople	25
@andilinks	24
@RonPaulNews	24
#ampats	24
#cnn	24
#jews	24
@First_Patriots	23
#patriot	23
#pjtv	23
@Liliaep	22
#nvsen	22
@BrnEyeSuss	21
@crispix49	21
@koopersmith	21
@Kriskxx	21
#Kagan	21
@blogging_tories	20
#cdnpoli	20
#fail	20
#nra	20
#roft	20

表5-4：出现在包含#JustinBiebe的tweet中最常见的实体

实体	频率
#justinbieber	1613
#JustinBieber	1379
@lojadoaltivo	354
@ProSieben	258
#JUSTINBIEBER	191
#Proform	191
http://migre.me/TJwj	191
#JustınBıeber	107
#nowplayıng	104
@justinbıeber	99
#music	88
#tickets	80
*@_Yassi_*	78
#musicmonday	78
#video	78
#Dschungel	74
#Celebrity	42
#beliebers	38
#BieberFact	38
@JustBieberFact	32
@TinselTownDirt	32
@rheinzeitung	28
#WTF	28
http://tinyual.com/343kax4	28
#Telezwerge	26
#Escutando	22
#justinBieber	22
#Restart	22
#TT	22
http://bit.ly/aARD4t	21

表5-4：出现在包含#JustinBiebe的tweet中最常见的实体（续）

实体	频率
http://bit.ly/b2Kc1L	21
#bieberblast	20
#Eclipse	20
#somebodytolove	20

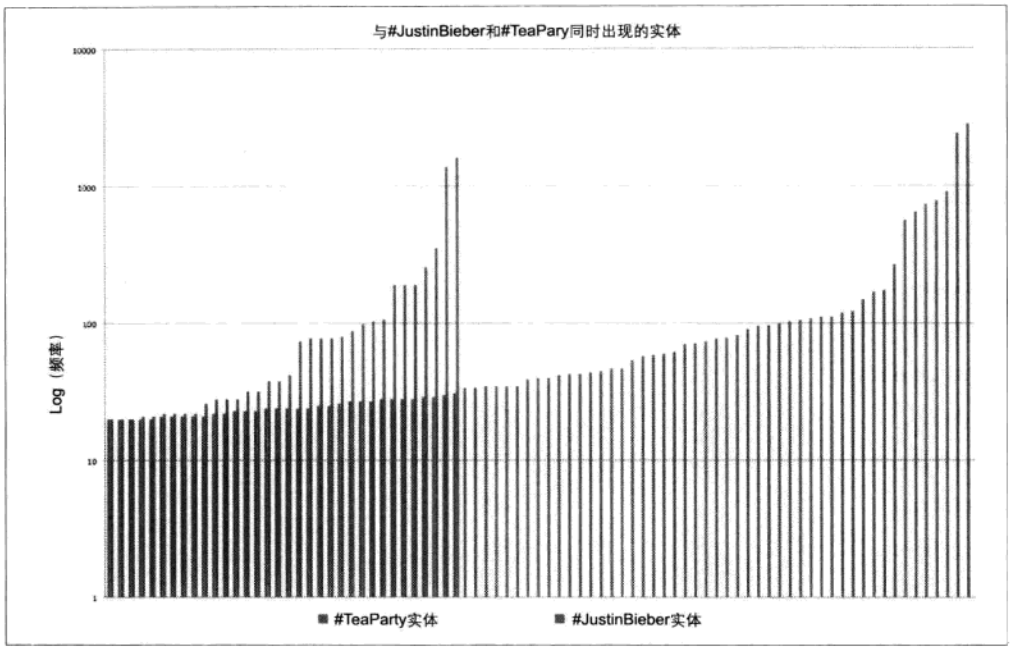

图5-4：共同出现在#JustinBieber和#TeaParty中实体的分布

显而易见的是#TeaParty tweets的曲线面积好像比#JustinBieber tweets的曲线面积更大，尾巴也更长[注7]（如果你可以把它称做尾巴的话）。因此，一眼看上去，#TeaParty tweet的hashtag平均数似乎大于#JustinBieber tweet的。下一节将探讨这一假设，但是在我们继续阅读之前，先仔细观察一下这些结果。对结果粗略的定性估计似乎表明：#TeaParty中编码到实体本身的信息似乎更丰富。例如，在#TeaParty实体中，我们会看到#oilspill、#Obama、#palin、#libertarian和@Drudge_Report（尤其值得一提）这样的主题。相反，

注7：“长尾”或“重尾”指的是统计分布的特征，其中曲线面积的重要部分（通常是50%以上）存在于它的尾巴上。在7.2节中，我们会在“齐普夫定律简要概述”中再次遇到这个概念。

最常出现的#JustinBieber实体只是#JustinBieber的简单变形，剩下的则是一些散乱并且无重点的hashtag。然而，请记住，这并不意外，因为#TeaParty是非常政治性的话题，而#JustinBieber是与流行文化和娱乐相关的话题。

我们还可以观察到：两个用户实体（*@lojadoaltivo*和*@Pro-Sieben*）出现在前几个结果中（高于“官方”*@Justinbieber*账号本身），与#JustinBieber共同出现的很多实体通常都是与演艺界相关的非英语单词或用户实体。

简要分析了这个定性评价后，让我们回到#TeaParty每条tweet中的hashtag是否确实多于#JustinBieber这个问题上来。

平均来说，#JustinBieber或#TeaParty，谁的tweet包含更多hashtag

示例5-13提供了计算每条tweet中hashtag平均数的一种可行方法，很容易将它用于search-justinbieber和search-teaparty数据库中，而不需要做任何其他工作。

对这两个数据库中结果的清点表明：#JustinBieber tweet的每条tweet中平均约有1.95个hashtag，而#TeaParty tweet的每条tweet中约有5.15个hashtag。#TeaParty tweet的hashtag数约为#JustinBieber tweet的2.5倍。虽然这并不是世界上最令人惊讶的发现，但是拥有稳定的数据资料点，并以此为基础来进一步研究或支持推测很有用：它们是可量化的结果，可以随着时间的推移被追踪，或者被其他人共享并重新评估。

虽然这种情况下的差别很惊人，但是要记住：收集的数据是众多Twitter用户通过搜索API，针对每个主题向我们提交的最新的3000条左右的tweet。虽然它可能是一个很好的指标，但它并没有统计意义。不管#TeaParty Twitter用户意识到了没有，他们非常相信公众分类法：显然他们对确信他们的内容很易于让“搜索API和像我们这样的数据收集者”获取并交叉引用感兴趣。

谁更常被转发：#JustinBieber还是#TeaParty

在本章的前面，我们合理地推测出高频率转发的tweet可能更有影响力、包含更多信息或本质上更像社论。“正在吃披萨”和“外国人刚刚降落在白宫前面的草坪上；我们都会死！#fail #apocalypse”这类tweet是极端的内容示例，它们相当不可能或相当可能被转发。如何比较#TeaParty和#JustinBieber被转发的tweet呢？再次分析搜索结果工作集中的*@mentions*会产生有趣的结果。表5-5和表5-6是最常转发#TeaParty和#JustinBieber tweet的用户删减结果，它们使用了频率参数为10的阈值。

表5-5：最常转发#TeaParty tweet的用户

实体	频率
@teapartyleader	10
@dhrxsol1234	11
@HCReminder	11
@ObamaBallBuster	11
@spitfiremurphy	11
@GregWHoward	12
@BrnEyeSuss	13
@Calroofer	13
@grammy620	13
@Herfarm	14
@andilinks	16
@c4Liberty	16
@FloridaPundit	16
@tlw3	16
@Kriskxx	18
@crispix49	19
@JIDF	19
@libertyideals	19
@blogging_tories	20
@Liliaep	21
@STOPOBAMA2012	22
@First_Patriots	23
@RonPauNews	23
@TheFlaCracker	24
@thenewdeal	25
@ResistTyranny	29
@ALIPAC	32
@Drudge_Report	38
@welshman007	39

表5-6：最常转发#JustinBieber tweet的用户

实体	频率
@justinbieber	14
@JesusBeebs	16
@LeePhilipEvans	16
@JustBieberFact	32
@TinselTownDirt	32
@ProSieben	122
@lojadoaltivo	189

如果针对每一话题，你都在约3000条tweet中运行示例5-4来作一些粗略分析的话，那么你会发现#TeaParty tweet中，差不多有1335条是转发的tweet，而#JustinBieber tweet中差不多只有763条是转发的tweet。#TeaParty转发的tweet条数几乎是#JustinBieber的两倍。你还可以观察到：与#JustinBieber的131条转发的tweet相比，拥有400多条转发的tweet的#TeaParty有一条较长的尾巴。从直观上而不是从严格的统计角度来说，这些可能是十分相关的指标，它们以有意义的方式判断不同的兴趣小组。#TeaParty用户似乎更一致地转发tweet内容；然而，转发#JustinBieber tweet内容的用户中，更常转发的是一些局外人。图5-5将表5-5和表5-6的值表示为简单的图。和图5-4一样，y轴采用对数刻度，通过将频率值压缩到更小的垂直空间中，使图更具可读性。

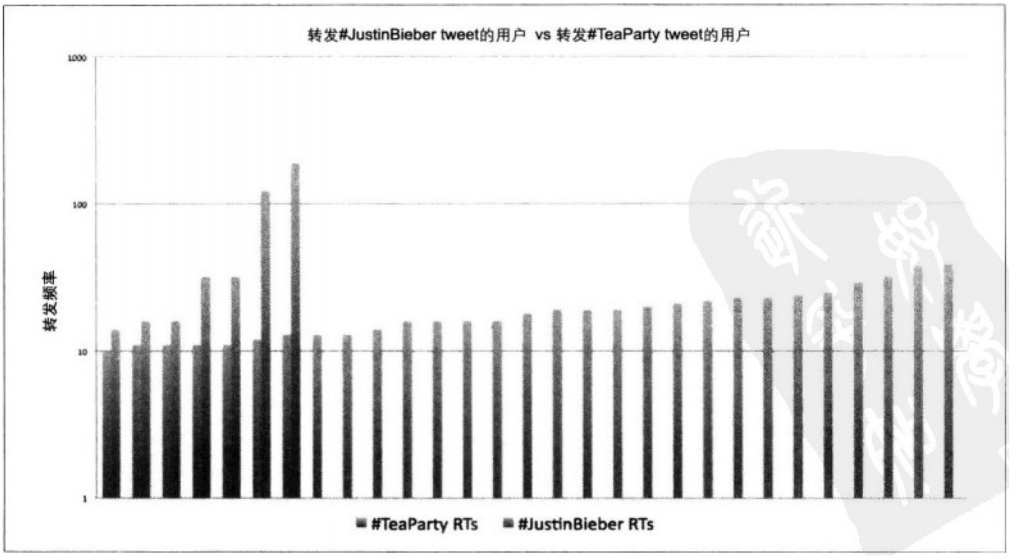

图5-5：转发了#JustinBieber和#TeaParty tweet的用户的分布

#TeaParty和#JustinBieber的tweet实体之间存在多少重叠

让你夜不能寐的最后一个问题是：从#TeaParty和#JustinBieber tweet中解析出来的实体之间存在多少重叠。借鉴第4章的一些想法，本质上我们查找的是这两个实体集的逻辑交集。虽然我们确实可以通过花时间修改现有的Python代码来计算它，但是将我们已有的脚本结果存放到两个文件中，将这两个文件名作为参数传递到一个可用脚本中可能更容易些，该脚本提供了计算任何行分隔的文件交集的通用功能。除了完成这项工作之外，这种方法也会提供给你随时阅读、随时与他人分享的典型产物。假设你在*nix shell中使用了*countentities-in-tweets.py*脚本，从示例5-4的#TeaParty和#JustinBieber输出中获取实体，以指定顺序存储它们：

```
#!/bin/bash

mkdir -p out
for db in teaparty justinbieber; do
    python the_tweet_count_entities_in_tweets.py search-$db 0 | \
    tail +3 | awk '{print $2}' | sort > out/$db.entities
done
```

运行完该脚本后，可以将这两个文件名传递给通用的Python程序来计算输出，如示例5-15所示。

示例5-15：计算文件中的行交集（the_tweet_compute_intersection_of_lines_in_files.py）

```
# -*- coding: utf-8 -*-

"""
Read in 2 or more files and compute the logical intersection of the lines in them
"""

import sys

data = {}
for i in range(1, len(sys.argv)):
    data[sys.argv[i]] = set(open(sys.argv[i]).readlines())

intersection = set()
keys = data.keys()
for k in range(len(keys) - 1):
intersection = data[keys[k]].intersection(data[keys[k - 1]])

msg = 'Common items shared amongst %s:' % ', '.join(keys).strip()
print msg
print '-' * len(msg)
for i in intersection:
    print i.strip()
```

#JustinBieber和#TeaParty共享的实体有些是可预见的，也是很有趣的。示例5-16列出了样例中的结果。

示例5-16：示例5-15中的样例结果

```
Common items shared amongst teaparty.entities, justinbieber.entities:
--------------------------------------------------------------------
#lol
#jesus
#worldcup
#teaparty
#AZ
#milk
#ff
#guns
#WorldCup
#bp
#News
#dancing
#music
#glennbeck
http://www.linkati.com/q/index
@addthis
#nowplaying
#news
#WTF
#fail
#toomanypeople
#oilspill
#catholic
```

#WorldCup、#worldcup和#oilspill出现在结果中并不奇怪，因为它们是很受欢迎的话题；然而，如果你不熟悉茶会运动的话，多少会对#teaparty、#glennbeck、#jesus和#catholic出现在共享的hashtag列表中感到奇怪。通过说明出现在每次搜索中的某些hashtag的频繁直接性，进一步分析可以很容易地确定这两个搜索之间的相关性到底有多强。从这些结果中得到的是：在与#JustinBieber相关的前20个最常见的实体中没有出现常见实体，所以它已经是一个指标了，可以指出它们不在#JustinBieber mention的频率分布的尾巴上。（是的，在列表中出现的#WTF和#fail，尤其是作为两个不同的群体之间的共同体有点有趣。遗憾的是，经历挫折是人类的共同点。）如果你想深入研究的话，重新使用示例5-7作为进一步的练习可以在tweet中执行全文索引，这样就可以按关键字检索了。

对大量tweet的可视化

比本章介绍的可视化Twitter数据更有趣的方法还有很多，但是我们可以通过一些扎实的、较明显的方法来完成练习 。我们特别考虑的是将tweet实体加载到标签云中，并用“图”可视化用户之间的“关系”。

使用高级的标签云可视化tweet

标签云是对tweet中提取的实体的可视化最显然的选择。你可以在Web上找到很多有趣的标签云部件来做所有辛勤的工作，它们都采用相同的输入——本质上是和本章通篇计算的频率分布相似的频率分布。但是当你可以使用可高度定制的基于Flash的旋转标签云时，为什么还要用普通的标签云可视化数据呢？刚好有很受欢迎的开源旋转标签云WP-Cumulus（*http://code.google.com/p/word-cumulus-goog-vis/wiki/UserGuide*）来展示。所需要的只是产生它所期望的简单输入格式，然后将这个输入格式提供给包含标准HTML样板的模板。

示例5-17只是对示例5-4的简单修改，它说明了产生简单的JSON结构（[term,URL,frequency]元组列表）的例程，可以将JSON结构输入到WP-Cumulus的HTML模板中。我们会在这些元组的URL端口输入空字符串，但是你可以在显示了包含实体的tweet列表的简单Web服务中运用你的想象力和超链接。（回想一下，示例5-7通过使用couchdb-lucene，提供了所有你需要的将它连接起来，来对存储在CouchDB中的tweet执行全文检索。）另一种选择可能是编写一个Web服务，将它链接到提供含有指定实体的tweet的URL中。

示例5-17：为使用WP-Cumulus的交互式标签云生成数据（the_tweet_tagcloud_code.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import webbrowser
import json
from cgi import escape
from math import log
import couchdb
from couchdb.design import ViewDefinition

DB = sys.argv[1]
MIN_FREQUENCY = int(sys.argv[2])

HTML_TEMPLATE = '../web_code/wp_cumulus/tagcloud_template.html'
MIN_FONT_SIZE = 3
MAX_FONT_SIZE = 20

server = couchdb.Server('http://localhost:5984')
db = server[DB]

# Map entities in tweets to the docs that they appear in

def entityCountMapper(doc):
    if not doc.get('entities'):
        import twitter_text
```

```
        def getEntities(tweet):

            # Now extract various entities from it and build up a familiar structure

            extractor = twitter_text.Extractor(tweet['text'])

            # Note that the production Twitter API contains a few additional fields in
            # the entities hash that would require additional API calls to resolve

            entities = {}
            entities['user_mentions'] = []
            for um in extractor.extract_mentioned_screen_names_with_indices():
                entities['user_mentions'].append(um)

            entities['hashtags'] = []
            for ht in extractor.extract_hashtags_with_indices():

                massage field name to match production twitter api
                ht['text'] = ht['hashtag']
                del ht['hashtag']
                entities['hashtags'].append(ht)

            entities['urls'] = []
            for url in extractor.extract_urls_with_indices():
                entities['urls'].append(url)

            return entities

        doc['entities'] = getEntities(doc)

    if doc['entities'].get('user_mentions'):
        for user_mention in doc['entities']['user_mentions']:
            yield ('@' + user_mention['screen_name'].lower(), [doc['_id'], doc['id']])
    if doc['entities'].get('hashtags'):
        for hashtag in doc['entities']['hashtags']:
            yield ('#' + hashtag['text'], [doc['_id'], doc['id']])

def summingReducer(keys, values, rereduce):
    if rereduce:
        return sum(values)
    else:
        return len(values)

view = ViewDefinition('index', 'entity_count_by_doc', entityCountMapper,
                      reduce_fun=summingReducer, language='python')
view.sync(db)

entities_freqs = [(row.key, row.value) for row in
                  db.view('index/entity_count_by_doc', group=True)]

# Create output for the WP-Cumulus tag cloud and sort terms by freq along the way

raw_output = sorted([[escape(term), '', freq] for (term, freq) in entities_freqs
                     if freq > MIN_FREQUENCY], key=lambda x: x[2])

# Implementation adapted from
# http://help.com/post/383276-anyone-knows-the-formula-for-font-s
```

```
min_freq = raw_output[0][2]
max_freq = raw_output[-1][2]

def weightTermByFreq(f):
    return (f - min_freq) * (MAX_FONT_SIZE - MIN_FONT_SIZE) / (max_freq
            - min_freq) + MIN_FONT_SIZE

weighted_output = [[i[0], i[1], weightTermByFreq(i[2])] for i in raw_output]

# Substitute the JSON data structure into the template

html_page = open(HTML_TEMPLATE).read() % \
                (json.dumps(weighted_output),)

if not os.path.isdir('out'):
    os.mkdir('out')

f = open(os.path.join('out', os.path.basename(HTML_TEMPLATE)), 'w')
f.write(html_page)
f.close()

print 'Tagcloud stored in: %s' % f.name

# Open up the web page in your browser

webbrowser.open("file://" + os.path.join(os.getcwd(), 'out',
        os.path.basename(HTML_TEMPLATE)))
```

这个代码清单中最引人注意的部分是下面公式（*http:help.com/post/383276-anyone-knows-the-formula-for-font-s*）的融入，它用于给云中的标签加权：

$$\frac{(\text{f-min_freq})*(\text{MAX_FONT_SIZE} - \text{MIN_FONT_SIZE})}{\text{max_freq-min_freq}} + \text{MIN_FONT_SIZE}$$

这个公式通过考虑正在讨论的词的频率和该数据最高、最低的频率值，来对词频加权，这样它们就可以在MIN_FONT_SIZE和MAX_FONT_SIZE之间线性压缩了。有很多变形形式可应用于该公式，而且包含对数也并不罕见。Kevin Hoffman的论文："In Search of the Perfect Tag Cloud"（*http://files.blog-city.com/files/J05/88284/b/insearchofperfecttagcloud.pdf*）提供了构思标签云各种设计决策的良好概述，如果你想深入了解标签云的话，这是一个有用的起点。

示例5-17引用的*tagcloud_template.html*文件相当无趣，可以在GitHub中得到其源代码（*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/web_code/wp_cumulus/tagcloud_template.html*）；它只是对带有标签云源代码股票示例的简单修改。这一页开头的脚本标签处理了所有困难的工作，你需要做的只是将一些数据提供给临时模板，该模板简单地使用了字符串替换来替代%s占位符。

图5-6和图5-7显示了共同出现在#JustinBieber和#TeaParty中的、频率高于30的tweet实体的标签云。它们之间最明显的区别是：与#JustinBieber云相比，#TeaParty标签云是多么拥挤。与查询项相关的其他主题的要点也是显而易见的。我们已经从图5-5中了解它了，但是标签云传达了类似的要点，还为启动提供了有用的交互功能。当然，什么都不能阻止你使用Google Chart Tools（*http://code.google.com/intl/zh-CN/apis/chart/interactive/docs/gallery.html*）这类工具创建交互式Ajax图。

图5-6：与#JustinBieber共同出现的tweet实体中的交互式3D标签云

对Twitter搜索结果中群集结构的可视化

本章前面的内容简要地比较了#JustinBieber和#TeaParty查询，这一节将从一个略微不同的角度介绍可视化，来进一步分析该查询。让我们通过使用我们之前收集的搜索结果、计算出现在这些tweet中的tweet作者和其他实体之间（如@mentions和#hashtags）的友谊、可视化这些关系，试着可视化#TeaParty和#JustinBieber Twitter用户的群集结构。除了对我们的示例产生额外的见解之外，这些技术也为其他情况提供了有用的起点，你脑中会存在这些情况有趣的对比。这个工作流的代码清单不会出现在这里，因为通过再次使用前面介绍的代码就可以轻易地创建它了。涉及的大体步骤包括：

- 计算与search-teaparty和search-justinbieber CouchDB数据库相关的唯一tweet作者和用户提及的昵称集。
- 通过Twitter的/friends/ids源，获取好友昵称的ID。
- 通过Twitter的/users/lookup资源解析好友ID的昵称（回忆一下，没有直接的方法查找好友的昵称，必须先收集ID值再解析它们）。
- 通过分析友谊、创建两个节点之间的边来构建networkx.Graph，其中每个方向上都存在“友谊”。
- 分析并可视化图。

图5-7：与#TeaParty共同出现的tweet实体中的交互式3D标签云

该脚本（*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/*）的结果是序列化的图文件，可以在解释器中打开并且搜索它，如示例5-18中的解释器会话所示。因为该脚本本质上只是前面示例的一些零碎的重用逻辑，所以文本并没

有包含它，但是可以在*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/the_tweet__get_friends_for_entity_mentions_in_search_results.py*中获取其代码。在#JustinBieber和#TeaParty的每次搜索的差不多3000条tweet上运行该脚本，可以得到该分析结果。此处省略了返回图中每个节点对应度[注8]的`nx.degree`调用输出，用图5-8的简单柱形图来直观地表示它。

示例5-18：使用解释器执行ad hoc图分析

```
>>> import networkx as nx
>>> teaparty = nx.read_gpickle("out/search-teaparty.gpickle")
>>> justinbieber = nx.read_gpickle("out/search-justinbieber.gpickle")
>>> teaparty.number_of_nodes(), teaparty.number_of_edges()
(2262, 129812)
>>> nx.density(teaparty)
0.050763513558431887
>>> sorted(nx.degree(teaparty)
... output omitted ...
>>> justinbieber.number_of_nodes(), justinbieber.number_of_edges()
(1130, 1911)
>>> nx.density(justinbieber)
>>> 0.0029958378077553165
>>> sorted(nx.degree(teaparty))
... output omitted ...
```

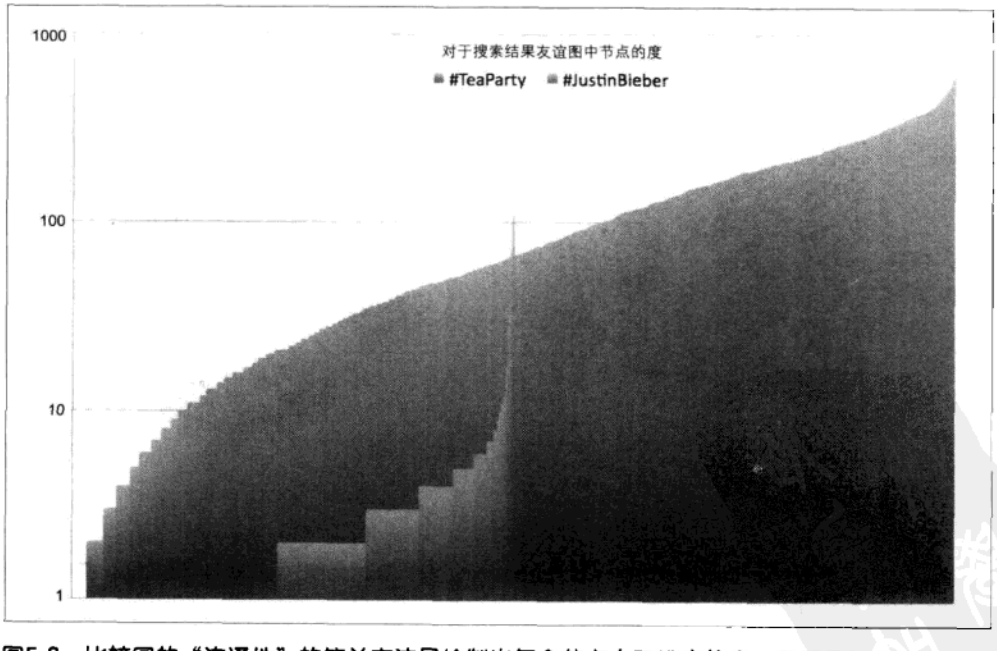

图5-8：比较图的“连通性”的简单方法是绘制出每个节点中已排序的度，然后叠加曲线

注8：　无向图中节点的度是边的总数。

无需任何可视化表现形式，解释器会话中的结果已经可以说明问题了。事实上，关键是无需查看可视化表现形式就可以讨论这些数字，因为如果没有投入大量时间，你通常无法轻易地可视化复杂的图。结果是#TeaParty图中节点的密度、节点与边的比例（用户之间的友谊）远远超过了#JustinBieber图中的。然而，观察结果本身不是最令人惊讶的。令人惊讶的是#TeaParty图中节点的高连通性。显然，#TeaParty是有较少关注的、激烈的政治运动，而#JustinBieber是有国际吸引力的、相对分散的娱乐话题。然而，#TeaParty结果曲线的总体分布和形状提供了一个直观和实际的视图，它说明了#TeaParty用户的关系实际上有多好。该图的2D可视化表现形式没有提供太多其他信息，但截止到现在，这些悬念可能会让你备受煎熬，那么还是让我们做个了断吧，请看图5-9。

回想一下，*nix用户可以使用nx.drawing.write_dot编写Graphviz输出，而Windows用户可能需要手动生成DOT语言输出。查看示例1-12和2-4。对于大型无向图来说，你会发现Graphviz的SFDP（scalable force directed placement）引擎（*http://www2.research.att.com/~yifanhu/PUB/peripheral.pdf*）极为重要。如何从命令行调用它来生成有用的稠密图布局的示例用法如下：

```
$sfdp -Tpng -
Oteaparty -Nfixedsize=true -Nlabel=' ' -Nstyle=filled -Nfillcolor=red -
Nheight=0.1 -Nwidth=0.1 -Nshape=circle -Gratio=fill -Gotputorder=edgesfirst -
Gratio=fill -Gsize='10!' -Goverlap=prism teaparty.dot
```

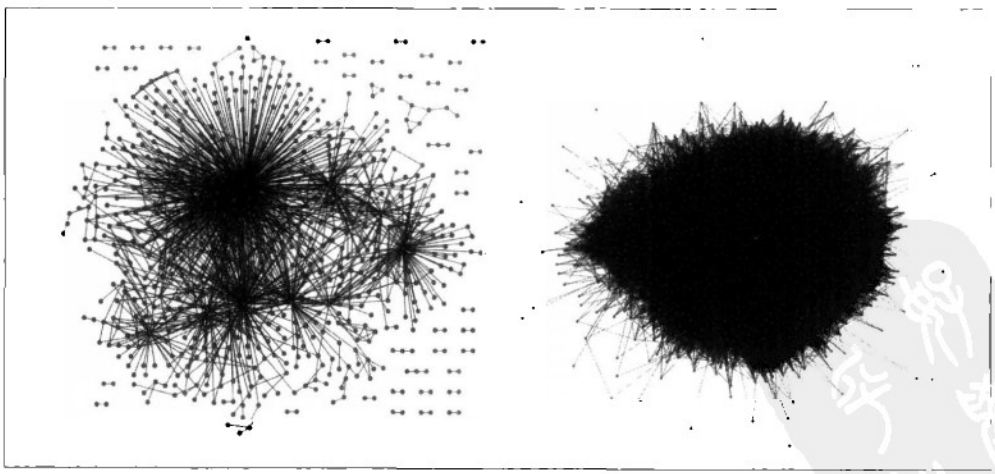

图5-9：展现了#JustinBieber（左）和#TeaParty（右）Twitter用户连通性的2D图；每条边表示至少在一个方向上存在一种友谊

小结

你可以使用tweet数据做很多显而易见的事，而且如果你有一点点创意的话，可能性会更多。本章我们只接触了一些皮毛。毫不夸张地说，整本书用于系统地产生更多可能性，很多关注tweet分析的小型企业可以通过将解决方案出售给客户进行的某类即席查询，获得数目可观的利润。下面是可以寻求的一些有趣的想法：

- 定义相似性度量，使用它来比较或进一步分析两个Twitter用户或用户组。本质上，你会打造一个形象，然后度量某个用户有多么适合该形象。例如，出现在#JustinBieber tweets中的某些hashtag、关键字（如，LOC、OMG等）或相似度量，是否比在与#gov20相关的这类更加理性的tweet中出现的次数更多？这个问题的答案虽然似乎是显而易见的，但是当你计算一个量化的答案时，你会学到什么有趣的知识呢？
- 哪些名人Twitter用户的tweet数量高，但是tweet实体关注度却很低？这会使他们成为漫谈吗？你知道一天泛泛地发100次tweet的那些家伙吗？相反，哪些名人Twitter用户的tweet中的hashtag关注度高？你能看出这两个群体之间明显的区别吗？
- 如果你有Twitter账户和合理数量的tweet，考虑分析自己的tweet，并比较你和其他用户的tweet流。在某一特定的范围内，你和不同的Twitter用户的相似性有多少？
- 我们没有讨论Twitter的列表API主题。你可以描述列表中的不同成员，并试着找到潜在的社交网络吗？例如，你可以找到允许你估计*@timoreilly/healthcare*和*@timoreilly/gov20*列表关注者之间的政治界限的一个度量吗？
- 按天来分析tweet量或tweet内容可能很有趣。这是很简单的练习，可以将SIMILE Timeline整合在一起（在3.4节讨论过）以便可视化。
- 准实时通信正在逐渐成为规范，Twitter提供了一种信息流API，通过该API可以很方便地获取它们的“消防水带”。查看在线信息流文档，考虑将tweepy作为良好的流媒体客户端。

第6章

LinkedIn：为了乐趣（和利润？）将职业网络聚类

本章将介绍挖掘存储在LinkedIn中数据价值的技术和注意事项，该网站是集中在专业和商业关系的流行且强大的社交网站。在你阅读本章时，非常鼓励你建立你自己的职业LinkedIn网络，但是即使你没有LinkedIn个人简介，你仍然可以应用我们研究其他领域的技术，这些技术提供了一种导出信息的方式，这种方式与查找通讯录类似。虽然LinkedIn最初看起来和其他社交网络一样，但是它的API提供的数据本身就具有不同的性质。一般来说，想要加入LinkedIn的人对它提供的商机感兴趣，而且想让他们的个人资料尽可能完美——这意味着确保他们提供非常私人性质的充足细节，用来传达重要的商务关系、工作详情等。

由于数据的敏感性，它的API和我们看到的在本书中其他API有所不同。例如，虽然通常你可以访问你的联系人的学历、之前的工作职位等所有有趣的细节，但是你不能确定任意的两个人是否“已连接”，缺少这种API并非偶然——做出这种决定是因为，LinkedIn的管理团队坚信你的职业网络数据本质上是私有的，并且它非常宝贵，不能像你的Twitter或Facebook上的好友那样任意供你利用和查看[注1]。

由于LinkedIn限制你访问你“一度人脉”的相关信息，并且不适合于作为图挖掘，因此它要求你对可利用的数据提各种问题。本章剩余的部分将介绍基本的聚类技术，它们可以帮助你回答以下几类问题：

- 你的哪些“人脉”与职位这类标准最为相似？
- 你的哪些“人脉”曾在你想去的公司工作过？

注1：LinkedIn的立场在作者和D.J.Patil之间的一次讨论中被传达，他是LinkedIn项目分析的首席科学家、首席安全专家和高级总监。

- 你的大多数“人脉”住在什么地方？

聚类的动机

由于LinkedIn数据的丰富性，你可以从你的职业网络中查询到一些很好的机会。然而，在实现回答这类问题的方案时，我们至少会反复遇到两个共同主题：

- 不管它们是职位、公司名称、专业领域，还是可以在自由文本中输入的其他字段，度量两个值（通常是字符串值）之间的相似性通常都是必要的。第7章正式介绍一些其他方法和注意事项来度量你可能也想审查的相似性。
- 为了用一个相似性度量将所有的项聚合为一个集合，理想的情况是将每个成员与其他所有成员作对比。因此，对于一个含有n个成员的集合来说，在最坏的情况下，需要执行n^2次相似性计算。计算机科学家将这种情况称为n^2问题，通常使用术语$O(n^2)$来描述它[注2]；可以把它念为“大O的n平方”问题。然而，对于非常大的n值来说，$O(n^2)$问题会变得很棘手。大多数时候，“棘手”这个词的使用意味着你必须等上好几年（或者甚至是数百年）才能得到该问题的解答——或者，至少对于想利用该方案解决问题来说，还是“太长”了。

这两个问题会出现在涉及近似匹配问题的所有讨论中[注3]，近似匹配是一个经过充分研究的问题，几乎存在于所有行业中。

近似匹配技术是任何合法数据挖掘工具带的基本组成部分，因为几乎在所有行业的任何部门（从国防情报局到银行的反欺诈侦测，再到当地园林绿化公司），都可能需要分析很多半结构化关系数据（不管负责人是否意识到了这一点）。通常会发生的是公司建立了收集某类信息的数据库，但并不是每一个字段都被枚举成预定义的有效响应。不管是由于某些字段不适合存储静态预定义值，导致该应用程序的用户界面逻辑设计不当，还是因为允许用户在文本框中输入任何值的用户体验很关键，结果总是相同的：你最终会以很多半结构化数据或者“脏记录”结束。虽然某个特殊字段可能有N个不同的字符串值，但是一些字符串值实际上是相同的概念。发生重复的原因有很多：错误的拼写、缩写或简写、单词的大小写差别等。

注2：从技术上来说，有时将一个问题的时间复杂度描述为$\Theta(n^2)$更精确，因为$O(n^2)$表示最坏的情况，而$\Theta(n^2)$表示对期望性能更严格的约束。也就是说，$\Theta(n^2)$表示它不只是最坏情况下的n^2问题，它也是最好情况下的n^2问题。然而，本章不是为了分析算法的运行时间复杂度（它本身就是一个棘手的问题），我们会继续使用更常见的$O(n^2)$符号描述最坏的情况。

注3：通常也被称为模糊匹配、聚类和/或重复数据删除。尝试在多个字段（记录）中执行近似匹配通常称为“记录链接”问题。

虽然不太明显，但这正是我们在挖掘LinkedIn数据时所面临的情况：LinkedIn成员可以把他们的职业信息输入为自由文本，这会导致一定数量的必然变化。例如，如果你想检查你的职业网络，尝试确定你的大多数“人脉”在哪儿工作，你就需要考虑公司名称的常见变化。即使是对最简单的公司名称，你也一定会遇到一些变化。例如，大多数人都知道“Google”是“Google，Inc.”的缩写，但是即使是这些命名约定的简单变化也必须在标准化工作中显式说明。在标准化公司名称时，良好的起点是首先考虑“，Inc.”、“，LLC”这类后缀。假设你有从LinkedIn导出的联系人的逗号分隔值（Comma-Seperated Value，CSV）文件，你可能会从一些基本的标准化开始，然后输出从柱形图中选择的实体，如示例6-1所示。

访问LinkedIn数据的两种主要方法是：将它导出为保存姓名、职位、公司和联系人信息这类基本信息的通讯录数据，或者使用LinkedIn API来以编程方式利用你的联系人的全部详细信息。虽然通过API可以像“一个已认证的用户在*http://linkedin.com*上浏览个人资料”那样访问你可见的所有内容，但通过导出通讯录信息，我们可以获得第一个练习需要的所有职位信息。（6.3节会使用LinkedIn API。）如果你不明白如何导出你的“人脉关系”，那么请查看图6-1，它提供了一些视觉线索，会告诉你如何导出你的“人脉关系”。我们将会使用Python的csv模块解析导出的数据；为了确保后面的代码清单的兼容性，选择“Outlook CSV”选项。

示例6-1：对通讯录数据中公司后缀的简单标准化（linkedin_analyse_companies.py）

```
# -*- coding: utf-8 -*-

import sys
import nltk
import csv
from prettytable import PrettyTable

CSV_FILE = sys.argv[1]

# Handle any known abbreviations,
# strip off common suffixes, etc.

transforms = [(', Inc.', ''), (', Inc', ''), (', LLC', ''), (', LLP', '')]

csvReader = csv.DictReader(open(CSV_FILE), delimiter=',', quotechar='"')
contacts = [row for row in csvReader]
companies = [c['Company'].strip() for c in contacts if c['Company'].strip() != '']

for i in range(len(companies)):
    for transform in transforms:
        companies[i] = companies[i].replace(*transform)

pt = PrettyTable(fields=['Company', 'Freq'])
pt.set_field_align('Company', 'l')
fdist = nltk.FreqDist(companies)
[pt.add_row([company, freq]) for (company, freq) in fdist.items() if freq > 1]
pt.printt()
```

然而，你需要更巧妙地处理更复杂的情况，如你可能会遇到公司名称的各种表示形式，如O'Reilly Media。例如，你可能会看到该公司名称表示为O'Reilly & Associates、O'Reilly Media、O'Reilly，Inc[注4]、或者只是O'Reilly。考虑职位时也存在相同的问题，只是这种情况可能更麻烦，因为职位有更多变化。表6-1列出了一些你可能在软件公司遇到的职位，其中包括一定量的自然变化。在列出的10个不同职位中，你可以找到几种不同的角色？

表6-1：科技产业的示例职位

职位
CEO
总裁/CEO
总裁&CEO
开发人员
软件开发人员
软件工程师
技术总监（CTO）
总裁
高级软件工程师

虽然极有可能会定义与CEO和总裁这类职称相等的别名或缩写列表，但是一般情况下，不太可能在所有领域中手动定义等同于软件工程师和开发人员这样的职称。然而，即使是对最坏情况下最复杂的字段，实现解决方案也不太难，将数据压缩成可以被专家评审的点，然后将它反馈给“可以以专家所采用的大致相同的方式应用”的程序。很多情况下，这就是组织喜欢的方式，因为它允许简单地在循环中插入它们来执行质量控制。

按职位将联系人聚类

既然你可以（但愿）更好地欣赏记录匹配问题的性质，让我们从LinkedIn中收集一些真实的数据，开始处理聚类吧。你对挖掘LinkedIn数据感兴趣的一些原因可能是你想要看看你的网络技术是否可以帮助你找到“合适的人群”，或者因为你想接触那些可能最适合

注4：如果刚开始时你认为这些听起来有点复杂，可以想想Dun & Bradstreet（*http://www.dnb.com/us/*）（公司信息的“名人录”）所承担的工作，该公司最大的挑战就是要维护全球范围内多语种的公司名录。

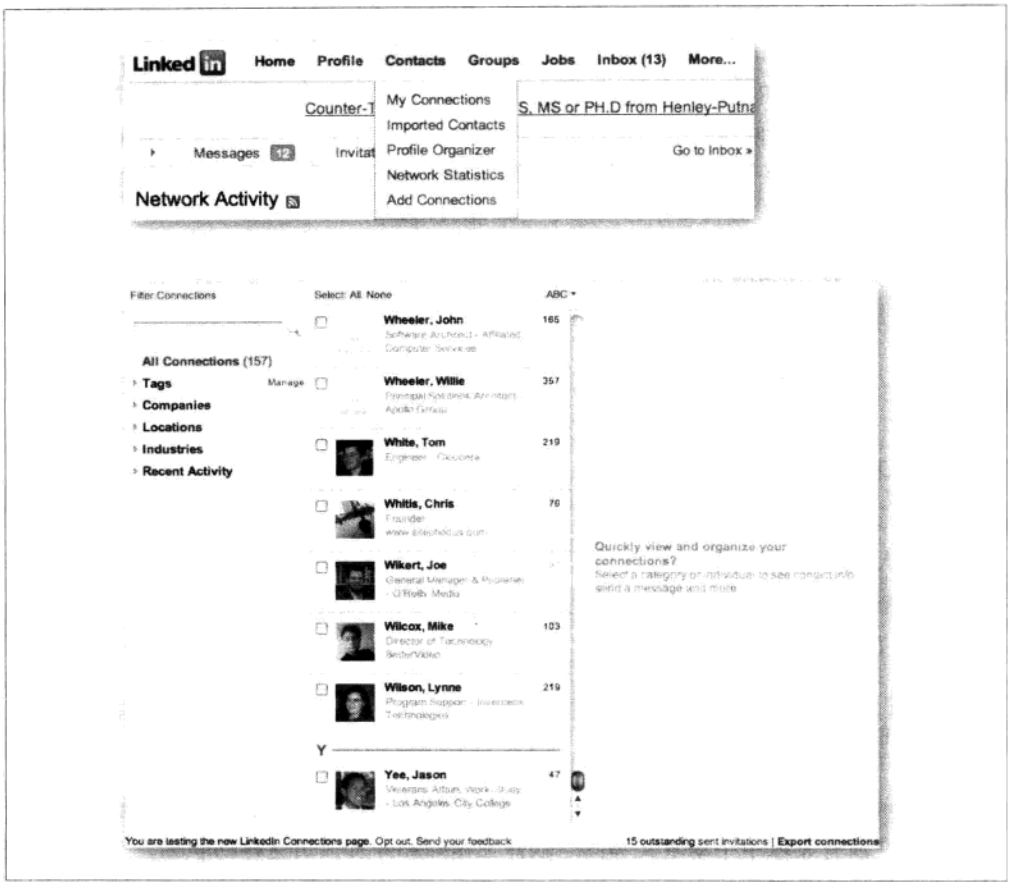

图6-1：要导出你的“人脉关系”，需要点击Contact菜单，从弹出的菜单中选择My Connections命令，然后点击屏幕下方的Export connections链接即可。

于“某种特定类型的业务查询或业务主题”的社会经济类型的联系人。本节剩余的部分将通过一些练习，系统地介绍一下按职位将联系人聚类的方法。

规范并统计职位的数量

当使用新数据集时，一个明显的起点是统计它们的总数，这里也不例外。本节介绍转换常见职位的模式，然后执行了基本的频率分析。

假设你有合理数量的已导出联系人，你可能会遇到的职位之间的细微区别实际上是令人惊讶的，但是在我们研究这些内容之前，让我们先整理一些用来创建一些规范记录数据模式的代码，得到按频率排序的基本库存。示例6-2检查了这些职位，输出了职位本身

和存在于它们本身中的个别标记的频率信息。如果还没有安装prettytable，就需要运行easy_install prettytable命令，可以用这个包来生成良好地格式化的表格输出。

示例6-2：标准化常见职位，计算它们的频率（linkedin_analyze_titles.py）

```
# -*- coding: utf-8 -*-

import sys
import nltk
import csv
from prettytable import PrettyTable

CSV_FILE = sys.argv[1]

transforms = [
    ('Sr.', 'Senior'),
    ('Sr', 'Senior'),
    ('Jr.', 'Junior'),
    ('Jr', 'Junior'),
    ('CEO', 'Chief Executive Officer'),
    ('COO', 'Chief Operating Officer'),
    ('CTO', 'Chief Technology Officer'),
    ('CFO', 'Chief Finance Officer'),
    ('VP', 'Vice President'),
    ]

csvReader = csv.DictReader(open(CSV_FILE), delimiter=',', quotechar='"')
contacts = [row for row in csvReader]

# Read in a list of titles and split apart
# any combined titles like "President/CEO"
# Other variations could be handled as well such
# as "President & CEO", "President and CEO", etc.

titles = []
for contact in contacts:
    titles.extend([t.strip() for t in contact['Job Title'].split('/')
                   if contact['Job Title'].strip() != ''])

# Replace common/known abbreviations

for i in range(len(titles)):
for transform in transforms:
titles[i] = titles[i].replace(*transform)
# Print out a table of titles sorted by frequency

pt = PrettyTable(fields=['Title', 'Freq'])

pt.set_field_align('Title', 'l')
titles_fdist = nltk.FreqDist(titles)
[pt.add_row([title, freq]) for (title, freq) in titles_fdist.items() if freq > 1]
pt.printt()
# Print out a table of tokens sorted by frequency

tokens = []
for title in titles:
    tokens.extend([t.strip(',') for t in title.split()])
```

```
pt = PrettyTable(fields=['Token', 'Freq'])
pt.set_field_align('Token', 'l')
tokens_fdist = nltk.FreqDist(tokens)
[pt.add_row([token, freq]) for (token, freq) in tokens_fdist.items() if freq > 1
 and len(token) > 2]
pt.printt()
```

总而言之，该代码读入了CSV记录，试图通过拆分使用了正斜杠（如“总裁/CEO”的职位）的组合职位并替换已知的缩写来将其标准化。除此之外，它只显示了包含在这些职位中的全职称和个别标记的频率分布的结果。这并不是什么新鲜事，但是作为起始模板来说，它是很有用的，它为你提供了如何分解数据的合理见解。样例结果如示例6-3所示。

示例6-3：示例6-2的样例结果

```
+--------------------------+------+
|           Title          | Freq |
+--------------------------+------+
| Chief Executive Officer  |  16  |
| Senior Software Engineer |  12  |
| Owner                    |   9  |
| Software Engineer        |   9  |

...

+-------------+------+
|    Token    | Freq |
+-------------+------+
| Engineer    |  44  |
| Software    |  31  |
| Senior      |  27  |
| Manager     |  26  |
| Chief       |  24  |
| Officer     |  12  |
| Director    |  11  |

...
```

有趣的是示例结果表明：基于精确匹配的最常见的职位是“CEO”，而职位中最常见的标记是“工程师”。虽然“工程师”并不是最常见的职位的组成标记，但它的确出现在很多职位中（如，几乎位于职位列表顶端的“高级软件工程师”和“软件工程师”）。分析职位或通讯录数据时，这恰恰是可以激发近似匹配或聚类算法需要的见解。下一节会进一步探讨它。

聚类常见的相似性度量

在获取一组字符串（现在是职位）并以一种有效的方法将它们聚类时，需要做的最实质性的决定是：使用哪种基本的相似性度量。有无数字符串相似性度量可供使用，选择最

适合的相似性度量在很大程度上取决于目标的性质。比较职位时，一些可能有帮助的最常见的相似性度量如下所示：

编辑距离

编辑距离，又称Levenshtein（*http://en.wikipedia.org/wiki/Vladimir_Levenshtein*）距离，是一种简单的度量，用于记录将一个字符串转换为另一个字符串时，插入、删除和替换的次数。例如，将“dad”转换为“bad”需要一次替换操作，会产生值1。NLTK通过nltk.metrics.distance.edit_distance函数实现编辑距离。请注意，两个字符串之间的实际编辑距离往往与计算编辑距离需要的操作次数不同；对于长度为*M*和*N*的字符串来说，计算编辑距离通常需要*M**N*次操作。

n元语法相似度

*n*元语法只是用于简单地表示文本中*n*个标记的所有可能的连续序列，它提供了计算搭配的基本数据结构。请查看请查看7.5节中*n*元语法和搭配的进一步讨论。

*n*元语法相似度有很多变化，但是考虑最简单的情况：计算两个字符串标记中所有可能的二元语法，通过统计两个字符串之间共同的二元语法来计算它们之间的相似度，如示例6-4所示。

示例6-4：使用NLTK计算解释器中的二元语法

```
>>> ceo_bigrams = nltk.bigrams("Chief Executive Officer".split(), pad_right=True, \
... pad_left=True)
>>> ceo_bigrams
[(None, 'Chief'), ('Chief', 'Executive'), ('Executive', 'Officer'),
('Officer', None)]
>>> cto_bigrams = nltk.bigrams("Chief Technology Officer".split(), pad_right=True, \
... pad_left=True)
>>> cto_bigrams
[(None, 'Chief'), ('Chief', 'Technology'), ('Technology', 'Officer'),
('Officer', None)]
>>> len(set(ceo_bigrams).intersection(set(cto_bigrams)))
2
```

请注意，关键字参数pad_right和pad_left的使用特意考虑了匹配的开头和结束标记。padding的作用是允许出现（None，‘Chief’）这样的二元语法，它们都是职位中重要的匹配。NLTK通过使用nltk.metrics.association模块中定义的BigramAssociationMeasures类和TrigramAssociationMeasures类，来提供相当全面的二元语法和三元语法计分函数。

雅克比距离

雅克比度量表示两个集合的相似度，它是由|集合1交集合2|/|集合1 ∩ 集合2|定义的。也就是说，它是两个集合中的不同项除以两个集合之间的共同项得到的结果。详细的讨论请查看7.5.1节。一般来说，可以通过使用包含一元文法（单个标记）的*n*元

语法计算雅克比相似度，来度量两个字符串的相似度。雅克比距离可以通过nltk.metrics.distance.jaccard_distance实现，实现方式如下：

```
len(X.union(Y)) - len(X.intersection(Y)))/float(X.union(Y))
```

其中，*X*和*Y*是项集。

MASI距离

MASI[注5]距离度量是雅克比相似度的加权版本，当集合之间存在部分重叠时，它通过调整得分来生成小于雅克比的距离。MASI距离在NLTK中定义为nltk.metrics.distance.masi_distance，实现方式如下：

```
1-float(len(X.intersection(Y)))/float(max(len(X),len(Y)))
```

其中，*X*和*Y*是项集。示例6-5和表6-2中的样例结果提供了一些数据点，它们有助于更好地理解这个度量。

示例6-5：使用NLTK中内置的距离度量来比较小型项集（linkedin_distances.py）

```
# -*- coding: utf-8 -*-

from nltk.metrics.distance import jaccard_distance, masi_distance
from prettytable import PrettyTable

fields = ['X', 'Y', 'Jaccard(X,Y)', 'MASI(X,Y)']
pt = PrettyTable(fields=fields)
[pt.set_field_align(f, 'l') for f in fields]

for z in range(4):
    X = set()
    for x in range(z, 4):
        Y = set()
        for y in range(1, 3):
            X.add(x)
            Y.add(y)
            pt.add_row([list(X), list(Y), round(jaccard_distance(X, Y), 2),
                        round(masi_distance(X, Y), 2)])
pt.printt()
```

请注意，当两个集合完全不相交或者相等时（即，当一个集合被另一个集合完全包含时[注6]），MASI距离和雅克比距离相同。然而，当两个集合部分重合时，MASI距离大于雅克比距离。值得考虑的是，如果两个任意职位之间的相似度（距离）超过一定的阈值，对于将它们计分和把它们聚类的任务，你是否认为MASI比雅克比更高效。表6-2的样例结果有助于了解MASI和传统的雅克比之间的区别。

注5：度量有关集值项的协议（Measuring Agreement on Set-Valued Item，MASI）。参见*http://www.cs.columbia.edu/~becky/pubs/lrec06masi.pdf*。

注6：`def subsumed(X,Y): return X.union_update(Y)==X or Y.union_update(X)==Y`

表6-2：对示例6-5产生的两个小型项集，比较雅克比和MASI距离

X	Y	雅克比(X,Y)	MASI(X,Y)
[0]	[1]	1.0	1.0
[0]	[1,2]	1.0	1.0
[0,1]	[1]	0.5	0.5
[0,1]	[1,2]	**0.67**	**0.5**
[0,1,2]	[1]	0.67	0.67
[0,1,2]	[1,2]	0.33	0.33
[0,1,2,3]	[1]	0.75	0.75
[0,1,2,3]	[1,2]	0.5	0.5
[1]	[1]	0.0	0.0
[1]	[1,2]	0.5	0.5
[1,2]	[1]	0.5	0.5
[1,2]	[1,2]	0.0	0.0
[1,2,3]	[1]	**0.67**	**0.67**
[1,2,3]	[1,2]	0.33	0.33
[2]	[1]	1.0	1.0
[2]	[1,2]	0.5	0.5
[2,3]	[1]	1.0	1.0
[2,3]	[1,2]	**0.67**	**0.5**
[3]	[1]	1.0	1.0
[3]	[1,2]	1.0	1.0

虽然还可以列出很多有趣的相似性度量，但是，一旦你更深入地了解了挖掘的数据，就很容易插入并尝试不同的相似性启发。下一节会逐步创建一段脚本，它使用MASI相似性度量来将职位聚类。

聚类的贪心方法

由于MASI对部分重叠词的偏爱，我们认为职位中的重叠是很重要的，让我们尝试将职位聚类，并将其与使用MASI距离的其他职位相对比，以此作为示例6-2的延续。示例6-6聚类了相似的职位，进而相应地显示了你的联系人。看一下该代码（特别是调用得到贪心决策的DISTANCE函数的嵌套循环），然后我们会讨论它们。

示例6-6：使用贪心启发来聚类职位（linkedin_cluster_contacts_by_title.py）

```
# -*- coding: utf-8 -*-

import sys
import csv
from nltk.metrics.distance import masi_distance

CSV_FILE = sys.argv[1]

DISTANCE_THRESHOLD = 0.34
DISTANCE = masi_distance

def cluster_contacts_by_title(csv_file):

    transforms = [
        ('Sr.', 'Senior'),
        ('Sr', 'Senior'),
        ('Jr.', 'Junior'),
        ('Jr', 'Junior'),
        ('CEO', 'Chief Executive Officer'),
        ('COO', 'Chief Operating Officer'),
        ('CTO', 'Chief Technology Officer'),
        ('CFO', 'Chief Finance Officer'),
        ('VP', 'Vice President'),
        ]

    seperators = ['/', 'and', '&']

    csvReader = csv.DictReader(open(csv_file), delimiter=',', quotechar='"')
    contacts = [row for row in csvReader]

    # Normalize and/or replace known abbreviations
    # and build up list of common titles

        all_titles = []
        for i in range(len(contacts)):
            if contacts[i]['Job Title'] == '':
                contacts[i]['Job Titles'] = ['']
                continue
            titles = [contacts[i]['Job Title']]
            for title in titles:
                for seperator in seperators:
                    if title.find(seperator) >= 0:
                        titles.remove(title)
                        titles.extend([title.strip() for title in title.split(seperator)
                                       if title.strip() != ''])

            for transform in transforms:
                titles = [title.replace(*transform) for title in titles]
            contacts[i]['Job Titles'] = titles
            all_titles.extend(titles)

    all_titles = list(set(all_titles))

    clusters = {}
```

```
    for title1 in all_titles:
        clusters[title1] = []
        for title2 in all_titles:
            if title2 in clusters[title1] or clusters.has_key(title2) and title1 \
                in clusters[title2]:
                continue
            distance = DISTANCE(set(title1.split()), set(title2.split()))
            if distance < DISTANCE_THRESHOLD:
                clusters[title1].append(title2)

# Flatten out clusters

    clusters = [clusters[title] for title in clusters if len(clusters[title]) > 1]

# Round up contacts who are in these clusters and group them together

    clustered_contacts = {}
    for cluster in clusters:
        clustered_contacts[tuple(cluster)] = []
        for contact in contacts:
            for title in contact['Job Titles']:
                if title in cluster:
                   clustered_contacts[tuple(cluster)].append('%s %s.'
                          % (contact['First Name'], contact['Last Name'][0]))

    return clustered_contacts

if __name__ == '__main__':
    clustered_contacts = cluster_contacts_by_title(CSV_FILE)

    for titles in clustered_contacts:
        common_titles_heading = 'Common Titles: ' + ', '.join(titles)
        print common_titles_heading

        descriptive_terms = set(titles[0].split())
        for title in titles:
            descriptive_terms.intersection_update(set(title.split()))
        descriptive_terms_heading = 'Descriptive Terms: ' \
            + ', '.join(descriptive_terms)
        print descriptive_terms_heading
        print '-' * max(len(descriptive_terms_heading), len(common_titles_heading))
        print '\n'.join(clustered_contacts[titles])
        print
```

该代码清单从使用常见的连词列表分离出综合职位开始，并标准化常见职位。然后，嵌套循环遍历所有的职位，并根据一个阈值MASI相似性度量将它们聚类。这种紧密的循环是清单中大部分实际操作发生的地方：它是所有职位互相比较的地方。如果相似度启发确定的任意两个职位之间的距离“足够近”，我们就可以贪婪地将它们集合起来。在此背景下，“贪婪”就意味着我们第一次就能确定一个可能适合某一聚类的项，继续前进并对其分配，而不需要进一步考虑是否有更合适的选择，或者如果有更合适的选择出现时，我们再想办法选择。虽然这种方法特别务实，但是它产生了非常合理的结果。显然，关键是选择有效的相似度启发，但是由于嵌套循环的性质，需要调用计分函数的次

数越少，结果越好。下一节将会更多地考虑这些内容，但是请注意，如果可能的话，我们会使用一些条件逻辑，尝试避免不必要的计算。

清单的剩余部分只用特定的职位查询联系人，并将它们分组显示，但是在计算聚类时，还存在一个有趣的细微差异：经常需要给每个聚类指定一个有意义的标签。奏效的方法就是通过对每个聚类中职称词语的逐个交集来计算标签，这似乎很合理，因为它是最明显的共同点。你的行程确实会随方法的不同而变化。

从这段代码中得到的结果类型估计会很有用，因为它们把可能在职位中共同承担责任的个体集合起来了。由于种种原因，这个信息可能很有用，不管你是计划包括“CEO专家组”的事件，试着计算出谁能尽力帮助你跳槽，还是试着根据你自己的职责确定你是否真正很好地连接到其他相似的职业。示例网络的缩略结果如示例6-7所示。

示例6-7：示例6-6的样例输出

```
Common Titles: Chief Executive Officer,
               Chief Finance Officer,
               Chief Technology Officer,
               Chief Operations Officer
Descriptive Terms: Chief, Officer
---------------------------------------
Gregg P.
Scott S.
Mario G.
Brian F.
Damien K.
Chris A.
Trey B.
Emil E.
Dan M.
Tim E.
John T. H.

...

Common Titles: Consultant,
               Software Engineer,
               Software Engineer
Descriptive Terms: Software, Engineer
---------------------------------------
Kent L.
Paul C.
Rick C.
James B.
Alex R.
Marco V.
James L.
Bill T.

...
```

自扩展聚类肯定不容易

重要的是要注意，在最坏的情况下，需要在刚才提到的代码中调用的嵌套循环的时间复杂度是$O(n^2)$，即，len(all_titles)*len(all_titles)次，该嵌套循环执行示例6-6中的DISTANCE计算。对于一个很大的n值来说，为了聚类而比较嵌套循环中的所有项并不是可扩展的方法，但是考虑到在你的职业网络中职位的唯一个数可能不太大，所以它不应该强加性能约束。这似乎没什么大不了的（毕竟，它只是一个嵌套循环），但是$O(n^2)$算法的关键是：要求处理输入集合而比较的次数会随着集合中项数的比例成倍剧增。例如，100个职位只要求1万次计分操作，而1万个职位要求1亿次计分操作。即使你能用很多硬件来处理它，但数学并不能很好地解决该问题，并且最终会导致曲解变形。

当面临大小成倍剧增这种情况时，你最初的反应可能是尝试尽可能缩小n值。但是通常你无法充分减小它，使你的方案像你的输入大小一样可伸缩，因为你的算法仍然是$O(n^2)$。你真正要做的是想出一种提供$O(k*n)$算法的方法，其中k比n小得多，它表示比n的增长速率慢得多的可控开销数量。但是，和其他工程决策一样，现实世界需要作出性能和质量的取舍，而且难以保持适当的平衡。实际上，很多高度精确地执行了可伸缩记录匹配的数据挖掘公司把它们的具体方法看做专有资料（商业机密），因为它们会产生一定的业务优势。（通常，它们不是真正解决问题的能力，而是以特定方式实现它的能力，这种方式可以应用到市场中。）

对于完全不能接受对于$O(n^2)$算法的情况，你可能会尝试的对可行示例的一种变通做法是重写嵌套循环，这样，对于计分函数中的任何一个随机样本，当样本大小为k时，时间复杂度将会有效地减小为$O(k*n)$。然而，对于接近于n的样本大小，运行时间开始接近于$O(n^2)$。示例6-8展示了抽样技术在代码中是如何体现的；之前清单的主要变化是粗体突出显示。重点是：对于外层循环的每次调用，我们都会执行更小的内层循环固定次数。

示例6-8：对示例6-6一部分的小修改，它加入了随机抽样来提高性能

```
# ...snip ...

all_titles = list(set(all_titles))
clusters = {}
for title1 in all_titles:
    clusters[title1] = []
    for sample in range(SAMPLE_SIZE):
        title2 = all_titles[random.randint(0, len(all_titles)-1)]
        if title2 in clusters[title1] or clusters.has_key(title2) and title1 \
            in clusters[title2]:
        continue
        distance = DISTANCE(set(title1.split()), set(title2.split()))
        if distance < DISTANCE_THRESHOLD:
        clusters[title1].append(title2)

# ...snip ...
```

你可能会考虑的另一种方法是随机抽取数据到n组中（其中，n是方法参数，一般小于或等于你集合中项数的平方根），在这些组中执行聚类，然后有选择性地合并输出。例如，如果你有100万个项，$O(n^2)$算法将会执行1万亿次逻辑操作，而将100万个项划分到1000个包含1000项的组中，将每一组的聚类只需要1亿次操作。（对于1000组的每一组，是1000 × 1000次比较。）1亿仍是一个很大的值，但是它比1万亿小三个数量级，这是不容忽视的。

除了抽样或分组之外，还可以在文献中找到很多其他方法，它们在减少问题的维度方面做得好得多。例如，当你要比较集合中的每一项时，最终用于避免对于较大n值$O(n^2)$情况的特定技术将会随着现实约束、你通过实验得到的见解和特定领域的知识而变化。当你考虑这些可能性时，要记住，机器学习领域提供了很多技术，它们通过使用各种概率模型和先进的采样技术，解决这些类型的规模问题。下一节介绍相当直观的、著名的聚类算法k均值，它是聚类多维空间的通用、非监督式方法。我们马上就会使用这种技术按地理位置聚类你的联系人。

智能聚类带来令人瞩目的用户体验

很容易陷入数据挖掘技术本身，而忽视了早先激励它们的商业案例。简单的聚类技术可以通过利用结果创造让人难以置信的令人瞩目的用户体验（到处都是），即使这些结果和我们刚才产生的职位一样简单。图6-2通过简单的树型控件，显示了对你的数据的强大的其他视图，这个树控件可以用做过滤搜索标准的导航面板或分面显示的一部分。假设你选择的基本相似性度量已经产生了有意义的聚类，在每一组的条目计数中显示逻辑群组中的数据可以化简查找信息的过程，可以增强几乎所有应用程序的直观工作流程，否则在其中查找结果需要很多浏览。

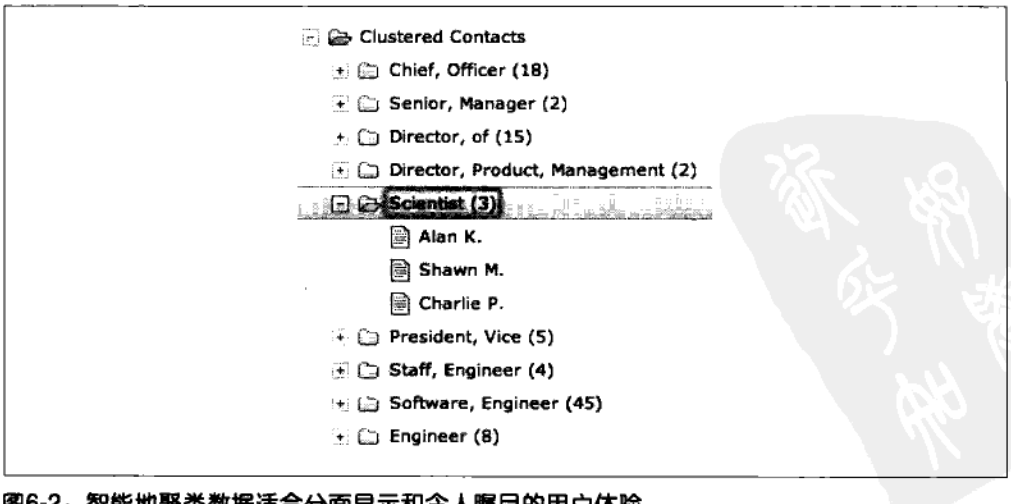

图6-2：智能地聚类数据适合分面显示和令人瞩目的用户体验

由于Ajax工具包和其他UI库的成熟，因此创建简单导航显示的代码变得出奇简单。例如，示例6-9显示了一个简单的网页，它是使用临时模板填充图6-2所示的Dojo树控件（*http://docs.dojocampus.org/dijit/Tree*）需要的所有内容。

示例6-9：示例6-2所示的Dojo树控件的模板——只添加数据，然后看它如何运行

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
        "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
    <title>Clustered Contacts</title>
    <link rel="stylesheet"
        href="http://ajax.googleapis.com/ajax/libs/dojo/1.5/dojo/resources/dojo.css">
    <link rel="stylesheet"
        href="http://ajax.googleapis.com/ajax/libs/dojo/1.5/dijit/themes/claro/claro.css">

    <script src="http://ajax.googleapis.com/ajax/libs/dojo/1.5/dojo/dojo.xd.js"
        type="text/javascript" djConfig="parseOnLoad:true"></script>

    <script language="JavaScript" type="text/javascript">
        dojo.require("dojo.data.ItemFileReadStore");
        dojo.require("dijit.Tree");
        dojo.require("dojo.parser");
    </script>

    <script language="JavaScript" type="text/javascript">
        var data = %s; //substituted by Python
    </script>

</head>
<body class="claro">
    <div dojoType="dojo.data.ItemFileReadStore" jsId="jobsStore"
        data="data"></div>

    <div dojoType="dijit.Tree" id="mytree" store="jobsStore"
        label="Clustered Contacts" openOnClick="true">
    </div>
</body>
</html>
```

代入模板的`data`值是一个简单的JSON结构，我们可以轻易地通过修改示例6-6的输出来获取它，如示例6-10所示。

示例6-10：整理示例6-6中计算出的clustered_contacts数据，产生Dojo树控件可以使用的JSON

```
import json
import webbrowser
import os

data = {"label" : "name", "temp_items" : {}, "items" : []}
for titles in clustered_contacts:
    descriptive_terms = set(titles[0].split())
    for title in titles:
```

```
            descriptive_terms.intersection_update(set(title.split()))
        descriptive_terms = ', '.join(descriptive_terms)

        if data['temp_items'].has_key(descriptive_terms):
            data['temp_items'][descriptive_terms].extend([{'name' : cc } for cc
                in clustered_contacts[titles]])
        else:
            data['temp_items'][descriptive_terms] = [{'name' : cc } for cc
                in clustered_contacts[titles]]

for descriptive_terms in data['temp_items']:
    data['items'].append({"name" : "%s (%s)" % (descriptive_terms,
        len(data['temp_items'][descriptive_terms]),),
                          "children" : [i for i in
                              data['temp_items'][descriptive_terms]]})

del data['temp_items']

# Open the template and substitute the data

if not os.path.isdir('out'):
    os.mkdir('out')

TEMPLATE = os.path.join(os.getcwd(), '..', 'web_code', 'dojo', 'dojo_tree.html')
OUT = os.path.join('out', 'dojo_tree.html')

t = open(TEMPLATE).read()
f = open(OUT, 'w')
f.write(t % json.dumps(data, indent=4))
f.close()

webbrowser.open("file://" + os.path.join(os.getcwd(), OUT))
```

创建以直观方式显示数据的用户体验具有超高的价值，它可以加强工作流程。和智能制作的分层显示一样简单的东西可以无意中激励用户在网站上花更多时间，发现比平常更多的信息，最终会认识到网站提供的服务具有更高价值。

分层聚类和*k*均值聚类

示例6-6介绍了聚类的贪心算法，但是你还必须知道其他两种常见的聚类技术：分层聚类和*k*均值聚类。从表面上看，分层聚类和我们前面使用的贪心启发相似，*k*均值聚类则完全不同。本章剩余的部分主要侧重于*k*均值算法，但是简要介绍一下这些方法使用的理论是值得的，因为你很可能会在文献分析和研究中碰到这些理论。这两种方法的完美实现可以通过easy_install cluter模块获取，所以请继续阅读，花点时间来安装它。

分层聚类

分层聚类是一种确定且详尽的技术，因为它可以计算所有项之间距离的全矩阵，然后遍历满足最小距离阈值的矩阵群集项。它是分层的，因为遍历矩阵和群集项自然会产生表

示项之间相对距离的树型结构。在文献中，这种技术可能称为凝聚（agglomerative），因为树的叶子表示正在聚类的项，而树的节点将这些项凝聚为群集。

这种方法与示例6-6中使用的方法类似，但从根本上来说，它们是不同的，它使用了贪心启发来对项聚类，而不是依次建立层次结构。同样地，分层聚类的实际时间（代码运行需要的时间）可能长一点，可能需要调整计分函数和距离阈值。然而，动态规划和其他精巧的记账技术可以大量节省实际的运行时间，这取决于实现。例如，假设两个项（如职位）之间的距离几乎肯定是对称的，那么应该只需要计算一半距离矩阵，而不是全矩阵。即使整个算法的时间复杂度仍然是$O(n^2)$，也只需要执行$0.5n^2$单位的工作，而不是n^2单位的工作。

如果我们重写示例6-6来使用cluster模块，那么执行聚类DISTANCE计算的嵌套循环将会被类似示例6-11的代码取代。

示例6-11：使用cluster.HierarchicalClustering代替贪心启发来修改示例6-6（linkedin_cluster_contacts_by_title_hac.py）

```
# ... snip ...

# Import the goods
from cluster import HierarchicalClustering

# Define a scoring function
def score(title1, title2):
    return DISTANCE(set(title1.split()), set(title2.split()))

# Feed the class your data and the scoring function
hc = HierarchicalClustering(all_titles, score)

# Cluster the data according to a distance threshold
clusters = hc.getlevel(DISTANCE_THRESHOLD)

# Remove singleton clusters
clusters = [c for c in clusters if len(c) > 1]

# ... snip ...
```

如果你对分层聚类的变形感兴趣，就一定要看看HierarchicalClustering类的setLinkageMethod方法，它提供了该类如何计算群集之间距离的微小变化。例如，可以指定群集之间的距离是否应该由计算任意两个群集之间的最短、最长或平均距离确定。根据数据的分布，选择不同的链接方法可能会产生完全不同的结果。图6-3显示了作为径向树布局和使用Protovis系统树图的专业网络的一部分（*http:vis.stanford.edu/protovis/*），它是基于HTML5的可视化工具包。径向树布局更节省空间，对于这个特定的数据集来说，它可能是更好的选择，但是如果你需要在一个更复杂的数据集中寻找树中每级之间的关系（相当于分层聚类中凝聚的每级），系统树图（*http://en.wikipedia.org/wiki/Dendrogram*）是一个好的选择。这里显示的两个可视化实际上只是图6-2中交互

式树型控件的静态具体化。正如它们显示的那样，在你能查看你的职业网络的简单图像时，就会显示出惊人的信息量[注7]。

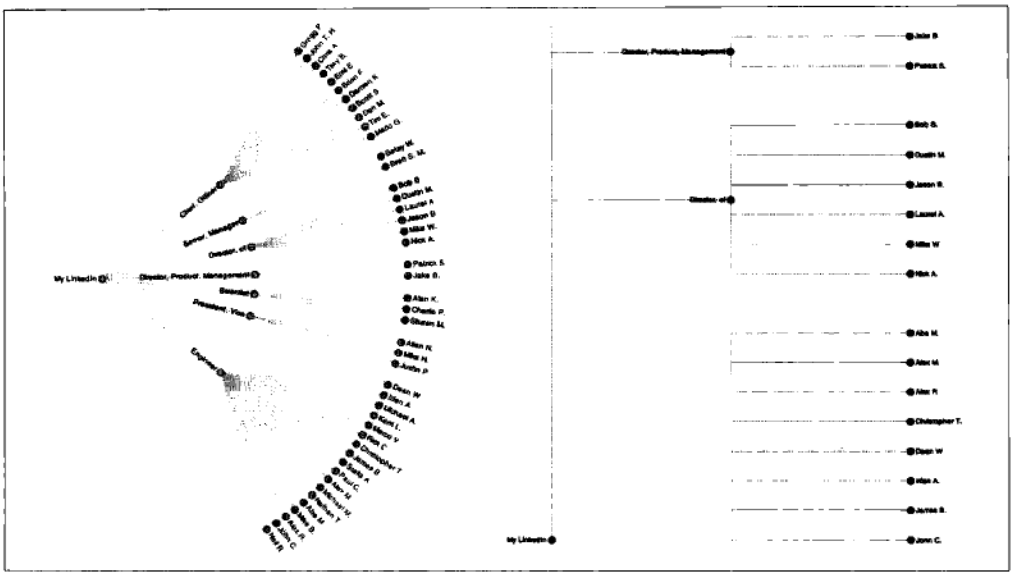

图6-3：按职位聚类的联系人的Protovis径向树布局（左），由系统树图表示的一部分相同数据（右）

*k*均值聚类

虽然分层聚类是一种可以详尽研究所有可能性的确定性技术，但是它是一种O(n^2)方法，而*k*均值聚类一般只需要执行O($k*n$)次。性能上的大量节省是以近似的结果为代价的，但是它们的潜力仍然不可限量。它的思想是通常有一个包含n个点的多维空间，可以通过以下的步骤将它们划分到k个聚类中：

1) 随机选择数据空间中的k个点作为计算k个聚类的初始值：$K_1,K_2,\cdots,K_k$。

2) 通过查找最近的K_n，把n个点都分配到聚类中——有效地创建了k个聚类，需要$k*n$次比较。

3) 对于k个聚类，分别计算它们的质心（*http://en.wikipedia.org/wiki/Centroid*），或是聚类的均值，重新给该值分配它的K_i值。（因此，需要在算法的每次迭代中计算“*k*均值”。）

注7：linkedin_cluster_contacts_by_title_hac.py（*https://github.com/ptwobrussell/mining-the-Social-Web/blog/master/python_code/linkedin_cluster_contacts_by_title_hac.py*）提供了对示例6-6的最低限度的修改，它说明了如何产生可以被Protovis使用的输出。

4) 重复步骤2)～3)，直到在迭代过程中聚类的所有成员都不再变化为止。一般来说，收敛需要的迭代相对较少。

因为乍看时，*k*均值可能并不那么直观，所以我们在图6-4中显示了该算法的每一步，可以在“Tutorial on Clustering Alogrithms”（*http://home.dei.polimi.it/matteucc/Clustering/tutorial_html/AppletKM.html*）中查看其具体步骤，它的特点是交互式Java applet。所使用的样本参数包含100个数据点，参数*k*的值为3，这意味着该算法会产生3个聚类。重要的是注意每一步中平方的位置，以及在算法向下运行时，这3个聚类中会包含哪些点。只需要9步就可以完成该算法。

虽然可以在二维或两千维空间中运行*k*均值算法，但是最常见的范围通常是数十维，而最常见的情况是二维或三维空间。当使用的空间维度相对较小时，*k*均值可能是有效的聚类技术，因为它的执行非常迅速，而且可以产生相当合理的结果。然而，确实需要挑选适当的*k*值，这通常是不明显的。下一节会说明如何获取联系人的更多补充个人信息，通过使用*k*均值，可以将这次讨论和从地理上聚类职业网络的后续练习联系起来。

获取补充个人信息

虽然本章前面的练习很有趣，但是你要尽早开始使用LinkedIn API来挖掘可用数据的整体丰富性。正如你期望的那样，在你登记获取授权证书的地方有基本的开发者选择性加入过程；你会想要访问*http://developer.linkedin.com*，详细研究所有细节，但是图6-5说明了大致的情况。一旦拥有了证书，整个过程就和其他需要执行OAuth dance访问API的社交网络相似了：请求API证书，返回标记和密钥，然后使用这些值来最终获得渴望已久的访问令牌，需要在发送请求时使用该令牌。当然，我们会把大部分繁琐的细节抽象出来，因为我们要使用`linkedin`模块来完成最乏味的工作。只需要运行`easy_install python-linkedin`命令即可。

安装好了linkedin包之后，就可以使用下面的脚本作为工作模板（示例6-12）登录，开始在API级别上访问职业网络。linkedin包仍然是不成熟的，所以当你阅读到这部分内容时，这个过程可能已经变得更精简了[注8]。

注8：从2010年10月开始，LinkedIn推出了OAuth 2实现。可以从*http://developer.linkedin.com/doc*中得到在1.6版本中实现的OAuth 1.0a 的`linkedin`模块的概述。

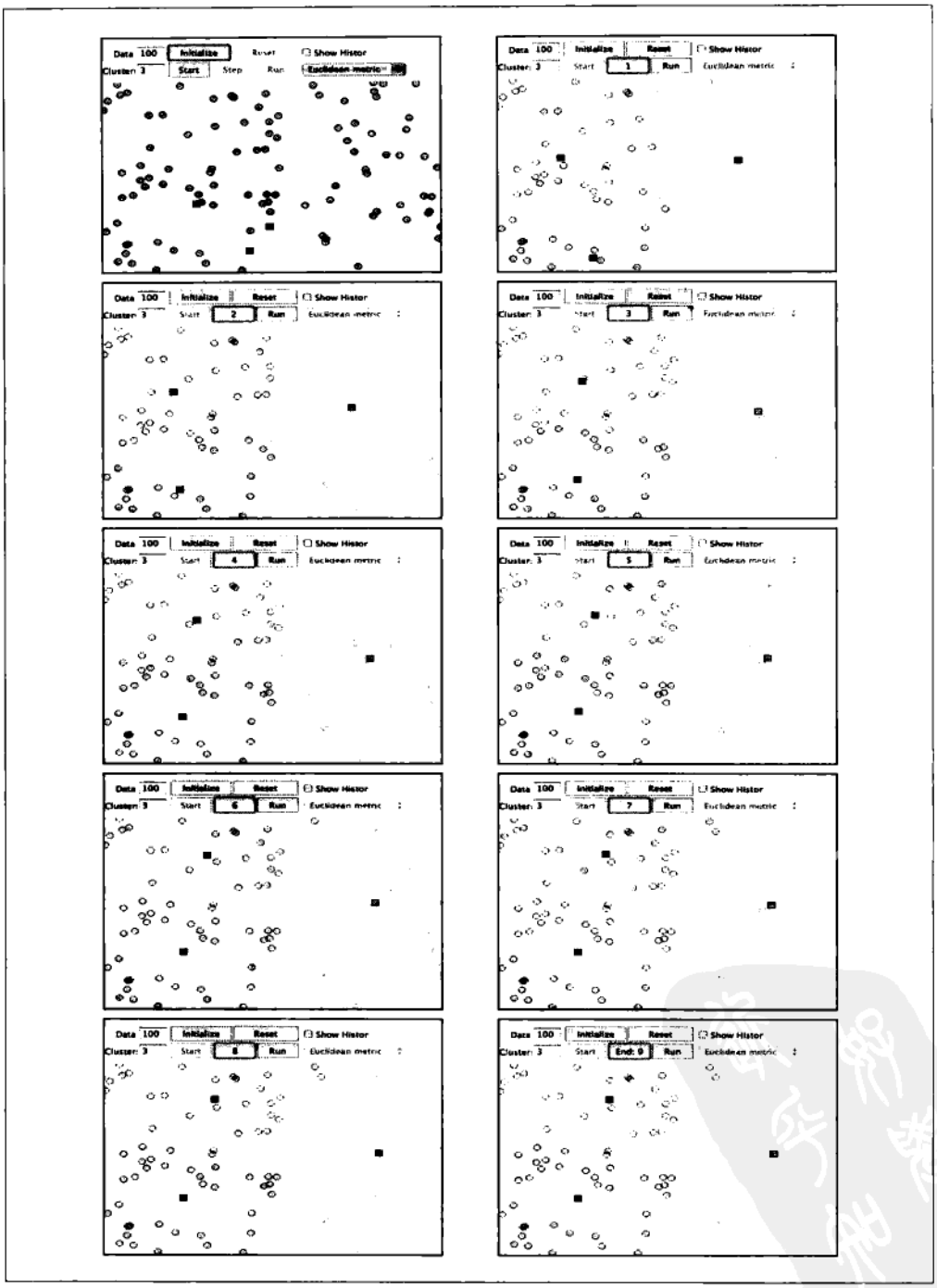

图6-4：使用100个点且k=3的k均值的进程

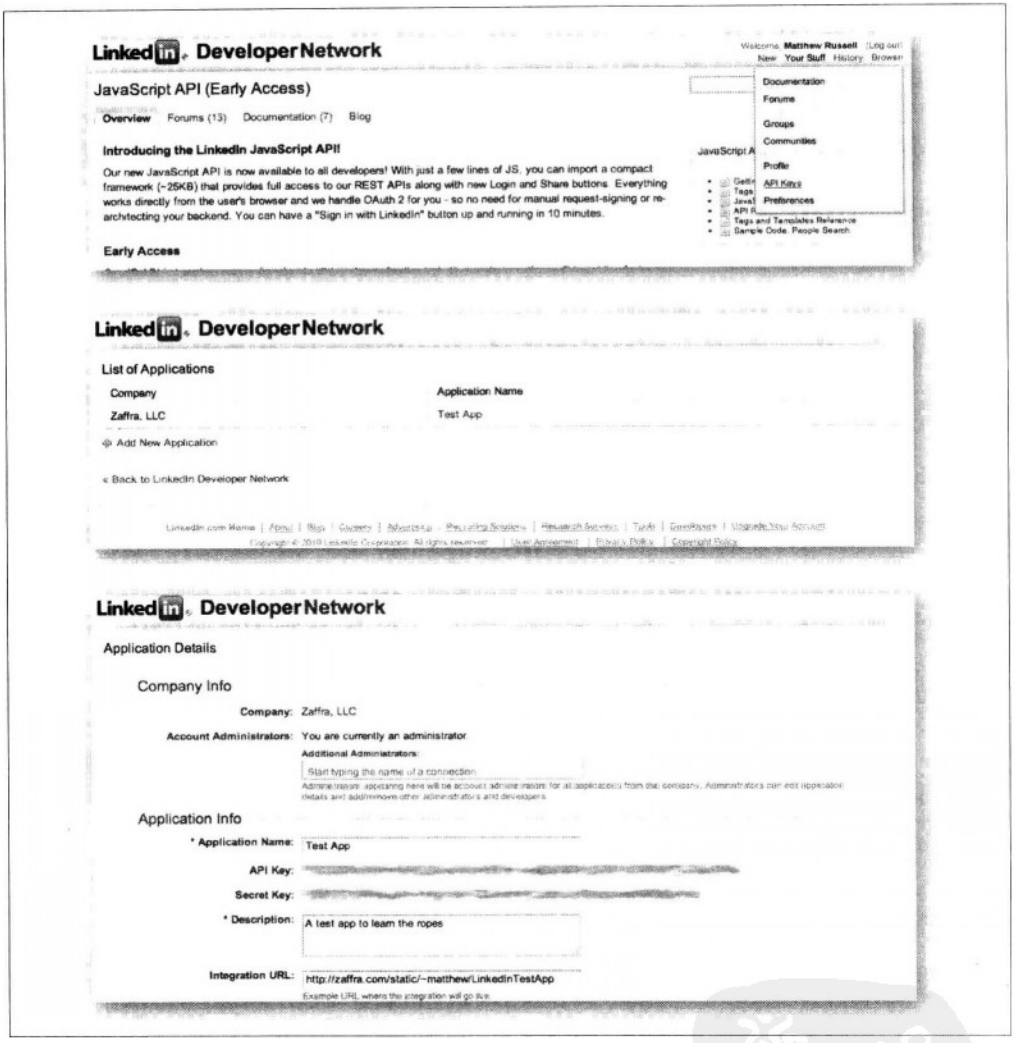

图6-5：获取LinkedIn API证书的基本步骤：从http://developer.linkedin.com中选择Your Stuff | API Keys（上），然后创建一个新的应用程序（中），最后设置应用程序参数，记录API Key和Secret Key（下）

示例6-12：获取LinkedIn联系人的补充个人信息（linkedin_get_connections.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import webbrowser
import cPickle
from linkedin import linkedin
```

```
KEY = sys.argv[1]
SECRET = sys.argv[2]

# Parses out oauth_verifier parameter from window.location.href and
# displays it for the user

RETURN_URL = 'http://miningthesocialweb.appspot.com/static/linkedin_oauth_helper.html'

def oauthDance(key, secret, return_url):
    api = linkedin.LinkedIn(key, secret, return_url)

    result = api.requestToken()

    if not result:
        print >> sys.stderr, api.requestTokenError()
        return None

    authorize_url = api.getAuthorizeURL()

    webbrowser.open(authorize_url)

    oauth_verifier = raw_input('PIN number, bro: ')

    result = api.accessToken(verifier=oauth_verifier)
    if not result:
        print >> sys.stderr, 'Error: %s\nAborting' % api.getRequestTokenError()
        return None

    return api

# First, do the oauth_dance

api = oauthDance(KEY, SECRET, RETURN_URL)

# Now do something like get your connections:

if api:
    connections = api.GetConnections()
else:
    print >> sys.stderr, 'Failed to aunthenticate. You need to learn to dance'
    sys.exit(1)

# Be careful - this type of API usage is "expensive".
# See http://developer.linkedin.com/docs/DOC-1112

print >> sys.stderr, 'Fetching extended connections...'

extended_connections = [api.GetProfile(member_id=c.id, url=None, fields=[
    'first-name',
    'last-name',
    'current-status',
    'educations',
    'specialties',
    'interests',
    'honors',
    'positions',
    'industry',
    'summary',
```

```
    'location',
    ]) for c in connections]

# Store the data

if not os.path.isdir('out'):
    os.mkdir('out')

f = open('out/linkedin_connections.pickle', 'wb')
cPickle.dump(extended_connections, f)
f.close()

print >> sys.stderr, 'Data pickled to out/linkedin_connections.pickle'
```

请注意，和Twitter不一样，LinkedIn要求你指定你自己的URL来捕获OAuth验证程序（访问令牌），我们需要用它来授权运行Python程序这样的命令行客户端。如果你很好奇的话，你可以在GitHub中得到示例6-13的网页模板，示例6-13的网页模板可以从GitHub（*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/web_code/linkedin/linkedin_oauth_helper.html*）中获取，它说明了如何提取信息，并以Twitter风格方式把它显示给用户。

示例6-13：提取LinkedIn OAuth验证程序的示例模板，并把它显示给你

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
        "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
    <title>LinkedIn OAuth Helper</title>

    <script src="http://ajax.googleapis.com/ajax/libs/dojo/1.5/dojo/dojo.xd.js"
        type="text/javascript"></script>
    <script type="text/javascript">
        dojo.ready(function() {
            var q = dojo.queryToObject(window.location.search.slice(1));
            dojo.byId("oauth_verifier").innerHTML = q.oauth_verifier;
        });
    </script>
</head>
<body>
    <div>
        Type the following identifier into the prompt in your terminal:
        <strong><span id="oauth_verifier"></span></strong>
    </div>
</body>
</html>
```

这基本上就是正常使用linkedin包需要知道的，但是使用API需要考虑一些其他重要的相关事项。最相关的也许是LinkedIn的速率流限制（*http://developer.linkedin.com/docs/DOC-1112*）比你在其他地方看到的更复杂一些。构建的应用程序有一个每日总速率限制，应用程序中的个人用户有速率限制，而且，在大多数情况下，作为开发者有另一个速率限制。除此之外，不同的速率限制取决于特定的API，以及应用程序的用户是否可

以访问他们自己的数据或网络中其他人的数据。一旦达到或超过该速率限制，就必须等到午夜太平洋标准时间（Pacific Standard Time，PST）来重置它。示例6-12说明了如何通过GetConnection获取用户的“个人基本资料”，以及如何通过在GetProfile文件中传送参数来获取全部公开的个人详细资料。通过长期运行的进程，速率限制并没有真正提供给你积累大量数据的能力，所以本章剩余的示例没有使用这些技术，如使用线程池来加速获取数据。你可以轻易坚持一分钟，或者如果你是喜欢一次花光所有钱的那种人的话，它可以用来扩大你的速率限制。

从地理上聚类网络

现在已经有了访问LinkedIn补充个人信息的技术诀窍和常见聚类算法的知识，剩下的就是介绍可以将它们集合起来的一种完美可视化。下一节在聚类职业联系人问题中使用了*k*均值，并且将它们绘制在Google Earth中。后一节介绍称为*Dorling Cartogram*的另一种可视化，它实际上是地理上聚类的泡泡图，可以让你轻易可视化生活在各个州的联系人。讽刺的是，它没有显式使用*k*均值这类聚类算法，但是它仍然可以产生映射到2D空间的直观结果，而且它还传送了地理聚类和频率信息。

使用Google Earth标识职业网络

如果你是Google Earth的粉丝，查看实际*k*均值的一个有趣示例是：通过将它放在地图或地球仪上，使用它来可视化并且聚类职业LinkedIn网络。除了通过可视化联系人的扩展方式获取的见解之外，还可以使用联系人、联系人的不同雇主、或者联系人居住的大城市来分析聚类。这三种方法都可以产生对于不同用途有用的结果。通过LinkedIn API，可以获取描述“Greater Nashville Area”这类主要大都市区域的位置信息，所有联系人居住在哪个区，整理方法非常适合将位置编码到坐标中，这样就可以在Google Earth这样的工具中绘制它们了。

为了执行可视化，必须做的事情主要包括：

- 从每个联系人的公开个人资料中解析出地理位置。示例6-12说明了如何获取这类信息。
- 把位置编码回坐标中。我们采用的方法是运行`easy_install geopy`命令，并且让它处理所有困难的工作。可以在线（*http://code.google.com/p/geopy/wiki/GettingStarted*）获取一本不错的入门指南；根据选择的地理编码器，可能需要从Google或Yahoo!这类服务提供商那里申请API密钥。
- 把地理坐标提供给cluster模块中的KMeansClustering类来计算聚类。

- 构建可以被提供给Google Earth这类可视化工具的KML。

一旦拥有了示例6-14中的基本工具，很多有趣的细微差别和变化都会成为可能。提到的linkedin_kml_utility非常无趣，它只执行一些XML整理；可以在GitHub中查看详情。

注意： 回想一下2.3节的内容，如果你不想下载和使用Google Earth的话，你可以将Google Earth指向一个可寻址的URL，该URL会指向一个KML文件。

示例6-14：将LinkedIn联系人的位置地理编码，并将它们导出到KML中（linkedin_geocode.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import cPickle
from urllib2 import HTTPError
from geopy import geocoders
from cluster import KMeansClustering, centroid

# A very uninteresting helper function to build up an XML tree

from linkedin__kml_utility import createKML

K = int(sys.argv[1])

# Use your own API key here if you use a geocoding service
# such as Google or Yahoo!

GEOCODING_API_KEY = sys.argv[2]

CONNECTIONS_DATA = sys.argv[3]

OUT = "clusters.kmeans.kml"

# Open up your saved connections with extended profile information

extended_connections = cPickle.load(open(CONNECTIONS_DATA))
locations = [ec.location for ec in extended_connections]
g = geocoders.Yahoo(GEOCODING_API_KEY)

# Some basic transforms may be necessary for geocoding services to function properly
# Here are a few examples that seem to cause problems for Yahoo. You'll probably need
# to add your own.

transforms = [('Greater ', ''), (' Area', ''), ('San Francisco Bay',
              'San Francisco')]

# Tally the frequency of each location

coords_freqs = {}
for location in locations:

    # Avoid unnecessary I/O
```

```
    if coords_freqs.has_key(location):
        coords_freqs[location][1] += 1
        continue
    transformed_location = location

    for transform in transforms:
        transformed_location = transformed_location.replace(*transform)
        while True:
            num_errors = 0
            try:

                # This call returns a generator

                results = g.geocode(transformed_location, exactly_one=False)
                break
            except HTTPError, e:
                num_errors += 1
                if num_errors >= 3:
                    sys.exit()
                print >> sys.stderr, e
                print >> sys.stderr, 'Encountered an urllib2 error. Trying again...'
        for result in results:

            # Each result is of the form ("Description", (X,Y))
            coords_freqs[location] = [result[1], 1]
            break

# Here, you could optionally segment locations by continent
# country so as to avoid potentially finding a mean in the middle of the ocean
# The k-means algorithm will expect distinct points for each contact so build out
# an expanded list to pass it

expanded_coords = []
for label in coords_freqs:
    ((lat, lon), f) = coords_freqs[label]
    expanded_coords.append((label, [(lon, lat)] * f)) # Flip lat/lon for Google Earth

# No need to clutter the map with unnecessary placemarks...

kml_items = [{'label': label, 'coords': '%s,%s' % coords[0]} for (label,
             coords) in expanded_coords]

# It could also be interesting to include names of your contacts on the map for display

for item in kml_items:
    item['contacts'] = '\n'.join(['%s %s.' % (ec.first_name, ec.last_name[0])
                                  for ec in extended_connections if ec.location
                                  == item['label']])

cl = KMeansClustering([coords for (label, coords_list) in expanded_coords
                       for coords in coords_list])

centroids = [{'label': 'CENTROID', 'coords': '%s,%s' % centroid(c)} for c in
             cl.getclusters(K)]

kml_items.extend(centroids)
kml = createKML(kml_items)
```

```
if not os.path.isdir('out'):
    os.mkdir('out')

f = open("out/" + OUT, 'w')
f.write(kml)
f.close()

print >> sys.stderr, 'Data pickled to out/' + OUT
```

警告： 作为LinkedIn个人信息一部分返回的位置值的一般形式为“Greater Nashville Area”，为了提取城市名称，需要一定量的割裂。这里介绍的方法并不完美，可能需要基于实际数据对其修改，以达到整体的准确度。

如示例6-6所示，包含在可视化结果的大部分工作都是数据处理样板。大多数有趣的细节都隐藏在清单末尾的KMeansClustering的getclusters()方法调用中。该方法说明了按地址将你的联系人分组，将它们聚类，然后使用聚类算法的结果计算质心。图6-6表示运行6-14代码产生的结果。

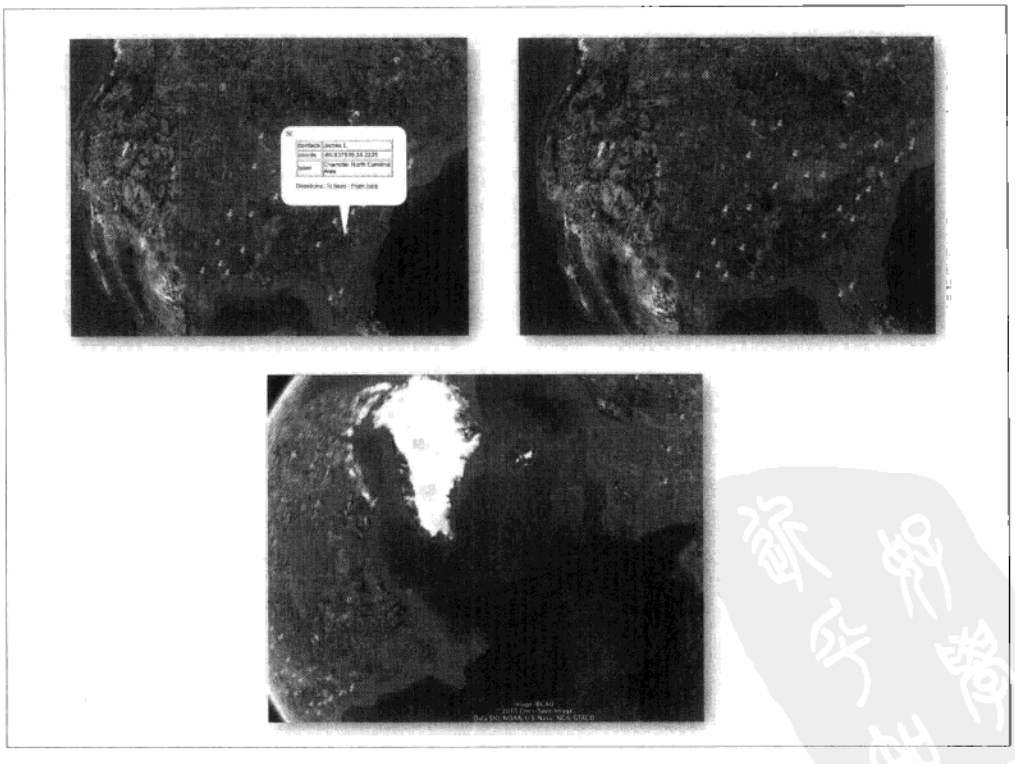

图6-6：从左上角到底部：1)按地址聚类联系人，这样你就可以轻松地看到谁住在哪个城市/谁在这个城市工作，2)寻找按k均值计算的3个聚类的质心，3)当试着寻找理想的会议地点时，不要忘了聚类可以横跨城市或者大陆

只可视化网络可能会很有趣，但是计算职业网络的地理质心也可能引入一些有趣的可能性。例如，你可能想要计算一系列区域研讨会或会议的备选地点。或许，如果你从事的是咨询行业，并且行程安排得很满的话，你可能想标注出一些适合于外出时租用的好位置。除了通过可视化职业网络的位置数据而引入的众多选项，地理聚类适合于很多其他可能性，如供应链管理和旅行商（*http://en.wikipedia.org/wiki/Travelling_salesman_problem*）这类问题。

使用Dorling Cartograms标识职业网络

第7章介绍的Protovis（*http://vis.stanford.edu/protovis/*）是基于尖端HTML5的一个可视化工具包，它包括一种称为Dorling Cartogram的可视化，后者实质上是地理聚类的泡泡图。而更传统的统计图可以通过将地图上一个州的地理界限变形来传送信息，Dorling Cartogram在地图上大约是州的实际位置处指定统一的形状（如圆形），使用圆的圆周（通常是颜色）来编码信息，如图6-7所示。它们都是强大的可视化工具，因为它们允许使用直觉来判断信息应该出现在2D地表测绘的什么地方，而且它们可以使用面积和颜色这样非常直观的属性来编码参数。

注意： Protovis也包括一些其他可以传送地理信息的可视化，热图（如heatmap）（*http://vis.stanford.edu/protovis/ex/heatmap.html*）、符号地图（*http://vis.stanford.edu/protovis/ex/symbol.html*）和地区分布图（*http://vis.stanford.edu/protovis/ex/choropleth.html*）。查看“A Tour Through the Visualization Zoo”（*http://queue.cm.org/detail.cfm?id=1805128*）来获取这些和可能有用的许多其他可视化概述。

假设你已经执行了示例6-14，并且已经成功地通过LinkedIn提供的地理数据对联系人进行了地理编码，稍微修改一下脚本就能产生不同的输出，这就是加强Protovis Dorling Cartogram可视化所要做的全部工作[注9]。对标准的Dorling Cartogram示例修改后的版本可以在GitHub中得到（*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/linkedin_create_dorling_cartogram_output.py*），而且会产生类似于图6-7的可视化。示例代码连接圆点来产生有用的可视化，但是，当点击某个特殊的州时，添加事件处理程序来显示联系信息就可实现这一可视化效果。正如我们在示例6-14之后提出的警告，这里实现的地理编码方法一定是不完善的，可能需要一些调整。

小结

本章覆盖了很多方面，介绍了基本的聚类技术，并以各种方式将它们应用到职业网络数据中。和其他章节一样，我们只接触了冰山的一角；还可以使用LinkedIn数据做很多其

注9：Protovis Dorling Cartogram可视化目前只能在美国实现处理位置信息。

他有趣的事，但本章并没有介绍这些内容。花点时间来研究一下你得到的补充个人信息。尝试把人们在哪儿工作和他们在哪儿上的学关联起来，并且/或者分析人们是否倾向于搬到和搬出某个地方，这很有趣，但是请记住，你的工具带包含很多不必包含geo数据或聚类的其他工具。如果你要继续研究geo数据，你可能想尝试的一个项目是最新出现的开源框架geodict（*http:petewarden.typepad.com/searchbrowser/2010/10/geodict-an-open-source-tool-for-extracting-locations-from-text.html*）。

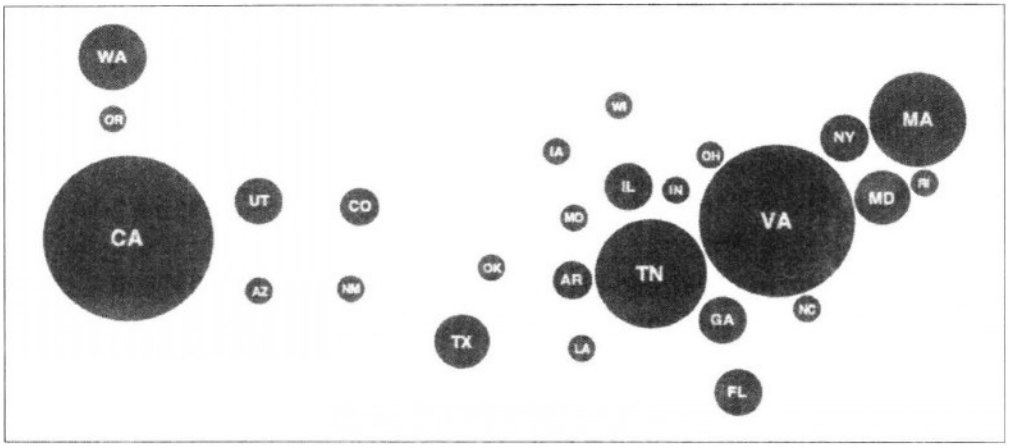

图6-7：使用圆面积来表示居住在各州的同事数量的Dorling Cartogram；使用颜色表示与州有关的失业率或你同事的工作转换率这类变量，这些都可以通过挖掘补充个人信息来确定

第7章

Google Buzz：TF-IDF、余弦相似性和搭配

本章将正式开始文本挖掘[注1]，这一章是本书的转折点。前面的章节主要集中在分析结构化或半结构化数据，如被编码成微格式的记录、人与人之间的关系，或tweet中特别标记的#hashtag。然而，本章通过介绍TF-IDF、余弦相似性和搭配检测这类信息检索（Information Retrieval，IR）理论基础，开始整理并了解文件中的文本信息。你可以从本章标题推断，Google Buzz是数据的主要来源，因为它本身就是社交的，且容易获取[注2]，而且有社交网络的很多潜力（*http://radar.oreilly.com/2010/02/google-buzz-re-invents-gmail.html*）。在本章结束前，我们也会研究如何利用Gmail数据。后面的几章会研究挖掘博客数据和其他自由文本资源，介绍作为其他形式的文本分析，如实体提取和摘要的自动生成。除了它填充了Twitter和博客之间的一个有趣利基这个事实之外，没有真正的理由在博客（第8章的主题）之前介绍Buzz，这样这个顺序就可以有助于从头到尾讲述故事。总的来说，你在本书任何一章中学到的文本挖掘技术都可以在其他章节中使用。

在可能的情况下，我们不必从头开始实现分析工具，但是当遇到了解文本挖掘必不可少的基础主题时，我们将“深入研究”它们。自然语言工具包（Natural Language Toolkit，NLTK）这种强大的技术提供了很多工具，我们已经在第1章的一些示例中使

注1：本书避免对“文本挖掘”、“非结构化数据分析”（Unstructured Data Analytics，UDA）或者“信息检索”这类常见短语隐含的区别作极细微的区分，只是简单地把它们看做相同的内容。

注2：也有点一语双关，因为Google最出名的就是它的搜索功能了，本章会进一步分析以文档为中心的基本搜索技术。

用过它了。它丰富的API套件起初可能会让你有点不知所措，但是不要担心：虽然文本分析是一个非常多样和复杂的研究领域，但是它有很多强大的基本原则，在你没有投入大量精力的情况下就可以完成很多工作。本章和下面的几章旨在进一步了解这些基础原则。

注意： 对NLTK的全面介绍不在本书范围内，但是可以在网上查阅《Natural Language Processing with Python:Analyzing Text with the Natural Language Toolkit》（O'Reilly）（*http://www.nltk.org/book*）。这本书的平装本可以在*http://oreilly.com/catalog/9780596516499*上购买。

Buzz=Twitter+博客（???）

拥有Gmail账号的人便同时拥有了Buzz账号。从产品的角度来看，Google Buzz似乎介于Twitter和博客之间（*http://radar.oreilly.com/2010/03/google-buzz-and-the-conversati.html*）。Buzz上的一次活动可以像一篇博文那样长，但是它和tweet一样，更加通俗。与Twitter类似，Buzz通常用来以最少的格式快速表达自己的想法，而且活动流建立在好友和关注者之间。事实上，很多Buzz用户会自定义他们的Buzz设置，这样Twitter中的活动就可以出现在他们的Buzz源中了（参见图7-1）。编写本书时，Buzz的API已经比Twitter更紧凑了，而且也没有制定速率限制。只要它们持续下去，这两个特性就都可以简化挖掘Buzz数据的工作。看Buzz如何发展到多少有点像Twitter和/或博客会很有趣。让我们从获取和挖掘Buzz的数据开始吧！值得花点时间熟悉一下Buzz如何通过公开源或Gmail账号中的“Buzz”导航链接工作。你的（和其他所有人的）公开Buzz源应该可以通过下面形式的URL得到：*http://www.googleapis.com/buzz/v1/activities/your_gmail_account_id/@public*。例如，可以从*http://www.googleapis.com/buzz/v1/activities/timoreilly/@public*中得到Tim O'Reilly的公开Buzz源。

Buzz提供了REST风格的API（*http://code.google.com/apis/buzz/v1/using_rest.html*），它包括3种基本资源：人、活动和评论。Buzz“活动”和“文章”的意思几乎相同，以后我们就会用基于文本的资源来考虑它们。（然而，一定要记住，关于Buzz的一件有趣的事是它如何将其他Google产品和服务中其他类型的富媒体（如相片和视频）无缝地集成到活动流中。）

如果你对REST风格的方式感兴趣的话，Google Buzz API文档（*http://code.google.com/apis/buzz/v1/using_rest.html*）描述了与Buzz交互REST风格的方式，但是如果你想了解REST的更多内容，并且想要看一些处理不那么明显的情况中的实际示例的话，《RESTful Web Services》（O'Reilly）这本书绝对值得你拥有（*http://oreilly.com/catalog/9780596529260*）。

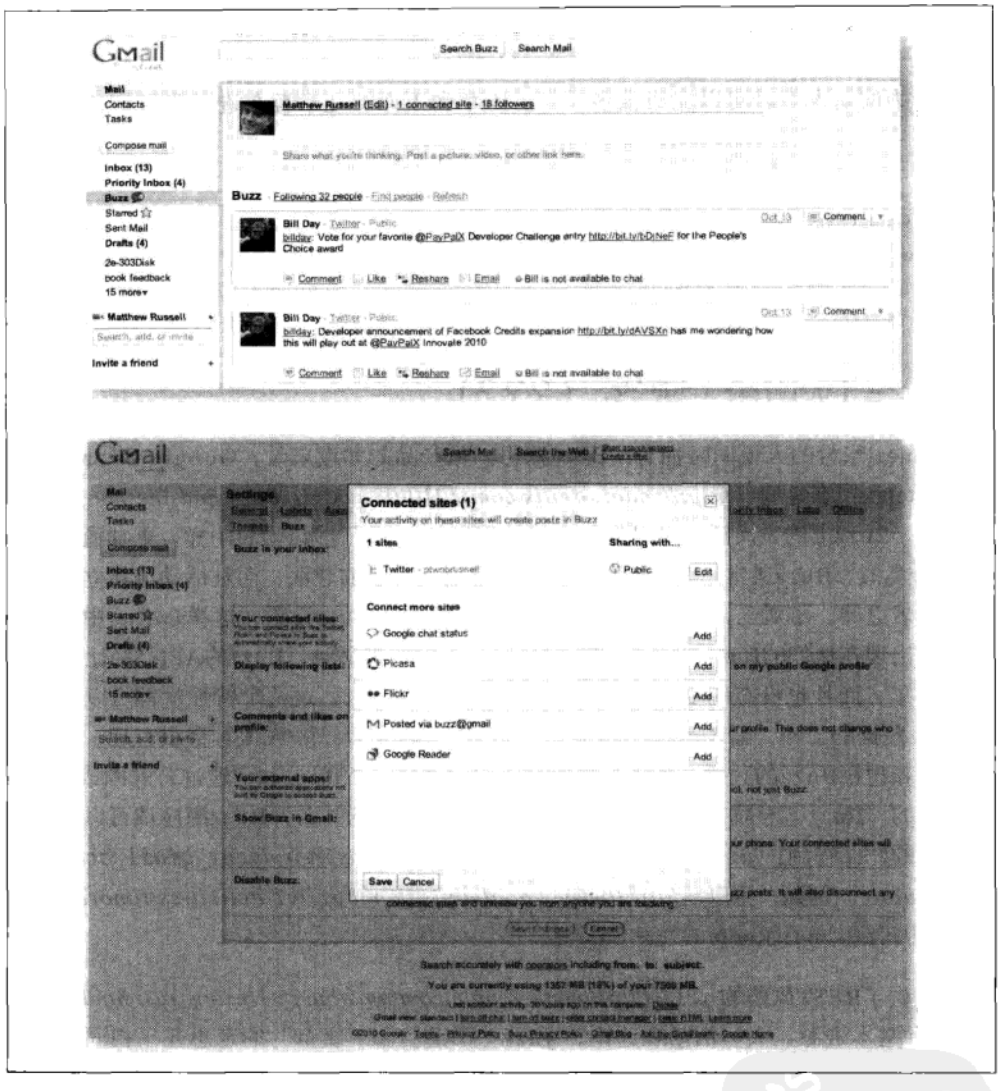

图7-1：Gmail账号设置允许轻松地把Twitter和其他服务中的活动集成到Buzz源中

可以从*http://code.google.com/p/buzz-python-client/*中得到基于Python的、叫做buzz的良好的Buzz客户端。可以通过运行`easy_install buzz-python-client`命令来安装它。

让我们从Tim O'Reilly的公开Buzz源中抓取一些数据吧！如果你发现你自己需要请求认证的资源，虽然`buzz`模块也支持OAuth，但是本章中我们只使用可以从公开的个人资料中得到的信息，所以我们并不需要使用OAuth。虽然从Google Buzz中挖掘社交关系数据可能会有很多有趣的可能性，但是我们会重点关注文章中的自由文本数据，而不是重新

建立或依赖第4章和第5章介绍的好友/关注者方法。 如果你对以这种方式与Buzz交互感兴趣的话，请查阅这两章。

Buzz数据作为源显示，你应该假设它包含标记、转义的HTML实体等。下面的清单提供了一个示例会话，它说明了如何使用buzz模块从100多篇公开的文章中抓取并提取适合挖掘使用的纯文本。虽然第一个示例说明了CouchDB作为存储介质的使用，但是本章将使用CouchDB进行数据持久化，这样我们就可以进一步研究文本挖掘技术，而不是讨论存储注意事项了。如果你对积极使用CouchDB和couchdb-lucene这类有用的附件感兴趣的话，可以重温一下前面的几章。

示例7-1创建了一个实用的数据结构，由标题、内容、评论和每篇Buzz文章的链接组成，把它存储在CouchDB数据库中。人们很容易忽视或者想当然，但是也要注意cleanHtml的重要性。用它来删除标记和转换HTML实体，以便尽可能使每篇文章的内容清晰且具有较强的可读性——非常重要的考虑因素是你不希望在你的分析例程中看到HTML实体这类东西。

示例7-1：抓取Google Buzz数据（buzz_get_buzz_posts_and_comments.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import buzz
from BeautifulSoup import BeautifulStoneSoup
from nltk import clean_html
import couchdb
import json

USER = sys.argv[1]

MAX_RESULTS = 100

# Helper function for removing html and converting escaped entities.
# Returns UTF-8

def cleanHtml(html):
    return BeautifulStoneSoup(clean_html(html),
            convertEntities=BeautifulStoneSoup.HTML_ENTITIES).contents[0]

client = buzz.Client()
posts_data = client.posts(type_id='@public',
                          user_id=USER,
                          max_results=MAX_RESULTS).data

posts = []
for p in posts_data:

    # Fetching lots of comments for lots of posts could take a little
    # bit of time. Thread pool code from mailboxes__CouchDBBulkReader.py could
    # be adapted for use here.
```

```
    comments = [{'name': c.actor.name, 'content': cleanHtml(c.content)} for c in
                p.comments().data]
    link = p.uri
    post = {
        'title': cleanHtml(p.title),
        'content': cleanHtml(p.content),
        'comments': comments,
        'link': link,
        }
    posts.append(post)

# Store out to a local file as json data if you prefer

if not os.path.isdir('out'):
    os.mkdir('out')

filename = os.path.join('out', USER + '.buzz')
f = open(filename, 'w')
f.write(json.dumps(posts))
f.close()

print >> sys.stderr, "Data written to", f.name

# Or store it somewhere like CouchDB like so...

# server = couchdb.Server('http://localhost:5984')
# DB = 'buzz-' + USER
# db = server.create(DB)
# db.update(posts, all_or_nothing=True)
```

既然已经有了抓取和存储Buzz数据的工具，让我们开始分析它吧！

使用NLTK处理数据

编写NLTK是为了让你在无前期大规模投入的情况下，轻松地研究数据，并开始形成一些印象。然而，在跳过这部分内容之前，考虑效仿示例7-2的解释器会话来了解NLTK开箱即用的强大功能。不要忘了当需要时，随时可以使用内置的`help`函数获得更多信息。例如，`help(nltk)`会提供NLTK包中的文档。同时也要记住，解释器会话中的一些功能并不能合并到生产软件中，因为输出是通过标准输出写入的，并不能被列表这样的数据结构获取。在这一方面，`nltk.text.concordance()`这类方法被认为是“演示功能”。说到这个话题，NLTK的很多模块都有一个`demo`函数，可以调用`demo`函数来了解如何使用它们提供的功能，这些`demo`函数的源代码是学习如何使用新API的一个好起点。例如，可以运行解释器中的`nltk.text.demo()`来了解更多的`nltk.text`模块提供的功能。下面将仔细研究这些演示功能是如何工作的。

注意：本章的所有示例（包括下面的解释器会话）都使用`split()`方法来切分文本。第8章将介绍更复杂的切词方法，它们可以生成更好的结果。

示例7-2：使用NLTK处理解释器中的Google Buzz数据。示例7-1在运行脚本的out目录中产生USER.buzz文件

```
>>> import nltk
>>> import json
>>> buzz_data = json.loads(open("out/timoreilly.buzz").read())
>>> all_content = " ".join([ p['content'] for p in buzz_data ])
>>> len(all_content)
42930
>>> tokens = all_content.split()
>>> text = nltk.Text(tokens)
>>> text.concordance("open")
Building index...
Displaying 22 of 22 matches:
operating system. Andy told him why open systems matter. "If google did not a
mulus payments for meaningful use of open healthcare records. It could also be
planning meeting at the Times (it's open to the public). It's great to see ho
me as the most influential voice in open source. http://www.mindtouch.com/blo
ff, I don't tweet or blog much about open source any more - though the subject
eply influenced by my thinking about open source. I came to articulate my idea
about the long term implications of open source and how it would reshape the
ons have also taken me away from the open source mainstream. It's only recentl
ecently that members of the hardcore open source and free software advocacy co
ntally changing the context in which open source exists, and that it hasn't do
er in favor of understanding how the open source community actually communicat
y remind people to attend OSCON, the open Source Convention, where the long te
he long term thinking that drives my open source position is leading us to cov
n is leading us to cover topics like open source and cloud computing, open sou
ike open source and cloud computing, open source hardware, and many other topi
ks than you can count), I do have an open source bully pulpit. It just isn't t
Reilly to Keynote at Computerworld's open Source Business Conference http://bi
hasn't built a personal brand in the open source space such that his company i
trum auction and their launch of the open Handset Alliance are among the compa
ng their phone even further from the open web. In the end, if the iPhone fails
s have weaker adoption dynamics than open systems, not because Eric Schmidt be
he Gov't has put in place around the open government directive. Any pushing yo
>>> text.collocations()
Building collocations list
open source; Economist Innovation; 1/4 cup; long term; Innovation
event.; Web 2.0; I'd love; free software; Annalee Saxenian; Nexus
One.; came back; Cauliflower Pancakes; Eric Schmidt; Open Source;
certainly true; olive oil; Andy Rubin; Mr. O' Reilly; O' Reilly said.;
tech support
>>> fdist = text.vocab()
Building vocabulary index...
>>> fdist["open"]
19
>>> fdist["source"]
14
>>> fdist["web"]
3
>>> fdist["2.0"]
3
>>> len(tokens)
6942
```

```
>>> len(fdist.keys()) # unique tokens
2750
>>> [w for w in fdist.keys()[:100] \
... if w.lower() not in nltk.corpus.stopwords.words('english')]
[u'-', u"it's", u'open', u'like', u'people', u'also', u'would', u"It's", u"don't",
u'source', u'Apple', u'one', u'Google', u'government', u"I'd", u'Microsoft',
u'many', u'great', u'innovation', u'make', u'think', u'companies', u'even', u'get',
u'good', u'lot', u'made', u'new', u'story', u'technology', u')', u"I'm", u'big',
u'going', u'long', u'love']
>>> [w for w in fdist.keys() if len(w) > 15 and not w.startswith("http")]
[u'\u201c#airportsecurity', u'Caravaggiomania,', u'administrations,',
u'anti-competitive.', u'apparentlycompared', u'entrepreneur-friendly.',
u'entrepreneur-unfriendly.', u'flyontime.us/m/lines/security', u'government/small',
u'interoperability', u'over-psychologized', u'super-impressive.',
u'technology-minded']
>>> len([w for w in fdist.keys() if w.startswith("http")])
35
>>> for rank, word in enumerate(fdist): print rank, word, fdist[word]
0 the 317
1 to 199
2 of 180
3 a 150
4 that 129
5 and 122
6 in 110
7 is 79
... output truncated ...
```

警告： 如果还没有安装NLTK的停用词数据，你可能需要运行`nltk.download('stopwords')`来下载它。如果可能的话，建议运行`nltk.download()`安装NLTK的所有数据。

解释器会话中最后的命令列出了频率分布中的词，它们是按频率排序的。不出所料，“the”、“to”和“of”（停用词）这类常用词是最常出现的，但是曲线急剧下降，而且该分布的尾巴很长。我们使用很小的Buzz数据样本，但是相同的属性对自然语言的频率分析也成立。著名的自然语言经验定律“齐普夫定律”（*http://en.wikipedia.org/wiki/Zipf's_law*）断言：语料库中的词频与它在频率表中的排序成反比。这意味着如果语料库中最常出现的词占词汇总数的*N*%，那么语料库中次常出现的词应该占词数的(*N*/2)%，第三常出现的词占词数的(*N*/3)%，依次类推。当绘制图形时，这种分布（即使是对于小样本数据）显示的曲线与两条轴都相交，如图7-2所示。重要的观察结果是这类分布的大部分区域都位于尾巴上，对于足够跨越一种语言的合理样本的语料库来说，尾巴总是太长。如果你要绘制这类分布的图，其中两条轴都按对数缩短了，那么对于一个标准样本大小来说，绘制出来的曲线总是接近于直线。

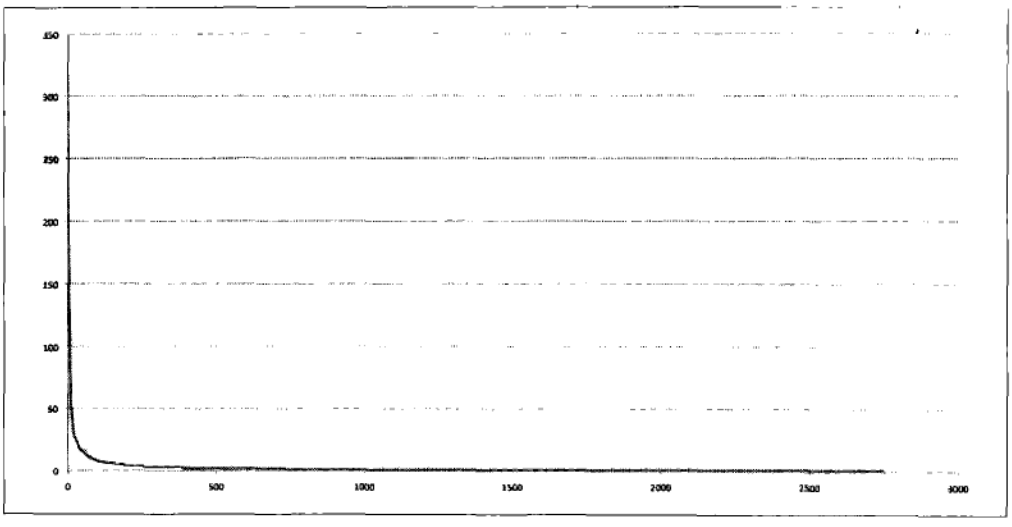

图7-2：出现在小Buzz数据样本中的词频分布，它与两条轴都相交

齐普夫定律可以让你进一步了解语料库中的词频分布应该是什么样的，它提供了一些有助于估计频率的经验法则。例如，如果你知道语料库中有100万（不唯一的）个词，而且你假设最常用的词（英语中通常是“the”）占总词数的7%[注3]，如果你考虑了频率分布中一些特殊词的话，你就可以推导出算法所执行的逻辑运算的总数。有时，这种“餐巾纸背后的简单算术”就是它需要的关于长时间运行的完整性检查假设，或者确定是否更容易处理足够大的数据集中的某些计算。

文本挖掘的基本原则

虽然为了深入了解文本数据，用严谨的方法来处理句子分割、切词、词块和实体检测这类自然语言处理（NLP）是必要的，但是它有助于第一次介绍信息检索理论的一些基本原则。本章剩余的部分将介绍一些更基本的方面，包括TF-IDF、余弦相似性度量和搭配检测的一些其他理论。第8章将提供NLP的深入讨论。

注意： 如果你想深入了解IR理论，可以从*http://nlp.stanford.edu/IR-book/information-retrieval-book.html*上获得《Introduction to Information Retrieved》的全文，它基本上提供了该领域所有的相关信息。

注3：“the”这个词占Brown语料库（*http://en.wikipedia.org/wiki/Brown_Corpus*）中全部标记的7%，只需知道它为语料库提供了一个合理的起点即可。

对TF-IDF的精彩介绍

信息检索领域非常广泛，它横跨了多个学科。现在我们只讨论TF-IDF，它是从语料库中提取相关文档最基本的技术之一。TF-IDF（Term Frequency-Inverse Document Frequency）表示词语的逆向文档频率，可以通过计算表示文档中词语相对重要性的归一化得分来查询语料库。从数学的角度来说，TF-IDF表示为词频和逆向文档频率的乘积：*tf_idf=tf*idf*，其中*tf*项表示一个词语在某个具体文档中的重要性，*idf*项表示一个词语在整个语料库中的重要性。将这两项相乘会产生说明这两个因素的得分，它已经成为有它存在的每个主要搜索引擎中不可或缺的一部分了。为了更直观地了解TF-IDF是如何工作的，让我们审查计算总分时涉及的所有计算。

为了方便说明，假设你有一个包含3个样本文档的语料库，通过简单地在空白处停顿来计算词语，如示例7-3所示。

示例7-3：本章中用于说明的样本数据结构

```
corpus = {
        'a' : "Mr. Green killed Colonel Mustard in the study with the candlestick. \
               Mr. Green is not a very nice fellow.",
        'b' : "Professor Plumb has a green plant in his study.",
        'c' : "Miss Scarlett watered Professor Plumb's green plant while he was away \
               from his office last week."
}

terms = {
        'a' : [ i.lower() for i in corpus['a'].split() ],
        'b' : [ i.lower() for i in corpus['b'].split() ],
        'c' : [ i.lower() for i in corpus['c'].split() ]
}
```

词频可以简单地表示为它在文本中出现的次数，但是更常见的情况是通过考虑文本中词语的总数来归一化它，这样总分就能表示与词频相关的文档长度了。例如，“green”（已经统一为小写字母）在`corpus['a']`中出现了两次，在`corpus['b']`中只出现了一次，所以如果频率是唯一的评分标准的话，那么`corpus['a']`的得分会更高。然而，如果要归一化文档长度，那么`corpus['b']`中“green”的词频得分(1/9)将会高于`corpus['a']`（2/19），因为`corpus['b']`的长度小于`corpus['a']`，即使“green”更常出现在`corpus['a']`中。因此，计算“Mr. Green”这类复合查询得分的常见技术是将每个文档中查询词的词频得分相加，返回按所有词频得分排序的文档。

上一段其实很好理解；例如，查询语料库中每个文档中的“Mr. Green”会返回表7-1所示的归一化得分。

表7-1：“Mr. Green”的样本词频得分

文档	tf(Mr.)	Tf(Green)	总和
Corpus ['a']	2/19	2/19	4/19(0.2105)
Corpus ['b']	0	1/9	1/9(0.1111)
Corpus ['c']	0	1/16	1/16(0.0625)

对于这个精心设计的示例来说，制定一个累积词频得分方案算出并返回corpus['a']（我们希望返回的文档），因为只有corpus['a']包含复合标记“Mr. Green”。然而，可能会出现很多问题，因为词频得分模式把所有文档都看做一个无序的词语集。例如，即使“Green Mr.”或“Green Mr. Foo”这两个复合标记都没有出现在样例语句中，但查询它们会返回与查询“Mr. Green”完全相同的结果。此外，通过利用末尾的标点符号没有正确处理这个事实，我们可以轻易设计出很多方案来说明词频排序方法中比较差的结果，而且有趣的标记周围的上下文没有被考虑到计算中。

当基于文档评分时，只考虑词频被证明是一个共同的问题，因为它没有考虑在很多文档中都很常见的词。也就是说，所有的词都均等地加权，而没有考虑它们实际的重要性。例如，“the green plant”包含停用词“the”，它歪曲了有利于corpus['a']的总词频得分，因为“the”和“green”都在文档中出现了两次。相反，“green”和“plant”在corpus['c']中只出现了一次。因此，得分会被分解为表7-2所示的那样，其中corpus['a']的排名比corpus['c']更相关，即使直觉可能让你相信理想的查询结果不是那样的。（然而，幸运的是，corpus['b']的排名仍然是最高的）

表7-2：“the green plant”的样本词频得分

文档	tf(the)	tf(green)	Tf(plant)	总和
corpus['a']	2/19	2/19	0	4/19(0.2105)
corpus['b']	0	1/9	1/9	2/9(0.2222)
corpus['c']	0	1/16	1/16	1/8(0.125)

NLTK这类工具包提供了停用词列表，可以用它们来过滤“the”、“a”、“and”这类词，但是请记住，也有很多词不包含在最好的停用词列表中，这类情况在专精的领域中仍然很常见。逆向文档频率度量是一个计算结果，它提供了语料库的通用归一化度量，而且通过考虑不断出现某个查询词的文档总数，说明文档集合中常用术语的出现。基于该度量的直觉知识是如果某个词在语料库中不常出现，那么它就会产生一个更大的值，它有助于解释我们刚才研究的停用词问题。例如，对样本文档语料库中“green”的查询返回的逆向文件频率得分应该低于“candlestick”，因为每个文档中都有“green”，而

“candlestick”只出现了一次。从数学角度来说，逆向文档频率计算的唯一差别是用对数将结果压制到一个压缩的范围中（如图7-3所示），因为通常将它与词频相乘作为比例因子。

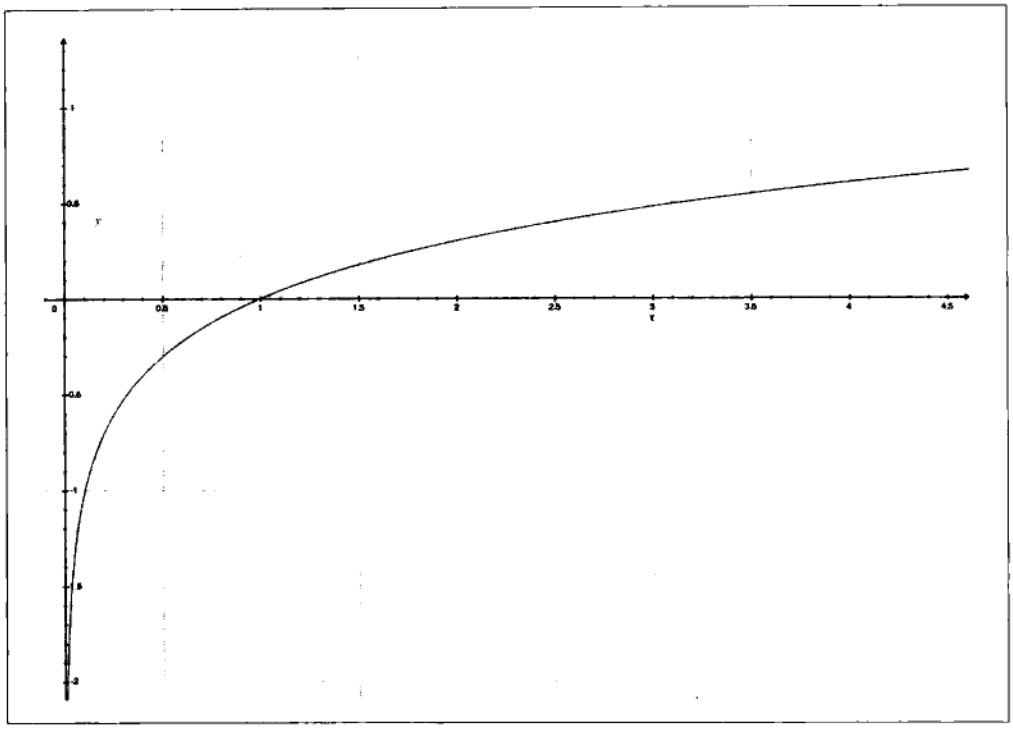

图7-3：对数将大范围的值压制到一个更加压缩的空间中

现在，我们已经回到了起点，而且涉及了计算多条查询得分的方法，它用于说明词语在文档中出现的频率、出现特定词的文档的长度和整个语料库文档中词语的整体唯一性。也就是说，`tf-idf=tf*idf`。示例7-4是该讨论的一次不太成熟的实现，它应该有助于巩固描述的概念。花一点时间来检查它，然后讨论一些示例查询。

示例7-4：在样本数据上运行TF-IDF（buzz_tf_idf.py）

```
# -*- coding: utf-8 -*-

import sys
from math import log

QUERY_TERMS = sys.argv[1:]

def tf(term, doc, normalize=True):
    doc = doc.lower().split()
    if normalize:
        return doc.count(term.lower()) / float(len(doc))
```

```
    else:
        return doc.count(term.lower()) / 1.0

def idf(term, corpus):
    num_texts_with_term = len([True for text in corpus if term.lower()
                               in text.lower().split()])

    # tf-idf calc involves multiplying against a tf value less than 0, so it's important
    # to return a value greater than 1 for consistent scoring. (Multiplying two values
    # less than 1 returns a value less than each of them)

    try:
        return 1.0 + log(float(len(corpus)) / num_texts_with_term)
    except ZeroDivisionError:
        return 1.0

def tf_idf(term, doc, corpus):
    return tf(term, doc) * idf(term, corpus)

corpus = \
    {'a': 'Mr. Green killed Colonel Mustard in the study with the candlestick. \
Mr. Green is not a very nice fellow.',
    'b': 'Professor Plumb has a green plant in his study.',
    'c': "Miss Scarlett watered Professor Plumb's green plant while he was away \
from his office last week."}

# Score queries by calculating cumulative tf_idf score for each term in query

query_scores = {'a': 0, 'b': 0, 'c': 0}
for term in [t.lower() for t in QUERY_TERMS]:
    for doc in sorted(corpus):
        print 'TF(%s): %s' % (doc, term), tf(term, corpus[doc])
    print 'IDF: %s' % (term, ), idf(term, corpus.values())
    print

    for doc in sorted(corpus):
        score = tf_idf(term, corpus[doc], corpus)
        print 'TF-IDF(%s): %s' % (doc, term), score
        query_scores[doc] += score
    print

print "Overall TF-IDF scores for query '%s'" % (' '.join(QUERY_TERMS), )
for (doc, score) in sorted(query_scores.items()):
    print doc, score
```

虽然我们研究的是一个非常小的样本，但是涉及的计算同样适用于更大的数据集。表7-3是对三个样本查询的程序输出的统一修改，它包括4个不同的词：

- “green”
- “Mr. Green”
- “the green plant”

即使词语的IDF计算的是整个语料库，但是在每个文档的基础中都显示了它们，这样你就可以通过浏览单行并把两个值相乘来轻易地验证TF-IDF结果了。而且，值得花几分钟时间来了解表中的数据，这样就可以了解计算工作的方式了。值得注意的是TF-IDF是多么强大，这是因为它不解释文档中词语的接近性或顺序。

表7-3：包含在TF-IDF样本查询中的计算，它是由示例7-4计算的

文档	tf(mr.)	tf(green)	tf(the)	tf(plant)
corpus['a']	0.1053	0.1053	0.1053	0
corpus['b']	0	0.1111	0	0.1111
corpus ['c']	0	0.0625	0	0.0625
文档	**tf(mr.)**	**idf(green)**	**idf(the)**	**idf(plant)**
corpus ['a']	2.0986	1.0	2 099	1 4055
corpus ['b']	2.0986	1.0	2.099	1.4055
corpus ['c']	2.0986	1.0	2.099	1.4055
文档	**tf(mr.)**	**tf-idf(green)**	**tf-idf(the)**	**tf-idf(plant)**
corpus ['a']	0.1053	0.1053	0.1053	0
corpus ['b']	0	0.1111	0	0.1111
corpus ['c']	0	0.0625	0	0.0625

与每个查询相同的结果如表7-4所示，基于每个文档汇总TF-IDF值。

表7-4：由示例7-4计算的，样本查询的总TF-IDF值

查询	corpus ['a']	corpus ['b']	corpus ['c']
green	0.1053	**0.1111**	0.0625
Mr.Green	0.1053+0.1053=**0.2105**	0+0.1111=0.1111	0+0.0625=0.0625
the green plant	0.1053+0.1053+0=0.2105	0+0.1111+0.1111=**0.2222**	0+0.0625+0 0625=0.1250

从定性的角度来说，查询结果非常有意义。corpus['b']文档是“green”查询的第一名，corpus ['a']是第二名。在这种情况下，决定因素是corpus ['b']的长度要比corpus ['a']小得多：归一化的TF得分倾向于只出现了一次“green”的corpus ['b']，即使“green”在corpus ['a']中出现了两次。因为三个文件中都出现了“green”，在这些计算中IDF词的有效影响是空的。然而，一定要注意，如果在IDF计算结果中返回了

0.0，而不是1.0，正如一些实现中所做的那样，那么三个文档中“green”的TF-IDF得分都是0.0。根据特定情况的不同，IDF得分是0.0要比1.0好。例如，如果有10万个文档，所有这些文档中都出现了“green”，你几乎可以肯定地认为它是停用词，而且想完全删除它对查询的影响。

为了便于说明，且因为我们正在处理简单的文档集，更细致的观察是通过在对数计算中增加1.0来调整示例7-4中提供的样本实现。不需要在计算中修改1.0，idf函数可能会返回小于1.0的值，这会导致在TF-IDF计算中将两个因子相乘。因为将两个因子相乘的结果小于它们本身，所以它其实是TF-IDF计算中容易被忽视的边缘情况。回想一下，TF-IDF计算中的直觉是我们希望能够以一种方式将两个词相乘，相对于不太相关的查询，它会始终对更相关的查询产生较大的TF-IDF得分。

使用TF-IDF查询Buzz数据

让我们在我们之前收集到的Buzz数据中应用TF-IDF，看看它作为查询数据的工具是如何工作的。NLTK提供了一些可以直接使用的抽象概念，所以如果你了解了基本理论，我们就没有什么可做的了。示例7-5的代码清单假设你把Buzz数据另存为JSON文件，它允许你传送用于按相关性对文档评分的多个查询词。

示例7-5：使用TF-IDF查询Google Buzz数据（buzz_tf_idf_nltk.py）

```
# -*- coding: utf-8 -*-

import sys
import json
import nltk

# Load in unstructured data from wherever you've saved it

BUZZ_DATA = sys.argv[1]
buzz_data = json.loads(open(BUZZ_DATA).read())

QUERY_TERMS = sys.argv[2:]

all_posts = [post['content'].lower().split() for post in buzz_data]

# Provides tf/idf/tf_idf abstractions

tc = nltk.TextCollection(all_posts)

relevant_posts = []

for idx in range(len(all_posts)):
    score = 0
    for term in [t.lower() for t in QUERY_TERMS]:
        score += tc.tf_idf(term, all_posts[idx])
    if score > 0:
        relevant_posts.append({'score': score, 'title': buzz_data[idx]['title'],
                               'link': buzz_data[idx]['link']})
```

```
# Sort by score and display results

relevant_posts = sorted(relevant_posts, key=lambda p: p['score'], reverse=True)
for post in relevant_posts:
    print post['title']
    print '\tLink: %s' % (post['link'], )
    print '\tScore: %s' % (post['score'], )
```

对Tim O'Reilly的Buzz文章中出现的“gov 2.0”的样例查询结果如示例7-6所示。

示例7-6：示例7-5的样例结果

```
Flattered but somewhat bemused by the metrics that name me as the most influential ...
    Link: http://www.google.com/buzz/107033731246200681024/C62oDoX5oYZ
    Score: 0.0224532639011
Interesting headline on AlleyInsider this morning. "Whoa! Did The Nook Have Better ...
    Link: http://www.google.com/buzz/107033731246200681024/hJLh5pf62Hf
    Score: 0.0128109579108
The Importance of Defaults I've been pushing the idea that one of the big lessons ...
    Link: http://www.google.com/buzz/107033731246200681024/C1Yh6MLmDeS
    Score: 0.0075401637989
```

给定一个检索词，把近50篇Buzz文章缩小到按相关性排序的3篇文章确实是正确的一步。尝试一些其他查询，定性检查产生的结果以便自己看看TF-IDF度量的优秀工作，请记住，这些得分的绝对值并不重要——真正重要的是按相关性查找并将文档排序的能力。然后，给定一个检索词，开始思考你可以通过调整或增强这个度量使其更有效的多数方法。（毕竟，每个数据处理者总得在他的职业生涯中完成这个练习。）作为练习留给读者的一个明显的改善是提取动词的词干，这样时态、语法角色等变化就可以解析相同的词干，更准确地说明相似性计算了。`nltk.stem`模块为几种常见的词干提取算法提供了易于使用的实现。

查找相似文档

一旦你查询到了有趣的文档，接下来要做的就是查找相似文档（参见图7-4）。虽然TF-IDF可以提供基于检索词缩小语料库的方法，但是余弦相似性是最常见的比较文档的方法，它是查找相似文档的本质。要了解余弦相似性，需要简要地介绍一下向量空间模型，这是下一节的主题。

向量空间模型和余弦相似性的理论

虽然我们一直强调TF-IDF模型将文档模型化为单词的无序集，模型化文档的另一种便捷的方法是使用一个称为向量空间的模型。向量空间模型的基本理论是：你有一个巨大的多维空间，其中每个文档都包含一个向量，两个向量之间的距离表示对应文档的相似性。向量空间模型最出色的部分是可以把查询表示为向量，通过查找与查询向量距离

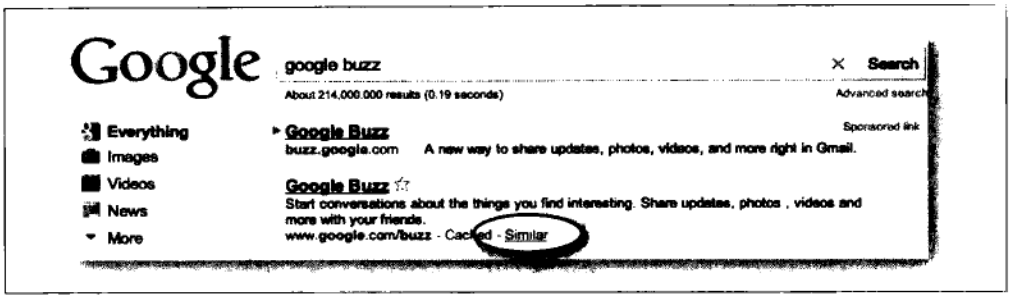

图7-4：一旦发现了你正在查找的文件后，就可以使用很多搜索引擎提供的“类似结果”功能

最短的文档向量，找出查询中最相关的文档。在一个短篇幅的节中介绍完这方面的内容是不可能的，如果你对文本挖掘或IR领域感兴趣的话，重要的是要对向量空间模型有个基本的了解。如果你对背景理论不感兴趣，想要直接查看实现细节的话，可以直接阅读下一节的内容。

警告： 本节假设你对三角学有基本的了解。如果你不太熟悉三角学技术，考虑把这一节作为你重新学习高等数学的好机会。

首先，它可能有助于确切说明什么是“向量”，因为各个领域的研究中与它相关的内容有很多细微变化。总的来说，向量就是一个数字列表，它表示与起点和大小相关的方向，大小是从起点开始的距离。可以通过在起点和N维空间中的某点之间绘制一条直线，很自然地用起点和该点之间的线段来表示一个向量。为了说明这一点，假设一个只由两个词(“Open”，“Web”)定义的文档，与它对应的向量是(0.45, 0.67)，其中向量的值是这个词的TF-IDF得分这类值。在向量空间中，该文档可以用二维空间中从起点(0, 0)延伸到点(0.45, 0.67)的线段来表示。根据x/y平面，x轴表示“Open”，y轴表示“Web”，从（0, 0）到(0.45, 0.67)的向量表示“正在讨论的文档”。然而，有趣的文档一般至少都包含几百个词，但是这些原则也适用于模型化这些高维空间中的文档；它只是较难可视化而已。

试着从可视化由两个要素表示的文档过渡到由3个维度表示的文件，如(“Open”，“Web”，“Government”)。然后考虑并承认以下事实：虽然它很难可视化，但是用向量表示你不能轻易绘制或看到的其他维度仍然是可能的。如果你能做到这一点，你就应该相信：可以用于二维空间的向量操作同样可以用于10维或367维空间中。图7-5表示三维空间中的一个示例向量。

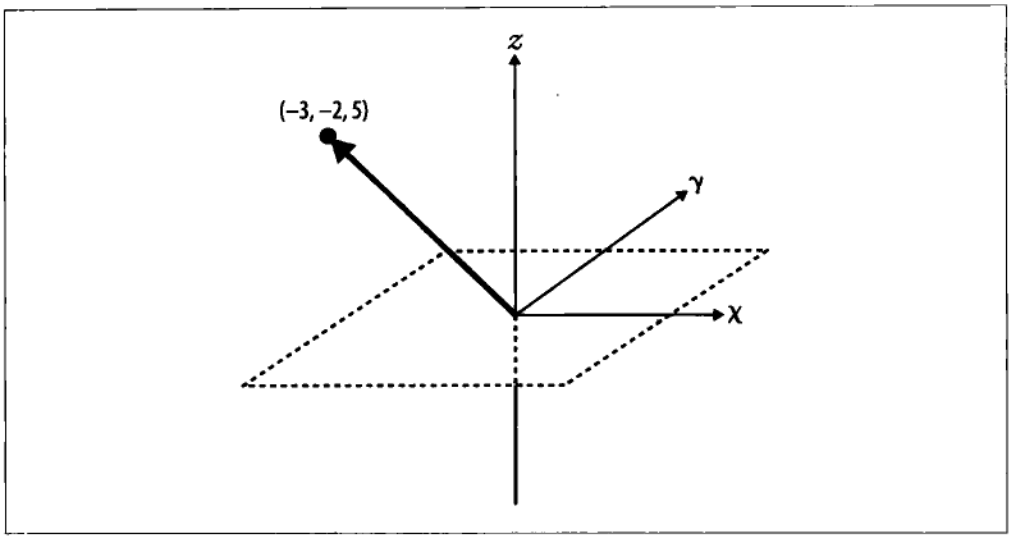

图7-5：在三维空间中绘制的一个示例向量，值为(−3，−2，5)

由于可以将文档模型化为以词为中心的向量，其中，文档中的每个词都由与它对应的TF-IDF得分表示，接下来的任务就是确定哪个度量最能表示两个文档之间的相似性。事实证明，两个向量夹角的余弦是比较它们的一个有效度量，它称为向量的余弦相似性。虽然还不是太直观，但是多年的研究已经证明：计算表示为词向量的文档的余弦相似性是一个很有效的度量。（虽然它有与TF-IDF相同的问题；查看7.7节来获取非常简要的说明。）对余弦相似性度量的严格证明不在本书范围内，但重点是两个向量之间夹角的余弦表示它们之间的相似性，等价于它们的单位向量的点积。从直观上来说，它有助于我们考虑：两个向量越接近，它们之间的夹角就会越小，因此它们之间夹角的余弦就会越大。两个相同向量的夹角是0°，相似性度量是1.0，而两个彼此正交的向量的夹角是90°，相似性度量是0.0。下面的草图试图证明这一点：

$\vec{doc1} \cdot \vec{doc2} = \|doc1\| \cdot \|doc2\| \cdot \cos\theta$	由三角学知识可知
$\frac{\vec{doc1} \cdot \vec{doc2}}{\|doc1\| \cdot \|doc2\|} = \cos\theta$	根据除法
$\hat{doc1} \cdot \hat{doc2} = \cos\theta$	由“单位向量”定义
$\hat{doc1} \cdot \hat{doc2} = Similarity(doc1, doc2)$	通过代入 （假设：$\cos\theta = Similarity(doc1, doc2)$）

回想一下，单位向量的长度是1.0（根据定义），比较文档相似性与单位向量的好处是它们已经将长度中实质性的变化归一化了。这节简短的数学课之后，你可能已经准备好查看实际的内容了。下一节介绍的全是这方面的内容。

使用余弦相似性将文章聚类

如果你不想记住上一节全部内容的话，只需要记住：要计算两个文档之间的相似性，你只需要生成每个文档的一个词向量，计算这些文档的单位向量的点积。NLTK提供了`nltk.cluster.util.cosine_distance(v1,v2)`函数来方便计算余弦相似性，所以很容易比较文档。如示例7-7所示，涉及的所有工作是产生适当的词向量；简而言之，它通过给向量的每个要素指定TF-IDF得分来计算给定文档对的词向量。然而，因为这两个文档的确切词汇表可能是不相同的，所以值为0.0的占位符必须留在每个词向量中，它们并不同时存在于两个文档中。最后的结果是你会以两个相同长度的向量结束，其中要素的排序相同，这些要素可以用来执行向量操作。

例如，假设document1包含词(A，B，C)，TF-IDF向量的对应权重为(0.10，0.15，0.12)，而document2包含词(C，D，E)，TF-IDF向量的对应权重为(0.05，0.10，0.09)。document1的导向量是（0.10，0.15，0.12，0.0，0.0），document2的导向量是(0.0，0.0，0.05，0.10，0.09)。这些向量都可以被传送给NLTK中的`cosine_distance`函数，后者生成余弦相似性。`cosine_distance`在内部使用`numpy`模块来有效地计算向量空间的点积，这就是结果。虽然这一节的代码重用了前面介绍的TF-IDF计算，但是计分函数可以是任何有用的度量。然而，TF-IDF（或者相关的变体）在很多实现中很常见，它提供了很好的起点。

示例7-7说明了一种使用余弦相似性来查找Buzz数据语料库中与所有文档最相似的文档的方法。它应该也适用于其他类型的非结构化数据，如博文、书等。

示例7-7：使用余弦相似性来查找相似的文档（buzz_cosine_similarity.py）

```
# -*- coding: utf-8 -*-

import sys
import json
import nltk

# Load in textual data from wherever you've saved it

BUZZ_DATA = sys.argv[1]
buzz_data = json.loads(open(BUZZ_DATA).read())

all_posts = [post['content'].lower().split() for post in buzz_data]

# Provides tf/idf/tf_idf abstractions for scoring

tc = nltk.TextCollection(all_posts)
```

```
# Compute a term-document matrix such that td_matrix[doc_title][term]
# returns a tf-idf score for the term in the document

td_matrix = {}
for idx in range(len(all_posts)):
    post = all_posts[idx]
    fdist = nltk.FreqDist(post)

    doc_title = buzz_data[idx]['title']
    link = buzz_data[idx]['link']
    td_matrix[(doc_title, link)] = {}

    for term in fdist.iterkeys():
        td_matrix[(doc_title, link)][term] = tc.tf_idf(term, post)

# Build vectors such that term scores are in the same positions...

distances = {}
for (title1, link1) in td_matrix.keys():

    distances[(title1, link1)] = {}
    (max_score, most_similar) = (0.0, (None, None))

    for (title2, link2) in td_matrix.keys():

        # Take care not to mutate the original data structures
        # since we're in a loop and need the originals multiple times

        terms1 = td_matrix[(title1, link1)].copy()
        terms2 = td_matrix[(title2, link2)].copy()

        # Fill in "gaps" in each map so vectors of the same length can be computed

        for term1 in terms1:
            if term1 not in terms2:
                terms2[term1] = 0

        for term2 in terms2:
            if term2 not in terms1:
                terms1[term2] = 0

        # Create vectors from term maps

        v1 = [score for (term, score) in sorted(terms1.items())]
        v2 = [score for (term, score) in sorted(terms2.items())]

        # Compute similarity among documents

        distances[(title1, link1)][(title2, link2)] = \
            nltk.cluster.util.cosine_distance(v1, v2)

        if link1 == link2:
            continue
        if distances[(title1, link1)][(title2, link2)] > max_score:
            (max_score, most_similar) = (distances[(title1, link1)][(title2,
                                         link2)], (title2, link2))

    print '''Most similar to %s (%s)
\t%s (%s)
```

```
\tscore %s
''' % (title1, link1,
           most_similar[0], most_similar[1], max_score)
```

如果你觉得余弦相似性的讨论很有趣，那么回想一下，最精彩的部分是查询向量空间与计算文档之间相似性的操作相同，只是后者值比较文档，而前者需要比较查询向量和文档向量。从实现方面来说，这意味着创建一个包含查询词的向量，并将它与语料库中的所有文档作比较。然而，需要注意的是，直接将查询向量与所有可能的文档向量比较并不是一个好主意，即使是对一个不大的语料库来说也是如此。你需要做一些非常好的工程决策，包括使用适当的索引来实现可扩展的解决方案。（在你开始创建下一个搜索引擎之前，只需要简单考虑一下。）

使用图形可视化将相似性可视化

和本书的其他所有内容一样，可视化项之间相似性的方法可能不止一种。本节介绍的方法是使用图状结构，其中文档之间的链接编码它们之间的相似性度量。这种情况为介绍Protovis中更多的可视化提供了很好的机会，它是基于尖端HTML5的可视化工具包，是由斯坦福大学可视化小组开发的。Protovis是由数据科学家专门设计的，它提供了常见的声明式语法，实现了介于高级接口与低级接口之间的中间接口。对示例7-7最低限度（乏味的）的修改是生成节点和边的一个集合，它可以用于生成数据样例总汇的可视化（*http:vis/stanford.edu/protovis/ex/*）。嵌套循环可以计算本章中Buzz数据可运行样例之间的相似度，项之间的联系是以简单的统计阈值标准为基础确定的。与整理数据和调整Protovis示例模板相关的细节不会在这里介绍，但是可以从网上下载相关代码（*https://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/buzz__cosine_similarity_protovis_output.py*）。

该代码生成了图7-6中的弧形图和图7-7中的矩阵图。如果节点之间有连接，那么弧形图会生成它们之间的弧形，根据节点的度缩放节点，将节点排序，这样就可以把聚类最小化了，而且很容易看到哪些节点的连接最多。标题是垂直显示的，但是可以省略它们，因为当鼠标悬停在一个节点上时，工具提示可以显示标题。点击节点可以打开一个新的浏览器窗口，它指向该节点表示的Buzz活动。事件处理程序等其他特性可以给基于相似性得分的弧形染色，调整这个可视化的参数，使它容易实现（度过下雨天的愉快方式）。

矩阵图是弧形图的互补显示，其中每个单元格的颜色强度根据相似性得分的相关强度变化；深一点的单元格表示更高的相似性。图7-7中矩阵的对角线已经省略了，因为即使对角线的相似性得分尽善尽美，我们也对这些单元格不感兴趣。悬停在单元格上会显示相似性得分的一个工具提示。矩阵图的优点是：表示连接的边之间没有混乱重叠的可能

性。弧形图中普遍存在的缺点是很难查找路径，但是本例中，路径查找并不是那么重要，而且，如果需要的话，它是可以找到的。根据你的个人喜好和你尝试可视化的数据，这两种或其中一种方法可能是适合的。

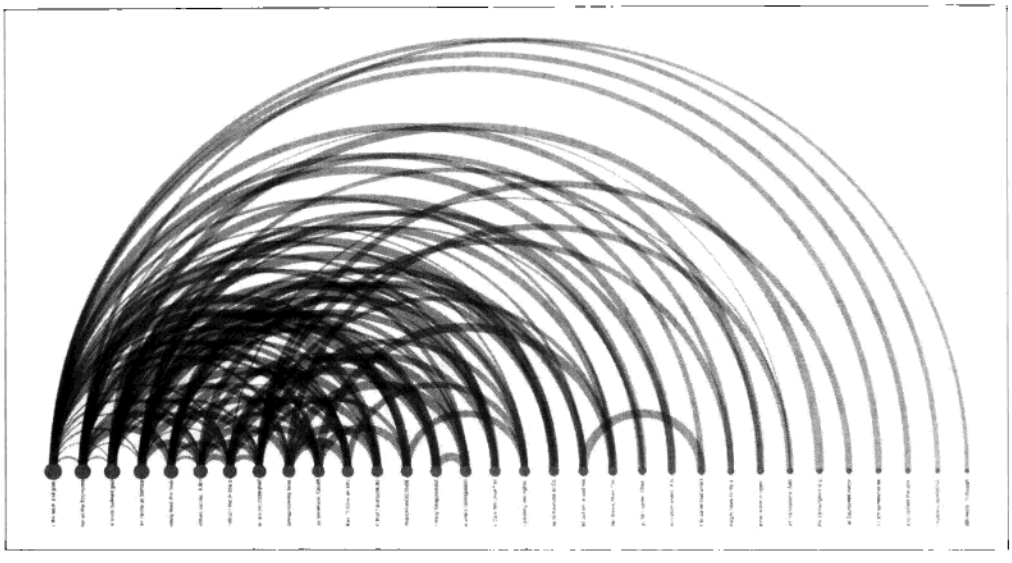

图7-6：显示Buzz活动之间连接的Protovis弧形图

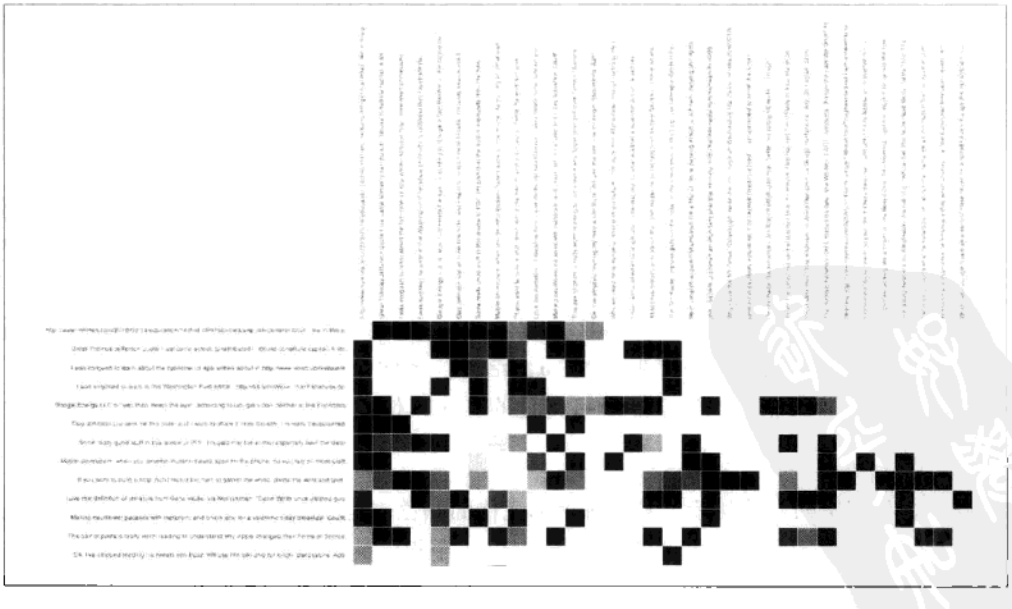

图7-7：显示Buzz活动之间连接的Protovis矩阵图

在二元语法中发Buzz

前面已经提到了，非结构化文本处理中最经常被忽视的问题是：当可以一次查看多个标记时，会获得大量信息，因为我们表达的很多概念都是短语，而不只是单个词。例如，如果有人告诉你文章中最常见的一些词是“open”、“source”和“government”，你能说该文本一定是关于“开源”和“开放政府”或一定不是“开源”和“开放政府”吗？如果你有作者或内容的先验知识，你也许可以猜出来，但是如果你完全依靠机器来试着对关于协同软件开发或转型政府的文档性质分类，那么你需要返回文本，以某种方式确定哪些词最常出现在“open”后面，也就是说，你想查找以“open”标记开头的搭配。

回想一下第6章，*n*元语法只是表示文本中*n*个标记所有可能的连续序列，它提供了计算搭配的基本数据结构。对于任意*n*值，总有(*n*-1)个*n*元语法，如果你考虑标记["Mr.", "Green", "killed", "Colonel", "Mustard"]序列的所有二元语法，会有4种可能性：[("Mr.", "Green"), ("Green", "killed"), ("killed", "Colonel"), ("Colonel", "Mustard")]。需要一个比例句更大的文本样本来确定搭配，但是假设你有背景知识或附加文本，下一步就是统计分析这些二元语法，以便确定它们中哪些可能是搭配。

警告： 值得注意的是：持久化*n*元语法模块需要*n*(*T*-1)个标记的（实际上是*Tn*）空间，其中*T*是正在讨论的标记数，*n*是由所需的*n*元语法定义的。作为示例，假设一个文档包含1000个标记，要求大约8KB的存储空间。保存文本中所有的二元语法需要的存储空间大约是原始存储空间的2倍，即16KB，因为要存储999 × 2个标记和系统开销。存储文本中所有的三元语法（998 × 3个标记加上系统开销）需要的存储空间大约是原始存储空间的3倍，即24KB。因此，在没有制定专门的数据结构或压缩方案的情况下，对于任何*n*值，*n*元语法的存储开销可以估计为任意*n*值的原始存储要求的*n*倍。

作为聚类中经常共同出现的词的技术，*n*元语法虽然很简单，但却很强大。对于任何一个*n*值，如果计算了所有的*n*元语法，那么不需要任何额外的工作，就可以发现出现在文本自身的一些有趣模式。例如，考虑一个足够长的文本的二元语法时，你可能会发现“Mr. Green”和“Colonel Mustard”这类专有名词和“开源”或“开放政府”这类概念。当你考虑常见的三元语法和略大于3的*n*元语法时，也会出现相似模式。事实上，以这种方法计算二元语法产生的结果实质上与在前面解释器会话中运行`collocations`函数的结果相同，只是一些附加统计分析会考虑生词的使用。和从本章前面的解释器会话（示例7-2）了解到的一样，NLTK用于计算*n*元语法、从文本中发现搭配、查找使用某个标记的上下文等，如示例7-8所示。

示例7-8：使用NLTK计算一个句子中的二元语法和搭配

```
>>> import nltk
>>> nltk.ngrams("Mr. Green killed Colonel Mustard in the study with the \
... candlestick. Mr. Green is not a very nice fellow.".split(), 2)
[('Mr.', 'Green'), ('Green', 'killed'), ('killed', 'Colonel'),
('Colonel', 'Mustard'), ('Mustard', 'in'), ('in', 'the'),
('the', 'study'), ('study', 'with'), ('with', 'the'),
('the', 'candlestick.'), ('candlestick.', 'Mr.'), ('Mr.', 'Green'),
('Green', 'is'), ('is', 'not'), ('not', 'a'), ('a', 'very'),
('very', 'nice'), ('nice', 'fellow.')]
>>> txt = nltk.Text("Mr. Green killed Colonel Mustard in the study with the\
... candletick. Mr. Green is not a very nice fellow.".split())
>>> txt.collocations()
Building collocations list
Mr. Green
```

回想一下，使用这类内置演示功能的唯一缺点是nltk.Text.collocations()这些函数无法返回可以存储和处理的数据结构。当遇到这种情况时，只需要查看源代码，通常很容易将它改编成自己需要的代码。示例7-9说明了可以如何计算一个标记集合的搭配和字母索引，以及如何保持结果的可控性。

示例7-9：使用NLTK、用与nltk.Text.collocations()演示功能相似的方法计算搭配(buzz_collocations.py)

```
# -*- coding: utf-8 -*-

import sys
import json
import nltk

# Load in human readable text from wherever you've saved it

BUZZ_DATA = sys.argv[1]
N = 25
buzz_data = json.loads(open(BUZZ_DATA).read())

all_tokens = [token for post in buzz_data for token in post['content'
              ].lower().split()]

finder = nltk.BigramCollocationFinder.from_words(all_tokens)
finder.apply_freq_filter(2)
finder.apply_word_filter(lambda w: w in nltk.corpus.stopwords.words('english'))
scorer = nltk.metrics.BigramAssocMeasures.jaccard
collocations = finder.nbest(scorer, N)

for collocation in collocations:
    c = ' '.join(collocation)
    print c
```

总而言之，该实现粗略地遵循NLTK的collocations()演示函数。它过滤掉了出现次数少于最低次数（本例中是2）的二元语法，然后使用一个计分度量来对结果排名。在这种情况下，计分函数是著名的雅克比指数，它是由nltk.metrics.BigramAssocMeasures.

jaccard定义的。BigramAssocMeasures类使用列联表（contingency table）来对共同出现在给定的二元语法中的短语排名，并比较它与可能出现在二元语法中的其他词。从概念上来说，雅克比指数度量集合的相似性，样本集是对出现在文本中的二元语法的具体比较。如何计算列联表和雅克比值的细节是一个高级主题，下一节提供了这些细节的延续话题，因为它们对较好地理解搭配检测至关重要。

然而，与此同时，让我们检查一下示例7-10。该示例显示了Tim的Buzz源的一些输出，因为使词组有意义的附加上下文，所以返回得分的二元语法比仅返回标记更强大，使得它看起来更明显。

示例7-10：示例7-9的样例结果

```
annalee saxenian
nexus one.
cafe standards
certainly true
eric schmidt
olive oil
open source
1/4 cup
free software
andy rubin
front page
mr. o' reilly
o' reilly said.
steve jobs
tech support
long term
web 2.0
"mr. o' reilly
personal brand
came back
cloud computing,
meaningful use
```

请记住，没有使用可以检查以大写字母为基础的专有名词文本的特别启发方式或手段，令人惊讶的是从数据中筛选出这么多专有名词和常用短语。结果中必然会有一定量不可避免的“噪音”，因为我们还没有清除标记中的标点符号，但是对于我们已经付出的少量工作，这样的结果已经相当完美了。是时候该说明一下了，即使采用了相当好的自然语言处理能力，仍然难以完全消除文本分析结果中的所有“噪音”。不要管这些“噪音”，寻找启发方式来控制它，直到你愿意付出巨大的努力来获取完美的结果为止，这样受过良好教育的人就可以把它们从文本中取出来了。

但愿这时候你做出的初步观察是只投入了很少的努力和时间，我们就已经能够使用另一种非常基本的技术来从一些自由文本数据中取出非常强大的意义了，结果似乎可以代表我们的推测是正确的。这是振奋人心的，因为它表明在任何人的Buzz（或者就此而言的

其他类型的非结构化文本）中应用相同的技术可能一样有益，它能够让你快速地浏览正在讨论的关键项。同样重要的是，虽然这种情况下的数据可能证实了你已经了解的Tim O'Reilly的一些事，但是你也可能学到一些新内容——其中之一就是显然他常常分享食谱，结果清单中的“olive oil”和“1/4 cup”可以说明这一点。虽然使用字母索引、正则表达式或者甚至是Python字符串类型中内置的find方法可以很容易找到与“olive oil”相关的文章，但是让我们利用在示例7-5中开发的代码和TF-IDF来查询“olive oil”。会返回什么呢？结果如下：

```
Making cauliflower pancakes with marjoram, and onion jam, for a valentine' s day…
    Link: http://www.google.com/buzz/107033731246200681024/h1tciv2psSP
    Score: 0.043766231649
```

这就是查询的结果。“olive oil”查询得到的是美味的菜花煎饼食谱。你实际已经从了解文本开始，缩小到一些使用了收集分析的有趣话题上，使用TF-IDF来查找最有趣话题的文本。这种情况下，你有理由使用余弦相似性来查找与菜花煎饼这篇文章最相似的文章（或者是你渴望调查研究的任何内容）。

列联表和计分函数

注意： 这一节将深入讨论BigramCollocationFinder（示例7-9中的雅克比计分函数）如何工作的技术细节。如果你第一次阅读本章或者你对这些细节不感兴趣，可以先跳过这部分内容，以后再看。

用于计算与二元语法有关的常见数据结构是列联表。列联表用于简洁地表示与二元语法中不同短语出现的各种可能性有关的频率。看一下表7-5中的加粗实体，其中*token*1表示二元语法中存在*token*1，~*token*1表示二元语法中不存在*token*1。

表7-5：列联表示例——斜体形式的值表示“旁注”，加粗形式的值表示二元语法变量的频率计数

	token1	~token1	
token2	**frequency(token1, token2)**	**frequency(~token1, token2)**	*frequency(*,token2)*
~token2	**frequency(token1, ~token2)**	**frequency(~token1, ~token2)**	
	frequency(token1,)*		*frequency(*,*)*

虽然有一些细节与哪些单元格对计算最重要有关，但愿不难看出表中间的4个单元格表示与二元语法中不同标记的出现有关的频率。这些单元格中的值可以计算用于给二元语

法计分并排名的不同相似性度量，这和之前介绍的雅克比指数情况相同，稍后会分析它。然而，首先简单讨论一下如何计算列联表中的词。

计算列联表中不同实体的方法与预先计算或已经得到的数据结构直接相关。如果假设只可以使用文本中不同二元语法的频率分布，那么计算*frequency(token1,token2)*的方法就是直接查找，但是怎么计算*frequency(~token1,token2)*呢？因为没有任何其他资料，所以需要扫描第二个空位中出现*token2*的所有单个二元语法，从该值中减去*frequency(token1,token2)*。（如果不是很明显，就花点时间自己证明这是正确的。）

然而，如果假设有一个可以统计文本（文本的一元语法）中每个标记出现的次数的频率分布，以及二元语法的频率分布，那么还可以使用一种包含两次查找和一次算术运算的廉价快捷方式。从bigram*(token1,token2)*出现的次数中减去*token2*作为一元语法出现的次数，得到的就是bigram*(~token1,token2)*出现的次数。例如，如果bigram*(~"mr.", "green")*出现了3次，unigram*("green")*出现了7次，那么bigram*(~"mr.", "green")*一定出现了4次（其中*~"mr."*表示“除了*'mr.'*之外的任何标记”）。在表7-5中，*frequency(*,token2)*表示一元语法*token2*，作为旁注，因为它将表的空白作为快捷方式。*frequency(token1,*)*的值以相同的方式帮助计算*frequency(token1,~token2)*，表达式*frequency(*,*)*指的是任何可能的一元语法，等于文本中的标记总数。已知*frequency(token1,token2)*、*frequency(token1,~token2)*和*frequency(~token1,token2)*，*frequency(~token1,~token2)*的计算必须使用*frequency(token1,~token2)*。

虽然讨论列联表似乎有些离题，但是它是理解不同计分函数的基础。例如，考虑雅克比指数。从概念上来说，它表示的是两个集合的相似性，定义为：

$$\frac{|Set1 \cap Set2|}{|Set1 \cup Set2|}$$

也就是说，它是两个集合中共同的项数除以并集中不同项总数的结果。值得花一些时间来思考这个简单但有效的计算方法。如果*Set1*和*Set2*相等，那么这两个集合的交集和并集相等，比值为1。如果两个集合完全不同，那么分子就会是0，最后的比值为0.0。其余的就是中间值了。适合特定二元语法的雅克比指数表示一个特定二元语法的频率和频率总数之间的比值，其中所有的二元语法都包含出现在二元语法中的短语。对于该度量的一种解释是：比率越高，*(token1,token2)*出现在文本中的概率就越大，因此，“token1 token2”搭配就越有可能表示有意义的概念。

最恰当的计分函数的选择通常依据基础数据特点的知识、一些直觉和运气来确定。`nltk.metrics.association`s中定义的大多数关联度量都已经在Christopher Manning和Hinrich Schuetze编写的《Foundations of Statistical Natural Language Processing》（MIT

Press）的第5章中讨论过了，可以在*http://nlp.stanford.edu/fsnlp/promo/colloc.pdf*上阅读这一章，并把它作为以下说明的有用的参考资料。本书不包括对这些度量的深入讨论，但是刚才提到的第5章提供了深入示例的详细说明。如果需要建立自己的搭配检测器，那么雅克比指数、Dice系数和似然比是很好的起点。下面的列表描述了这些术语和一些其他关键术语：

原始频率

顾名思义，原始频率表示某个*n*元语法频率除以所有*n*元语法频率的比。它用于检查一个文本中某个搭配的整体频率。

雅克比指数

Jaccard指数是度量两个集合之间相似性的比值。适用于搭配，它被定义为给定的搭配频率与搭配总数的比值，这些搭配中至少包含一个词。它可以用于确定给定的短语是否可以形成一个搭配，以及排名这些可能的搭配的可能性。使用与前面的解释一致的标记，在数学上这个公式的定义为：

$$\frac{\text{frequency(token1,token2)}}{\text{frequency(token1,token2)} + \text{frequency(~token1,token2)} + \text{frequency(token1,~token2)}}$$

Dice系数

Dice系数与雅克比指数相似。它们最基本的区别是Dice系数在集合之间一致性的权重是雅克比指数的两倍。数学上把它定义为：

$$\frac{\text{2*frequency(token1,token2)}}{\text{frequency(*,token2)+frequency(token1,*)}}$$

在数学上也可以把它表示为：

$$\text{Dice} = \frac{2 * \text{Jaccard}}{1 + \text{Jaccard}}$$

当更重要的是分数偏差有利于出现“token1 token2”搭配的示例时，你可能会选择使用该度量替代雅克比指数。

似然比

这个度量是另一种假设检验方法，用于度量可能形成搭配的词之间的独立性。一般情况下，它是比卡方检验更适宜搭配发现的方法，它可以很好地处理包含很多罕见搭配的数据。包含在NLTK实现的、计算搭配中的似然估计计算方法假设了一个二项分布（*http://en.wikipedia.org/wiki/Binomical_distribution*），其中决定分布的参数由搭配出现的次数和组成词来计算。

卡方检验

和t检验一样，该度量常用于检验两个变量之间的独立性，可以用来度量两个标记是否是基于统计显著性Pearson卡方检验的搭配。一般来说，使用t检验和卡方检验的区别并不明显。卡方检验的优点是：和t检验不同，它没有假设基本的标准正态分布；正是由于这个原因，我们更常使用卡方检验。

t检验

t检验一般用于假设检验，因为它适用于*n*元语法分析，所以可以用t检验来检验两个短语是否为搭配这个假设。按照t检验的规范，该计算的统计过程使用了标准分布。与原始频率相比，t检验的优点是：t检验考虑了与它的组成部分有关的二元语法频率。这种特性有利于对搭配优势排名。t检验的缺点是：它必须假设搭配的基本概率分布是标准的，但通常情况下并不是这样的。

点式互信息

点式互信息（Point Mutual Information，PMI）用于在知道某个词的相邻词的值时，度量可以从这个词中获取多少信息。换一种方式来说，就是一个词可以告诉你另一个词的多少相关信息。讽刺的是（在目前讨论的上下文中），在计算PMI时包含的计算过程会导致它的高频词得分低于低频词，这刚好与预期效果相反。因此，它适合于度量独立性，但不适合度量相关性（也就是说，它不是计算搭配的理想选择）。同时它也表示，稀疏数据是PMI得分的“绊脚石”，似然比这类技术的性能往往比它好。

利用Gmail

Google Buzz是你可以挖掘到“干净”的文本数据的好资源，但是它只是众多起点之一。因为本章展示的是Google技术，所以这一节只提供了如何利用Gmail数据的简要概述，这样就可以挖掘收件箱中上万封邮件中的文本了。如果你还没有阅读第3章，只要记住它专注于邮件分析，但关注的主要是邮件消息中的结构化数据功能，如邮件线程中的参与者。第3章介绍的技术也可以用于Gmail邮件中，反之亦然。

使用OAuth访问Gmail

在2010年初，Google就宣布可以使用OAuth访问Gmail中的IMAP和SMTP（*http://googlecode.blogspot.com/2010/03/oauth-access-to-imapsmtp-in-gmail.html*）。这是一份重要的公告，因为它正式打开了“Gmail作为平台”的大门，允许第三方开发者创建可以访问Gmail数据的应用程序，而不需要你提供给他们你的用户名和密码。这一节并不介绍Google OAuth的特殊实现Xoauth是如何工作（查阅4.1节中OAuth的简单介绍）的。相反，它关注于了解这些基本知识，这样就可以访问Gmail数据了，包括以下简单步骤：

- 在Gmail账号设置中选择“转发和POP/IMAP”标签中的“启用IMAP”选项。
- 访问Google邮箱Xoauth工具wiki网页（*http://code.google.com/p/google-mail-xoauth-tools/wiki/XoauthDotPyRunThrough*），下载xoauth.py命令行工具，遵循指示生成“匿名”用户的OAuth令牌和密钥[注4]。
- 通过easy_install oauth2安装python-oauth2，使用示例7-11中的模板建立连接。

示例7-11：使用OAuth连接到IMAP的一个模板（buzz_gmail_template.py）

```
# -*- coding: utf-8 -*-

import sys
import oauth2 as oauth
import oauth2.clients.imap as imaplib

# See http://code.google.com/p/google-mail-xoauth-tools/wiki/
# XoauthDotPyRunThrough for details on xoauth.py

OAUTH_TOKEN = sys.argv[1] # obtained with xoauth.py
OAUTH_TOKEN_SECRET = sys.argv[2] # obtained with xoauth.py
GMAIL_ACCOUNT = sys.argv[3] # example@gmail.com

url = 'https://mail.google.com/mail/b/%s/imap/' % (GMAIL_ACCOUNT, )

# Standard values for GMail's Xoauth
consumer = oauth.Consumer('anonymous', 'anonymous')
token = oauth.Token(OAUTH_TOKEN, OAUTH_TOKEN_SECRET)

conn = imaplib.IMAP4_SSL('imap.googlemail.com')
conn.debug = 4 # set to the desired debug level
conn.authenticate(url, consumer, token)

conn.select('INBOX')

# access your INBOX data
```

一旦能访问邮件数据，下一步就是抓取并分析一些邮件数据了。

抓取并分析邮件消息

IMAP（Internet Mail Access Protocol，交互式邮件存取协议）是相当讲究和复杂的协议，但好在你无需深入了解它就可以检索并抓取邮件消息了。可以从*http://www.doughellmann.com/PyMOTW/imaplib/*中得到兼容imaplib的示例，比较常见的操作是检索邮件。有很多构建IMAP查询的方法。如何搜索某个特定用户的邮件的示例是conn.search(NONE, '(FROM "me")')，其中None是字符集的可选参数，'(FROM "me")'是查

注4：如果你只是要处理自己的Gmail数据，使用xoauth.py产生的匿名用户证书就可以了；在合适的时机，你也可以注册并创建“受信任的”客户端应用程序（*http://code.google.com/intl/zh-CN/apis/accounts/docs/RegistrationForWebAppsAuto.html*）。

找你自己发送的邮件（Gmail把“me”当作验证用户）的查找命令。在主题中搜索包含“foo”的邮件的命令是'(SUBJECT "foo")'，可以从RFC 3501的Section 6.4.4（*http://www.faqs.org/rfcs/rfc3501.html*）中了解更多可能性，其中定义了IMAP规范。imaplib将返回的搜索响应表示为元组，它是由状态代码和包含在列表中、用空格隔开的邮件ID字符串组成的，如('OK', ['506 527 566'])。可以解析这些ID值来获取兼容的RFC 822（*http://www.faqs.org/rfcs/rfc822.html*）邮件消息，可惜的是，将邮件消息的内容解析为可使用的形式还需要很多其他工作。幸运的是，只需要做些小修改，我们就可以重用示例3-3的代码，它使用email模块来把邮件解析为更可读的形式，来处理必须从每条邮件中获取可用文本的无趣的email解析代码。

示例7-12：从一个搜索中提取返回的Gmail邮件正文的简单工作流（buzz_search_and_parse_mail.py）

```
# -*- coding: utf-8 -*-

import oauth2 as oauth
import oauth2.clients.imap as imaplib

import os
import sys
import email
import quopri
import json
from BeautifulSoup import BeautifulSoup

# See http://code.google.com/p/google-mail-xoauth-tools/wiki/
#     XoauthDotPyRunThrough for details on xoauth.py

OAUTH_TOKEN = sys.argv[1] # obtained with xoauth.py
OAUTH_TOKEN_SECRET = sys.argv[2] # obtained with xoauth.py
GMAIL_ACCOUNT = sys.argv[3] # example@gmail.com
Q = sys.argv[4]

url = 'https://mail.google.com/mail/b/%s/imap/' % (GMAIL_ACCOUNT, )

# Authenticate with OAuth

# Standard values for GMail's xoauth implementation
consumer = oauth.Consumer('anonymous', 'anonymous')
token = oauth.Token(OAUTH_TOKEN, OAUTH_TOKEN_SECRET)
conn = imaplib.IMAP4_SSL('imap.googlemail.com')
conn.debug = 4
conn.authenticate(url, consumer, token)

# Select a folder of interest

conn.select('INBOX')

# Repurpose scripts from "Mailboxes: Oldies but Goodies"

def cleanContent(msg):

    # Decode message from "quoted printable" format
```

```
    msg = quopri.decodestring(msg)

    # Strip out HTML tags, if any are present

    soup = BeautifulSoup(msg)
    return ''.join(soup.findAll(text=True))

def jsonifyMessage(msg):
    json_msg = {'parts': []}
    for (k, v) in msg.items():
        json_msg[k] = v.decode('utf-8', 'ignore')

    # The To, CC, and Bcc fields, if present, could have multiple items
    # Note that not all of these fields are necessarily defined

    for k in ['To', 'Cc', 'Bcc']:
        if not json_msg.get(k):
            continue
        json_msg[k] = json_msg[k].replace('\n', '').replace('\t', '').replace('\r'
                , '').replace(' ', '').decode('utf-8', 'ignore').split(',')

        try:
            for part in msg.walk():
                json_part = {}
                if part.get_content_maintype() == 'multipart':
                continue
            json_part['contentType'] = part.get_content_type()
            content = part.get_payload(decode=False).decode('utf-8', 'ignore')
            json_part['content'] = cleanContent(content)

            json_msg['parts'].append(json_part)
    except Exception, e:
        sys.stderr.write('Skipping message - error encountered (%s)' % (str(e), ))
    finally:
        return json_msg

# Consume a query from the user. This example illustrates searching by subject

(status, data) = conn.search(None, '(SUBJECT "%s")' % (Q, ))
ids = data[0].split()

messages = []
for i in ids:
    try:
        (status, data) = conn.fetch(i, '(RFC822)')
        messages.append(email.message_from_string(data[0][1]))
    except Exception, e:
        'Print error fetching message %s. Skipping it.' % (i, )

jsonified_messages = [jsonifyMessage(m) for m in messages]

# Separate out the text content from each message so that it can be analyzed

content = [p['content'] for m in jsonified_messages for p in m['parts']]

# Note: Content can still be quite messy and contain lots of line breaks and other quirks
```

```
if not os.path.isdir('out'):
    os.mkdir('out')

filename = os.path.join('out', GMAIL_ACCOUNT.split("@")[0] + '.gmail.json')
f = open(filename, 'w')
f.write(json.dumps(jsonified_messages))
f.close()

print >> sys.stderr, "Data written out to", f.name
```

一旦成功地解析出Gmail邮件正文之后，就需要一些其他工作来“净化”文本，它适合不错的显示或将会在第8章介绍的先进的NLP，但是不需要特别努力地“净化”文本，因为对搭配分析来说，它已经足够“干净”了。事实上，可以直接把示例7-12的结果提供给示例7-9来从搜索结果中产生一个搭配列表。非常有趣的可视化练习是创建一个图形，绘制基于共有的二元语法数的消息之间的链接，是由自定义度量确定的。

在中断之前试着创建一个搜索引擎……

虽然本章希望能让你深入了解如何提取非结构化文本中的有用信息，但是从理论和工程实践的方面来说，我们只接触了最基本的概念。信息检索是价值数百万美元的大产业，所以你只能想象进入理论和实现中的组合投资数，从规模上增强Google和Yahoo!这类搜索引擎。这一节试着确保你意识到了TF-IDF、余弦相似性和本章介绍的其他概念的一些固有的局限性，而且希望它有利于形成你对这个领域的整体看法。

虽然TF-IDF是很容易使用的强大工具，但是它在我们的具体实现中存在一些重要的限制条件，虽然为了方便，我们忽略了它们，但是在实际应用中，你应该考虑它们。其中一个最基本的因素是把它作为单词集合的文档，这意味着文档和查询中词的顺序并不重要。例如，如果我们没有实现逻辑来考虑词序或把查询解释为相对于独立词的短语，查询“Green Mr.”将返回与“Mr. Green”相同的结果。但是很显然，词出现的词序是很重要的。

即使你执行了*n*元语法分析来说明搭配和词序，但基本问题还是TF-IDF假设具有相同文本值的标记的意思相同。然而，很显然，我们并不需要这种情况。你选择的所有谐音都是反例，这种情况还有很多，即使是意思相同的单词，如果使用它们的上下文不同，表示的意义也可能有所不同。基于TF-IDF原则的传统关键字搜索技术和更先进的语义搜索引擎之间的主要区别是语义搜索引擎必定允许你通过定义上下文，以某种方式限制你的搜索关键词。例如，可以指定正在搜索的词被解释为人、地点、结构或其他特定类型的实体。在特定的上下文中限制搜索关键词是目前非常活跃的研究领域。

余弦相似性与TF-IDF有很多相同的缺点。它并不考虑文档的上下文或*n*元语法分析中的词序，它假设向量空间中彼此邻近的词必定相似，但实际情况并非总是如此。最明显的

反例是同音异义词，它们可能意味着不同的内容，但是会被解释为同一个词，因为它们有相同的文本值。余弦相似性的特别实现也取决于将TF-IDF得分作为它计算文档中词的相关重要性的手段，所以TF-IDF错误会产生一系列影响。

你可能也意识到了：在分析非结构化文本的过程中可能要管理很多烦人的细节，而且这些细节对于最新的实现来说很重要。例如，字符串比较是区分大小写的，所以规范化词是很重要的，这样就可以尽量准确地计算频率了。但是，盲目地把词规范化为小写也可能会使情况更复杂，因为某些单词和短语的大小写很重要。“Mr. Green”和“Web 2.0”是两个值得考虑的示例。就“Mr. Green”而言，保持“Green”的首字母大写可能是有利的，因为它可能为查询算法提供了有用的线索，它指的不是形容词，可能是名词的一部分。当在第8章讨论NLP时，我们会再次谈到该主题，因为它最终是使用了“Green”的上下文，使用“词袋”方法会丢失上下文，而使用更先进的NLP解析则有可能保留该上下文。

另一个比TF-IDF的特点更根深蒂固的注意事项是我们使用的标记文字的分裂可能会在标记上留下结尾标点符号，它可能影响制表频率。例如，在先前的示例中，`corpus['b']`以“study”标记结束，与出现在`corpus['a']`中的“study”标记不同（有人可能更愿意查询该标记）。在这种情况下，标记上的结尾句点会影响TF和IDF计算。

注意： 你可能会考虑提取词的词干，这样同形词的常见变体基本上会被视为相同的词，而不是不同的词。对各种好的词干提取实现试试`nltk.stem`包。

最后，要考虑大量的工程注意事项来思考你是否应该决定实现你计划在生产情况中考虑的解决方案。索引和缓存的使用是在大型数据集中获取合理查询次数的关键考虑因素。在Hadoop[注5]这类批量处理系统中，甚至是在价格合理的云基础设施（如Amazon的弹性计算云）上分析实实在在的大量文本数据的能力，可能很贵，需要一个中等公司的预算。

小结

本章介绍了一些IR理论基础：TF-IDF、余弦相似性和搭配。由于Google这类搜索供应商的巨大能力，很容易忘记这些基本的搜索工具曾经出现过。然而，了解它们会了解搜索中普遍被接受的现状的假设和限制，同时还要明确区分最新出现的以实体为核心的技术。（第8章将介绍脱离本章中这些工具的基本范式，如果你还没有阅读过这些材料，

注5： 如果你对Hadoop感兴趣的话，可能想试试Dumbo，它是允许在Python中编写和运行Hadoop程序的项目。

应该会更明显地感受到这些差异。）如果你想尝试将本章的技术应用到Web（一般而言）中，你可能想要试试Scrapy（*http://scrapy.org*），它是一种易用和成熟的网络抓取和爬行框架。

第8章

博客及其他：自然语言处理（等）

本章将尽量介绍自然语言处理（National Language Processing，NLP），并把它应用到博客的非结构化数据中。本着前几章的精神，我们尝试用最少的细节来让你对本身就很复杂的话题有个基本的了解，同时我们也提供了足够的技术钻研，这样你就可以立即挖掘数据了。虽然我们经常走捷径，并采用帕累托式方法（给你20%的关键技术，你可以完成80%的工作），但是本章中我们走的捷径尤其明显，因为NLP实在是太复杂了。任何一本书或者小型多卷集书籍的任何一章都不可能完全介绍这些内容。这一章是一个实际的介绍，它给你足够的信息来完成从文件中自动提取摘要和提取重要实体列表这类工作，但是我们不会太深入这个主题，需要多篇论文才能把它弄清楚。

虽然你不一定非得在学习本章之前阅读第7章，但是我建议你最好先阅读一下。要完全了解NLP，首先要具备对TF-IDF、向量空间模型等内容基本优缺点的正确认识。在这一方面，这两章之间的联系要强于本书的其他章。本章使用的特殊数据源是博客，但是和第7章一样，可以使用任何文本资源。博客正好是社交网络的主要产品（*http://radar.oreilly.com/2010/10/why-blogging-still-matters.html*），所以它本身就很适合于文本挖掘。而且，现在博客文章和传统文章之间的区别正在变得越来越模糊（*http://www.slate.com/id/2271184/pagenum/all/*）。

NLP：帕累托式介绍

本章开篇主要是尝试说明NLP难度的说明性讨论，可以让你深入了解它与前几章中介绍的技术的不同。然而，这一节会使用一些示例代码来说明该技术。

句法与语义

回想一下第7章，TF-IDF和余弦相似度最基本的缺点是这两个模型本身就不要求深入了解数据的语义。与此相反的是，第7章的示例能利用非常基本的句法，用空格分割单词（token）来把其他不透明的文档分解为一系列单词（token），并使用频率和简单的统计相似性度量来确定数据中的哪些单词（token）可能是重要的。虽然你可以使用这些技术做一些非常了不起的事情，但是它们并没有真正告诉你某个文档上下文中出现的单词（token）的意义。包含“fish”或“bear”这类同形异义词（*http://en.wikipedia.org/wiki/Homograph*）[注1]的句子就是一个很好的例子；它们既可以是名词，也可以是动词。

NLP本身就很复杂，甚至很难做到相当不错，在常用语言的大型集合中完全掌握它很可能会是本世纪的问题。毕竟，对NLP的完全掌握实际上是完全掌握图灵测试（*http://en.wikipedia.org/wiki/Turing_test*）的代名词，对于最仔细的观察者来说，实现该功能的计算机程序说明了不可思议的类人智能。虽然结构化或半结构化源基本上是记录的集合，这些记录给了可以被直接分析的每个字段一些假想意义，即使是最简单的任务，处理自然语言数据也有更多细致的考虑。例如，让我们假设给了你一个文档，并要求你统计其中句子的总数。如果你是人类并且基本了解英语语法的话，这是非常简单的任务，但是对于机器来说，这就完全是另一回事了，它要求复杂而详细的指令集来完成这个任务。

值得庆幸的是：只要数据的结构相对合理，机器就可以检测出其断句位置，不但速度很快，而且准确率也很高。然而，即使你已经准确地检测到了所有句子，你仍然不了解这些句子中词或短语的用法。例如，我们看一下20世纪90年代的经典用语：“That's the bomb”（*www.nydailynews.com/archives/entertainment/1997/06/08/1997-06-08_that_s_the_bomb__kids__slang.html*）[注2]，你就知道是怎么回事了。这是一个可以从语法上分析的句子，几乎只由一个主语、一个谓语和一个宾语组成。然而，如果没有上下文，即使完全了解句子的组成（你无法知道“bomb”的含义），它可能是“真的很酷的某事”，也可能是具有巨大破坏能力的核武器。重点是：即使了解了某个句子的结构，你仍然需要句子之外的其他上下文来解释它。在此例中，你需要解析代词“that”指的到底是什么[注3]，然后推断“that”是否是危险的武器。因此，作为极度广泛的概括，我们可以说

注1：同音异义词是同形异义词的特殊情况。如果两个词拼写相同，它们就是同形异义词。如果两个词拼写和发音都相同，它们就是同音异义词。由于某种原因，即使是“同形异义词”的误用，语法中更常见的还是“同音异义词”。

注2：查看《城市辞典》中bomb的定义（*http://www.urbandictionary.com/define.php?term=bomb*）。

注3：解析代词的过程称为指代消解（*http://en.wikipedia.org/wiki/Anaphora_(linguistics)*），这个主题不在本书讨论的范围内。

NLP基本上是关于使用由符号的有序集组成的不透明文档，这些符号遵循正确的句法和定义明确的语法，并最终推导出与这些符号相关的语义。

简短的思考练习

让我们回到大多数NLP流程的第一步：检测句子来说明NLP中包含的复杂性。似乎很容易高估简单的基于规则的启发式方法的作用，而且重要的是通过一个练习来让你了解关键问题是什么，不要把时间浪费在尝试重蹈覆辙上。

解决句子检测问题的最初尝试可能只是统计句子中句号、问号和感叹号的总数。这可能是开始时最明显的启发式方法，但是它相当不成熟，可能会产生很高的误差范围。看一下下面的（非常明确的）控诉：

> "Mr. Green killed Colonel Mustard in the study with the candlestick.Mr. Green is not a very nice fellow."

通过在标点符号（本例中是句号）上分隔来简单地切分这个句子，会产生下面的结果：

```
>>> txt = "Mr. Green killed Colonel Mustard in the study with the \
... candlestick. Mr. Green is not a very nice fellow."
>>> txt.split(".")
['Mr', 'Green killed Colonel Mustard in the study with the candlestick',
'Mr', 'Green is not a very nice fellow', '']
```

显而易见的是：不综合考虑上下文或高级信息，只是盲目地按句号断句来执行句子检测是不够的。在本例中，问题是英语中常见的缩写词"Mr."的使用。虽然第7章对该样本的*n*元语法分析已经说明"Mr.Green"实际上是一个搭配或词块（包含空格的复合标记），但是如果要分析数量更大的文本，就不难想象难以检测以搭配出现为基础的其他边界情况。提前想一下，同样值得指出的是：使用简单的逻辑在句子中查找关键主题也并不容易完成。作为聪明的人类，你可以很容易地推断出样本中的关键主题可能是"Mr. Green"、"Colonel Mustard"、"the study"和"the candlestick"，但是训练机器得出相同的结果是一项复杂的任务。可能会出现一些明显的可能性，如使用正则表达式检测'首字母大写'，构建常见缩写词列表解析出专有名词，将这个逻辑的一些变形应用到查找句末边界的问题中，以防你陷入困境。

当然，这些努力会对一些示例有效，但是对于任意的英语文本，误差范围会是怎样呢？算法能在多大程度上容忍用法不标准的英语文本；文本消息或tweet这类高度浓缩的信息；或（夹杂）其他浪漫的语言，如西班牙语、法语或意大利语？这里没有简单的答案，这就是文本分析在可访问的文本数据几乎每秒都在增加这样的时代中如此重要的原因。指出这些并不是让你灰心丧气。提到它们只是为了激励你在困难时继续努力，因为

这是一个非常困难的领域，目前还没有人完全攻克它。即使你觉得沮丧，你也应该很快高兴起来，因为NLTK确实在很多包含任意文本的情况下都表现良好，在下一节中我们将会看到它的表现。

使用NLTK的典型NLP流程

这一节交互地使用NLTK和解释器会话执行NLP。我们要使用的NLP流程是典型的，类似于下面的高级流程：

- 句末(End of Sentence，EOS)检测→
- 切词→
- 词性（Part of Speech，POS）标记→
- 分块→
- 提取

我们会使用下面的样例文本来说明："Mr. Green killed Colonel Mustard in the study with the candlestick. Mr.Green is not a very nice fellow."请记住，即使你已经阅读了该文本，了解了它是由两句话组成的，但是对于机器来说，现在它只是一个不透明的字符串值。让我们来更详细地查看这些步骤：

EOS检测

这一步把一段文本分解成一个有意义的句子集合。因为句子通常是代表思维的逻辑单元，所以它们往往都包含非常适合于进一步分析的可预见句法。你看到的大多数NLP流程都是从这一步开始的，因为切词（下一步）操作的是单句。虽然把文本分解成段或节可能会增加某些分析类型的值，但是它不可能帮助EOS检测整体任务。在解释器中，使用NLTK解析句子的过程如下：

```
>>> import nltk
>>> txt = "Mr. Green killed Colonel Mustard in the study with the candlestick.\
... Mr. Green is not a very nice fellow."
>>> sentences=nltk.tokenize.sent_tokenize(txt)
>>> sentences
['Mr. Green killed Colonel Mustard in the study with the candlestick. ',
'Mr. Green is not a very nice fellow. ']
```

我们会在下一节更多地讨论`sent_tokenize`底层发生了什么。现在，我们将会接受出现在任何文本中的正确的句子检测的字面意义——对在可能是标点符号的字符处停顿的明显改善。

切词

这一步操作的是单个句子，把它们分割成单词（token）。仿照示例解释器会话，需要执行以下操作：

```
>>> tokens = [nltk.tokenize.word_tokenize(s) for s in sentences]
>>> tokens
[['Mr. ', 'Green', 'killed', 'Colonel', 'Mustard', 'in', 'the', 'study',
  'with', 'the', 'candlesick', '. '],
['Mr. ', 'Green', 'is', 'not', 'a', 'very', 'nice', 'fellow', '. ']
```

请注意，对于这个简单的示例，切词和在空格上分割所作的操作一样，只是它能正确地区分句末标记（句号）。然而，在后面一节中我们可以看到，如果我们给它机会的话，它能做的其实更多，而且我们已经知道，句号是句子结束的标记还是作为缩写词的一部分之间的差别有时候也是重要的。

POS标记

这一步把词性信息分配给每个单词（token）。在示例解释器会话中，通过另外一步来运行单词（token），用标签来装饰它们：

```
>>> pos_tagged_tokens = [nltk.pos_tag(t) for t in tokens ]
>>> pos_tagged_tokens
[[('Mr.', 'NNP'), ('Green', 'NNP'), ('killed', 'VBD'), ('Colonel', 'NNP'),
 ('Mustard', 'NNP'), ('in', 'IN'), ('the', 'DT'), ('study', 'NN'),
 ('with', 'IN'), ('the', 'DT'), ('candlestick', 'NN'), ('.', '.')],
[('Mr.', 'NNP'), ('Green', 'NNP'), ('is', 'VBZ'), ('not', 'RB'), ('a', 'DT'),
 ('very', 'RB'), ('nice', 'JJ'), ('fellow', 'JJ'), ('.', '.')]]
```

你可能无法直观地了解所有这些标签，但是它们确实表示词性信息。例如，‘NNP’表示这个单词是作为名词短语一部分的一个名词，‘VBD’表示一般过去时中的一个动词，‘JJ’表示一个形容词。宾州树库项目（*http://www.cis.upenn.edu/~treebank/*）提供了可返回的词性标签的全面总结（*http://www.ling.upenn.edu/courses/Fall_2003/ling001/penn_treebank_pos.html*）。随着POS标记的完成，分析显然会变得非常强大。例如，通过使用词性分析，可以把名词组块作为名词短语的一部分，然后试着讨论它们可能是哪类实体（例如，人、地方、组织等。）

分块

这一步包含分析一个句子中所有被标记的单词（token），聚集表示逻辑概念的复合单词——是和系统地分析搭配完全不同的方法。可以通过NLTK的`chunk.RegexpParser`来定义一种自定义语法，但是这部分内容不在本书的范围内，查看《Natural Language Processing with Python》（O'Reilly）的第9章可以获得全部细节，可以在*http://nltk.googlecode.com/svn/trunk/doc/book/ch09.html*中查阅它。此外，NLTK还提供了把分块和命名实体提取结合起来的一个函数，它是下一步的内容。

提取

这一步包含每个分块的分析，以及把这些分块进一步标记为命名实体，如人、组织、位置等。解释器中NLP的继续执行如下：

```
>>> ne_chunks = nltk.batch_ne_chunk(pos_tagged_tokens)
>>> ne_chunks
[Tree('S', [Tree('PERSON', [('Mr.', 'NNP')]), Tree('PERSON', [('Green', 'NNP')]),
('killed', 'VBD'),
Tree('ORGANIZATION', [('Colonel', 'NNP'), ('Mustard', 'NNP')]), ('in', 'IN'),
('the', 'DT'), ('study', 'NN'), ('with', 'IN'), ('the', 'DT'),
('candlestick', 'NN'), ('.', '.')]),
Tree('S', [Tree('PERSON', [('Mr.', 'NNP')]), Tree('ORGANIZATION',
[('Green', 'NNP')]), ('is', 'VBZ'), ('not', 'RB'),
('a', 'DT'), ('very', 'RB'), ('nice', 'JJ'), ('fellow', 'JJ'), ('.', '.')])]
>>> ne_chunks[0].draw() # You can draw each chunk in the tree
```

不要太专注于试图精确解释上面的树输出的意义。简而言之，它把一些标记组成块，并且试着把它们分成某些类型的实体。（你可以看出：它把“Mr. Green”识别为人，遗憾的是，它把“Colonel Mustard”分类为组织。）图8-1说明了对于nltk.batch_ne_chunk中的结果调用draw()的效果。

同样有趣的是我们将会继续讨论NLP的复杂性，为NLP产生最新的NLP栈或者甚至深入研究NLTK真的不是我们的目的。这一节的背景旨在加大任务的难度，如果你想进一步研究该主题，鼓励你阅读前面提到的《Natural Language Processing with Python》（*http://www.nltk.org/book*）或网上的其他资源。顺便提一句，如果你的业务（或想法）依赖于真正最新的NLP栈，认真考虑从商业或学术机构购买“成品”的许可证，而不是尝试自己开发。你可以使用开源软件独立完成很多工作，而且它是一个好的起点，但是和其他事情一样，如果你必须取得需要特定咨询业务的重大改进，涉及的投资将会是巨大的。NLP仍然是非常活跃的研究领域，而且最前沿的技术还未形成商品。

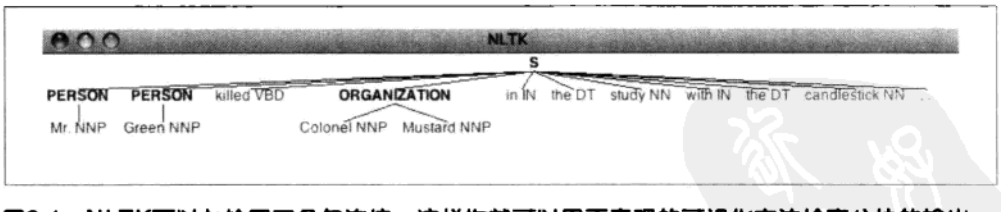

图8-1：NLTK可以与绘图工具包连接，这样你就可以用更直观的可视化方法检查分块的输出，而不是直接查看解释器中的原始文本

除非另有说明，否则本章剩余的部分都假设你也“按现在的样子”使用NLTK。（如果你是计算机语言学这一方面的博士，那么你就更有能力根据你自己的需要修改NLTK，而且可能阅读了更多学术性材料。）

随着对NLP简短介绍的结束，现在让我们开始挖掘一些博客数据吧！

使用NLTK检测博客中的句子

当创建NLP栈时，句子检测可能是你要考虑的第一个任务，所以从创建栈开始是非常有意义的。即使你没有完成流程中剩余的任务，只靠EOS检测也能产生一些强大的可能性（如文档摘要），我们将会把它作为后续练习。但是，我们首先需要抓取一些优质的博客数据。让我们使用行之有效的feedparser模块来抓取O'Reilly Radar博客（*http://radar.oreilly.com*）中的一些文章，如果你还没有安装它，执行easy_install feedparser即可。示例8-1的清单抓取了一些文章，并把它们保存在作为简单旧式JSON的本地文件中，因为本章的其他操作都不依靠CouchDB这类更先进的存储介质的功能。和往常一样，可以把这些文章存储到任何地方。

示例8-1：通过解析源来获取博客数据（blogs_and_nlp_get_feed.py）

```
# -*- coding: utf-8 -*-

import os
import sys
from datetime import datetime as dt
import json
import feedparser
from BeautifulSoup import BeautifulStoneSoup
from nltk import clean_html

# Example feed:
# http://feeds.feedburner.com/oreilly/radar/atom
FEED_URL = sys.argv[1]

def cleanHtml(html):
    return BeautifulStoneSoup(clean_html(html),
                              convertEntities=BeautifulStoneSoup.HTML_ENTITIES).contents[0]

fp = feedparser.parse(FEED_URL)

print "Fetched %s entries from '%s'" % (len(fp.entries[0].title), fp.feed.title)

blog_posts = []
for e in fp.entries:
    blog_posts.append({'title': e.title, 'content'
                       : cleanHtml(e.content[0].value), 'link': e.links[0].href})

if not os.path.isdir('out'):
    os.mkdir('out')

out_file = '%s__%s.json' % (fp.feed.title, dt.utcnow())
f = open(os.path.join(os.getcwd(), 'out', out_file), 'w')
f.write(json.dumps(blog_posts))
f.close()

print >> sys.stderr, 'Wrote output file to %s' % (f.name, )
```

从有信誉的来源中获取非结构化文本可以给我们提供假设良好的英语语法的优势，但愿

这也意味着NLTK的即装即用句子检测器可以正常运行。找不到比处理一些代码更好的方法来看看到底发生了什么，所以让我们继续前进，看一下示例8-2中的代码清单。它提供了sent_tokenize和word_tokenize方法，它们分别是NLTK目前推荐的句子检测器（sentence detector）和分词器（word tokenizer）的别名。稍后我们会简单讨论一下这个清单。

示例8-2：使用NLTK的NLP工具解析博客数据（blogs_and_nlp_sentence_detection.py）

```
# -*- coding: utf-8 -*-

import sys
import json
import nltk

# Load in output from blogs_and_nlp__get_feed.py

BLOG_DATA = sys.argv[1]
blog_data = json.loads(open(BLOG_DATA).read())

# Customize your list of stopwords as needed. Here, we add common
# punctuation and contraction artifacts

stop_words = nltk.corpus.stopwords.words('english') + [
    '.',
    ',',
    '--',
    '\'s',
    '?',
    ')',
    '(',
    ':',
    '\'',
    '\'re',
    '"',
    '-',
    '}',
    '{',
    ]

for post in blog_data:
    sentences = nltk.tokenize.sent_tokenize(post['content'])

    words = [w.lower() for sentence in sentences for w in
            nltk.tokenize.word_tokenize(sentence)]

    fdist = nltk.FreqDist(words)

    # Basic stats

    num_words = sum([i[1] for i in fdist.items()])
    num_unique_words = len(fdist.keys())

    # Hapaxes are words that appear only once

    num_hapaxes = len(fdist.hapaxes())
```

```
    top_10_words_sans_stop_words = [w for w in fdist.items() if w[0]
                                    not in stop_words][:10]

print post['title']
print '\tNum Sentences:'.ljust(25), len(sentences)
print '\tNum Words:'.ljust(25), num_words
print '\tNum Unique Words:'.ljust(25), num_unique_words
print '\tNum Hapaxes:'.ljust(25), num_hapaxes
print '\tTop 10 Most Frequent Words (sans stop words):\n\t\t', \
       '\n\t\t'.join(['%s (%s)'
       % (w[0], w[1]) for w in top_10_words_sans_stop_words])
print
```

你想知道的第一件事可能是sent_tokenize和word_tokenize调用。NLTK提供了一些“分词”选项，但是它通过这些别名提供了最好的“建议”。编写这部分内容时（任何时候都可以使用pydoc再次检查它），句子检测器是PunkSentenceTokenizer，分词器是TreebankWordTokenizer。让我们简单地介绍一下它们吧！

PunktSentenceTokenizer在很大程度上依赖能够把缩写词检测为搭配模式的一部分，它通过考虑标点符号用法的共同模式，使用一些正则表达式来试着智能地解析句子。对PunktSentenceTokenizer内部结构逻辑的完全解释超出了本书的范围，但是Tibor Kiss和Jan Strunk的学术性论文：“Unsupervised Multilingual Sentence Boundary Detection”（*http://www.linguistics.ruhr-uni-bochum.de/~strunk/ks2005FINAL.pdf*）讨论了它的方法，具有很强的可读性，它值得你花点时间来仔细阅读一下。正如我们一会儿看到的，使用尝试提高它精度的示例文本来实例化PunktSentenceTokenizer是可能的。所使用的基础算法的类型是非监督式学习算法（unsupervised learning algorithm）；它不要求你以任何方式显式标记示例训练数据。相反，该算法检查出现在文本本身的某些特性，如大写字母的使用、共同出现的标记等，来获取将文本分解成句子的合适参数。

虽然NLTK的WhitespaceTokenizer会是我们介绍的最简单的分词器，它通过按空格分解文本创建单词，但是你已经熟悉盲目按空格分解的缺点了。相反，NLTK目前推荐TreebankWordTokenizer，它是操作句子并使用相同的约定作为宾州树库项目的分词器[注4]。可能会让你措手不及的一件事是TreebankWordTokenizer的分词器（*http://www.cis.upenn.edu/~treebank/tokenization.html*）会做一些不太明显的事，如在缩写和所有格形式的名词上分别标记要素。例如，分析句子“I'm hungry”，将会产生不同的要素“I”和“'m”，保留主语和“I'm”动词之间的区别。和你想象的一样，是时候开始仔细检查句子中主语和动词之间关系的高级分析了，对这类语法信息的“细粒度”访问可能非常有价值。

注4：“treebank”（树库）是一个专有名词，指的是使用先进的语言信息专门标记的一个语料库。事实上，把这个语料库称为“树库”的原因是为了强调它是由很多句子组成的一个库（集合），已经把它们解析为遵循特定语法的树。

注意： 如果你在使用先进的分词器（如NLTK的TreebankWordTokenizer或PunktWordTokenizer）方面有很多困难，那么最好使用默认的WhitespaceTokenizer，直到你确定是否值得投资使用更先进的分词器为止。事实上，有时候使用更简单的分词器可能很有利。例如，对经常内联URL的数据使用先进的分词器可能不是一个好主意，因为这些分词器不能识别即装即用的URL，将会把它们错误地分解为多个单词（token）。实现自定义分词器超出了本书的范围，但是如果你想要尝试自定义一个分词器的话，可以从网上找到很多资源。

已知句子解析器（sentence tokenizer）和分词器，首先可以把文本解析为句子，然后把每个句子都解析为单词（token）。请注意，虽然这种方法非常直观，但是它有一个致命点，因为句子检测器产生的错误会向前传播，可能会约束其他NLP栈产生的质量上限。例如，如果句子解析器错误地分解了出现在“Mr. Green killed Colonel Mustard in the study with the candlestick”这段文本中的“Mr.”后面某个句子的句号，那么可能无法从文本中提取出“Mr. Green”实体，除非存在专门的修复逻辑。它完全依赖整个NLP栈的复杂度，以及它如何解释误差传播。即装即用的PunkSentenceTokenizer是在宾州树库项语料库中训练的，可以良好地执行。语法分析的最终目标是实例化一个便捷的FreqDist对象，该对象需要一个单词列表。示例8-2中剩余的代码是一些常用的NLTK API的简单用法。

这一节旨在让你熟悉创建NLP流程的第一步。与此同时，我们开发了一些试图表示一些博客数据的度量。我们的流程并不包含词性标记或分块，但是它可以让你基本了解一些概念，让你思考包含的某些细微问题。虽然我们确实可以简单地在空格上分割、统计词数、统计结果和从数据中获取很多信息，但是不久以后你就会很高兴你采取这些初始步骤来深入了解数据。为了说明你刚才学到的一种可能的应用，下一节会介绍一个简单的文档摘要算法，它只依赖于句子分割和频率分析。

对文件的总结

虽然并不是那么显而易见，但是能够执行相当好的句子检测作为挖掘非结构化数据的NLP方法的一部分，使得一些非常强大的文本挖掘功能成为可能，如对文档摘要的虽然不成熟但是非常合理的尝试。有很多可能性和方法，但是完全开始使用数据的其中一种最简单的方法是回到《IBM研究与发展年报》（IBM Journal）1958年4月的专题。在“The Automatic Creation of Literature Abstracts”这篇文章中，H.P. Luhn描述了归结为过滤句子的技术，这些句子包含最常出现的单词，这些单词是彼此邻近的。

这篇学术论文很容易理解，而且非常有趣；Luhn实际上描述了他如何准备穿孔卡，以便使用不同的参数运行各种测试！令人惊讶的是考虑在一个廉价的商业硬件中，我们用几十行Python代码可以实现什么，他可能用了好长时间把这段代码编写到庞大的主机中。

示例8-3提供了文档摘要的Luhn算法的基本实现。下一节会对该算法进行简要分析。在跳到这个讨论之前，让我们先花一点时间浏览一下这段代码，看看你是否能确定它的工作原理。

示例8-3：文档摘要算法（blogs_and_nlp_summarize.py）

```
# -*- coding: utf-8 -*-

import sys
import json
import nltk
import numpy

N = 100 # Number of words to consider
    CLUSTER_THRESHOLD = 5 # Distance between words to consider
    TOP_SENTENCES = 5 # Number of sentences to return for a "top n" summary

# Approach taken from "The Automatic Creation of Literature Abstracts" by H.P. Luhn

def _score_sentences(sentences, important_words):
        scores = []
        sentence_idx = -1

        for s in [nltk.tokenize.word_tokenize(s) for s in sentences]:

            sentence_idx += 1
            word_idx = []

            # For each word in the word list...
            for w in important_words:
                try:
                    # Compute an index for where any important words occur in the sentence
                word_idx.append(s.index(w))
            except ValueError, e: # w not in this particular sentence
                pass

        word_idx.sort()

        # It is possible that some sentences may not contain any important words at all
        if len(word_idx)== 0: continue

        # Using the word index, compute clusters by using a max distance threshold
        # for any two consecutive words

        clusters = []
        cluster = [word_idx[0]]
        i = 1
        while i < len(word_idx):
            if word_idx[i] - word_idx[i - 1] < CLUSTER_THRESHOLD:
                cluster.append(word_idx[i])
            else:
                clusters.append(cluster[:])
                cluster = [word_idx[i]]
            i += 1
        clusters.append(cluster)
```

```
        # Score each cluster. The max score for any given cluster is the score
        # for the sentence

        max_cluster_score = 0
        for c in clusters:
            significant_words_in_cluster = len(c)
            total_words_in_cluster = c[-1] - c[0] + 1
            score = 1.0 * significant_words_in_cluster \
                * significant_words_in_cluster / total_words_in_cluster

            if score > max_cluster_score:
                max_cluster_score = score

        scores.append((sentence_idx, score))

    return scores

def summarize(txt):
    sentences = [s for s in nltk.tokenize.sent_tokenize(txt)]
    normalized_sentences = [s.lower() for s in sentences]

    words = [w.lower() for sentence in normalized_sentences for w in
             nltk.tokenize.word_tokenize(sentence)]

    fdist = nltk.FreqDist(words)

    top_n_words = [w[0] for w in fdist.items()
            if w[0] not in nltk.corpus.stopwords.words('english')][:N]

    scored_sentences = _score_sentences(normalized_sentences, top_n_words)

    # Summaization Approach 1:
    # Filter out non-significant sentences by using the average score plus a
    # fraction of the std dev as a filter

    avg = numpy.mean([s[1] for s in scored_sentences])
    std = numpy.std([s[1] for s in scored_sentences])
    mean_scored = [(sent_idx, score) for (sent_idx, score) in scored_sentences
                   if score > avg + 0.5 * std]

    # Summarization Approach 2:
    # Another approach would be to return only the top N ranked sentences

    top_n_scored = sorted(scored_sentences, key=lambda s: s[1])[-TOP_SENTENCES:]
    top_n_scored = sorted(top_n_scored, key=lambda s: s[0])

    # Decorate the post object with summaries

    return dict(top_n_summary=[sentences[idx] for (idx, score) in top_n_scored],
                mean_scored_summary=[sentences[idx] for (idx, score) in mean_scored])

if __name__ == '__main__':

    # Load in output from blogs_and_nlp__get_feed.py

    BLOG_DATA = sys.argv[1]
    blog_data = json.loads(open(BLOG_DATA).read())

    for post in blog_data:
```

```
post.update(summarize(post['content']))

print post['title']
print '-' * len(post['title'])
print
print '-------------'
print 'Top N Summary'
print '-------------'
print ' '.join(post['top_n_summary'])
print
print '-------------------'
print 'Mean Scored Summary'
print '-------------------'
print ' '.join(post['mean_scored_summary'])
print
```

我们会使用Tim O'Reilly的Rador文章："The Louvre of the Industrial Age"（*http://radar.oreilly.com/2010/07/louvre-industrial-age-henry-ford.html*）来作为示例输入/输出。它大约有460个单词，在这里转载了它，这样就可以比较清单中两个摘要尝试的示例输出了：

> This morning I had the chance to get a tour of The Henry Ford Museum in Dearborn, MI, along with Dale Dougherty, creator of Make: and Makerfaire, and Marc Greuther, the chief curator of the museum. I had expected a museum dedicated to the auto industry, but it's so much more than that. As I wrote in my first stunned tweet, "it's the Louvre of the Industrial Age."
>
> When we first entered, Marc took us to what he said may be his favorite artifact in the museum, a block of concrete that contains Luther Burbank's shovel, and Thomas Edison's signature and footprints. Luther Burbank was, of course, the great agricultural inventor who created such treasures as the nectarine and the Santa Rosa plum. Ford was a farm boy who became an industrialist; Thomas Edison was his friend and mentor. The museum, opened in 1929, was Ford's personal homage to the transformation of the world that he was so much a part of. This museum chronicles that transformation.
>
> The machines are astonishing——steam engines and coal-fired electric generators as big as houses, the first lathes capable of making other precision lathes (the makerbot of the 19th century), a ribbon glass machine that is one of five that in the 1970s made virtually all of the incandescent lightbulbs in the world, combine harvesters, railroad locomotives, cars, airplanes, even motels, gas stations, an early McDonalds' restaurant and other epiphenomena of the automobile era.
>
> Under Marc's eye, we also saw the transformation of the machines from purely functional objects to things of beauty. We saw the advances in engineering—the materials, the workmanship, the design, over a hundred years of innovation. Visiting The Henry Ford, as they call it, is a truly humbling experience. I would never in a hundred years have thought

of making a visit to Detroit just to visit this museum, but knowing what I know now, I will tell you confidently that it is as worth your while as a visit to Paris just to see the Louvre, to Rome for the Vatican Museum, to Florence for the Uffizi Gallery, to St. Petersburg for the Hermitage, or to Berlin for the Pergamon Museum. This is truly one of the world's great museums, and the world that it chronicles is our own.

I am truly humbled that the Museum has partnered with us to hold Makerfaire Detroit on their grounds. If you are anywhere in reach of Detroit this weekend, I heartily recommend that you plan to spend both days there. You can easily spend a day at Makerfaire, and you could easily spend a day at The Henry Ford. P.S. Here are some of my photos from my visit. (More to come soon. Can't upload many as I'm currently on a plane.)

使用平均分和标准差过滤句子，会产生大约170个单词的摘要：

This morning I had the chance to get a tour of The Henry Ford Museum in Dearborn, MI, along with Dale Dougherty, creator of Make: and Makerfaire, and Marc Greuther, the chief curator of the museum. I had expected a museum dedicated to the auto industry, but it's so much more than that. As I wrote in my first stunned tweet, "it's the Louvre of the Industrial Age. This museum chronicles that transformation. The machines are astonishing-steam engines and coal fired electric generators as big as houses, the first lathes capable of making other precision lathes (the makerbot of the 19th century), a ribbon glass machine that is one of five that in the 1970s made virtually all of the incandescent lightbulbs in the world, combine harvesters, railroad locomotives, cars, airplanes, even motels, gas stations, an early McDonalds' restaurant and other epiphenomena of the automobile era. You can easily spend a day at Makerfaire, and you could easily spend a day at The Henry Ford.

另一种摘要算法会产生约90个单词的缩略结果，它只考虑了前*N*个句子（在本例中，*N*=5）。它更简洁，但是仍然可以说它是一个很好的信息提取：

This morning I had the chance to get a tour of The Henry Ford Museum in Dearborn, MI, along with Dale Dougherty, creator of Make: and Makerfaire, and Marc Greuther, the chief curator of the museum. I had expected a museum dedicated to the auto industry, but it's so much more than that. As I wrote in my first stunned tweet, "it's the Louvre of the Industrial Age. This museum chronicles that transformation. You can easily spend a day at Makerfaire, and you could easily spend a day at The Henry Ford.

和其他涉及分析的情况一样，可以从与全文有关的摘要的视觉检查中了解它。输出几乎可以被所有Web浏览器打开的标记格式和调整该代码最后一部分一样简单，它通过执行输出来替换字符串。示例8-4说明了一种可能性。

示例8-4：增加示例8-3的输出来产生适合分析摘要算法结果的HTML标记（blogs_and_nlp_summarize_markedup_output.py）

```
# -*- coding: utf-8 -*-
```

```
import os
import sys
import json
import nltk
import numpy
from blogs_and_nlp__summarize import summarize

HTML_TEMPLATE = """<html>
    <head>
        <title>%s</title>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
    </head>
    <body>%s</body>
</html>"""

if __name__ == '__main__':

    # Load in output from blogs_and_nlp__get_feed.py

    BLOG_DATA = sys.argv[1]
    blog_data = json.loads(open(BLOG_DATA).read())

    # Marked up version can be written out to disk

    if not os.path.isdir('out/summarize'):
        os.makedirs('out/summarize')

    for post in blog_data:

        post.update(summarize(post['content']))

        # You could also store a version of the full post with key sentences markedup
        # for analysis with simple string replacement...

        for summary_type in ['top_n_summary', 'mean_scored_summary']:
            post[summary_type + '_marked_up'] = '<p>%s</p>' % (post['content'], )
            for s in post[summary_type]:
                post[summary_type + '_marked_up'] = \
                post[summary_type + '_marked_up'].replace(s, \
                  '<strong>%s</strong>' % (s, ))

            filename = post['title'] + '.summary.' + summary_type + '.html'
            f = open(os.path.join('out', 'summarize', filename), 'w')
            html = HTML_TEMPLATE % (post['title'] + \
              ' Summary', post[summary_type + '_marked_up'],)

            f.write(html.encode('utf-8'))
            f.close()

            print >> sys.stderr, "Data written to", f.name
```

输出结果是文档的全文，其中组成摘要的句子以粗体形式高亮表示，如图8-2所示。和探讨摘要的替代技术一样，浏览器标签之间的快速浏览可以让你直观地了解摘要技术之间的相似性。这里说明的主要差别是文档中间很长（和描述性）的句子，以“The machines are astonishing”开始。

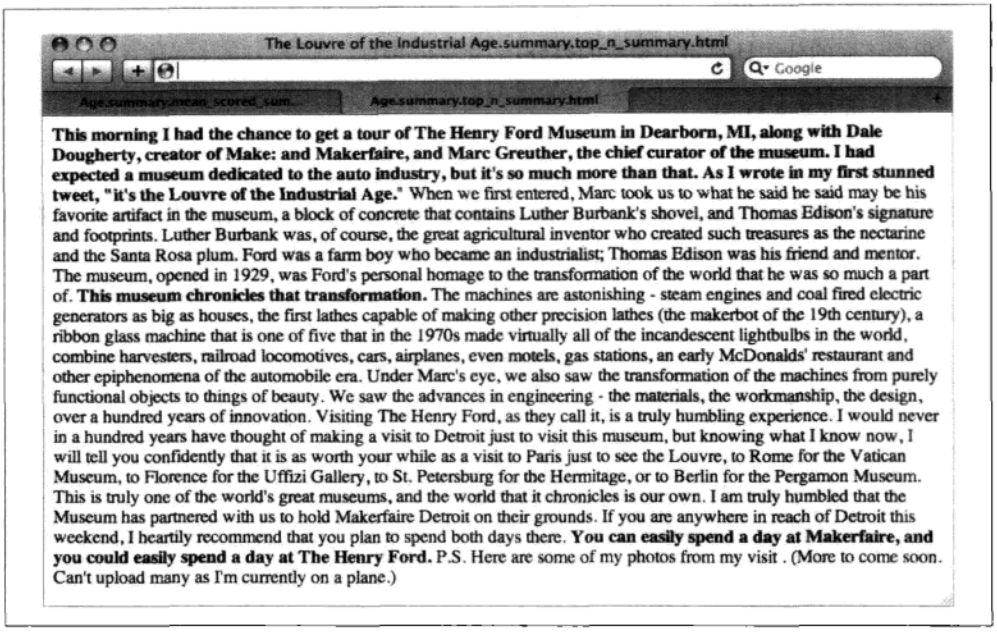

图8-2：O'Reilly Radar博文中的文本，其中摘要算法确定的重点句子由粗体字表示

下一节是关于Luhn算法的简短讨论。

对Luhn摘要算法的分析

Luhn算法的基本前提是文档中的重点句是包含经常出现的单词的句子。然而，值得指出一些细节。首先，并不是所有经常出现的单词都是重要的；一般来说，停用词是填充器，几乎不值得分析。请记住，虽然我们确实可以在样例实现中过滤常见的停用词，但是对于给定的博客或范围，使用附加先验知识创建一个停用词自定义列表是可能的，这可能会进一步增强这个算法或假设已经过滤了停用词的其他算法的效力。例如，专门谈论棒球的博客可能会经常使用“棒球”这个词，应该考虑将它加入一个停用词列表中，即使它不是通用的停用词。（作为旁注，对某个数据源，把TF-IDF合并到计分函数中作为说明这个领域中常用词用法的方法会很有趣。）

假设已经尝试消除了停用词，算法的下一步是选择合适的N值，选择前N个词作为分析的基础。请注意，这个算法潜在的假设是这前N个词能充分地描述文档的特征，对于文档中的任何两个句子来说，包含更多这些单词的句子会被认为更具描述性。确定了文档中的“重点词”后，剩余的工作就是对每个句子应用启发式方法，过滤句子的一些子集来把它作为文档的概要或摘要。需要使用`score_sentences`函数来计算每个句子的得分。这就是代码清单中最有趣的部分。

为了计算每个句子的得分，score_sentences中的算法对聚类单词应用了简单的距离阈值，根据下面的公式计算每个聚类的得分：

$$\frac{(\text{聚类中有意义的词})^2}{\text{聚类中的总词数}}$$

每个句子的最终得分与出现在句子中的任何聚类的最高得分相等。让我们考虑一个例句的score_sentences中涉及的大体步骤，来看看该方法是如何执行的：

输入：例句

```
['Mr.', 'Green', 'killed', 'Colonel', 'Mustard', 'in', 'the', 'study',
'with', 'the', 'candlestick', '.']
```

输入：重点词列表

```
['Mr.', 'Green', 'Colonel', 'Mustard', 'candlestick']
```

输入/假设：聚类阈值（距离）

```
3
```

中间计算：聚类检测

```
[ ['Mr.', 'Green', 'killed', 'Colonel', 'Mustard'], ['candlestick'] ]
```

中间计算：聚类得分

```
[ 3.2, 1 ] # Computation: [ (4*4)/5, (1*1)/1]
```

输出：句子得分

```
3.2 # max([3.2, 1])
```

在score_sentences中完成的实际工作只是记账来检测句子中的聚类。一个聚类被定义为包含两个或更多重要词的句子，其中每个重要的词都是与其最近的邻居的距离阈值。虽然Luhn的论文建议距离阈值取4或5，但是为了简单起见，我们在示例中用了3；因此，‘Green’和‘Colonel’之间的距离被充分地桥接起来了，第一个检测的阈值由句子中的前5个词组成。请注意，‘study’也出现在了重要词列表中，整个句子都作为聚类（除了最后的标点符号）。

计算了每个句子的得分之后，剩下的就是确定把哪些句子作为原文摘要返回。样例实现提供了两种方法。第一种方法通过计算得分的均值和标准差，使用统计阈值来过滤句子，而后一种方法只是简单地返回了前N个句子。根据数据的性质，里程可能会有所不同，但是使用任意一种方法，你都应该可以调整参数来得到合理的结果。使用前N个句子的好处是你会深入了解摘要的最大长度。如果很多句子的得分相对接近，使用均值和标准差可能会返回更多句子。

Luhn算法容易实现，对被描述为整个文档中经常出现的词有效。然而，请记住，和上一章探讨的很多IR方法一样，Luhn算法并没有尝试从更深的语义层次上理解数据。它直接计算摘要作为经常出现词的函数，而且它在如何计算句子得分方面也并不十分复杂。而且，和TF-IDF的情况一样，这使得可执行的以及似乎可以在随机抽取的博客数据中执行操作更惊人。

当权衡实现一个更复杂的方法的利弊时，值得反思一下改进Luhn算法产生的这类合理摘要需要的工作量。有时，一个不成熟的启发式方法是你实现你的目标真正需要的。然而，平时你可能需要一些更新的技术。棘手的部分是计算从不成熟的启发式方法到最新的解决方案的成本效益分析。我们中的很多人往往会对包含的相对工作量过于乐观。

以实体为中心的分析：对数据的深层了解

本章一直在暗示：更深入了解数据的分析方法可能比只是把每个单词作为不透明符号的方法更强大。但是对数据的“更深入了解”真正的意思是什么呢？一种解释是能够检测文档中的实体，并使用这些实体作为分析的基础，而不是以文档为中心的分析方法，其中包含关键字搜索或将搜索输入解释成特定类型的实体并相应地定制结果。虽然你没有从这些方面考虑过它，但是这恰恰是WolframAlpha这类新兴技术在表示层所做的。例如，在WolframAlpha中搜索“tim o'reilly”会返回意味着理解了“搜索的实体是一个人”这样的结果；不是只返回一个包含关键字的文档列表（参阅图8-3）。不管使用了哪种技术来实现这个目标，由此产生的用户体验都会更强大，因为这些结果符合更能满足用户期望的格式。

虽然考虑以实体为中心的分析的各种可能性超出了本章的范围，但是介绍一种从文档中提取实体的方法非常适合，且它属于我们讨论的范围，稍后会用它来做各种分析。假设本章前面介绍的NLP示例流程，你可能采取的一种方法是简单地从文档中提取名词和名词短语），并作为出现在文件中的实体索引它们——重要的基本假设是名词和名词短语（或一些精心构造的子集）适合作为有趣的实体。这确实是个合理的假设，而且是以实体为中心的分析的一个良好的起点，下面的示例清单会说明这一点。请注意，按照宾州树库约定注解的结果，以‘NN’开头的任何标记都是名词或名词短语的一部分。宾州书库标签的完整清单可以从*http://bulba.sdsu.edu/jeanette/thesis/PennTags.html*上得到。

示例8-5分析了单词中使用的POS标签，并且把名词和名词短语标识为实体。在数据挖掘说法中，查找文本中的实体称为实体提取。

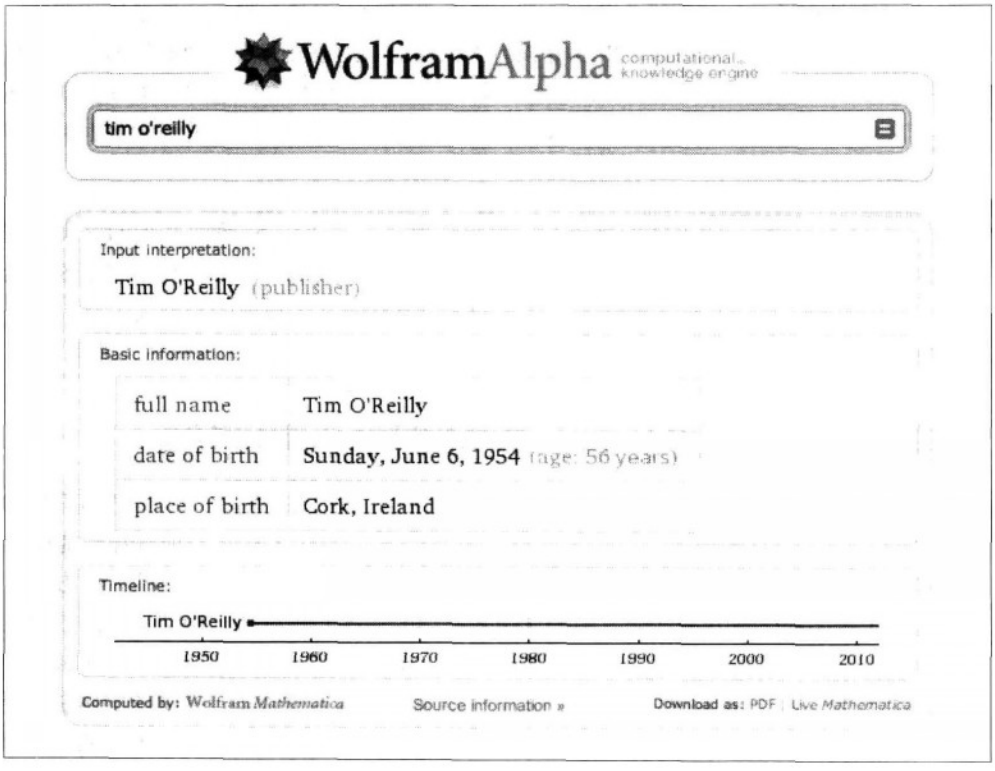

图8-3：使用WolframAlpha查询“tim o'reilly”的示例结果

示例8-5：使用NLTK从文本中提取实体（blogs_and_nlp_extract_entities.py）

```
# -*- coding: utf-8 -*-

import sys
import nltk
import json

# Load in output from blogs_and_nlp__get_feed.py

BLOG_DATA = sys.argv[1]
blog_data = json.loads(open(BLOG_DATA).read())

for post in blog_data:

    sentences = nltk.tokenize.sent_tokenize(post['content'])
    tokens = [nltk.tokenize.word_tokenize(s) for s in sentences]
    pos_tagged_tokens = [nltk.pos_tag(t) for t in tokens]

    # Flatten the list since we're not using sentence structure
    # and sentences are guaranteed to be separated by a special
    # POS tuple such as ('.', '.')

    pos_tagged_tokens = [token for sent in pos_tagged_tokens for token in sent]
```

```
    all_entity_chunks = []
    previous_pos = None
    current_entity_chunk = []
    for (token, pos) in pos_tagged_tokens:

        if pos == previous_pos and pos.startswith('NN'):
            current_entity_chunk.append(token)
        elif pos.startswith('NN'):
            if current_entity_chunk != []:

                # Note that current_entity_chunk could be a duplicate when appended,
                # so frequency analysis again becomes a consideration

                all_entity_chunks.append((' '.join(current_entity_chunk), pos))
            current_entity_chunk = [token]

        previous_pos = pos

    # Store the chunks as an index for the document
    # and account for frequency while we're at it...

    post['entities'] = {}
    for c in all_entity_chunks:
        post['entities'][c] = post['entities'].get(c, 0) + 1

    # For example, we could display just the title-cased entities

    print post['title']
    print '-' * len(post['title'])
    proper_nouns = []
    for (entity, pos) in post['entities']:
        if entity.istitle():
            print '\t%s (%s)' % (entity, post['entities'][(entity, pos)])
    print
```

注意： 从8.2节中“提取”的描述中我们可以知道，NLTK提供了试着从POS标记的单词中提取命名实体的nltk.batch_ne_chunk函数。欢迎你直接使用这个功能，但是你可能会发现，你得到的好处可能会随着支持实现的即装即用模块而变化。改善与NLTK有关的实现的讨论超出了本书的范围。

这个清单的示例输出如示例8-6所示。它提供了非常有意义的结果，通过智能博客平台来给出对标签的建议。对于示例中使用的更大的语料库来说，标签云也是可视化数据的一个明显的“候选对象”。（回想一下5.4.2节，它介绍了一种好用且非常强大的标签云，可以改编它，然后在这里使用它。）

示例8-6：示例8-5的示例结果

```
The Louvre of the Industrial Age
--------------------------------
    Paris (1)
    Henry Ford Museum (1)
    Vatican Museum (1)
    Museum (1)
```

```
Thomas Edison (2)
Hermitage (1)
Uffizi Gallery (1)
Ford (2)
Santa Rosa (1)
Dearborn (1)
Makerfaire (1)
Berlin (1)
Marc (2)
Makerfaire (1)
Rome (1)
Henry Ford (1)
Ca (1)
Louvre (1)
Detroit (2)
St. Petersburg (1)
Florence (1)
Marc Greuther (1)
Makerfaire Detroit (1)
Luther Burbank (2)
Make (1)
Dale Dougherty (1)
Louvre (1)
```

通过更盲目地分析句子的词法特点（如大写的使用），我们已经发现了专用名词的相同列表了吗？也许，但是请记住，该技术也可以抓取没有明确表明大小写的名词和名词短语。大写字母确实是通常值得探讨的文本的一个重要特征，但是在全部都是小写字母的示例文本中（例如，“chief curator”、“locomotives”和“lightbulbs”），也有很多其他有趣的实体。

虽然实体列表并不能像我们之前计算的摘要那样，尽可能有效地传达文本的大意，然而标识这些实体对分析及其重要，因为它们有语义层次的含义，而且不是经常出现的词。事实上，显示在示例输出中的大多数专用名词的频率都很低。然而，它们是很重要的，因为它们在文本中有实质的含义，即，它们是人、地点、事情或想法，它们通常都是数据中的实质性信息。

现在把动词考虑进去并计算主谓宾形式的三元组是向前迈出的又一个重要步骤，这样你就能知道哪些实体正在与其他哪些实体交互，以及这些交互的性质。这种三元组适合于可视化文档的对象图，浏览速度将大大超过阅读这些文档本身。更好的是，想象采用来自文档集中的多幅对象图，合并它们来获取更大的语料库。这项技术是非常活跃的研究领域，而且几乎适合任何信息过载的问题。但是要说明的是，一般情况下，它是一个极其折磨人的问题。

假设一个POS标签器已经从句子中标识出了以下词性，并生成如下的输出：[('Mr.', 'NNP'), ('Green', 'NNP'), ('killed', 'VBD'), ('Colonel', 'NNP'), ('Mustard', 'NNP'), ...]，存储('Mr. Green', 'killed', 'Colonel Mustard')这类“主谓宾”三

元组的索引很容易计算。然而，这种情况的实质是使用这种简单的级别，你不太可能遇到真正的POS标记数据——除非你计划挖掘少儿读物（对于初学者来说，这不是一个坏主意）。例如，把本章前面的博文的第一句话作为你可能转化为一幅对象图的一组任意可行的数据，考虑从NLTK中生成标签：

> This morning I had the chance to get a tour of The Henry Ford Museum in Dearborn, MI, along with Dale Dougherty, creator of Make: and Makerfaire, and Marc Greuther, the chief curator of the museum.

你希望从上面的句子中提取出来的最简单的三元组是：('I', 'get', 'tour')，但是即使你得到了这个结果，也不能说明Dale Dougherty也参加了这次参观，或Mark Greuther也参加了。POS标记的数据也能很清楚地说明得到这些解释并不那么简单，因为这句话有着非常丰富的结构：

```
[(u'This', 'DT'), (u'morning', 'NN'), (u'I', 'PRP'), (u'had', 'VBD'),
(u'the', 'DT'), (u'chance', 'NN'), (u'to', 'TO'), (u'get', 'VB'), (u'a',
'DT'), (u'tour', 'NN'), (u'of', 'IN'), (u'The', 'DT'), (u'Henry', 'NNP'),
(u'Ford', 'NNP'), (u'Museum', 'NNP'), (u'in', 'IN'), (u'Dearborn', 'NNP'),
(u',', ','), (u'MI', 'NNP'), (u',', ','), (u'along', 'IN'), (u'with',
'IN'), (u'Dale', 'NNP'), (u'Dougherty', 'NNP'), (u',', ','), (u'creator',
'NN'), (u'of', 'IN'), (u'Make', 'NNP'), (u':', ':'), (u'and', 'CC'),
(u'Makerfaire', 'NNP'), (u',', ','), (u'and', 'CC'), (u'Marc', 'NNP'),
(u'Greuther', 'NNP'), (u',', ','), (u'the', 'DT'), (u'chief', 'NN'),
(u'curator', 'NN'), (u'of', 'IN'), (u'the', 'DT'), (u'museum', 'NN'), (u'.',
'.')]
```

这种情况下，最新的解决方案能否生成有意义的三元组是值得怀疑的，因为谓语“had the chance to get a tour”的复杂特性，而且参与了这次参观的其他人被列在了加在句子最后的短语中。构建这些三元组的战略超出了本书的范围，但是理论上来说，你应该能使用合理准确的POS标记信息来尝试一下。这个任务的困难之处并不是要让你灰心丧气，通常是要提供对NLP复杂性的实际看法，这样你就能知道当你决定解决计算一般情况下的三元组问题时，你会遇到什么。它可能意味着大量的工作，但结果是非常值得的。

所有的这一切都说明，好消息是通过从文本中提取实体，并把它们作为分析的基础，你实际上可以做很多事，这之前已经说明过了。你可以非常轻松地从每个句子基础上的文本生成三元组，其中每个三元组的“谓语”是表示主语和宾语“交互”的类属关系的概念。示例8-7是示例8-5的重构，后者收集了每个句子基础上的实体，它可能非常有利于计算作为上下文窗口使用的句子的实体之间的交互。

示例8-7：查找实体之间的交互（blogs_and_nlp_extract_interactions.py）

```
# -*- coding: utf-8 -*-

import sys
import nltk
import json

def extract_interactions(txt):
    sentences = nltk.tokenize.sent_tokenize(txt)
    tokens = [nltk.tokenize.word_tokenize(s) for s in sentences]
    pos_tagged_tokens = [nltk.pos_tag(t) for t in tokens]

    entity_interactions = []
    for sentence in pos_tagged_tokens:

        all_entity_chunks = []
        previous_pos = None
        current_entity_chunk = []

        for (token, pos) in sentence:

            if pos == previous_pos and pos.startswith('NN'):
                current_entity_chunk.append(token)
            elif pos.startswith('NN'):
                if current_entity_chunk != []:
                    all_entity_chunks.append((' '.join(current_entity_chunk),
                            pos))
                current_entity_chunk = [token]

            previous_pos = pos

        if len(all_entity_chunks) > 1:
            entity_interactions.append(all_entity_chunks)
        else:
            entity_interactions.append([])

    assert len(entity_interactions) == len(sentences)

    return dict(entity_interactions=entity_interactions,
                sentences=sentences)

if __name__ == '__main__':

    # Read in output from blogs_and_nlp__get_feed.py

    BLOG_DATA = sys.argv[1]
    blog_data = json.loads(open(BLOG_DATA).read())

    # Display selected interactions on a per-sentence basis

    for post in blog_data:

        post.update(extract_interactions(post['content']))

        print post['title']
        print '-' * len(post['title'])
        for interactions in post['entity_interactions']:
            print '; '.join([i[0] for i in interactions])
        print
```

该清单的结果如示例8-8所示，它强调了非结构化数据分析特性中非常重要的内容：它很混乱！

示例8-8：示例8-7的示例输出

```
The Louvre of the Industrial Age
--------------------------------
morning; chance; tour; Henry Ford Museum; Dearborn; MI; Dale Dougherty; creator;
Make; Makerfaire; Marc Greuther; chief curator

tweet; Louvre

"; Marc; artifact; museum; block; contains; Luther Burbank; shovel; Thomas Edison…

Luther Burbank; course; inventor; treasures; nectarine; Santa Rosa

Ford; farm boy; industrialist; Thomas Edison; friend

museum; Ford; homage; transformation; world

machines; steam; engines; coal; generators; houses; lathes; precision; lathes;
makerbot; century; ribbon glass machine; incandescent; lightbulbs; world; combine;
harvesters; railroad; locomotives; cars; airplanes; gas; stations; McDonalds;
restaurant; epiphenomena

Marc; eye; transformation; machines; objects; things

advances; engineering; materials; workmanship; design; years

years; visit; Detroit; museum; visit; Paris; Louvre; Rome; Vatican Museum; Florence;

Uffizi Gallery; St. Petersburg; Hermitage; Berlin

world; museums

Museum; Makerfaire Detroit

reach; Detroit; weekend

day; Makerfaire; day
```

结果中一定量的“噪音”几乎是不可避免的，但是认识到高度可理解且有用的结果（即使它们确实包含不少噪音）是一个非常有价值的目标。实现几乎无噪音的原始结果需要的工作量是巨大的。事实上，在大多数情况下，因为自然语言固有的复杂性和现有的大多数可用工具包（如NLTK）的局限性，这几乎是不可能实现的。如果你能提出数据领域的某种假设或有噪音性质的专业知识，你或许可以设计有效的启发式方法，而不必承担某些信息丢失的风险。但这是一个难以实现的提议。

而且，这些交互确实提供了一定量有价值的“要点”。例如，你对“morning; chance; tour; Henry Ford Museum; Dearborn; MI; Dale Dougherty; creator; Make; Makerfaire; Marc Greuther; chief curator”的解释会多接近原句的含义呢？

和我们之前对摘要研究的情况一样，显示可以直观浏览的标记也是很方便的。对脚本输出的简单修改就是产生图8-4中结果所需要的全部工作。

示例8-9：对示例8-7中脚本的修改（blogs_and_nlp_extract_interactions_markedup_output.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import nltk
import json
from blogs_and_nlp__extract_interactions import extract_interactions

HTML_TEMPLATE = """<html>
    <head>
        <title>%s</title>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
    </head>
    <body>%s</body>
</html>"""

if __name__ == '__main__':

    # Read in output from blogs_and_nlp__get_feed.py

    BLOG_DATA = sys.argv[1]
    blog_data = json.loads(open(BLOG_DATA).read())

    # Marked up version can be written out to disk

    if not os.path.isdir('out/interactions'):

        os.makedirs('out/interactions')

    for post in blog_data:

        post.update(extract_interactions(post['content']))

        # Display output as markup with entities presented in bold text

        post['markup'] = []

        for sentence_idx in range(len(post['sentences'])):

        s = post['sentences'][sentence_idx]
        for (term, _) in post['entity_interactions'][sentence_idx]:
            s = s.replace(term, '<strong>%s</strong>' % (term, ))

        post['markup'] += [s]

    filename = post['title'] + '.entity_interactions.html'
    f = open(os.path.join('out', 'interactions', filename), 'w')
    html = HTML_TEMPLATE % (post['title'] + ' Interactions', ' '.join(post['markup']),)
    f.write(html.encode('utf-8'))
    f.close()

    print >> sys.stderr, "Data written to", f.name
```

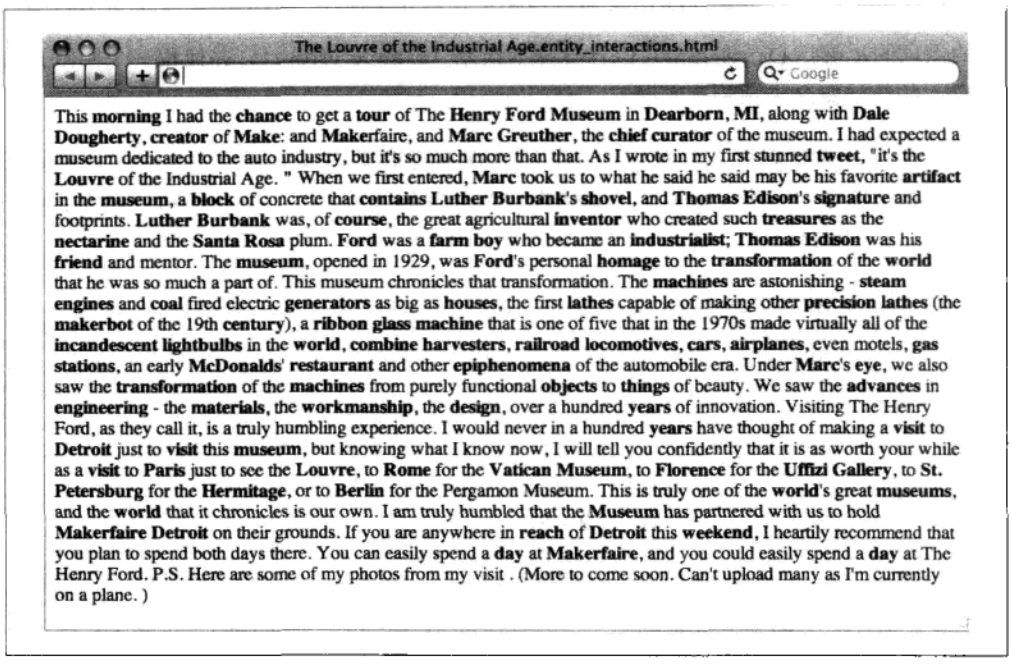

图8-4：简单的HTML输出，用粗体字标识出实体，这样就可以轻易地浏览它的关键概念的内容了

作为对“有兴趣的读者”的考虑，值得执行更多的分析来标识大段文本的交互集合并查找交互中的并发事件。你也可以简单地修改第1章中的一些Graphviz或Protovis输出来可视化这些交互对应的图，其中边不一定有任何标签。示例文件*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/python_code/introduction__retweet_visualization.py*是一个好的起点。

即使不知道交互的具体性质，只知道主语和宾语也有很大的价值。而且如果你觉得野心勃勃，且习惯于一定程度的混乱和精确度，你一定要尝试填写缺少的动词。

分析的质量

在做了很多数据挖掘之后，最后你想做的就是开始量化你的分析的质量。文本挖掘尤其如此。例如，如果你开始自定义从非结构化文本中提取实体的基本算法，那么你如何才能了解你的算法有关结果的质量的性能呢？虽然你可以手动检查一个小型语料库的结果，并且调整该算法，直到你满意为止，但是你仍然需要花时间确定你的分析在更大的语料库或不同类的文档中是否运行良好——因此，我们需要更自动化的过程。

明显的起点是：随机抽取一些文档，创建一个实体“黄金集合”，你相信它们对于从它

们中提取并使用该列表作为评估基础的好算法是至关重要的。根据你的严格程度和满足你的需求的足够信心，你甚至可以计算样本误差（sample error），并使用一个叫做置信区间[注5]的统计概念来预测真实误差（true error）。

然而，为了计算精确度，你应该计算的、以你的提取器和黄金集合为基础的计算到底是什么呢？度量精确度很常见的计算称为*F*1得分（*F*1 Score），它是用两个称为准确率（precision）和召回率（recall）[注6]的概念定义的：

$$F=2\times\frac{\text{准确率}\times\text{召回率}}{\text{准确率}+\text{召回率}}$$

其中：

$$\text{准确率}=\frac{TP}{TP+FP}\text{，}\quad\text{召回率}=\frac{TP}{TP+FN}$$

在现在的情况下，准确率是表示“误报率”（False Positive，FP）正确性的一个度量，召回率是表示“判断为真的正确率”（True Positive，TP）完整性的一个度量。下面的列表说明了与现在的讨论相关的术语的含义，以免不熟悉它们或将它们混淆：

判断为真的正确率（真正，TP）
: 被正确地标识为术语的实体

误报率（假正，FP）
: 被标识为实体，却不是实体的术语

判断为假的正确率（真负，True Negative，TN）
: 没有被标识为实体，实际上也不是实体的术语

漏报率（假负，False Negative，FN）
: 是实体，却没被标识的术语

注5：置信区间（*http://en.wikipedia.org/wiki/Confience_interval*）的更多信息可以随时在网上找到。

注6：更确切地说，*F*1被认为是准确率和召回率的调和平均数，其中任何两个数：*x*和*y*的调和平均数被定义为：

$$H=\frac{2xy}{x+y}$$

可以通过阅读调和数（*http://en.wikipedia.org/wiki/Harmonic_number*）的定义来了解“调和”是什么意思。

由于准确率是量化“误报率”正确性的度量，因此它被定义为准确率=*TP*/(*TP*+*FP*)。从直观上来说，如果“误报率”的值为0，那么这个算法的正确性就是完美的，准确率为1.0。相反，如果“误报率”很高，并且开始达到或超过“判断为真的正确率”，准确率就会很小，准确率会接近于0。作为完整性的度量，回召率被定义为*TP*/(*TP*+*FN*)，如果“漏报率”为0，就表示完美的召回率，生成的值为1.0。当“漏报率”增加时，召回率会接近于0。请注意，按照定义，当准确率和召回率都很完美时，*F*1的值为1.0，当准确率和召回率都很差时，*F*1的值为0.0。当然，你会发现：至于你是否想要提高准确率或召回率，通常需要权衡，因为两者通常很难兼得。如果你思考它（因为包含在“误报率”和“漏报率”中的权衡），它会非常有意义（见图8-5）。

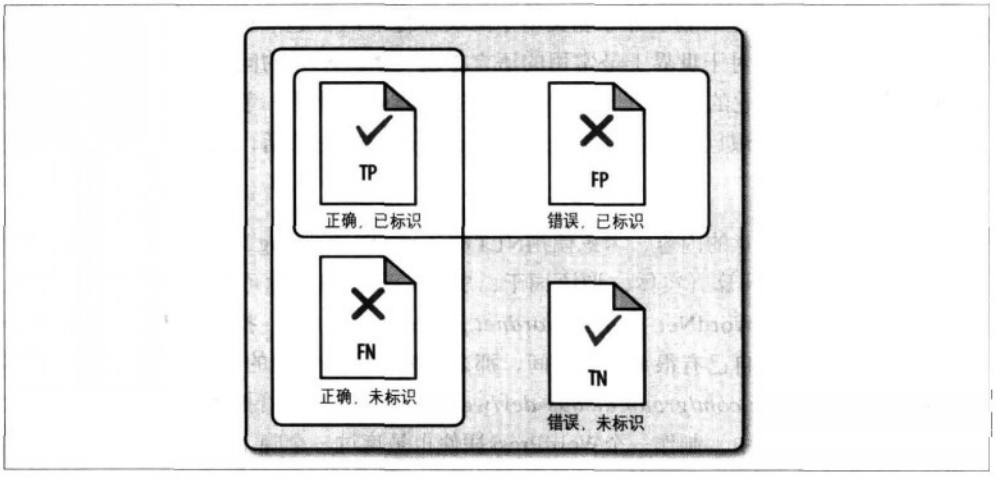

图8-5：“判断为真的正确率”、“误报率”、“判断为假的正确率”和“漏报率”的直观感受

要正确地看待这一切，让我们考虑经典的句子：“Mr. Green killed Colonel Mustard in the study with the candlestick”，并假设一个专家已经确定这句话的关键实体是“Mr. Green”、“Colonel Mustard”、“study”和“candlestick”。假设你的算法只别出了这4个术语，那么你的“判断为真的正确率”为4，“误报率”为0，“判断为假的正确率”（“killed”，“with”，“the”，“in”，“the”）是5，“漏报率”是0。这是完美的准确率和完美的召回率，产生的*F*1得分为1.0。如果这是你第一次遇到这些术语，那么把不同的值代入准确率和召回率公式中是很简单、很值得做的练习。例如，如果你的算法识别出了“Mr. Green”、“Colonel”、“Mustard”和“candlestick”，那么准确率、召回率和*F*1得分分别是多少呢？

另外，有趣的是：根据监督学习算法，了解到在NLP空间中，被商业企业使用的最引人注目的技术栈使用了先进的统计模型来处理自然语言。监督学习算法（supervised learning algorithm）本质上是一种可以给模型提供[(input1, output1), (input2,

output2), ..., (inputN, outputN)]形式的训练样本的方法，这样该模型就能精确地预测三元组了。棘手的部分是确保受过训练的模型可以很好地概括还没有遇到的输入。如果该模型在训练数据中运行良好，在没有见过的样本中不能良好地运行，那么通常说它遭受到了过渡拟合训练数据的问题。度量一个模型效能的常见方法称为交叉验证（cross-validation）。在这种方法中，训练数据的一部分内容（如，1/3）专门用于测验模型，剩余的数据用于训练模型。

小结

本章介绍了先进的非结构化数据分析的本质，说明了如何使用NLTK来探讨第7章介绍的句子解析，将NLP流程的剩余部分和文本中的实体提取组合起来。计算机语言学领域仍然是新生学科，而且对于世界上最常用的语言来说，解决NLP的问题仍然是这个世纪的问题。把NLTK推到它的极限，当你需要更高的性能或质量时，考虑深入研究一些学术文献。这确实是一项艰巨的任务，但是如果你有兴趣解决它的话，它是一个真正有价值的问题。

如果你想详细叙述本章的内容，考虑使用NLTK的wordstemming工具，试着使用以示例8-7为基础的代码来计算（实体、谓语词干、实体）三元组。你可能想要研究一下你应该尽早使用的工具：WordNet（*http://wordnet.princeton.edu/*），找出三元组中项的额外含义。如果你发现你自己有很多空闲时间，那么研究一下受欢迎的评论API，如DISQUS（*https://www.google.com/group/disqus-dev/web/api-1-1*），试着把我们介绍的NLP技术整合到博文的评论流中。制作一个WordPress插件也是度过一个晚上或周末的好方法，该插件能智能表明以从博文草稿中提取出来的实体为基础的标签。

第9章

Facebook：一体化的奇迹

从社交网络的角度来看，Facebook真的是一个奇迹。由于超过5亿用户可以更新他们的公开状态，让他们的朋友即时了解他们的所做所想，就像发邮件一样以最流行的方式相互交换更长的消息，参与实时交谈，组织、分享他们的照片，表明地理位置，并使用网站完成一些其他事情，所以2010年年底，Facebook取代Google成为访问最多的网站一点儿也不奇怪（*http://techcrunch.com/2010/12/29/hitwise-facebook-overtakes-google-to-become-most-visited-website-in-2010/*）。图9-1把Google和Facebook的访问数字并列放入表中，这样你就能直观地比较它们的访问量了。这让人感到非常兴奋，因为存在很多老用户的地方就会存在很多有趣的数据。本章会利用Facebook非常强大的API挖掘数据，找到你最经常联系的好友，基于共同兴趣聚类你的好友，快速了解你的社交网络中的用户正在讨论什么。

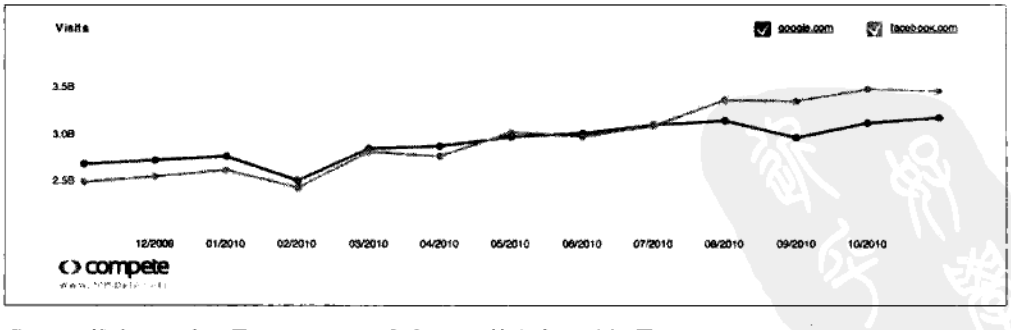

图9-1：截止2010年8月，Facebook和Google的所有月访问量

我们会从对常见Facebook API的概述开始，然后快速过渡到编写利用这些API的脚本，这样我们就可以分析并可视化你的社交数据了。我们在前面使用的技术几乎都可以应用到Facebook数据中，因为Facebook平台是如此丰富、多元。和本书的其他章节一样，

我们不可能包含所有的方面：一本书只能专注于开发很多有趣的应用程序和数据挖掘工具，可以在这样丰富的平台上构建它们，来获取你最亲密的人的详细资料。和其他丰富的开发平台一样，确保本章中用于生产应用的技术考虑到了用户的隐私，记得定期查看Facebook的开发者方针政策（*http://developers.facebook.com/policy/*），这样当你向其他人推出你的应用程序时，就可以避免一些意想不到的情况。

利用社交网络数据

这一节提供了如何在桌面应用程序中完成Facebook的OAuth 2.0流以获取访问令牌的概述，然后迅速过渡到数据收集的示例。为了使本章尽量简单化，我们不会讨论创建Facebook应用程序：很多在线教程可以教你如何创建Facebook应用程序，介绍Facebook应用程序开发需要Google App Engine（GAE）这类服务器平台概述，因为Facebook应用程序必须托管在你自己的服务器环境中。然而，本章所介绍的这些脚本的GAE版本可以从*http://github.com/ptwobrussell/Mining-the-Social-Web/tree/master/web_code/facebook_gae_demo_app*上下载得到。它很容易部署，是一个真正的Facebook应用程序，一旦准备把脚本转化为可部署的应用程序，就可以把它作为起点。

在10分钟内从零到访问令牌

和其他支持OAuth的应用程序一样，需要从获取授权使用的应用程序ID和密钥、选择开发者社区和创建一个“应用程序”开始。下面的列表总结了主要步骤，图9-2提供了一些视觉线索：

- 首先，如果你还没有Facebook账号，那么你需要申请一个。只需要在*http://facebook.com*中注册就可以加入了。
- 然后，需要访问*http://www.facebook.com/developers*来安装开发者应用程序，通过点击这个请求来安装这个应用程序。
- 安装好开发者应用程序后，可以点击“Set Up New Application”按钮来创建应用程序。
- 一旦完成安全检查后，就可以使用应用程序ID和密钥来完成Facebook的OAuth 2.0实现所涉及的步骤了，需要填写提供的表单来指定应用程序的网站设置。只要输入你计划用做托管应用程序的URL作为网站网址，包含相同的域作为网站域名。Facebook使用网站设置作为OAuth流的一部分，如果它们没有正确填写，你会在OAuth dance期间收到一条错误消息。

虽然不太明显，但是如果要返回应用程序，最简单的方法可能是回到*http://facebook.com/developers*（需要登录）。

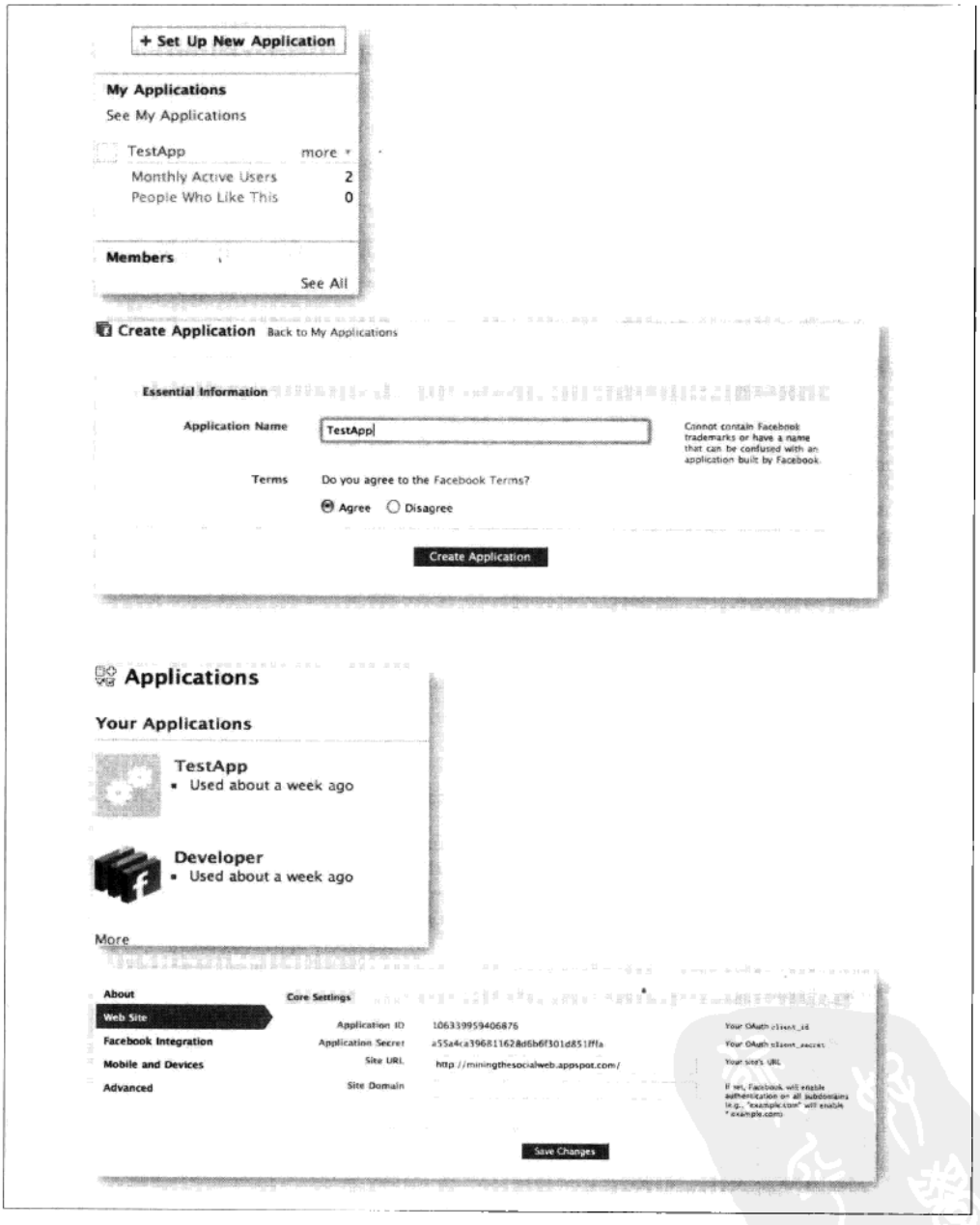

图9-2：从上到下依次为：a)点击http://facebook.com/developers中的按钮创建一个新的应用程序，b)完成应用程序命名并确认服务条款的对话框，c)应用程序已经出现在应用程序列表中了，而且 d)应用程序的设置，包括OAuth 2.0应用程序ID和密钥

处理好应用程序注册的所有细节后，下一步就是编写处理认证、返回可以用来访问API的访问令牌的脚本。这个过程中的整体流程比我们在包含Twitter和LinkedIn这些章节中看到的简单。我们的脚本会弹出一个Web浏览器，输入Facebook账号，然后他会显示一个可以在提示符中复制/粘贴的特定码（访问令牌），这样就可以把它保存在磁盘中，并在以后的查询中使用它。示例9-1说明了这个过程，它只是“桌面应用程序认证”（Desktop Application Authentication）中（*http://developers.facebook.com/docs/authentication/desktop*）描述的工作流的粗略实现。简单查看Facebook的认证（*http://developers.facebook.com/docs/authentication/*）文档可能很有帮助；如果你还没有阅读4.1节，返回去阅读一下这一节内容。然而，请注意，示例9-1中实现桌面应用程序的流程要比包含在Web应用程序认证中的流程简单。

示例9-1：获取桌面应用程序的OAuth 2.0访问令牌（facebook_login.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import webbrowser
import urllib

def login():

    # Get this value from your Facebook application's settings

    CLIENT_ID = ''

    REDIRECT_URI = \
        'http://miningthesocialweb.appspot.com/static/facebook_oauth_helper.html'

    # You could customize which extended permissions are being requested on the login
    # page or by editing the list below. By default, all the ones that make sense for
    # read access as described on http://developers.facebook.com/docs/authentication/
    # are included. (And yes, it would be probably be ridiculous to request this much
    # access if you wanted to launch a successful production application.)

    EXTENDED_PERMS = [
        'user_about_me',
        'friends_about_me',
        'user_activities',
        'friends_activities',
        'user_birthday',
        'friends_birthday',
        'user_education_history',
        'friends_education_history',
        'user_events',
        'friends_events',
        'user_groups',
        'friends_groups',
        'user_hometown',
        'friends_hometown',
        'user_interests',
```

```
    'friends_interests',
    'user_likes',
    'friends_likes',
    'user_location',
    'friends_location',
    'user_notes',
    'friends_notes',
    'user_online_presence',
    'friends_online_presence',
    'user_photo_video_tags',
    'friends_photo_video_tags',
    'user_photos',
    'friends_photos',
    'user_relationships',
    'friends_relationships',
    'user_religion_politics',
    'friends_religion_politics',
    'user_status',
    'friends_status',
    'user_videos',
    'friends_videos',
    'user_website',
    'friends_website',
    'user_work_history',
    'friends_work_history',
    'email',
    'read_friendlists',
    'read_requests',
    'read_stream',
    'user_checkins',
    'friends_checkins',
    ]

args = dict(client_id=CLIENT_ID, redirect_uri=REDIRECT_URI,
            scope=','.join(EXTENDED_PERMS), type='user_agent', display='popup'
            )

webbrowser.open('https://graph.facebook.com/oauth/authorize?'
                + urllib.urlencode(args))

# Optionally, store your access token locally for convenient use as opposed
# to passing it as a command line parameter into scripts...

access_token = raw_input('Enter your access_token: ')

if not os.path.isdir('out'):
    os.mkdir('out')

filename = os.path.join('out', 'facebook.access_token')
f = open(filename, 'w')
f.write(access_token)
f.close()

print >> sys.stderr, \
        "Access token stored to local file: 'out/facebook.access_token'"

return access_token
```

```
if __name__ == '__main__':
    login()
```

你想知道的一个重要细节可能是EXTENDED_PERMS的定义，一定需要一个简单的说明。你第一次尝试登录应用程序时，它会通知你应用程序正在请求大量的扩展权限，这样你就可以非常灵活地访问你可以使用的数据了（见图9-3）。Facebook的认证文档（*http://developers.facebook.com/docs/authentication/*）中描述了扩展权限的细节，但是默认情况下，应用程序只能访问用户个人资料中的基本数据（如姓名、性别和用户头像），必须获得显式授权才能访问其他数据。这里要观察的奥妙是可以通过Facebook账号，查看好友的某些细节，如他们“喜欢”什么或者他们正在进行的活动是什么，但是应用程序只有在得到显式的授权后，才可以访问这些细节。也就是说，登录facebook.com后看到的好友个人资料和通过API请求信息返回的数据是不同的。这是因为当你登录facebook.com后，使用平台返回数据的是Facebook（不是你），但是当你创建你的应用程序时，请求数据的是你（作为开发者）。

警告： 应用程序不需要扩展权限来访问数据，但是试着访问一下也无妨，结果可能会返回空数据对象，而不是显式的错误消息。

警告： Facebook平台的文档在不断发展，你可能无法通过它们了解扩展权限的所有内容。例如，为了访问好友的宗教或政治信息（你的friends_religion_politics扩展权限），这些好友必须安装你的应用程序，对这些数据有显式认证的访问权限，也可以通过他们的user_religion_politics扩展权限来访问数据。

现在，已经配备好了有权访问你所有数据的全新访问令牌，是时候做一些更有趣的事情了。

Facebook的查询API

你可能已经听说了，Facebook开发者生态系统很复杂且在不断发展中[注1]，它充满了很多曲折，而且包含Web中最先进的隐私控制（*http://www.facebook.com/privacy/explanation.php*）。除了一组标准的REST API（*http://developers.facebook.com/docs/reference/rest/*）和更先进的Facebook Query Language（*http://developers.facebook.com/docs/reference.fql/*）（FQL）（一种以与SQL类似的方式查询数据的类SQL语言）之外，2010年4月，Facebook还在F8大会上发布了Graph API和开放图谱协议（Open Graph Protocol，OGP）（*http://opengraphprotocol.org*）。简而言之，OPG是一种通过把一些RDFa（*http://www.*

注1： 标签和留意开发路线图（*http://developers.facebook.com/roadmap*）保持更新是一个好主意。

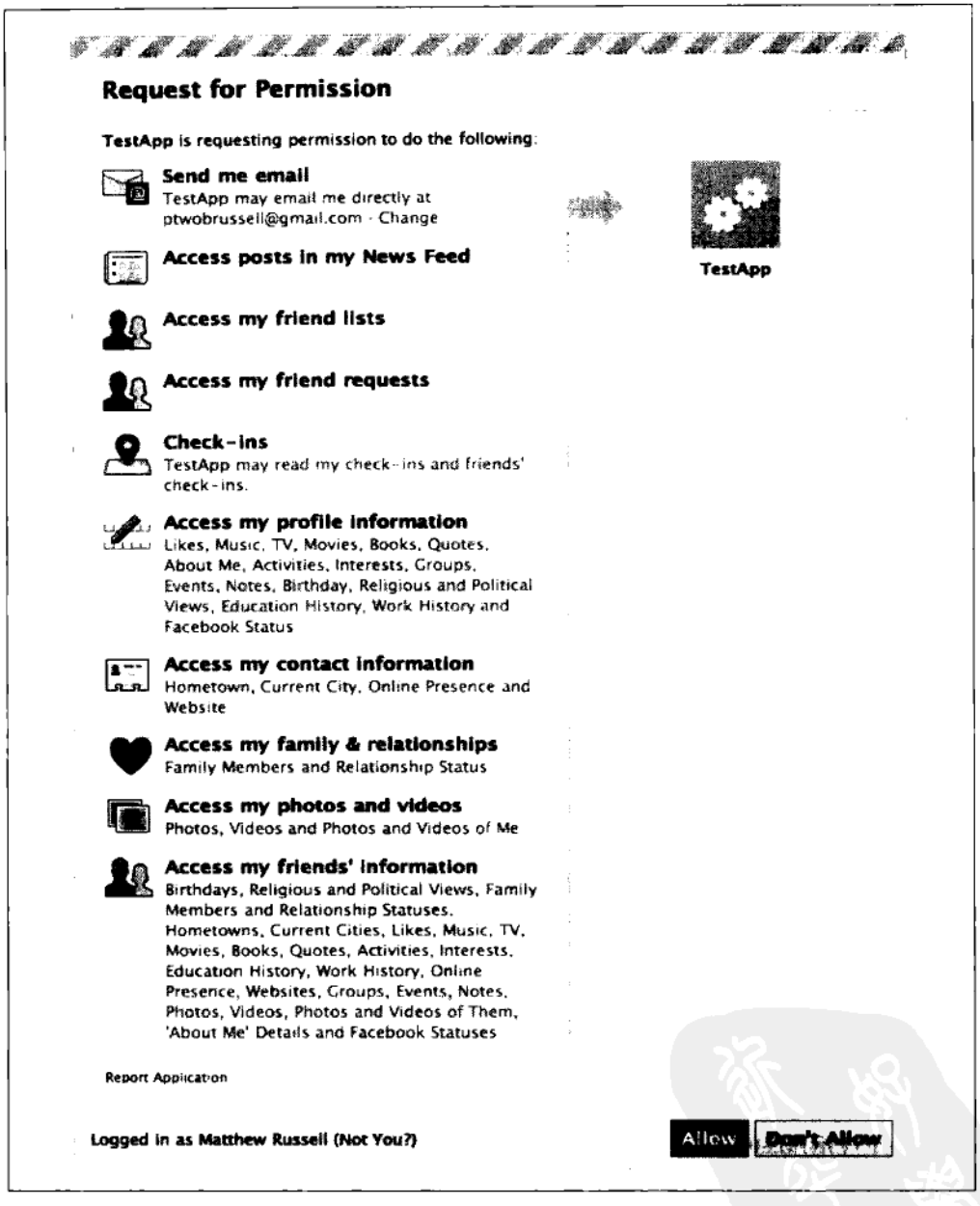

图9-3：页面初始化时加载的所有脚本（Kitchen Sink）：如果一个应用程序请求访问所有内容的权限，你会看到这个样本对话框

w3.org/TR/xhtml-rdfa-primer/）（一会儿介绍它）元数据插入网页中，让你能够把丰富的社交图中的网页当作对象的机制，而Graph API是用于查询“图”的一种简单直观的机

制。每个对象都有特定的类型。编写这部分内容时，Graph API支持下面的对象类型，如Graph API Reference（*http://developers.facebook.com/docs/reference/api/*）所述：

相册（album）
　　相册

应用程序（application）
　　在Facebook平台上注册的个人应用程序

签到（checkin）
　　通过Facebook Places“签到”

事件（event）
　　Facebook事件

群组（group）
　　Facebook群组

链接（link）
　　共享链接

笔记（note）
　　Facebook笔记

页面（page）
　　Facebook页面

照片（photo）
　　个人照片

博文（post）
　　个人资料订阅中的一个实体

状态消息（status message）
　　用户留言板上的状态消息

订阅（subscription）
　　一个应用程序的个人订阅，获取一个对象类型的实时更新

用户（user）
　　用户个人资料

视频（video）
　　个人的视频

Graph API Reference（*http://developers.facebook.com/docs/reference/api/*）包含每个对象类型的详细文档，它描述了属性和你期望存在于每个对象中的关系的类型。

示例9-2是文档中典型的示例，该文档说明了如何把“The Rock”上IMDB的页面转换为开放图谱协议（OGP）中的对象，并把这个对象作为使用命名空间的XHTML文档的一部分。一旦大规模地实现了这些元数据，它们就会有巨大的潜力，因为它们使*http://www.imdb.com/title/tt0117500*这类URL能以机器可读的方式明确地表示任何网页（不管它是人、公司还是产品），进一步加强了语义网的愿景。

示例9-2：开放图谱协议的样例RDFa

```
<html xmlns:og="http://ogp.me/ns#">
<head>
<title>The Rock (1996)</title>
<meta property="og:title" content="The Rock" />
<meta property="og:type" content="movie" />
<meta property="og:url" content="http://www.imdb.com/title/tt0117500/" />
<meta property="og:image" content="http://ia.media-imdb.com/images/rock.jpg" />
...
</head>
...
</html>
```

考虑OGP的可能性时，要有远见和创造性，但是要记住：它是全新的，而且仍然在发展中。由于它通常都会涉及语义网和Web标准，因此对使用“开放式”（*http://techcrunch.com/2010/04/23/facebook-open-graph/*）的惊慌失措已经开始了，而规范的各种操作指南仍然在制定中（*http:/groups.google.com/group/open-graph-protocol/browse-thread/thread/cco3368efod12c1a*）。实质上它只是单一厂商的努力，其实它与Web早期的meta元素的能力差不多。实际上，与其说OGP是一种标准，不如说它是一个“白点”，但是它仍然有改变的潜力，作为未来进展和出现的创新，会发生很多令人兴奋的事。第10章会回到语义网的话题上来，到时候我们会讨论它的巨大潜力。现在让我们返回去进一步分析：通过创建一个简单的Facebook应用程序，如何使用Graph API来挖掘社交数据。

注意： 因为名称上的相似性，很容易混淆Google的社交图谱API（*http://socia/graph-resource.googlecode.com/svn/trunk/samples/findyours.html*）和Facebook的Graph API，但是它们是完全不同的。

Graph API其实非常简单：替代*http(s)://graph.facebook.com/ID* URI中某个对象的ID来抓取该对象的细节。例如，抓取Web浏览器中的*http://graph.facebook.com/http://www.imdb.com/title/tt0717500* URL会返回示例9-3中的响应。

示例9-3：对http://graph.facebook.com/http://www.imdb.com/titile/tt0117500的开放图谱查询的样例响应

```
{
    "id": "114324145263104",
    "name": "The Rock (1996)",
    "picture": "http://profile.ak.fbcdn.net/hprofile-ak-snc4/hs344.snc4/41581...jpg",
    "link": "http://www.imdb.com/title/tt0117500/",
    "category": "Movie",
    "description": "Directed by Michael Bay. With Sean Connery, Nicolas Cage, ...",
    "likes" : 3
}
```

如果检查*http://www.imdb.com/title/tt0117500* URL的源，你会发现响应中的字段与页面的meta标签中的数据相对应，这并不是巧合。响应简单查询的丰富元数据的交付是OGP工作方式背后的整体构思。让它变得更有趣的是：通过在请求中添加查询字符串参数metadata=1，显式请求页面中一个对象的其他元数据。对*https://graph.facebook.com/114324145263104?metadata=1*查询的样例响应如示例9-4所示。

示例9-4：对包括可选的元数据的http://graph.facebook.com/http://www.imdb.com/title/tt011700?metadata=1的开放图谱查询的样例响应

```
{
    "id": "118133258218514",
    "name": "The Rock (1996)",
    "picture": "http://profile.ak.fbcdn.net/hprofile-ak-snc4/..._s.jpg",
    "link": "http://www.imdb.com/title/tt0117500",
    "category": "Movie",
    "website": "http://www.imdb.com/title/tt0117500",
    "description": "Directed by Michael Bay. With Sean Connery, Nicolas Cage, ...",
    "likes": 3,
    "metadata": {
       "connections": {
          "feed": "http://graph.facebook.com/http://www.imdb.com/title/tt0117500/feed",
          "posts": "http://graph.facebook.com/http://www.imdb.com/title/tt0117500/posts",
          "tagged": "http://graph.facebook.com/http://www.imdb.com/title/tt0117500/tagged",
          "statuses": "http://graph.facebook.com/http://www.imdb.com/title/...",
          "links": "http://graph.facebook.com/http://www.imdb.com/title/tt0117500/links",
          "notes": "http://graph.facebook.com/http://www.imdb.com/title/tt0117500/notes",
          "photos": "http://graph.facebook.com/http://www.imdb.com/title/tt0117500/photos",
          "albums": "http://graph.facebook.com/http://www.imdb.com/title/tt0117500/albums",
          "events": "http://graph.facebook.com/http://www.imdb.com/title/tt0117500/events",
          "videos": "http://graph.facebook.com/http://www.imdb.com/title/tt0117500/videos"
       },
       "fields": [
          {
             "name": "id",
             "description": "The Page's ID. Publicly available. A JSON string."
          },
          {
             "name": "name",
             "description": "The Page's name. Publicly available. A JSON string."
          },
```

```
        {
          "name": "category",
          "description": "The Page's category. Publicly available. A JSON string."
        },
        {
          "name": "likes",
          "description": "\\* The number of users who like the Page..."
        }
      ]
    },
    "type": "page"
}
```

metadata.connections中的项是图中其他节点的指针，可以抓取它来访问其他有趣的数据。例如，可以沿着“照片”链接获取与电影相关的照片，并且还可能沿着这些链接找到是谁张贴的照片，以及与照片相关的评论内容。如果你还没想到它，你也仍然是图谱中的一个对象。试着访问相同的URL前缀，但是以自己的Facebook ID或用户名作为URL上下文，并亲眼看一下结果。由于你是你自己的社交网络的逻辑中心，因此本章剩余的部分会重新详细地讨论它。下一节深入研究Graph API查询，后一节深入研究FQL查询。

一次探讨Graph API的一个关系

Facebook的Graph API使用官方Python SDK（*http://github.com/facebook/python-sdk/*）；只需要运行easy_install facebook-python-sdk命令，就可以使用它了。有一些很好的示例（*http://github.com/facebook/python-sdk/tree/master/examples/*）揭示如何使用该模块快速完成OAuth dance，并创建在Google应用程序引擎这类平台上托管的成熟的Facebook应用程序。我们会缩小到GraphAPI类（在*facebook.py*中定义）的一些特定端口，并在独立的脚本中使用它们。这些方法如下：

- `get_object(self, id, **args)`

  示例：`get_object("me", metadata=1)`

- `get_objects(self, id, **args)`

  示例：`get_objects(["me", "some_other_id"], metadata=1)`

- `get_connections(self, id, connection_name, **args)`

  示例：`get_connections("me", "friends", metadata=1)`

- `request(self, path, args=None, post_args=None)`

  示例：`request("search", {"q" : "programming", "type" : "group"})`

注意：与其他社交网络不同，Facebook似乎没有明确发布关于API速率限制的指导方针。虽然API似乎非常有效，但是为了安全起见，你仍然需要精心设计你的应用程序，尽可能少使用

API，并处理任何/所有错误情况。截至2010年底，你可能会发现的与指导方针最相关的是论坛中的开发者讨论（*http://forum.developers.facebook.net/viewtopic.php?pid=221976*）。

为了返回与一个对象相关的关系及该对象本身的细节，你可能使用的最常见的（通常是唯一的）关键字参数是`metadata=1`。看一看示例9-5，它介绍了`GraphAPI`类，并使用了它的`get_objects`方法来查询“编程小组”。它传送了结果集大小的重要特征，在很多类型的请求中，你都可能会得到该结果集。

示例9-5：查询“编程”小组的开放图谱（facebook_graph_query.py）

```
# -*- coding: utf-8 -*-

import sys
import json
import facebook
import urllib2
from facebook__login import login

try:
    ACCESS_TOKEN = open('out/facebook.access_token').read()
    Q = sys.argv[1]
except IOError, e:
    try:

        # If you pass in the access token from the Facebook app as a command line
        # parameter, be sure to wrap it in single quotes so that the shell
        # doesn't interpret any characters in it

        ACCESS_TOKEN = sys.argv[1]
        Q = sys.argv[2]
    except:
        print >> sys.stderr, \
            "Could not either find access token in 'facebook.access_token' or parse args."
        ACCESS_TOKEN = login()
        Q = sys.argv[1]

LIMIT = 100

gapi = facebook.GraphAPI(ACCESS_TOKEN)

# Find groups with the query term in their name

group_ids = []
i = 0
while True:
    results = gapi.request('search', {
        'q': Q,
        'type': 'group',
        'limit': LIMIT,
        'offset': LIMIT * i,
        })
    if not results['data']:
        break
```

```
    ids = [group['id'] for group in results['data'] if group['name'
            ].lower().find('programming') > -1]

    # once groups stop containing the term we are looking for in their name, bail out

    if len(ids) == 0:
        break
    group_ids += ids

    i += 1
if not group_ids:
    print 'No results'
    sys.exit()

# Get details for the groups

groups = gapi.get_objects(group_ids, metadata=1)

# Count the number of members in each group. The FQL API documentation at
# http://developers.facebook.com/docs/reference/fql/group_member hints that for
# groups with more than 500 members, we'll only get back a random subset of up
# to 500 members.

for g in groups:
    group = groups[g]
    conn = urllib2.urlopen(group['metadata']['connections']['members'])
    try:
        members = json.loads(conn.read())['data']
    finally:
        conn.close()
    print group['name'], len(members)
```

查询“编程小组”的抽样结果如示例9-6所示，结果集的上界接近于500并非巧合。正如代码注释中说明的那样，FQL文档指出：当查询group_member表时，结果的总条数会被限制在500之内。遗憾的是，Graph API文档仍然在发展中，编写这部分内容时，还没有提供类似的警告（虽然希望很快就能提供它们）。在统计群组成员数时，重点是你经常会使用大小合理的随机抽样。9.2.1节描述另一种情况，其中出现了某种意外的结果截断，以及如何通过调度多个查询来解决这个问题。

示例9-6：示例9-5的样例结果

```
Graffiti Art Programming 492
C++ Programming 495
Basic Programming 495
Programming 215
C Programming 493
C programming language 492
Programming 490
ACM Programming Competitors 496
programming 494
COMPUTER PROGRAMMING 494
Programming with Python 494
Game Programming 494
ASLMU Programming 494
```

```
Programming 352
Programming 450
Programmation - Programming 480
```

封装了本章的大多数示例代码并使用了相同基本模式的样例web应用程序（*http://miningthesocialweb.appspot.com/*）托管在GAE上，你可以在编写你自己的代码之前阅读一下它。图9-4说明了查询“编程”小组的结果。回想一下，如果对你来说，安装并完全自定义GAE支持的Facebook应用程序是比从本地控制台运行脚本更好的选择，那么你可以这么做。在生产率方面，一旦准备维持快速的开发周期，最好的方法可能就是通过本地脚本开发，然后把这些功能合并到GAE代码库中。

使用FQL交叉分析数据

正如你在前一节学到的，编写与Graph API交互的简单例程并不会有太多开销，因为图中的对象很简单，传送给你可以按原样使用的URL来查看对象的链接。然而，对于更先进的查询类型或某些工作流来说，你可能会发现FQL更适合这个问题。*http://developers.facebook.com/docs/reference/fql/*中存在大量的FQL文档。我们不会太深入探讨这个问题，因为在线文档很权威，而且随着平台不断变化，但如果你熟悉SQL语法的话，FQL是可以预见的。FQL查询的形式是：`select [fields] from [table] where [conditions]`，但是由于使用了各种限制条件，FQL成为了SQL中精心挑选的子集。例如，`from`子句中只可以出现一个表名，`where`子句中的条件是有限的（但通常足够了），必须被标记为FQL文档的索引字段。从5万英尺的水平在Facebook平台上执行FQL查询的角度来看，所需要的只是把FQL查询发送到两个API端点中的任何一个：*https://api.facebook.com/method/fql.query*或*https://api.facebook.com/method/fql.multiquery*。稍后会讨论它们两个的区别。

示例9-7说明了一个简单的FQL查询，它抓取姓名、性别和当前登录用户的好友的关系状态。

示例9-7：把用户和关系数据捆绑在一起的嵌套FQL查询

```
select name, sex, relationship_status from user where uid in
    (select target_id from connection where source_id = me() and target_type =
'user')
```

这个嵌套查询首先通过执行子查询来运行：

```
select target_id from connection where source_id = me() and target_type='user'
```

它会产生一个用户ID值列表。特殊的`me()`指令是一个非常方便的快捷方式，它与当前登录用户的ID相对应，`conection`表（*http://developers.facebook.com/docs/reference/fql/*

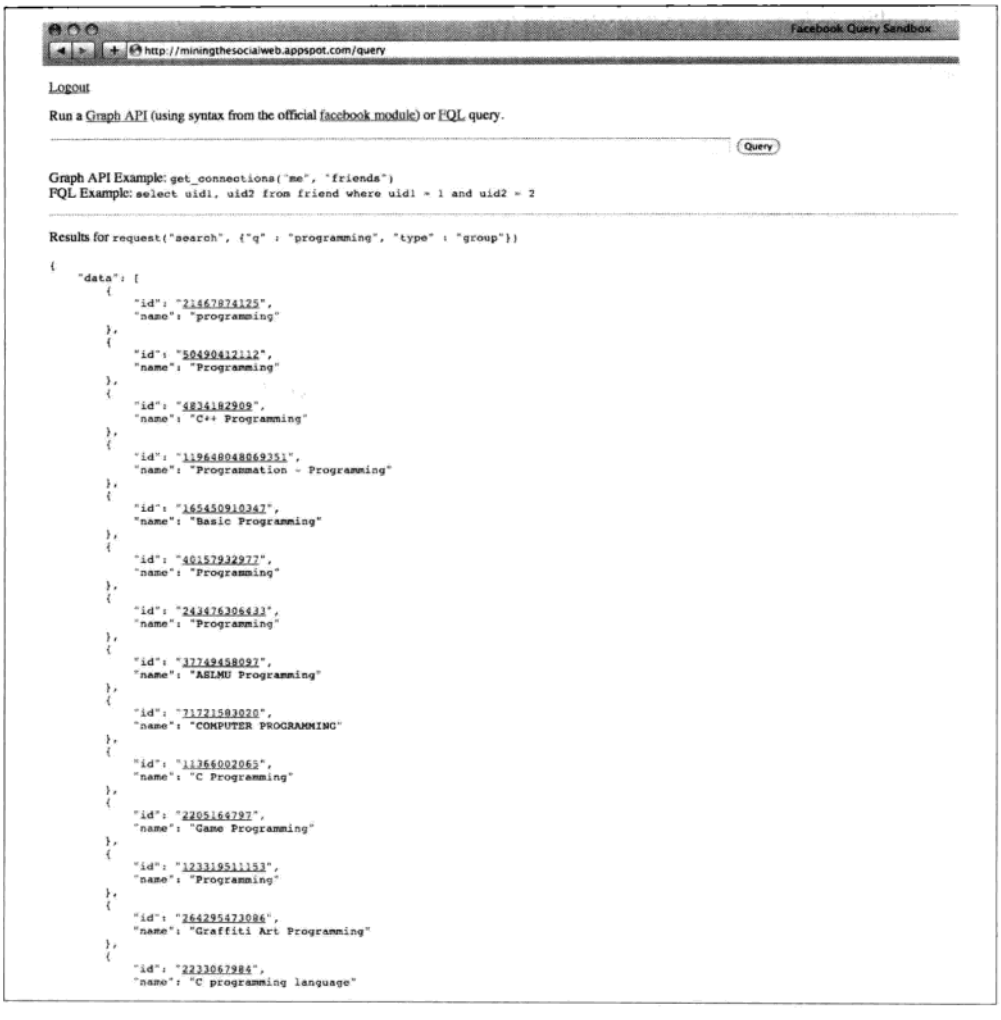

图9-4：查询“编程”小组的结果——可以点击这些链接抓取到图的下一层

connection）旨在允许查询，在其中查找当前登录用户的好友。请注意，虽然connection表中存储的大多数连接是用户之间的，但是用户和其他对象类型之间也可能存在其他连接类型，如'page '，所以target_type过滤器的存在是很重要的。稍后再评估外部查询，结构如下：

```
select name, sex, relationship_status from user where uid in(…)
```

从user（*http://developers.facebook.com/docs/reference/fql/user*）表中查询。FQL的user（*http://developers.facebook.com/docs/reference/fql/user*）表含有大量信息，可以对你的好友进行很多类有趣的分析。自己试一下吧！

FQL查询的结果集的一般形式如示例9-8所示。

示例9-8：查询的FQL结果

```
[
    {
        "name": "Matthew Russell",
        "relationship_status": "Married",
        "sex": "male"
    },
    ...
]
```

FQL多查询（multiquery）的运行方式与此相同，只是可以运行多个查询，将查询结果作为使用散列符号表名的参照。示例9-9是与前面的嵌套循环等效的FQL多查询。

示例9-9：把用户和关系数据捆绑在一起的FQL多查询

```
{
    "name_sex_relationships" : "select name, sex, relationship_status from user \
        where uid in (select target_id from #ids)",
    "ids" : "select target_id from connection where source_id = me() \
        and target_type = 'user'"
}
```

请注意，虽然嵌套查询返回单一的对象列表，但是FQL多查询会返回这两个元素的结果，如示例9-10所示。

示例9-10：示例FQL多查询结果

```
[
    {
        "fql_result_set": [
            {
                "target_id": -1
            },
            ...
        ],
        "name": "ids"
    },
    {
        "fql_result_set": [
            {
                "name": "Matthew Russell",
                "relationship_status": "Married",
                "sex": "male"
            },
            ...
        ],
        "name" : "name_sex_relationships"
    }
    ]
```

从编程方式来说，查询逻辑非常简单，可以被封装到一个小类中。示例9-11说明了可以从命令行中提取查询并运行查询的一个FQL类。下面是一些可以试运行的样例查询：

```
$ python facebook_fql_query.py 'select name, sex, relationship_status
from user where uid in (select target_id from connection
where source_id = me())'

$ python facebook_fql_query.py '{" name_sex_relationships": "select name,
sex, relationship_status from user where uid in (select target_id from #ids)",
"ids": "select target_id from connection where source_id = me()"}'
```

示例9-11：使用一个小型Python类抽象封装FQL查询（facebook_fql_query.py）

```
# -*- coding: utf-8 -*-

import sys
from urllib import urlencode
import json
import urllib2
from facebook_login import login

class FQL(object):

    ENDPOINT = 'https://api.facebook.com/method/'

    def __init__(self, access_token=None):
        self.access_token = access_token

    def _fetch(cls, url, params=None):
        conn = urllib2.urlopen(url, data=urlencode(params))
        try:
            return json.loads(conn.read())
        finally:
            conn.close()

    def query(self, q):
        if q.strip().startswith('{'):
            return self.multiquery(q)
        else:
            params = dict(query=q, access_token=self.access_token, format='json')
            url = self.ENDPOINT + 'fql.query'
            return self._fetch(url, params=params)

    def multiquery(self, q):
        params = dict(queries=q, access_token=self.access_token, format='json')
        url = self.ENDPOINT + 'fql.multiquery'
        return self._fetch(url, params=params)

# Sample usage...

if __name__ == '__main__':
    try:
        ACCESS_TOKEN = open('out/facebook.access_token').read()
        Q = sys.argv[1]
    except IOError, e:
        try:
```

```
            # If you pass in the access token from the Facebook app as a command line
            # parameter, be sure to wrap it in single quotes so that the shell
            # doesn't interpret any characters in it. You may also need to escape
            # the # character

            ACCESS_TOKEN = sys.argv[1]
            Q = sys.argv[2]
        except IndexError, e:
            print >> sys.stderr, \
                "Could not either find access token in 'facebook.access_token'
                    or parse args."
            ACCESS_TOKEN = login()
            Q = sys.argv[1]

    fql = FQL(access_token=ACCESS_TOKEN)
    result = fql.query(Q)
    print json.dumps(result, indent=4)
```

作为本章源代码的示例GAE应用程序（*http://mining the social web.appspot.com/*）检测并运行了FQL查询和Graph API查询，所以你可以把它当作尝试FQL的场所。现在已经有了一些执行查询的基本结构，下一节会遍历一些创建数据驱动可视化和UI控件的用例。

对Facebook数据的可视化

这一节规定了一些模板，介绍了一些分析与可视化Facebook数据的良好起点。如果你一直跟着我们的节奏，那么你现在已经有了获取平台提供的所有功能需要的基本工具。这一节探讨的练习类型包括：

- 可视化社交网络中所有的共同友谊。
- 可视化特定小组和任意标准（如性别）的共同友谊。
- 创建一个简单的数据驱动游戏，它可以让你基于好友们的现居住地和家乡来标识他们。

这些可能的列表仍在继续，现在已经拥有了必要的完整样板，这样你就可以轻松地定制将要编写的脚本用于解决其他问题了。

对整个社交网络的可视化

这一节包括从Facebook账号中抓取友谊信息、使用JavaScript InfoVis Toolkit（*http://thejit.org/*）（JIT），以有趣且有用的方式可视化它的全过程。JIT提供了一些精彩的示例模板，可以被轻松地定制为交互式可视化。

使用RGraphs进行可视化

RGraph（*http://en.wikipedia.org/wiki/Radial_tree*）[注2]是一种网络可视化，它通过布置同心圆中从圆心开始的节点来组织显示。RGraphs可以在很多可视化工具包中使用，包括JIT。值得花点时间研究一下JIT的RGraph示例来熟悉它提供的巨大可能性。

注意： 6.2.4节介绍的Protovis的节点树（Node-Link tree）本质上是相同的核心可视化，但是JIT也是值得拥有的工具，它提供了更多交互式的即装即用示例，你可以轻易地找到并运行它们。多点选择并不是坏事。

正如你所熟知的，大多数用于连接存在于原始Facebook数据的抓取和这些数据的可视化之间的点的工作都是整理，对于把它转换为可以被可视化使用的格式来说，这是必要的。可以被RGraph和其他JIT图可视化接受的一种数据格式是一种可预见的JSON结构，它是由对象列表组成的，每个对象代表一个节点和它的邻节点。示例9-12传达了一些基本概念，这样你就可以知道我们的最终目标是什么了。这个结构表示“Matthew”与其他三个由邻节点确定的ID值标识的节点相连。在本例中，`data`字段提供了“Matthew”的更多额外信息，以人类可读的标签传达了他的三个关系，以及看似归一化的人气指数。

示例9-12：可以被JIT的RGraph可视化使用的示例输入

```
[
    {
        "adjacencies": [
            "2",
            "3",
            "4",
            ...
        ],
        "data": {
            "connections": "Mark<br>Luke<br>John",
            "normalized_popularity": 0.0079575596817,
            "sex" : "male"
        },
        "id": "1",
        "name": "Matthew"
    },
    ...
    ]
```

让我们计算存在于你的朋友网中的共同友谊（mutual friendship），并把该数据可视化为RGraph。也就是说，我们会计算存在于你的朋友网中的所有友谊，它会告诉我们在你的

注2：　通常也称为径向图、径向树、径向地图，以及不必包括“径向”或“半径”的其他说法。

网络中谁最受欢迎，以及谁没有被很好地连接。了解这两点内容会有潜在的价值。从抽象层次来说，我们只需要运行一些FQL查询来获取数据：一个用来计算你的好友的ID，另一个用来连接图中的好友，最后的查询用来抓取图中的所有相关细节，如姓名、生日等。下面的查询说明了这个想法：

获取好友ID

```
q = "select target_id from connection where \
source_id = me() and target_type = 'user'"
my_friends = [str(t['target_id']) for t in fql.query(q)]
```

计算共同的友谊

```
q = "select uid1, uid2 from friend where uid1 in (%s) and uid2 in (%s)" %
(",".join(my_friends), ",".join(my_friends),)
mutual_friendships = fql(q)
```

抓取其他详细资料来装饰图

```
q = "select uid, first_name, last_name, sex from user where uid in (%s)" %
(",".join(my_friends),)
names = dict([(unicode(u["uid"]), u["first_name"] + " " +
u["last_name"][0] + ".") for u in fql(q)])
```

这就是重点，但是你可能会发现一个没有得到良好归档的细节：在第二步计算共同的友谊时，你会发现，即使你传送了合适数量的ID值，也会返回任意截断的结果。这是合理的，因为你在要求Facebook平台代表你执行逻辑上为$O(n^2)$的操作（相互比较你的好友的嵌套循环），但事实是会返回任意数据，而不是接收到一条错误消息，友好地提示你输入了较少的数据，这可能会让你措手不及。

警告： 和世界上其他任何事情一样，Facebook平台并不完美。但是做的好的地方要称赞，需要时也要提交错误报告。

幸运的是，可以修复这种情况：只要通过几个查询分批输入数据，然后聚焦它们即可。在代码中，表示方式如下：

```
mutual_friendships = []
N = 50
for i in range(len(my_friends)/N +1):
    q = "select uid1, uid2 from friend where uid1 in (%s) and uid2 in (%s)" %
        (",".join(my_friends), ",".join(my_friends[i*N:(i+1)*N]),)
    mutual_friendships += fql(query=q)
```

除了这个潜在的阻碍之外，包含在产生RGraph可视化的期望输出中的其他逻辑非常简

单，遵循刚才介绍的模式。示例9-13说明了一个可行的示例，它把所有的内容都聚在一起，产生了可以用做其他分析的JavaScript输出。

示例9-13：为JIT的RGraph可视化获取并整理朋友数据（facebook_get_friends_rgraph.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import json
import webbrowser
import shutil
from facebook__fql_query import FQL
from facebook__login import login

HTML_TEMPLATE = '../web_code/jit/rgraph/rgraph.html'
OUT = os.path.basename(HTML_TEMPLATE)

try:
    ACCESS_TOKEN = open('out/facebook.access_token').read()
except IOError, e:
    try:

        # If you pass in the access token from the Facebook app as a command line
        # parameter, be sure to wrap it in single quotes so that the shell
        # doesn't interpret any characters in it. You may also need to escape
        # the # character

        ACCESS_TOKEN = sys.argv[1]
    except IndexError, e:
        print >> sys.stderr, \
            "Could not either find access token in 'facebook.access_token' or parse args."
        ACCESS_TOKEN = login()

fql = FQL(ACCESS_TOKEN)

# get friend ids

q = \
    'select target_id from connection where source_id = me() and target_type =\'user\''
my_friends = [str(t['target_id']) for t in fql.query(q)]

# now get friendships among your friends. note that this api appears to return
# arbitrarily truncated results if you pass in more than a couple hundred friends
# into each part of the query, so we perform (num friends)/N queries and aggregate
# the results to try and get complete results
# Warning: this can result in a several API calls and a lot of data returned that
# you'll have to process

mutual_friendships = []
N = 50
for i in range(len(my_friends) / N + 1):
    q = 'select uid1, uid2 from friend where uid1 in (%s) and uid2 in (%s)' \
        % (','.join(my_friends), ','.join(my_friends[i * N:(i + 1) * N]))
    mutual_friendships += fql.query(q)
```

```
# get details about your friends, such as first and last name, and create an accessible map
# note that not every id will necessarily information so be prepared to handle those cases
# later

q = 'select uid, first_name, last_name, sex from user where uid in (%s)' \
    % (','.join(my_friends), )
results = fql.query(q)
names = dict([(unicode(u['uid']), u['first_name'] + ' ' + u['last_name'][0] + '.'
             ) for u in results])

sexes = dict([(unicode(u['uid']), u['sex']) for u in results])

# consolidate a map of connection info about your friends.

friendships = {}
for f in mutual_friendships:
    (uid1, uid2) = (unicode(f['uid1']), unicode(f['uid2']))
    try:
        name1 = names[uid1]
    except KeyError, e:
        name1 = 'Unknown'
    try:
        name2 = names[uid2]
    except KeyError, e:
        name2 = 'Unknown'

    if friendships.has_key(uid1):
        if uid2 not in friendships[uid1]['friends']:
            friendships[uid1]['friends'].append(uid2)
    else:
        friendships[uid1] = {'name': name1, 'sex': sexes.get(uid1, ''),
                             'friends': [uid2]}

    if friendships.has_key(uid2):
        if uid1 not in friendships[uid2]['friends']:
            friendships[uid2]['friends'].append(uid1)
    else:
        friendships[uid2] = {'name': name2, 'sex': sexes.get(uid2, ''),
                             'friends': [uid1]}

# Emit JIT output for consumption by the visualization

jit_output = []
for fid in friendships:
    friendship = friendships[fid]
    adjacencies = friendship['friends']

    connections = '<br>'.join([names.get(a, 'Unknown') for a in adjacencies])
    normalized_popularity = 1.0 * len(adjacencies) / len(friendships)
    sex = friendship['sex']
    jit_output.append({
        'id': fid,
        'name': friendship['name'],
        'data': {'connections': connections, 'normalized_popularity'
                 : normalized_popularity, 'sex': sex},
        'adjacencies': adjacencies,
        })
```

```
# Wrap the output in variable declaration and store into
# a file named facebook.rgraph.js for consumption by rgraph.html

if not os.path.isdir('out'):
    os.mkdir('out')

# HTML_TEMPLATE references some dependencies that we need to
# copy into out/

shutil.rmtree('out/jit', ignore_errors=True)

shutil.copytree('../web_code/jit',
                'out/jit')

html = open(HTML_TEMPLATE).read() % (json.dumps(jit_output),)
f = open(os.path.join(os.getcwd(), 'out', 'jit', 'rgraph', OUT), 'w')
f.write(html)
f.close()

print >> sys.stderr, 'Data file written to: %s' % f.name

# Write out another file that's standard JSON for additional analysis
# and potential use later (by facebook_sunburst.py, for example)

json_f = open(os.path.join('out', 'facebook.friends.json'), 'w')
json_f.write(json.dumps(jit_output, indent=4))
json_f.close()

print 'Data file written to: %s' % json_f.name

# Open up the web page in your browser

webbrowser.open('file://' + f.name)
```

警告： 如果你的朋友网很大，你可能需要通过一个有意义的标准使得结果更易管理，筛选你要可视化的朋友。示例9-16是通过交互地提示用户输入，按组筛选的示例。

如果一切都按计划进行，那么你就会拥有整个社交网络的一个交互式可视化。图9-5说明了大型社交网络的交互式可视化是什么样的。下一节会介绍一些方法，用于缩小输出以便使其更容易管理。

使用Sunburst进行可视化

Sunburst可视化（*http://www.cc.gatech.edu/gvu/ii/sunburst*）是表示树这类层次结构的一种空间填充可视化。它的名字来源于它与旭日图像的相似性，如图9-6中所示的日本军旗。JIT的Sunburst（*http://thejit.org/static/v20/Jit/Examples/Sunburst/example2.html*）实现和它的美一样强大。和RGraph一样，它使用一个简单的图状JSON结构，并提供方便的事件处理程序。虽然Sunburst可视化与RGraph使用的数据结构基本相同，但其布局几乎可以立马生成更多的洞察信息。你很容易就可以看到树中中间节点的相关度，它是以Sunburst中每一层占用多大面积为基础的。例如，要用JIT的示例实现来呈现一个非常大

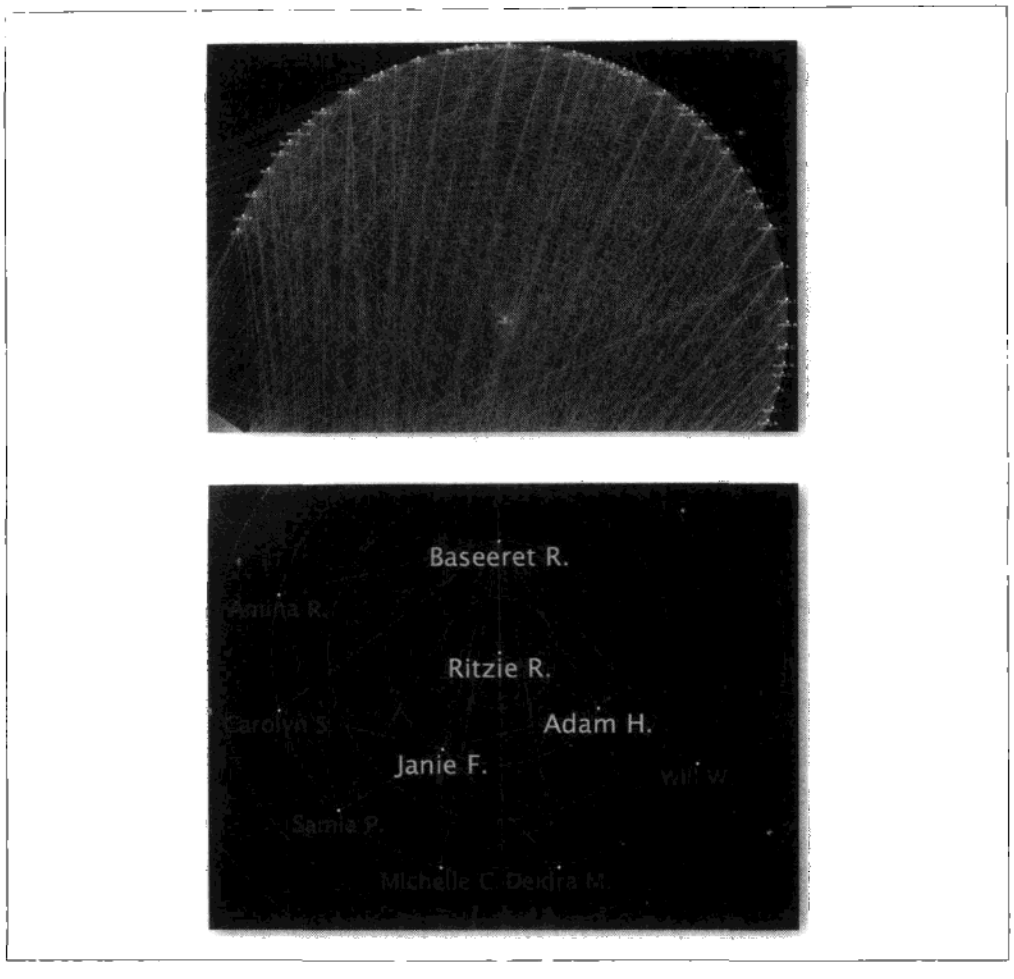

图9-5：使用500多人组成的大型网络的Facebook数据，通过示例9-13计算得到的示例RGraph（上图）和一个共同好友的较小的RGraph，其共同好友是某个小组的成员，由本章稍后会介绍的示例9-16计算得到（下图）

的社交网络的一个有用的交互式可视化并不困难，只要这些人数的相对份额能与给定的标准（例如，性别）相匹配即可。事实上，这正是图9-6所描绘的。它表明这个社交网络中大概2/3的成员是女性，每个成员都并排显示在扇形图中。

示例9-14说明了如何获取示例9-13的输出并转换它，这样它就能被JIT Sunburst使用了，后者按性别细分社交网络。请注意`$angularWidth`参数是被树的每层所使用的角的相对度量，被该脚本用来按比例显示扇形图（*http://en.wikipedia.org/wiki/Circular_sector*）。每个好友占用的区域根据他们的知名度变化，并且提供了一个直观的可视化指示器。防止

API访问性别信息而设置隐私的好友的性别值是一个空字符串，被简单地忽略了[注3]。当悬停在某一部分时，可视化的交互版本也会显示一个工具提示，表明某人的共同好友，如图9-6所示，这样就可以具体到某个人，并了解更多细节了。

示例9-14：获取并整理JIT的Sunburst可视化（facebook_sunburst.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import json
import webbrowser
import shutil
from copy import deepcopy

HTML_TEMPLATE = '../web_code/jit/sunburst/sunburst.html'
OUT = os.path.basename(HTML_TEMPLATE)

# Reuses out/facebook.friends.json written out by
# facebook__get_friends_rgraph.py
DATA = sys.argv[1]
data = json.loads(open(DATA).read())

# Define colors to be used in the visualization
# for aesthetics

colors = ['#FF0000', '#00FF00', '#0000FF']

# The primary output to collect input

jit_output = {
    'id': 'friends',
    'name': 'friends',
    'data': {'$type': 'none'},
    'children': [],
    }

# A convenience template

template = {
    'id': 'friends',
    'name': 'friends',
    'data': {'connections': '', '$angularWidth': 1, '$color': ''},
    'children': [],
    }

i = 0
for g in ['male', 'female']:

    # Create a gender object

    go = deepcopy(template)
    go['id'] += '/' + g
```

注3：　也通常称为“pie piece”。

```
    go['name'] += '/' + g
    go['data']['$color'] = colors[i]

    # Find friends by each gender

    friends_by_gender = [f for f in data if f['data']['sex'] == g]
    for f in friends_by_gender:

        # Load friends into the gender object

        fo = deepcopy(template)
        fo['id'] = f['id']
        fo['name'] = f['name']
        fo['data']['$color'] = colors[i % 3]
        fo['data']['$angularWidth'] = len(f['adjacencies']) # Rank by global popularity
        fo['data']['connections'] = f['data']['connections'] # For the tooltip

        go['children'].append(fo)

    jit_output['children'].append(go)
    i += 1

# Emit the output expected by the JIT Sunburst

if not os.path.isdir('out'):
    os.mkdir('out')

# HTML_TEMPLATE references some dependencies that we need to
# copy into out/

shutil.rmtree('out/jit', ignore_errors=True)

shutil.copytree('../web_code/jit',
                'out/jit')

html = open(HTML_TEMPLATE).read() % (json.dumps(jit_output),)
f = open(os.path.join(os.getcwd(), 'out', 'jit', 'sunburst', OUT), 'w')
f.write(html)
f.close()

print 'Data file written to: %s' % f.name

# Open up the web page in your browser

webbrowser.open('file://' + f.name)
```

虽然示例实现值是按性别和受欢迎程度对好友分组的，但是可以产生代表多层次树并且可使用的JSON对象，其中每层都对应一个不同的标准。例如，可以先按性别可视化，然后按“关系状态”获得这两个变量如何与你的社交网络相互关联的直观感受。

使用电子表格（老式方法）进行可视化

虽然你可能会设想自己在浏览器中创建很多具有巨大视觉吸引力的可视化，但是你更可能想用老方法：在电子表格中可视化数据。假设你把上一节中增强了RGraph的JSON输出保存在一个本地文件中，那么你就可以很轻松地处理这些数字，产生一种简单的CSV

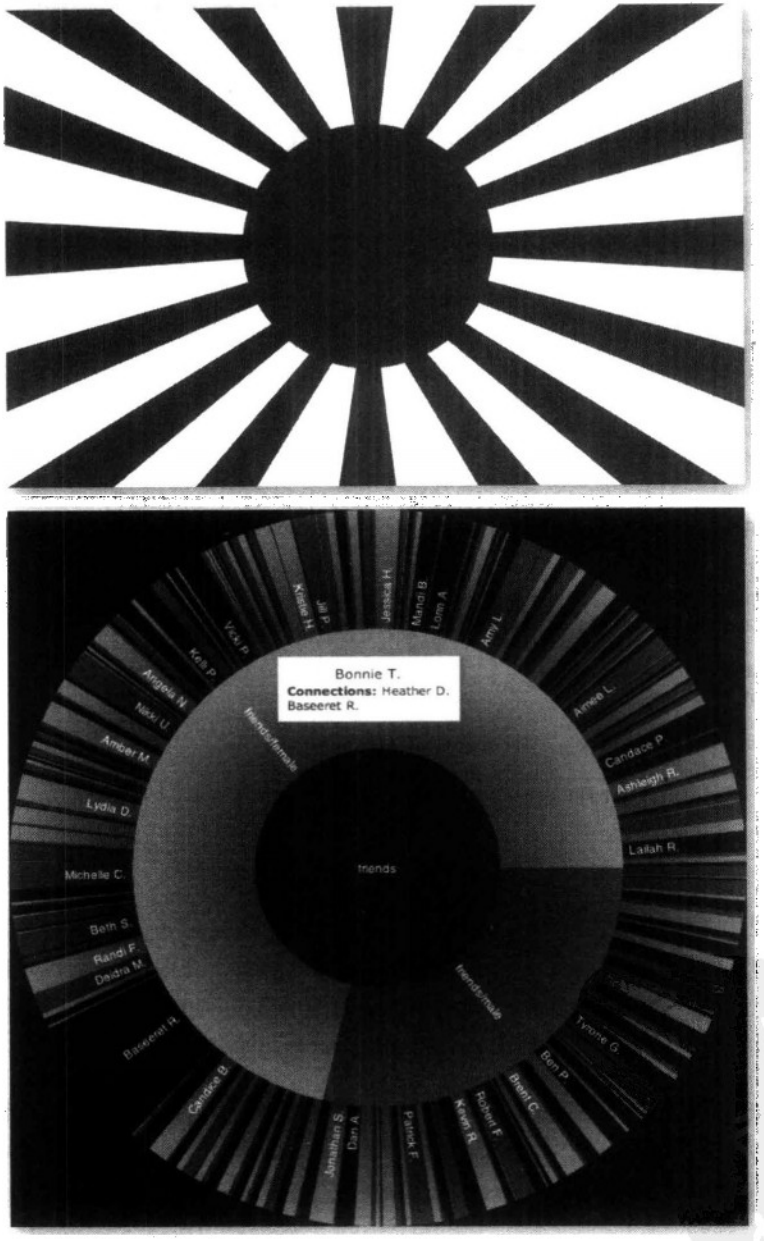

图9-6：Sunburst可视化的名字来源于它与太阳图像的明显相似性，如日本军旗（上图）。这种特殊的可视化说明朋友网大约2/3是女性，大约1/3是男性（下图）

格式，可以把它加载到一个电子表格中，快速地了解友谊分布看起来如何，即，显示社交网络中每个好友受欢迎程度的直方图。示例9-15说明了如何快速地把数据转化为容易使用的CSV格式。

示例9-15：导出数据，这样很容易就可以把它加载到电子表格中来分析了（facebook_popularity_spreadsheet.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import json
import operator

# Reuses out/facebook.friends.json written out by
# facebook__get_friends_rgraph.py
DATA = open(sys.argv[1]).read()
data = json.loads(DATA)

popularity_data = [(f['name'], len(f['adjacencies'])) for f in data]
popularity_data = sorted(popularity_data, key=operator.itemgetter(1))

csv_data = []
for d in popularity_data:
    csv_data.append('%s\t%s' % (d[0], d[1]))

if not os.path.isdir('out'):
    os.mkdir('out')

filename = os.path.join('out', 'facebook.spreadsheet.csv')
f = open(filename, 'w')
f.write('\n'.join(csv_data))
f.close()

print 'Data file written to: %s' % filename
```

把数据可视化为直方图会产生有趣的结果，因为它明确显示了“网络中人们的连接方式”。例如，如果网络连接完美，那么分布就会是平坦的。在图9-7中，你可以看到：网络中第二受欢迎的人的关系大约是网络中最受欢迎的人的一半，网络中剩余的关系大致遵循有一条长尾巴的齐普夫分布[注4]，它非常符合对数趋势线。

对于非常大且多样化的社交网络来说，对数分布是可预测的。有一些关系亲密的人是不可避免的，与最受欢迎的个人相比，大多数人的关系相对较少。按照帕累托原则，或80-20法则（*http://en.wikipedia.org/wiki/Pareto_principle*），我们可以说“20%的人有80%的朋友。”

注4：　7.2节介绍了这个概念。

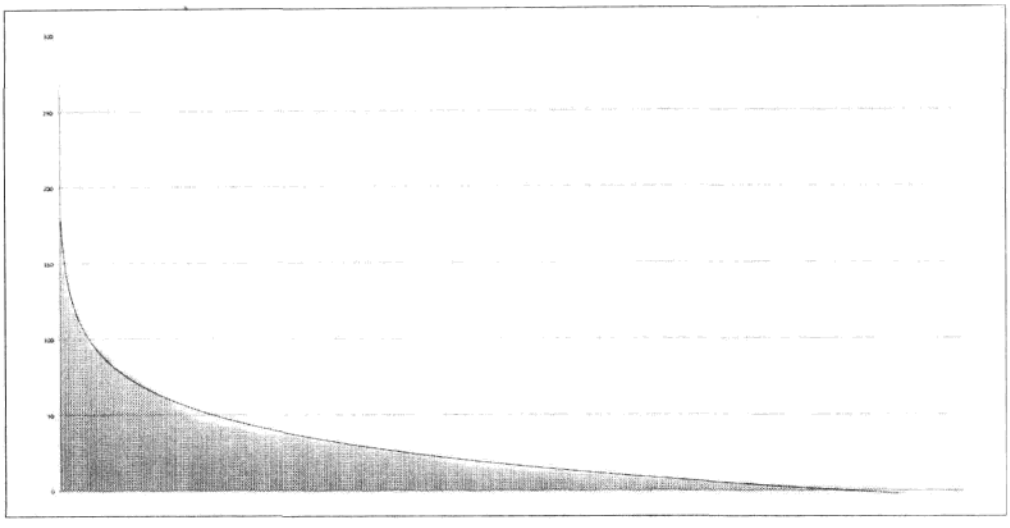

图9-7：在Facebook好友网络中受欢迎程序的分布和对数趋势线（为了保护无辜，省略了名字）

可视化分组中的共同友谊

示例9-13中提供的示例实现尝试可视化社交网络中的所有共同友谊。如果在一个支持canvas元素的现代浏览器中运行它的话，JIT甚至可以很好地支持巨大图。示例实现中包含了一个单击事件处理程序：它显示了图中当前选中的节点的连接和标准的人气得分（计算方法为：关系数除以网络中的关系总数）。查看整个朋友网肯定是有趣的，但是很快你就想要按一些有意义的标准筛选好友，分析更小的图。

如果目标是为一个小型Web应用程序创建一个好看的用户界面，那么可以有很多优秀的筛选选项，但是我们会采用最简单可行的方法，根据制定的分组标准来筛选示例9-13产生的JSON输出。群组是最合乎逻辑的起点，示例9-16说明了如何实现这一附加功能。从你的全球朋友网中过滤输出的好处是：你会保留网络剩余部分的完整的“connections”数据，所以即使某人的其他朋友不在小组中，你也仍然可以了解他还连接到哪些朋友。请记住，其他组分析的可能性是巨大的：你可以分析你是否与你的男性好友关系更为密切，执行4.3.1节描述的派系分析，或者很多其他可能性。

示例9-16：获取并整理数据来可视化特某个组的共同朋友（facebook_filter_rgraph_output_by_group.py）

```
# -*- coding: utf-8 -*-

import os
import sys
import json
```

```
import facebook
import webbrowser
import shutil
from facebook__fql_query import FQL
from facebook__login import login

HTML_TEMPLATE = '../web_code/jit/rgraph/rgraph.html'
OUT = os.path.basename(HTML_TEMPLATE)

# Reuses out/facebook.friends.json written out by
# facebook__get_friends_rgraph.py

DATA = sys.argv[1]
rgraph = json.loads(open(DATA).read())

try:
    ACCESS_TOKEN = open('out/facebook.access_token').read()
except IOError, e:
    try:

        # If you pass in the access token from the Facebook app as a command line
        # parameter, be sure to wrap it in single quotes so that the shell
        # doesn't interpret any characters in it. You may also need to escape
        # the # character.

        ACCESS_TOKEN = sys.argv[2]
    except IndexError, e:
        print >> sys.stderr, \
            "Could not either find access token in 'facebook.access_token' or parse args."
        ACCESS_TOKEN = login()

gapi = facebook.GraphAPI(ACCESS_TOKEN)

groups = gapi.get_connections('me', 'groups')

# Display groups and prompt the user

for i in range(len(groups['data'])):
    print '%s) %s' % (i, groups['data'][i]['name'])

choice = int(raw_input('Pick a group, any group: '))
gid = groups['data'][choice]['id']

# Find the friends in the group

fql = FQL(ACCESS_TOKEN)
q = \
    """select uid from group_member where gid = %s and uid in
(select target_id from connection where source_id = me() and target_type = 'user')
""" \
    % (gid, )

uids = [u['uid'] for u in fql.query(q)]

# Filter the previously generated output for these ids

filtered_rgraph = [n for n in rgraph if n['id'] in uids]
```

```
# Trim down adjancency lists for anyone not appearing in the graph.
# Note that the full connection data displayed as HTML markup
# in "connections" is still preserved for the global graph.

for n in filtered_rgraph:
    n['adjacencies'] = [a for a in n['adjacencies'] if a in uids]

if not os.path.isdir('out'):
    os.mkdir('out')

# HTML_TEMPLATE references some dependencies that we need to
# copy into out/

shutil.rmtree('out/jit', ignore_errors=True)

shutil.copytree('../web_code/jit',
                'out/jit')

html = open(HTML_TEMPLATE).read() % (json.dumps(filtered_rgraph),)
f = open(os.path.join(os.getcwd(), 'out', 'jit', 'rgraph', OUT), 'w')
f.write(html)
f.close()

print 'Data file written to: %s' % f.name

# Open up the web page in your browser

webbrowser.open('file://' + f.name)
```

曾经在同一个小学上学的一群人的一幅样本图如图9-5所示。

比较不同类型的组之间的关系、确定哪些好友参加了你所在的大多数组无疑是有趣的。但是不要把你的时间都花在这件事情上；还有很多其他可能性。

注意： 如果你正在使用支持最新且最好的canvas元素的现代浏览器，你可能会改编下面JavaScript代码把图另存为一幅图像：

```
//grab the canvas element
var canvas = document.getElementById("infovis-canvas");
//now prompt a file download
window.location = canvas.toDataURL("image/png");
```

我的朋友都到哪里去了（一个由数据驱动的游戏）

有很多可以在分析中关联的有趣变量，但是每个人都喜欢偶尔玩一款好玩的游戏。这一节为你可能玩的简单游戏奠定了基础，来看看你对你的朋友了解多少，通过对他们分组，而并列他们的家乡和现居住地。我们会重新使用6.2.3节的树控件作为这个实例的一部分，而且大部分工作都会被投入到要求把FQL查询的数据转化为合适的格式中。因为这个苦差事多半都很乏味，所以我们将通过只显示FQL查询和最终格式来保存一些树。和平常一样，完整的源代码可以从*http://github.com/ptwobrussell/Mining-the-Social-Web/*

*blob/master/python_code/linkedin__get_friends_current_locations_and_hometowns.py*中得到。

我们为了获取姓名、现居住地和家乡而运行的FQL查询很简单，它与前面的FQL查询非常相似：

```
q = """select name, current_location, hometown_location from user where uid in
       (select target_id from connection where source_id = me())"""
results = fql(query=q)
```

一旦你付出了巨大的努力将其转化为合适的格式，就可以得到示例9-17所示的提供给树控件的最终格式，示例9-18是生成它的Python代码。

示例9-17：为Dojo树控件使用而产生的目标JSON

```
{
    "items": [
        {
            "name": " Alabama (2)",
            "children": [
                {
                    "state": " Alabama",
                    "children": [
                        {
                            "state": " Tennessee",
                            "name": "Nashville, Tennessee (1)",
                            "children": [
                                {
                                    "name": "Joe B."
                                }
                            ]
                        }
                    ],
                    "name": "Prattville, Alabama (1)",
                    "num_from_hometown": 1
                }
            ]
        },
        {
            "name": " Alberta (1)",
            "children": [
                {
                    "state": " Alberta",
                    "children": [
                        {
                            "state": " Alberta",
                            "name": "Edmonton, Alberta (1)",
                            "children": [
                                {
                                    "name": "Gina F."
                                }
                            ]
                        }
```

```
                ],
                "name": "Edmonton, Alberta (1)",
                "num_from_hometown": 1
            }
        ]
    },
    ...
],
"label": "name"
}
```

最终的控件类似于图9-8，它是首先按朋友的现居住地，然后按他们的家乡分组的一个层次排列。Jess C.现在居住在Tuscaloosa，AL，但是是在Princeton，WV长大的。虽然我们正在关联两个无害的变量，但是这个练习帮助你快速确定你的朋友大多住在哪里，了解谁离开了家乡，谁一直留在家乡。不难想象更有趣的偏差，或介绍毕业的大学、职业联系或婚姻状况这类其他变量的分面显示。

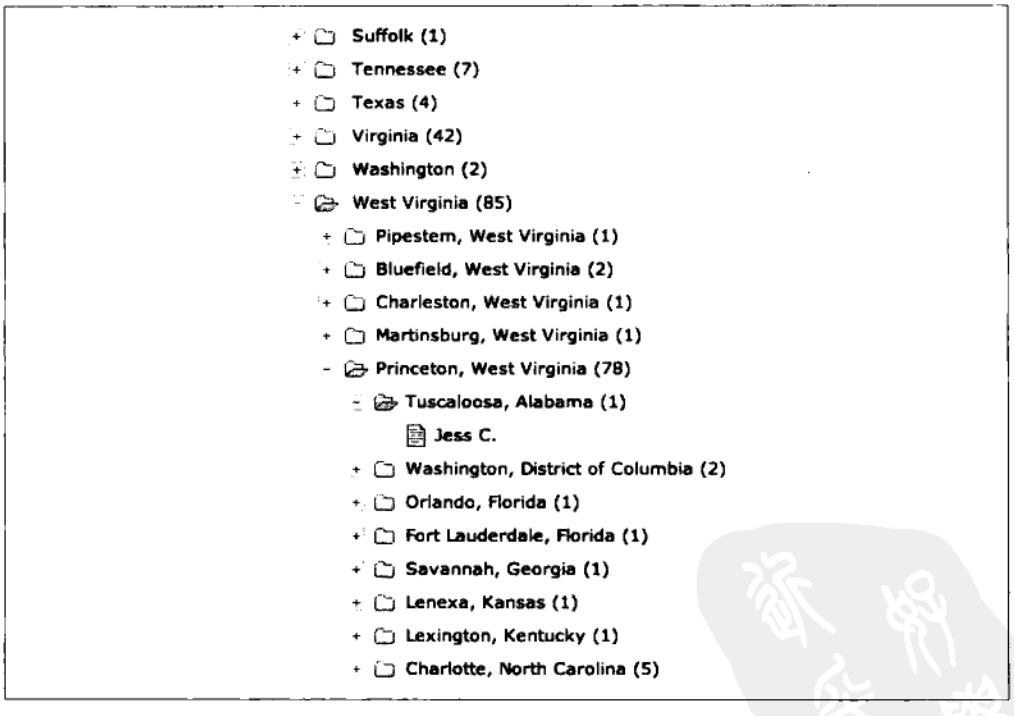

图9-8：你有多了解你的朋友呢？通过玩这款非常流行的游戏：《他们现在在哪里？》，找出答案，这款游戏信息量大，且非常好玩

简单的FQL查询用来抓取基本数据，但是还包括一些积累数据项以填充分层树控件的工作。树控件的源代码与6.2.3节的代码是相同的，为了可视化数据必要的是把它抓取到文件中，然后把树控件指向它。用户体验的一个有趣改善可能是把Google Maps和该控件

结合起来，这样你就可以在地图上快速找到你不熟悉的场所了。如果你想深入分析或使用其他方法聚类的话，把年龄和性别信息添加到这个显示中可能也是很有趣的。生成一些与6.4节描述相似的KML，用Google Maps可视化它可能是值得考虑的另一种可能性，这取决于你的目标。

示例9-18：获取数据，计算示例9-17中显示的目标JSON（facebook_get_friends_current_locations_and_hometowns.py）

```
import sys
import json
import facebook
from facebook__fql_query import FQL
from facebook__login import login

try:
    ACCESS_TOKEN = open("facebook.access_token").read()
except IOError, e:
    try:
        # If you pass in the access token from the Facebook app as a command-line
        # parameter, be sure to wrap it in single quotes so that the shell
        # doesn't interpret any characters in it. You may also need to escape
        # the # character.
        ACCESS_TOKEN = sys.argv[1]
    except IndexError, e:
        print >> sys.stderr, "Could not either find access token" + \
            in 'facebook.access_token' or parse args. Logging in..."
        ACCESS_TOKEN = login()

# Process the results of the following FQL query to create JSON output suitable for
# consumption by a simple hierarchical tree widget:

fql = FQL(ACCESS_TOKEN)

q = \
    """select name, current_location, hometown_location from user where uid in
        (select target_id from connection where source_id = me() and target_type =
            'user')"""
results = fql.query(q)

# First, read over the raw FQL query and create two hierarchical maps that group
# people by where they live now and by their hometowns. We'll simply tabulate
# frequencies, but you could easily grab additional data in the FQL query and use it
# for many creative situations.

current_by_hometown = {}
for r in results:
    if r['current_location'] != None:
        current_location = r['current_location']['city'] + ', ' \
            + r['current_location']['state']
    else:
        current_location = 'Unknown'

    if r['hometown_location'] != None:
        hometown_location = r['hometown_location']['city'] + ', ' \
            + r['hometown_location']['state']
```

```
    else:
        hometown_location = 'Unknown'

    if current_by_hometown.has_key(hometown_location):
        if current_by_hometown[hometown_location].has_key(current_location):
            current_by_hometown[hometown_location][current_location] += \
                [r['name']]
        else:
            current_by_hometown[hometown_location][current_location] = \
                [r['name']]
    else:
        current_by_hometown[hometown_location] = {}
        current_by_hometown[hometown_location][current_location] = \
            [r['name']]

# There are a lot of different ways you could slice and dice the data now that
# it's in a reasonable data structure. Let's create a hierarchical
# structure that lends itself to being displayed as a tree.

items = []
for hometown in current_by_hometown:
    num_from_hometown = sum([len(current_by_hometown[hometown][current])
                             for current in current_by_hometown[hometown]])
    name = '%s (%s)' % (hometown, num_from_hometown)
    try:
        hometown_state = hometown.split(',')[1]
    except IndexError:
        hometown_state = hometown
item = {'name': name, 'state': hometown_state,
        'num_from_hometown': num_from_hometown}

item['children'] = []
for current in current_by_hometown[hometown]:
    try:
current_state = current.split(',')[1]
except IndexError:
current_state = current

item['children'].append({'name': '%s (%s)' % (current,
                        len(current_by_hometown[hometown][current])),
                        'state': current_state, 'children'
                        : [{'name': f[:f.find(' ') + 2] + '.'}
                        for f in
                        current_by_hometown[hometown][current]]})

# Sort items alphabetically by state. Further roll-up by state could
# be done here if desired.

item['children'] = sorted(item['children'], key=lambda i: i['state'])
items.append(item)

# Optionally, roll up outer-level items by state to create a better user experience
# in the display. Alternatively, you could just pass the current value of items in
# the final statement that creates the JSON output for smaller data sets.

items = sorted(items, key=lambda i: i['state'])
all_items_by_state = []
```

```
grouped_items = []
current_state = items[0]['state']
num_from_state = items[0]['num_from_hometown']
for item in items:
    if item['state'] == current_state:
        num_from_state += item['num_from_hometown']
        grouped_items.append(item)
    else:
        all_items_by_state.append({'name': '%s (%s)' % (current_state,
                num_from_state), 'children': grouped_items})
        current_state = item['state']
        num_from_state = item['num_from_hometown']
        grouped_items = [item]
all_items_by_state.append({'name': '%s (%s)' % (current_state,
                          num_from_state), 'children': grouped_items})

# Finally, emit output suitable for consumption by a hierarchical tree widget

print json.dumps({'items': all_items_by_state, 'label': 'name'},
indent=4)
```

把留言板数据可视化为（旋转的）标签云

和非结构化数据的其他源代码一样，分析留言板上或新闻供稿中使用的语言是一个有趣的主题。在网上可以找到很多标签云控件，它们都采用了同样的输入——实质上是频率分布。但是当你可以使用可定制的、交互式标签云时，为什么要使用普通的标签云可视化数据呢？回想一下，恰好有一个称为WP-Cumulus（*http://code.google.com/p/word-cumulus-goog-vis/wiki/UserGuide*）开放源代码旋转标签云有不错的表现[注5]。让它工作所需要的只是产生它预计的简单输入格式，把这种输入格式提供给包含标准HTML样板的模板。为了简洁起见，这里就不再重复这个样板了。请查看*http://github.com/ptwobrussell/Mining-the-Social-Web/blob/master/web_code/dojo/facebook.current_locations_and_hometowns.html*。

示例9-19列出了一些获取新闻数据网页的最小逻辑，并计算简单的JSON结构，它是可以提供给HTML模板的[term，URL，frequency]元组列表。我们会给这些元组的URL传送空字符串，但是只需要一点额外的工作，你就可以引入附加逻辑了，该逻辑用于维护哪些词出现在哪些文章中的地图。这样标签云中的数据就可以指回源代码了。请注意，因为我们正在从这个示例的Graph API中抓取多页数据，所以我们简单地选择直接与你的Facebook留言板中的API端点交互。

示例9-19：获取并整理可视化为WP-Cumulus标签云的数据（facebook_tag_cloud.py）

```
# -*- coding: utf-8 -*-
```

注5：　5.4.1节介绍了它。

```
import os
import sys
import urllib2
import json
import webbrowser
import nltk
from cgi import escape
from facebook_login import login

try:
    ACCESS_TOKEN = open('out/facebook.access_token').read()
except IOError, e:
    try:

        # If you pass in the access token from the Facebook app as a command line
        # parameter, be sure to wrap it in single quotes so that the shell doesn't
        # interpret any characters in it. You may also need to escape the # character

        ACCESS_TOKEN = sys.argv[1]
    except IndexError, e:
        print >> sys.stderr, \
            "Could not find local access token or parse args. Logging in..."
        ACCESS_TOKEN = login()

BASE_URL = 'https://graph.facebook.com/me/home?access_token='
HTML_TEMPLATE = '../web_code/wp_cumulus/tagcloud_template.html'
OUT_FILE = 'out/facebook.tag_cloud.html'
NUM_PAGES = 5
MIN_FREQUENCY = 3
MIN_FONT_SIZE = 3
MAX_FONT_SIZE = 20

# Loop through the pages of connection data and build up messages

url = BASE_URL + ACCESS_TOKEN
messages = []
current_page = 0
while current_page < NUM_PAGES:
    data = json.loads(urllib2.urlopen(url).read())
    messages += [d['message'] for d in data['data'] if d.get('message')]
    current_page += 1
    url = data['paging']['next']

# Compute frequency distribution for the terms

fdist = nltk.FreqDist([term for m in messages for term in m.split()])

# Customize a list of stop words as needed

stop_words = nltk.corpus.stopwords.words('english')
stop_words += ['&', '.', '?', '!']

# Create output for the WP-Cumulus tag cloud and sort terms by freq along the way

raw_output = sorted([[escape(term), '', freq] for (term, freq) in fdist.items()
                     if freq > MIN_FREQUENCY and term not in stop_words],
                     key=lambda x: x[2])
```

```
# Implementation adapted from
# http://help.com/post/383276-anyone-knows-the-formula-for-font-s

min_freq = raw_output[0][2]
max_freq = raw_output[-1][2]

def weightTermByFreq(f):
    return (f - min_freq) * (MAX_FONT_SIZE - MIN_FONT_SIZE) / (max_freq
            - min_freq) + MIN_FONT_SIZE

weighted_output = [[i[0], i[1], weightTermByFreq(i[2])] for i in raw_output]

# Substitute the JSON data structure into the template

html_page = open(HTML_TEMPLATE).read() % (json.dumps(weighted_output), )

f = open(OUT_FILE, 'w')
f.write(html_page)
f.close()

print 'Date file written to: %s' % f.name

# Open up the web page in your browser

webbrowser.open('file://' + os.path.join(os.getcwd(), OUT_FILE))
```

图9-9表示一些样例结果。该标签云中的数据是在Boise State和Virginia Tech足球比赛时收集的。新闻供稿中最常见的词是“I”并不足为奇。总的来说，生成人们在你的留言板上讨论什么的一个快速可视化并不需要付出很多努力，你可以轻易地扩展它来获取其他文本数据源的要点。然而，如果你想追求具有有趣内容的标签云的自动构建，你可能需要考虑融合更先进的筛选技术和本书前面介绍过的NLP技术。可见，你的目标是可能是实体的短语，而不是执行简单的频率分析。回想一下，第8章提供了如何完成这一目标的简单概述。

小结

你可以在Facebook上做你能想象到的关于社交的任何事情，通过强大的API提供的大量数据提供了创建智能且惊人的数据驱动应用程序的巨大商机。我们在本章接触到的Facebook平台的粗略处理只是冰山一角，但愿它能提供很多有用的工具，你可以用这些工具来进一步分析，而且你可能成为下一个强大的（数据驱动）Facebook应用程序的创造者。

作为进一步练习，你可以考虑用标签云可视化你的好友的留言板或群组留言板中的非结构化文本，但是使用第8章介绍的以实体为中心的分析来代替本章使用的分词方案。根据好友发表的状态或评论总数，通过Graph API研究结构化数据来查找网络中最能聊的人是谁也是很有趣的。另一个好的想法是试着按好友“喜欢什么”来聚类他们，然后分析数据透视表（*http://en.wikipedia.org/wiki/Pivot_table*）中的喜好或把他们绘制成图。

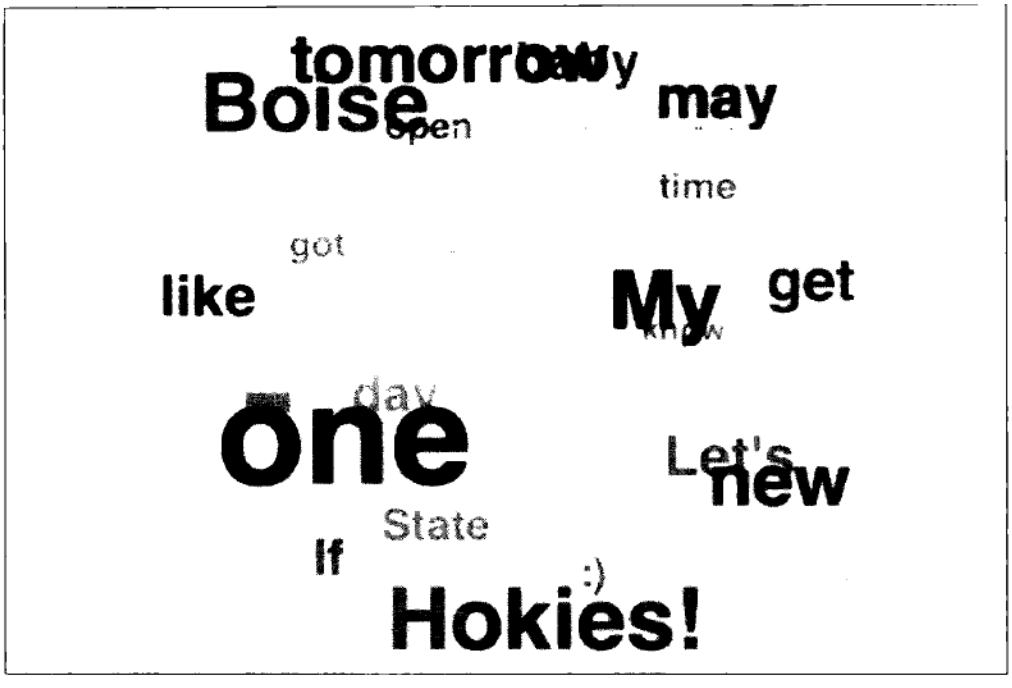

图9-9：旋转的标签名是高度可定制的，并且不费吹灰之力就可以启动和运行

第10章

语义网：简短的讨论

虽然前面的几章试图提供社交网络的概述，鼓励你挖掘数据，但是以简短的语义网补充说明结束是恰当的。这个简单的讨论并不打算以复述有趣的邮件列表讨论、博客文章和记录Web起源的其他信息源、它如何改变了我们近20年的生活，以及语义网一直是这一构想的一部分结束。然而，它确实想要让你参加类似鸡尾酒的讨论，虽然掩饰了这些问题的很多广度和深度，但愿能激起你对即将出现的可能性的兴趣。

发展中的变革

让我们从分析“语义网”开始。由于Web是有关共享信息的，因此语义网的实际定义是“有导致某种行为的足够的含意”[注1]，我们有理由推断：语义网主要是以某种有意义的方式表示知识。但是让我们采用另外一步，不要假设是人类使用了它表示的信息。让我们考虑可以实现的可能性，如果信息以一种机器完全可理解的方式共享——一种不太明确的方式，Web机器人这样的相当熟练的用户代理可以提取、解释、使用该信息作出重要的决定。在这个方向上我们已经有所作为：例如，第2章讨论了微格式如何使某些领域变得可能，第9章关注了Facebook是如何积极自主地用其开放图谱协议将一个显式的图结构整合到Web中的。在我们的梦想变得遥不可及之前，让我们停下来仔细考虑一下我们该如何完成这些工作。

互联网只是网络的网络[注2]，从技术的角度来说，它吸引人的地方是建立在更低级协议之

注1：在Toby Segaran、Jamie Taylor和Colin Evans编写的《Programming the Semantic Web》（*http://oreilly.com/cato log/9780596153823*）（O'Reilly）中定义。

注2：互联网意味着“共有或合作网络。”

上的越来越高级的协议层最终如何生成容错的全球计算基础结构。在网上活动中，我们依靠每天几十条协议，而无须考虑它们。然而，有一个时常会考虑的协议：HTTP，你输入到浏览器中的每个URL的前缀，大量超文本文档（HTML网页）的启用协议，把它们全部连接起来成为我们熟悉的Web。但是你早就知道了，Web并不只是超文本；它包含各种嵌入式技术，如JavaScript、Flash和新兴的HTML5资产（如音频和视频流）。所有的这些高交互性都使得“超文本”听起来有些过时，是不是？

我们通过现代浏览器（包括移动或平板设备中的浏览器）使用HTTP交互的文档、平台和应用程序的网络世界的概念非常模糊，但是它可能非常接近大部分人对“Web”的理解。在一定程度上，2004年出现的Web 2.0思维过程背后的动机是更精确地定义日益模糊的Web是什么和它正在成为什么的概念。沿着这条思路，一些人以为Web来自它的开端，直到高度交互式的Web应用程序和用户协作的当今时代作为Web 1.0，富互联网应用程序（Rich Internet Application，RIA）和协作的当前时代作为Web 2.x时代，尚未到来的语义空间时代作为Web 3.0（查看表10-1）。Web 3.0是什么意思目前还没有真正的共识，但是对这一主题的大多数讨论一般包括“语义网”短语和被机器在网络规模中还不能实现的方式使用和作用于的信息的概念。例如，机器仍然难以提取并推断网络中现成文档中包含的事实。关键字搜索和启发式方法一定可以提供相关的搜索结果列表，但是仍然要求人类智慧来解释并合成文档本身中的信息。Web 3.0和语义网是否相同仍然有待求证；但是，通常认为术语语义网指的是与我们已经知道并热爱的网络相像的网络，但是现在机器已经可以在某种粒度级别中提取并作用于包含在文件中的信息。

表10-1：各种Web表现形式/时代以及它们的优点

表现形式/时代	优点
互联网	应用程序协议，如SMTP、FTP、BitTorrent、HTTP等
Web 1.0	大部分是静态HTML网页和超链接
Web 2.0	平台、协作、丰富的用户体验
社交网络（Web 2.x???）	人们和他们在虚拟和现实世界的社交连接和活动
Web 3.0（语义网）	机器可理解的大量内容

人不可能只靠事实生活

语义网络表示知识的基本构造称为三元组，它是表示事实的一种非常直观和自然的方法。例如，我们多次考虑的句子（“Mr. Green killed Colonel Mustard in the study with candlestick”），被表示为（*Mr. Green*，*killed*，*Colonel Mustard*）这种三元组，其中三元组的成分是句子的主语、谓语和宾语。资源描述框架（Resource Description

Framework，RDF）是定义和允许交换三元组的语义网模型。RDF具有高度可扩展性，因为它提供了表示知识的基础，它也可以用来定义称为本体（ontology）的专业词汇，它提供了为特定领域建模的精确语义。不只是提到的具体语义网技术，如RDF（*http://www.w3.org/RDF/*）、RDFa（*http://www.w3.org/TR/xhtml-rdfa-primer/*）、RDF模式（*http://www.w3.org/TR/rdf-schema*）和OWL（*http://www.w3.org/TR/owl2-overview/*）会超过最后时刻的范围，但是我们会研究一个高级示例，它试图解释语义网相关的一些渲染。

开放世界与封闭世界假说

在Prolog[注3]这类逻辑编程语言中的推理方式与RDF栈这类其他方式之间的有趣差异是它们是否作出关于宇宙的开放世界或封闭世界假说。Prolog这类逻辑编程语言和大多数传统数据库系统假设的是封闭世界，而RDF技术一般假设开放世界。在封闭世界中，没有明确告知你的都被认为是假的；而开放世界中，你不知道的事情可以更恰当地当做未定义的（说“未知”的另一种方式）。区别是假设开放世界的推理不排除没有明确在知识库中表示的事实，而Prolog编程语言或大多数数据库系统封闭世界的推理会排除没有明确表示的事实。此外，在假设封闭世界的系统中合并矛盾知识通常会引发错误，而假设开放世界的系统可能会尝试得出新推论，这些推论会以某种方式协调矛盾信息。和你想象的一样，开放世界系统非常灵活，可能会导致某些非常有趣的难题；当合并不同的知识库时，这种可能会变得尤为突出。

直觉上，你可以这样考虑它：系统基于封闭世界推理假说作出推断，它们得到的数据是完整的，从添加新事实时以前的所有事实仍然成立的意义上来说，它们是典型的非单调推理。与此相反，开放世界系统不假设他们数据的完整性，而且是单调的。正如你可能想象到的，存在比较两个假说优点的实质性辩论。如果有人对语义网感兴趣，你至少应该意识到这个问题。对于与RDF相关的，W3C文档的官方表述如下[注4]：

> 为了便于互联网规模上的操作，RDF是一个开放世界架构，它允许任何人描述任何资源。一般来说，它并不假设可以得到所有资源的完整信息。RDF并不阻止任何人作出无意义或与其他陈述，或人们看到的世界不一致的断言。使用RDF的应用程序的设计者应该意识到这一点，可以让自己的应用程序忍受不完整或不一致的信息来源。

注3：非常鼓励你试试Prolog这类真正的基于逻辑的编程语言，它是在一个专门设计的范式中编写的，这样你就可以表示知识，并从已有的事实中推断新信息了。GNU Prolog（*http://www.gprolog.org/*）是一个良好的起点。

注4：查看*http://www.w3.org/TR/rdf-concepts/*。

如果你想进一步仔细研究这个话题的话，你也可以看看Peter Patel-Schneider和Ian Horrock的《Position Paper: A Comparison of Two Modelling Paradigms in the Semantic Web》（*http://www2006.org/programmer/files/xhtml/4015-patel-schneider/4015-patel-schneider-xhtml.html*）。不管你是否决定深入研究这个主题，都要记住：网络中得到的数据是不完整的，假设封闭的世界（也就是说，考虑所有未知的信息都是错误的）迟早会造成严重后果。

使用FuXi推断开放世界

设计RDF模式和OWL这类基本语言是为了可以使用精确的词汇，以机器可读的方式表示三元组（Mr. Green，killed，Colonel Mustard）这类事实，它是完全实现语义网的必要不充分条件。一般来说，如果你拥有了事实集，下一步就是执行对事实的推理，得出基于这些事实的结论。形式推理概念至少要追溯到古希腊亚里士多德的三段论，对于机器如何利用这个明显的关系在过去的50年并没有被对人工智能感兴趣的研究人员所忽视。充满Jena（*http://jena.sourceforge.net/*）和Sesame（*http://www.openrdf.org/*）这类企业级选项并且基于Java的景观似乎是重量级行动存在的地方，但是幸运的是，我们有一些基础的选项来使用Python。

你可能会遇到的具有推理能力的支持Python的最佳选择是Fuxi（*http://code.google.com/p/fuxi/*）。Fuxi是语义网中强大的逻辑推理系统，它使用一个叫做正向链接（*http://en.wikipedia.org/wiki/Forward_chaining*）的技术从现有信息中推断出新信息。你可以从一组事实出发，通过使用一组逻辑规则，从已知的事实中推断出新事实；重复这一过程，直到某个特定的结论被证明或反驳，或无法推断更多事实为止。据说FuXi传递的正向链接类型是“两种声音”，因为产生的任何新事实都是正确、完整的，因为任何正确的事实最终都可以被证明。对命题式和一阶逻辑的全面讨论可以轻易地组成一本书，如果你想要深入研究的话，Stuart Russell和Peter Norvig编写的经典教材《Artificial Intelligence:A Modern Approach》（Prentice Hall）可能是最全面的资源了。

为了说明FuXi这类系统可以提供的推理能力类型，让我们考虑著名的示例——亚里士多德的三段论[注5]，它提供了一个包含“苏格拉底是人”和“凡人都会死”这种事实的知识库，它允许你推断“苏格拉底会死”。虽然这个问题看起来很繁琐，但是要记住，当可以得到更多事实，而且这些事实会产生新的事实，而它们又会产生新事实时，产生“苏格拉底会死”的新事实的确定性算法会以相同的方式工作。例如，考虑一个稍微复杂的知识库，它包含一些附加事实：

- 苏格拉底是人

注5：　现代说法中，三段论常称为一个“含义”。

- 凡人都会死
- 只有上帝住在奥林匹克山
- 所有凡人都喝威士忌
- 查克·诺里斯住在奥林匹克山

如果陈述了上面的知识库，然后提出“苏格拉底喝威士忌吗？”这个问题，在你可以推断后续的“苏格拉底喝威士忌”之前，你首先要推断的事实是“苏格拉底会死”。为了说明所有这些如何在代码中运行，考虑Notation3（*http://www.w3.org/DesignIssues/Notation3*）（N3）中表示的相同知识库，如示例10-1所示。N3是一种简单而强大的语法，它表示RDF中的事实和规则。虽然RDF有很多不同的表示格式，但是很多语义网选择使用N3，因为它的可读性和可表达性使它容易理解。

示例10-1：用N3表示的小型知识库

```
#Assign a namespace for logic predicates
@prefix log: <http://www.w3.org/2000/10/swap/log#> .

#Assign a namespace for the vocabulary defined in this document
@prefix : <MiningTheSocialWeb#> .

#Socrates is a man
:Socrates a :Man.

@forAll :x .

#All men are mortal: Man(x) => Mortal(x)
{ :x a :Man } log:implies { :x a :Mortal } .

#Only gods live at Mt Olympus: Lives(x, MtOlympus) <=> God(x)
{ :x :lives :MtOlympus } log:implies { :x a :god } .
{ :x a :god } log:implies { :x :lives :MtOlympus } .

#All mortals drink whisky: Mortal(x) => Drinks(x, whisky)
{ :x a :Man } log:implies { :x :drinks :whisky } .

#Chuck Norris lives at Mt Olympus: Lives(ChuckNorris, MtOlympus)
:ChuckNorris :lives :MtOlympus .
```

使用--ruleFacts选项运行FuXi会告诉它从输入源程序中解析事实，可以指定--rules选项，计算源程序中的其他事实。如果从命令行运行FuXi的话，你应该会看到与示例10-2相似的输出。请注意，在执行完easy_install fuxi命令后，路径中才会出现FuXi。

示例10-2：从命令行运行示例10-1中知识库的FuXi结果

```
$ FuXi --rules=foo.n3 --ruleFacts

@prefix _7: <file:///Users/matthew/MiningTheSocialWeb#>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
```

```
_7:ChuckNorris a _7:god.

_7:Socrates a _7:Mortal;
_7:drinks _7:whisky.
```

该程序的输出告诉我们最初的知识库中没有明确指明以下事情：查克·诺里斯是上帝，苏格拉底是凡人，苏格拉底喝威士忌。虽然对于大多数人来说，很显然就能推断出这些事实，但是用机器推断它们就是另一回事了——这会使得事情很刺激。

警告： 需要注意的是对查克·诺里斯事实的随意断言（即使是在样本知识库的上下文中）被证明有害你的身体健康或你的电脑生活[注6]。已经正式警告过你了。

如果这个简单的示例可以激励你，那么一定要深入研究FuXi和语义网拥有的潜力。可以说语义网比社交网络更先进、更复杂，研究它肯定是非常值得的追求——尤其是你因推理带给社交数据的可能性而兴奋。

期望

虽然本书故意省略了语义网的讨论可能已经造成了社交网络和语义网之间随意和严格的分歧这种印象，但分歧其实是相当模糊且不断变化的。这可能是Web上社交数据的必然扩散，与各方发表的从食品网到LinkedIn、Facebook的开放图谱协议微格式这样的举措相结合，你自己这样的数据挖掘者的创造性努力大大加快了语义网的实现，这可能与15年前的炒作相同。社交数据的扩散可能是对语义网发展的巨大推动，它可能会使代理商做出能代表我们的重要行动。我们还没有达成目标，但是要抱着希望！或者，用另一种说法，至少考虑一下伊壁鸠鲁的箴言，他或许会这么说："不要因为期望你没有的就破坏你拥有的；但是要记住，你现在拥有的曾经是你期望的东西。"

注6： 参见*http://www.chucknorrisfacts.com*。

作者介绍

马修·罗塞尔（Matthew A. Russell），Digital Reasoning Systems公司的技术副总裁和Zaffra公司的负责人，是热爱数据挖掘、开源和Web应用技术的计算机科学家。他也是《Dojo: The Definitive Guide》（O'Reilly出版社）的作者。在Linkedin上联系他或在Twitter关注@ptwobrussell，可随时了解他的最新动态。

封面介绍

本书封面上的动物是土拨鼠（学名Marmota monax），又称美洲旱獭（woodchuck：该名字源自其Algonquin叫法“wuchak”）。土拨鼠与美国/加拿大2月2日的土拨鼠节相关。民俗认为，如果这一天土拨鼠从它的洞里出来，而且能看到它的影子的话，那么冬天还会持续6周。该说法的支持者说，啮齿动物预测的准确性为75%～90%。很多城市中都有著名的土拨鼠天气预报员，包括美国庞克瑟托克小镇上的菲尔（比尔·默瑞于1993年在电影《土拨鼠节》中所扮演的主角）。

这个传说可能源于土拨鼠是整个冬季都冬眠的少数几种物种之一。作为食草动物，土拨鼠主要依靠植物、浆果、坚果、昆虫和人类园林作物在夏天贮存脂肪，这也导致很多人认为它是害虫的原因。然后，它们会在冬天挖一个洞穴，从10月到来年的3月它们都会一直呆在那里（虽然在温带地区，或者假设，如果它们是与它们齐名的节日的关注重心，它们可能会更早出现）。

土拨鼠是松鼠家族中最大的成员，大约为16～26英寸长，4～9磅重。它们都有弯曲、厚厚的爪子作为理想的挖掘工具，而且有两层皮毛：密集的灰色绒毛和较长毛发的浅色外层，二者提供了对自身的保护。

土拨鼠遍及加拿大的大部分地区和美国北部地区，生活在开阔的空间和林地富足的地方。它们能爬树和游泳，但经常生活在离洞穴不远的地面上，洞穴用来睡觉、养育后代并保护自己。这些洞穴通常有2～5个入口，长达46英尺的地道。

封面图片来自于Wood的电影《Animate Creatures》。

[General Information]
书名=社交网站的数据挖掘与分析
作者=
页数=302
出版社=
出版日期=
SS号=12960063
DX号=
URL=http://book2.duxiu.com/bookDetail.jsp?dxNumber=000008240698&d=F42263DB04D4B0865FC32CAA49480835